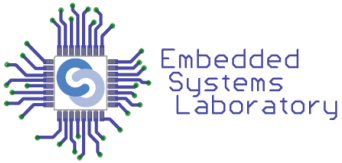


PROIECTAREA CU MICROPROCESOARE

Facultatea de Automatică și Calculatoare
Universitatea Politehnica București

Sper că sunteți aici din cauza asta

POO, PP, PA

- Cum ajunge un program scris într-un limbaj de nivel înalt să fie executat pe mașina de calcul?
- Ce se intâmplă la mijloc?
- Cum putem să proiectăm un procesor complex pornind de la porți logice și conexiuni?

CN1 & 2

Perspectiva programatorului asupra felului în care funcționează un sistem de calcul

Perspectiva Architectului de Sistem:
Cum să proiectez un sistem de calcul care să satisfacă cerințele de design.

Perspectiva inginerului HW asupra felului în care merge un calculator

About me

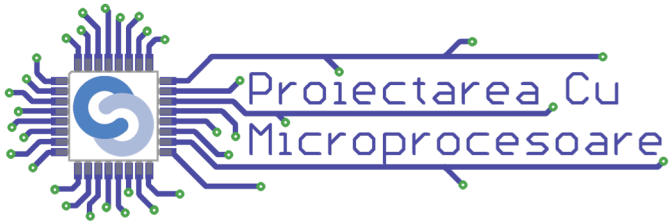
Dan Ștefan Tudose

- dan.tudose@cs.pub.ro
- Office: ED422
- Office Hours: luni 5:00-6:00pm
- https://elf.cs.pub.ro/wsn/wiki/dan.tudose
- PhD in Computer Science
- Research & teaching:
  - Computer architecture, hardware/software interaction
  - Embedded and Pervasive Computing
  - Wireless Sensor Networks
  - Low Power Computing Architectures, Energy Harvesting
  - Fault tolerance

dan.tudose@cs.pub.ro

Echipa de Asistenți

http://cs.curs.pub.ro/wiki/pm/administrative/echipapm



Iulian Mateșică
Marius Andrei
Răzvan Bărbascu
Vlad Corneci
Dorin Ioniță
Cristi Trancă
Ana Drăgan
Alexandru Predescu
Ovidiu Știru
Mihaela Lungoci
Adrian Mocanu
Adrian Stratulat
Adriana Drăghici
Andrei Bîrlică
Andrei Voinescu
Daniel Ghilință
Ioana Popescu

Important Stuff

Curs: Moodle

Laboratoare: http://cs.curs.pub.ro/wiki/pm/

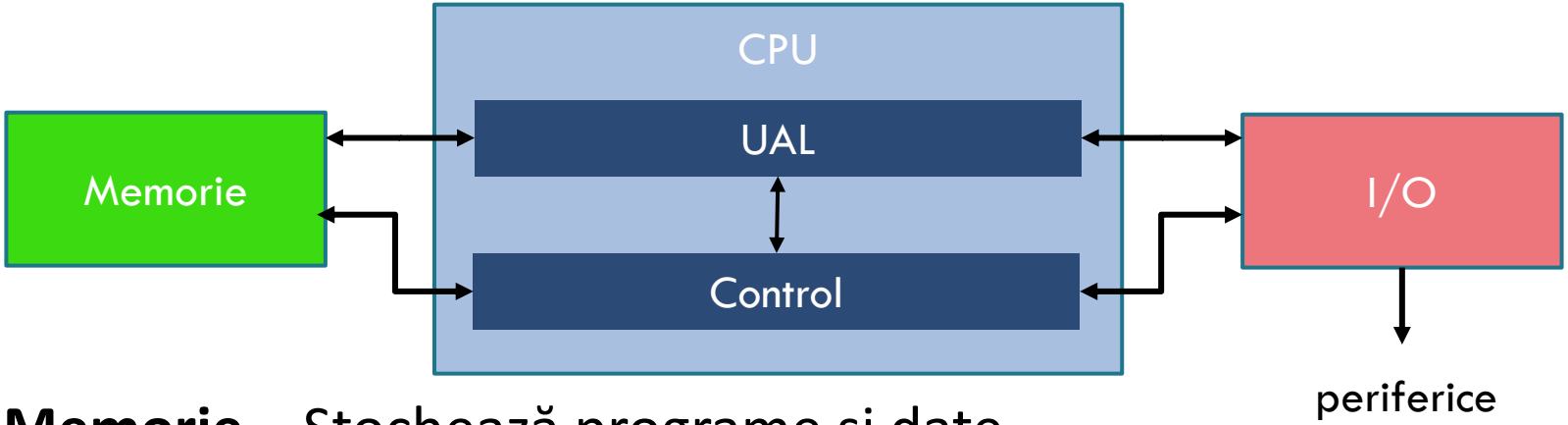
Notare:

- 1 punct activitatea laborator
- 1 punct lucrare laborator
- 3 puncte proiect
- 5 puncte examen final

Cerințe minime pentru a promova:

- minim 6 prezente la laborator
- minim 2.5 puncte din cele 5 puncte din examenul final
- Activitate la curs!

Organizarea generală a unui calculator



Memorie – Stochează programe și date
CPU – Central Processing Unit
UAL – Unitate Aritmetică și Logică
Control – Secvențiază transferurile de date
I/O – comunică cu exteriorul

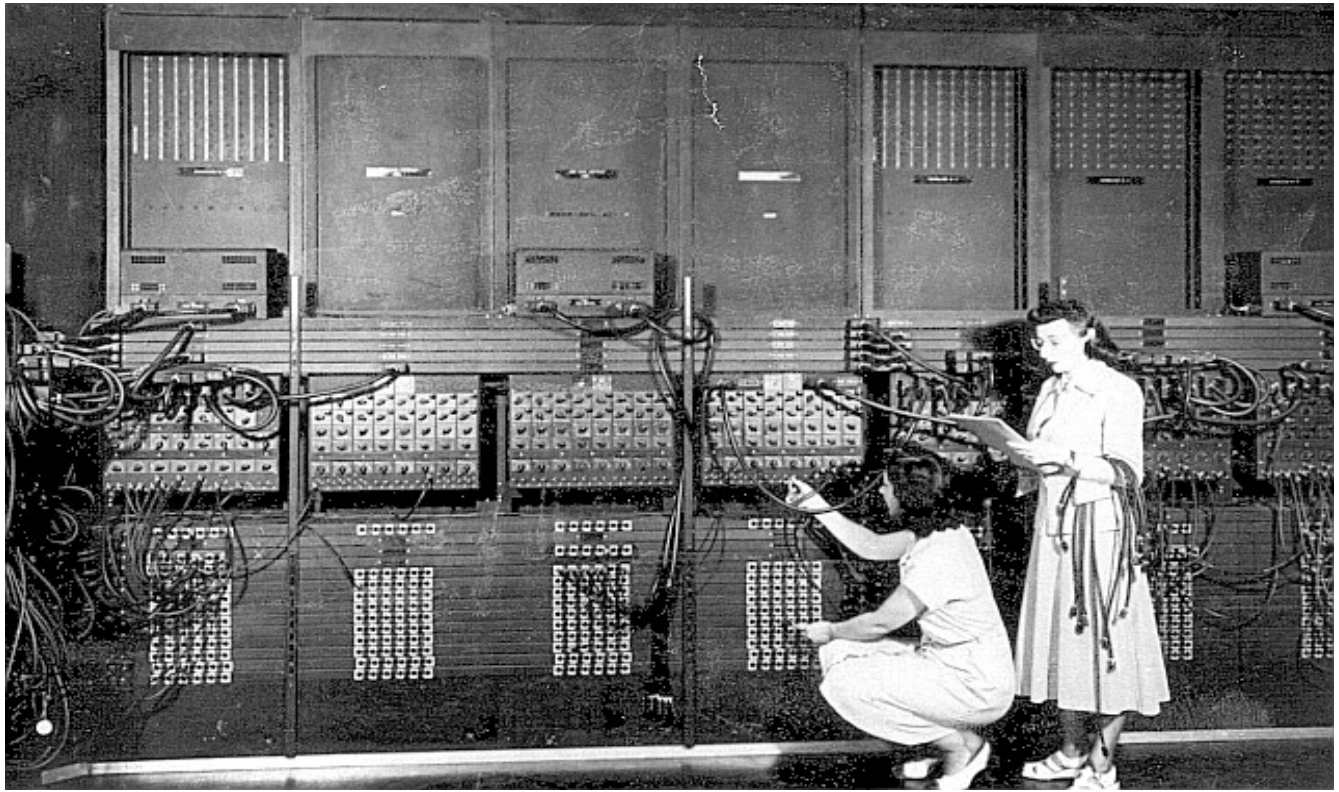
Arhitectura sistemelor de calcul

Organizarea clasică (von Neumann) a unei mașini de calcul:

- Programe stocate sub formă de instrucțiuni
- ”General-purpose”
- Datele și instrucțiunile stocate în memorie
- Instrucțiunile executate în ordine

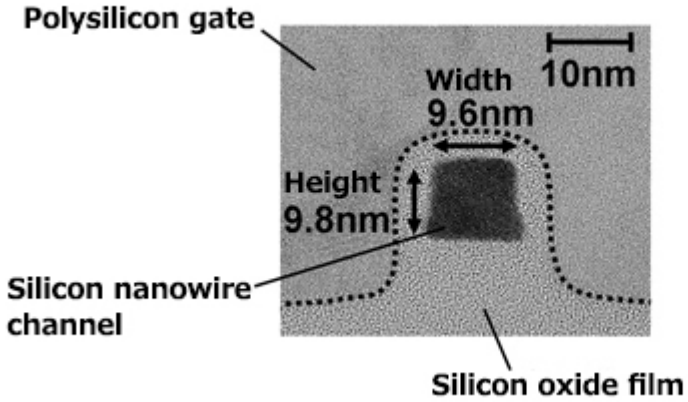
Arhitectura unui sistem este determinată atât de hardware cât și de software.

Programarea calculatoarelor înainte de von Neumann

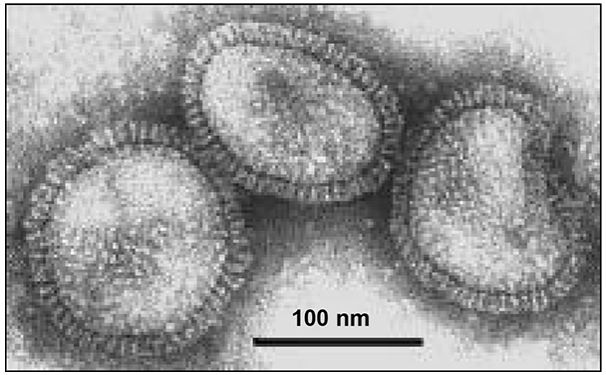


Putting things into perspective

10 µm – 1971
6 µm – 1974
3 µm – 1977
1 µm – 1985
600 nm – 1994
250 nm – 1997
130 nm – 2001
90 nm – 2004
65 nm – 2006
32 nm – 2010
22 nm – 2012
14 nm – 2014
10 nm – 2017
7 nm – ~2018
5 nm – ~2020

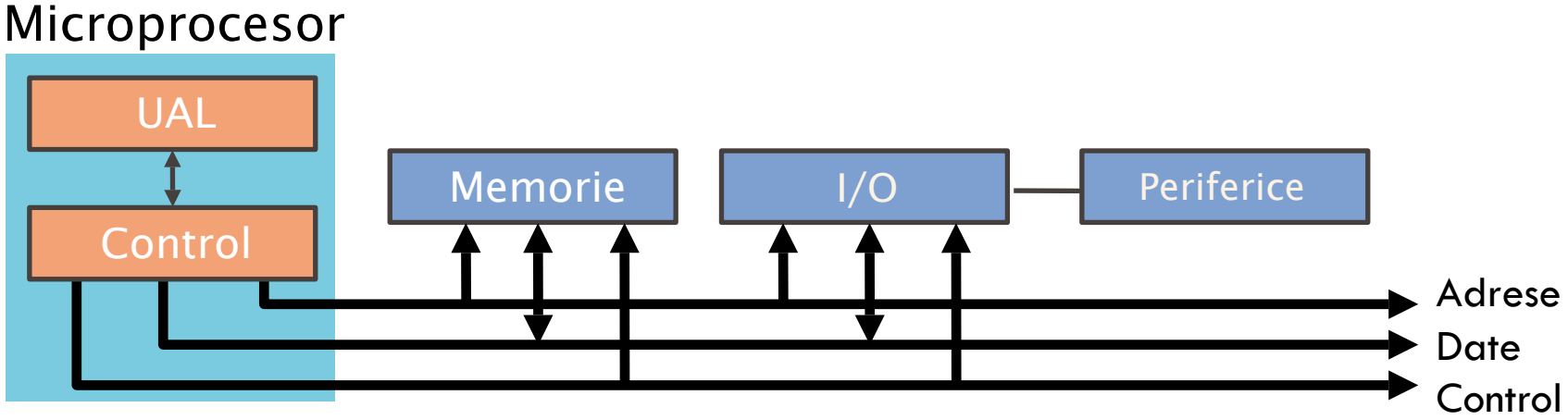


Tranzistor - 10nm



Influenza A virus – 100nm

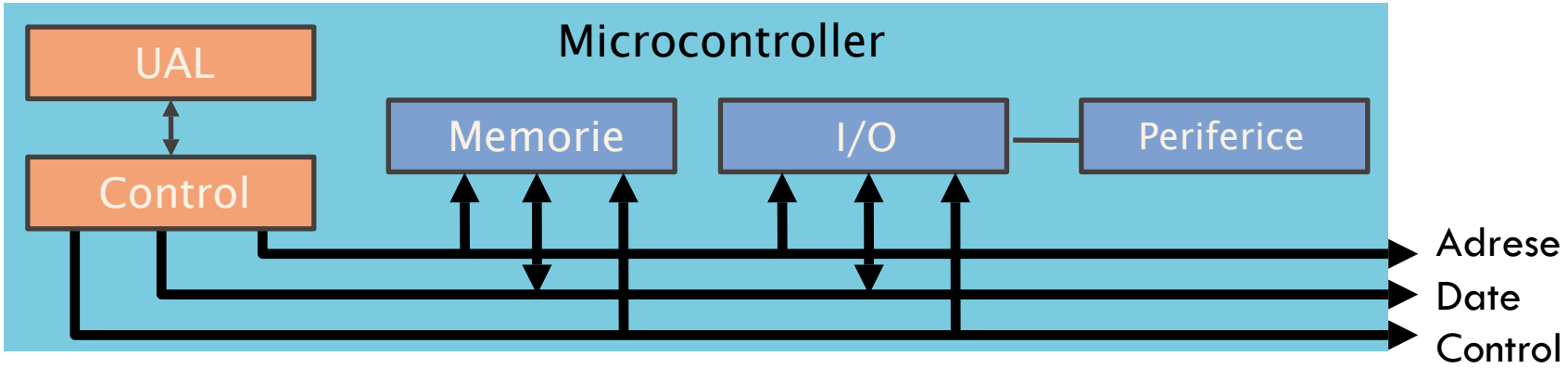
Microprocesoare



High performance, general purpose, pentru PC & workstations
Instruction decode and control, arithmetic/logic operations, registers, timing, external control
Cost tipic: $75 - $500

Cerere anuală: zeci de milioane

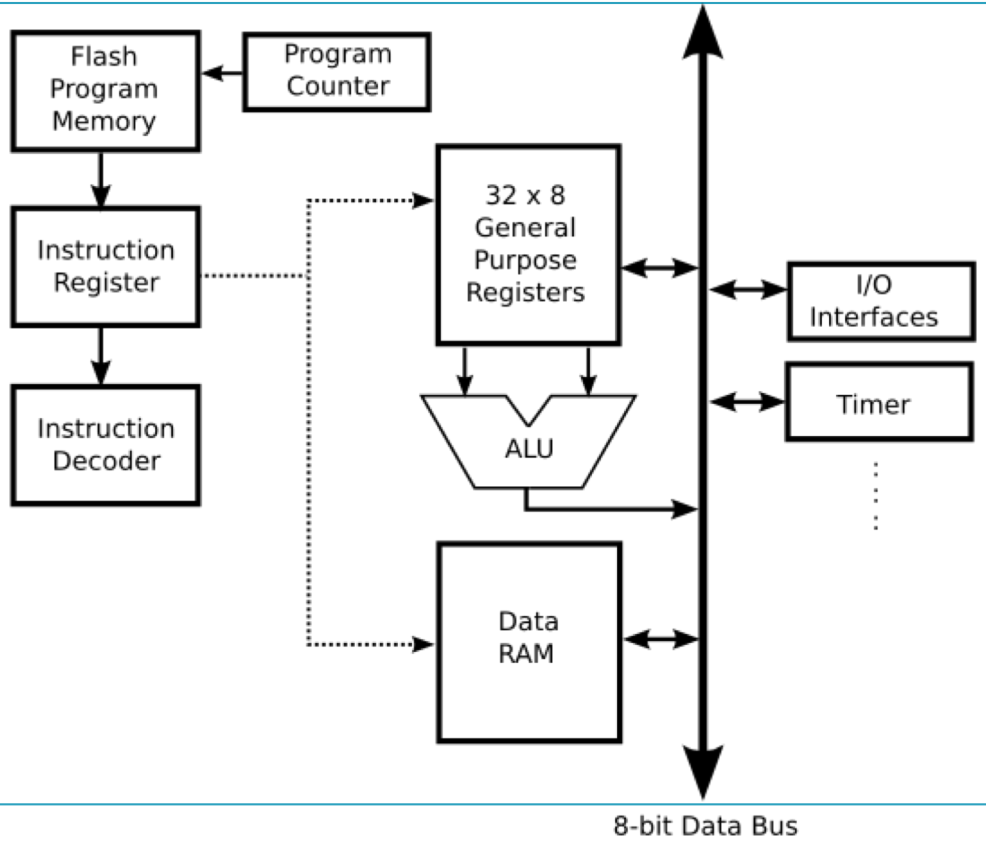
Microcontrollere



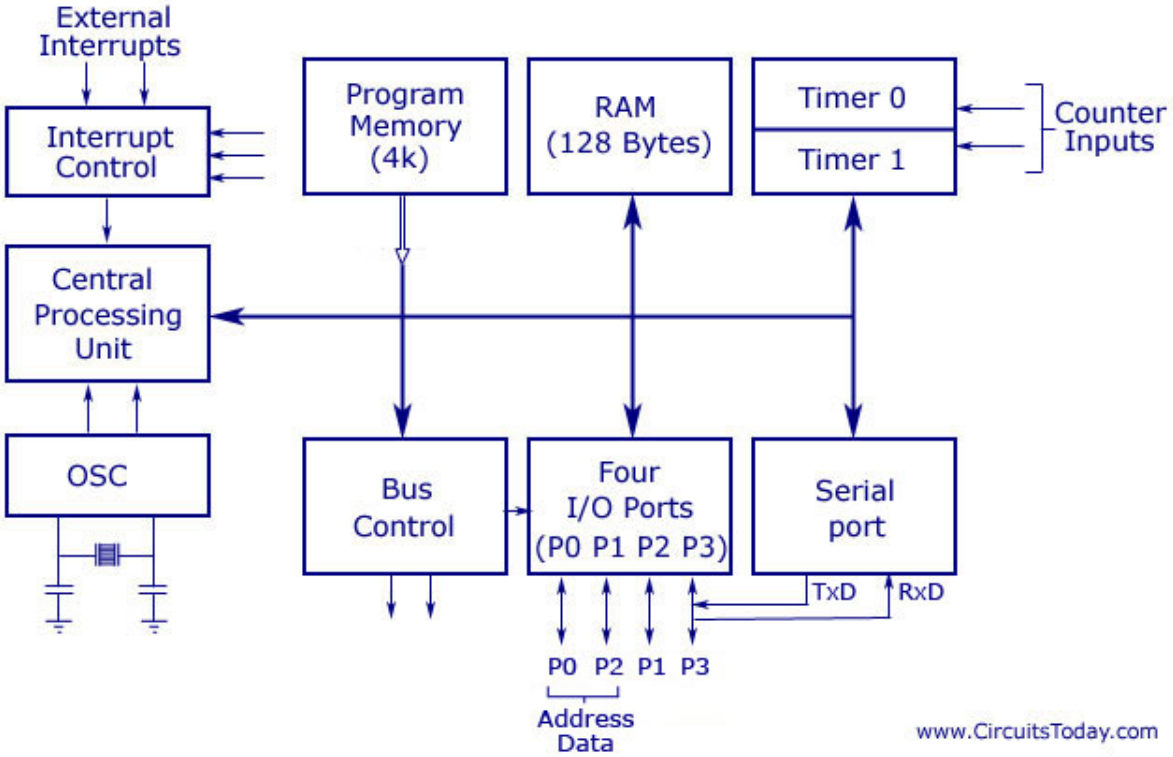
Nivel mult mai mare de integrare pentru controlul embedded
Funcții de microprocesor, plus memorie și periferice on-chip (ex. porturi, timere)
"Swiss army knife" al tehnologiei
Cost tipic: $1- $25

Cerere anuală: miliarde!

Baseline Microcontroller

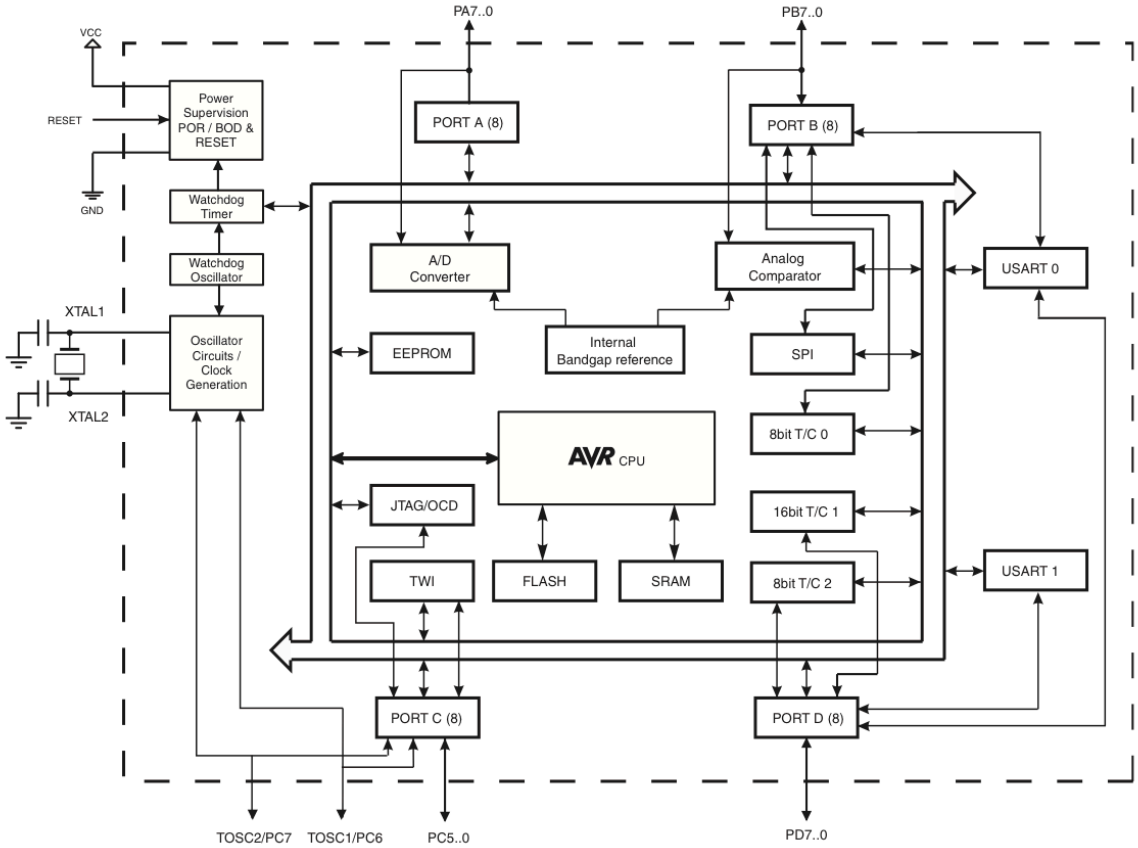


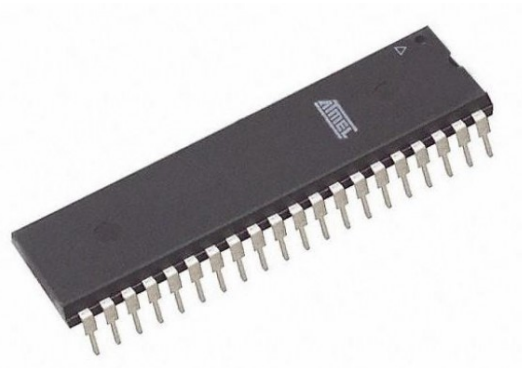
8051 - Intel


External Interrupts
Interrupt Control
Program Memory (4k)
RAM (128 Bytes)
Timer 0
Timer 1
Counter Inputs
Central Processing Unit
OSC
Bus Control
Four I/O Ports (P0 P1 P2 P3)
Serial port
TxD
RxD
P0 P2 P1 P3
Address Data
www.CircuitsToday.com

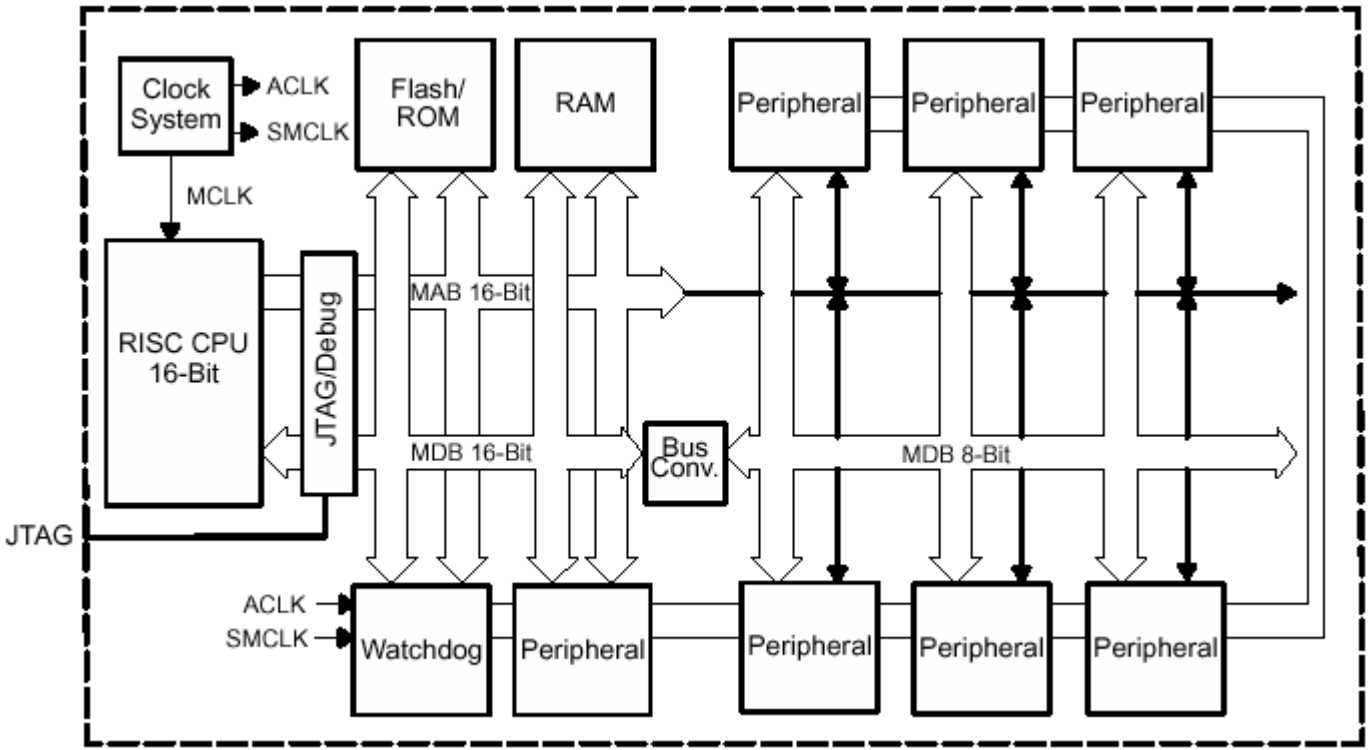
P8051AH 0649
8736041
93X3903 Y15
INTEL '80

ATmega324 - Atmel

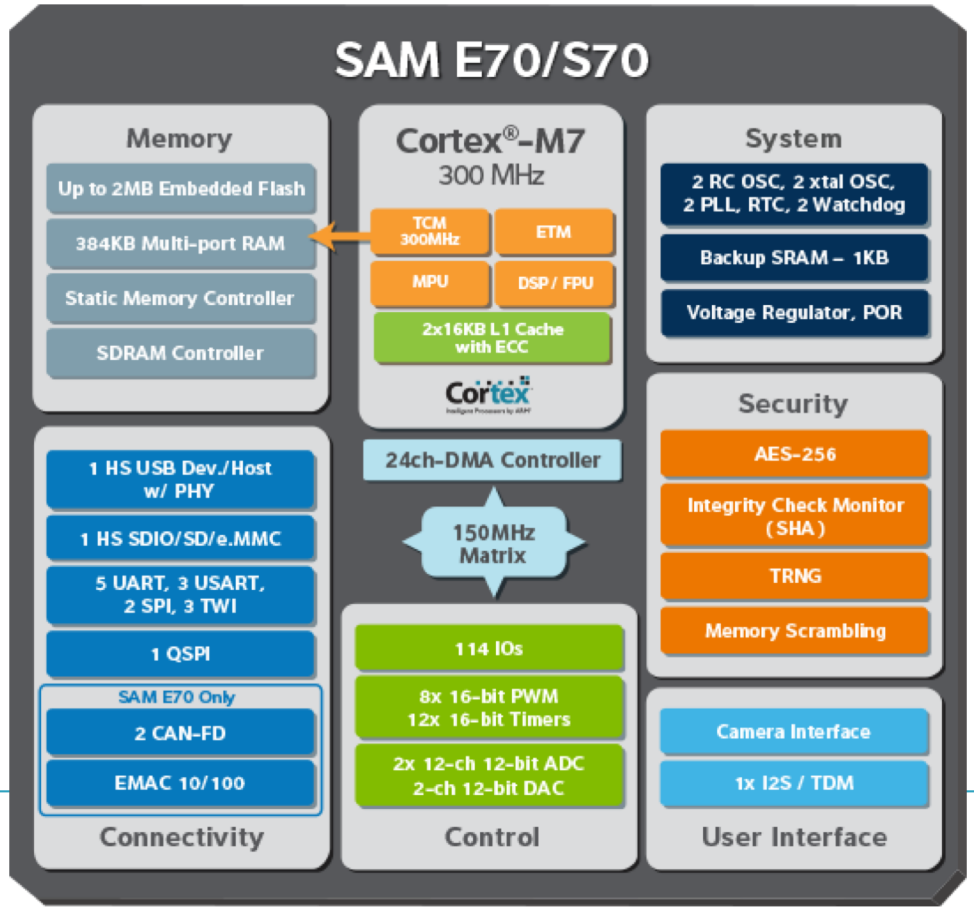




MSP430 – Texas Instruments



SAM E70 - Atmel

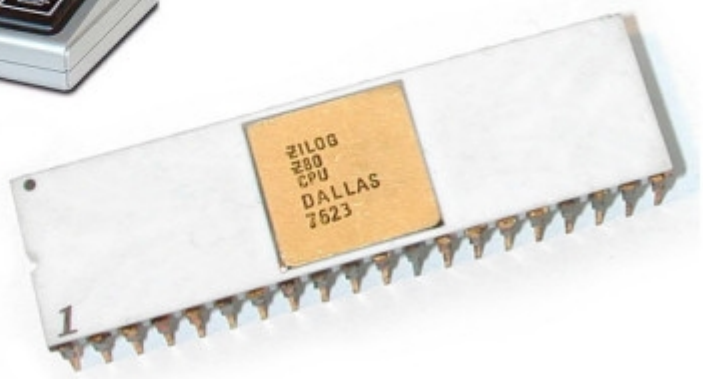


World-Changing Microprocessors

Zilog Z80

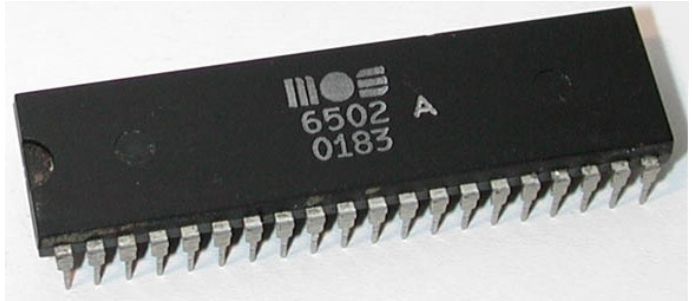
- Released 1976
- 8-bit architecture
- Enhanced clone of Intel 8080
- Nintendo Game Boy
- Sega Master System
- Texas Instruments graphing calculators
- Still sold for embedded applications

Zilog



MOS Technology 6502 (1975)

- 6502 - inexpensive and capable
- Intel 8080 sold for $149
- 6502 sold for only $25
- **Apple II (1977)**
- Atari 2600
- Nintendo Entertainment System (NES)



Motorola 68000

- Released 1980
- 16/32 bits
- UNIX workstations
- Apple Macintosh

MOTOROLA

AIM PowerPC 601

- 1992
- Apple, IBM & Motorola ”unholly alliance”
- Break the Intel + Microsoft monopoly
- Used in Apple Macintosh from 1994 to 2006
- Nintendo Wii
- Xbox 360
- Sony PlayStation 3 (part of Cell processor)

A Brief History of Intel

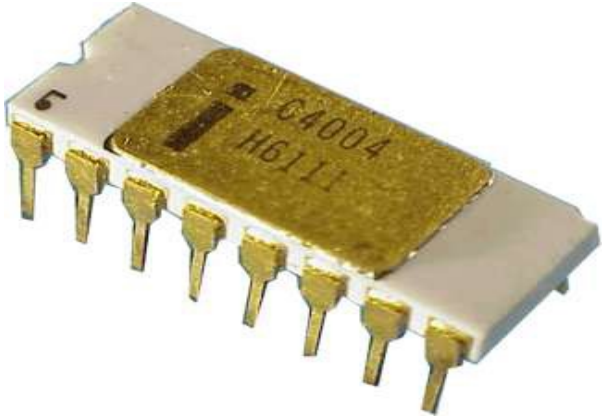
Intro

- Fairchild Semiconductors (founded in 1957) invented the first IC in 1959
- In 1968, **Robert Noyce, Gordon Moore, Andrew Grove** resigned from Fairchild Semiconductors
- They founded their own company **Intel** (Integrated Electronics)
- Intel grown from 3 man start-up in 1968 to industrial giant by 1981

➢ It had 20,000 employees and $188 million revenue.

4-BIT MICROPROCESSORS

INTEL 4004



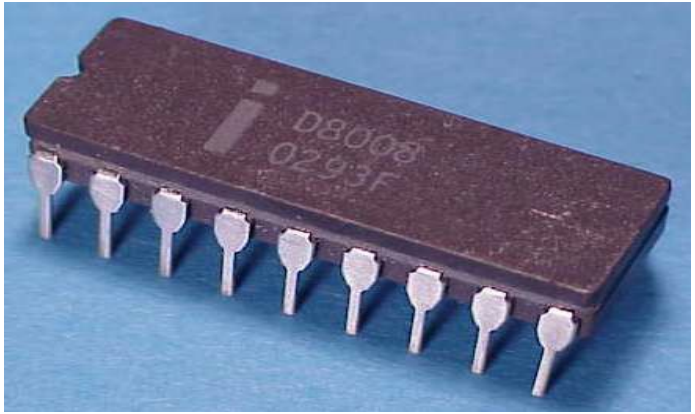
- Introduced in 1971
- First Intel uP
- Word width: 4-bit
- 2300 transistors
- Clock: 108KHz/500/740
- 46 instructions
- Registers: 16 x 4-bit
- Stack: 12 x 4-bit
- Address space
  - 1Kb of program, 4Kb of data

INTEL 4040

- Introduced in 1974
- 60K instructions/sec.
- 3000 transistors
- Support chips:
  - 4201 – Clock Generator 500 to 740 kHz using 4 to 5.185MHz crystals
  - 4308 – 1KB ROM
  - 4207 – General Purpose byte Output port
  - 4209 – General Purpose byte Input port
  - 4211 – General Purpose byte I/O port
  - 4289 – Standard Memory Interface
  - 4702 – 256 byte UVEPROM
  - 4316 – 2KB ROM
  - 4101 – 256 4-bit word RAM

8-BIT MICROPROCESSORS

INTEL 8008



- Introduced in 1972
- First 8-bit uP
- Word width: 8-bit
- Clock: 800KHz
- 3500 transistors
- 48 instructions
- Registers: 6 x 8-bit
- Stack: 17 x 7-bit
- Address space: 16KB

INTEL 8080

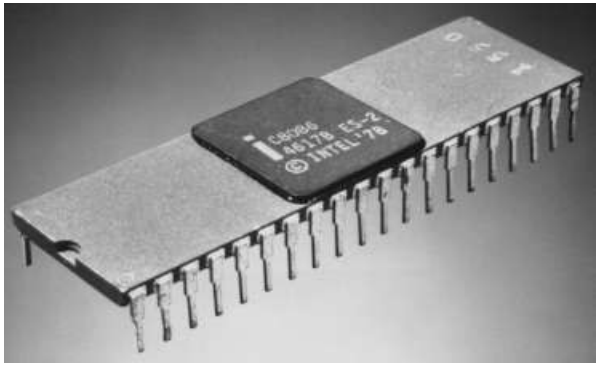
- Introduced in 1974.
- Clock speed 2 MHz.
- It had 6,000 transistors
- 10 times faster than 8008.
- Clock: 2M-3MHz
- Address space: 64KB
- Registers: 6 x 8-bit
- Distinct Address & Data Buses
- IO ports, Stack pointer

INTEL 8085

- Introduced in 1976
- Binary compatible with 8080
- Clock speed 3MHz
- 8-bit data bus 16-bit address bus
- 6,500 transistors.
- 64 KB of addressable memory
- 246 instructions
- Over 100 million copies sold

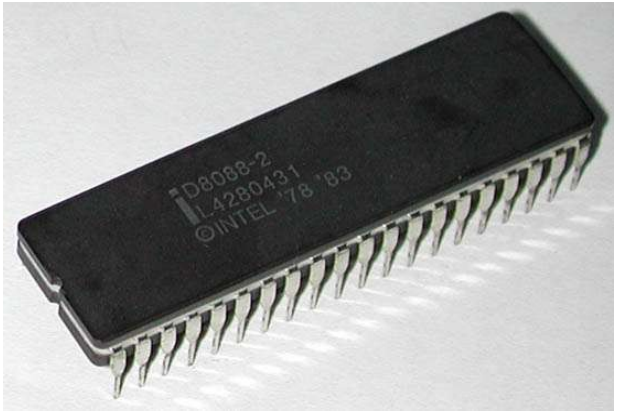
16-BIT MICROPROCESSORS

INTEL 8086



- Introduced in 1978
- First 16-bit µP
- 4.77 MHz, 8 MHz and 10 MHz
- 16-bit data bus and 20-bit address bus
- 29,000 transistors
- 2.5 million instructions per second
- Could access 1 MB of memory
- 22,000 instructions
- ***Multiply*** and ***Divide*** instructions

INTEL 8088



- Introduced in 1979.
- Created as a cheaper version of Intel's 8086
- 16-bit processor with an 8-bit external bus
- 2.5 million instructions per second
- This chip became the most popular in the computer industry when IBM used it for its first PC.

INTEL 80186 & 80188

- Introduced in 1982
- Clock speed was 6 MHz
- 80188 was a cheaper version of 80186 with an 8-bit external data bus
- Additional components like:
  - Interrupt Controller
  - Clock Generator
  - Local Bus Controller
  - Counters

INTEL 80286

- Introduced in 1982
- Clock speed 8 MHz
- 16-bit data bus and 24-bit address bus
- 16 MB of addressable memory
- 4 million instructions per second

32-BIT MICROPROCESSORS

INTEL 80386

- Introduced in 1986
- First 32-bit µP
- 32-bit data bus & 32-bit address bus
- 4 GB of addressable memory
- 2,750,000 transistors
- Clock speed varied from 16 MHz to 33 MHz depending upon the various versions.
- Different versions:
  - 80386 DX
  - 80386 SX
  - 80386 SL

The best selling microprocessor in history.

INTEL 80486

- Introduced in 1989
- 1.2 million transistors
- 8 KB of cache
- Clock speed varied from 16 MHz to 100 MHz depending upon the various versions.
- Five different versions
  - 80486 DX
  - 80486 SX
  - 80486 DX2
  - 80486 SL
  - 80486 DX4

INTEL PENTIUM

- Introduced in 1993.
- Originally named 80586
- Clock speed 66 MHz
- Its data bus is 32-bit and address bus is 32-bit
- Could address 4 GB of memory
- Could execute 110 million instructions per second
- Cache memory
  - 8KB Data
  - 8KB Instructions

INTEL PENTIUM PRO

- Introduced in 1995
- 21 million transistors
- Primarily used in server systems
- Cache memory:
  - 8KB for instructions
  - 8KB for data
- L2 cache of 256KB

INTEL PENTIUM II

- Introduced in 1997
- Clock speed was 233MHz to 500MHz
- Could execute 333 million instructions per second
- MMX technology was supported
- L2 cache & processor were on one circuit

INTEL PENTIUM II XEON

- Introduced in 1998
- Designed for servers
- Clock speed was 400MHz to 450MHz
- L1 cache of 32KB & L2 cache of 512KB, 1MB or 2 MB
- It could work with 4 Xeons in same system

INTEL PENTIUM III

- Introduced in 1999
- Clock speed varied from 500MHz to 1.4GHz
- 9.5 million transistors

INTEL PENTIUM IV

- Introduced in 2000
- Clock speed from 1.3GHz to 3.8GHz
- L1 cache of 32KB & L2 cache of 256KB
- 42 million transistors

INTEL DUAL CORE

- Introduced in 2006
- Hybrid32-bit/64-bit µP
- Both the cores have there own internal bus and L1 cache, but share the external bus and L2 cache
- SMT: Simultaneously Multi- Threading
- E.g.: Adobe Photoshop

64-BIT MICROPROCESSORS

INTEL CORE 2

- Introduced in 2006.
- Clock speed from 1.2GHz to 3GHz
- 291 million transistors
- It has 64 KB of L1 cache per core and 4 MB of L2 cache
- Launched in three different versions:
  - Intel Core 2 Duo
  - Intel Core 2 Quad
  - Intel Core 2 Extreme

INTEL CORE I7

- Introduced in 2008
- 4 physical cores
- Clock speed from 2.66GHz to 3.33GHz
- 781 million transistors
- 64 KB of L1 cache per core, 256 KB of L2 cache and 8 MB of L3 cache

INTEL CORE I5

- Introduced in 2009
- 4 physical cores
- Clock speed is from 2.40GHz to 3.60GHz
- 781 million transistors
- 64 KB of L1 cache per core, 256KB of L2 cache and 8MB of L3 cache

INTEL CORE I3

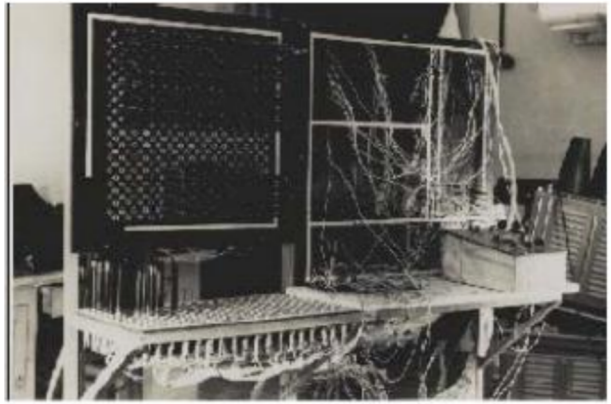
- Introduced in 2010
- 2 physical cores
- Clock speed is from 2.93GHz to 3.33GHz
- 781 million transistors
- 64 KB of L1 cache per core, 512KB of L2 cache and 4MB of L3 cache

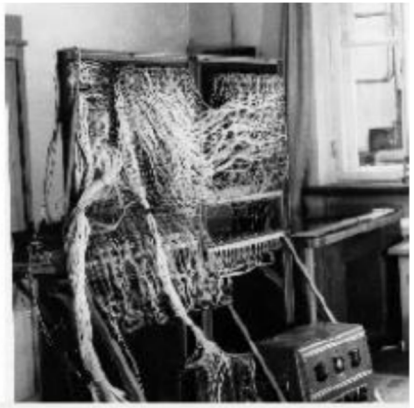
COMPUTER RESEARCH @ UPB

Calculatoare proiectate de membrii Departamentului Calculatoare @ UPB

Electrointegrator pentru soluționarea unor probleme de câmp.

Adrian Petrescu (1963)





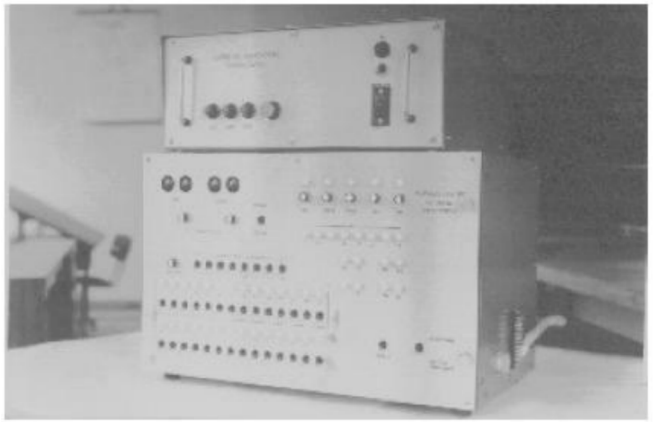
MAC-1 Calculator Analogic cu 30 de amplificatoare operaționale.

Adrian Petrescu, Petre Dimo, Ivan Sipos (1965)

Calculatoare proiectate de membrii Departamentului Calculatoare @ UPB

Microcalculatorul MC-1

Adrian Petrescu, Nicolae Țăpuș, Trandafir Moisa (1972)



Microcalculatorul FELIX MC-8

Adrian Petrescu, Nicolae Țăpuș, Trandafir Moisa (1975)

Calculatoare proiectate de membrii Departamentului Calculatoare @ UPB

Microcalculatorul FELIX M-18

Adrian Petrescu, Nicolae Țăpuș, Trandafir Moisa (1978)

Microcalculatorul aMIC

Adrian Petrescu, Francisc Iacob (1984)

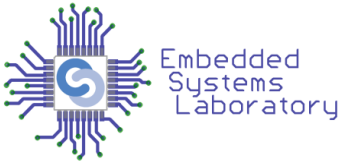
Calculatoare proiectate de membrii Departamentului Calculatoare @ UPB

Microcalculatorul FELIX HC-85

Adrian Petrescu, Francisc Iacob (1985)

Microcalculatorul FELIX-PC

Adrian Petrescu, Nicolae Țăpuș, Trandafir Moisa, Irina Athanasiu (1985)



PROIECTAREA CU MICROPROCESOARE

ARHITECTURA X86

Facultatea de Automatică și Calculatoare
Universitatea Politehnica București

Contents

- Overview
- Hardware architecture
- Software architecture

Overview

- The name **x86 refers to instruction set** which is used in Intel's 8086 CPU
- This name is used in processors from Intel, AMD, VIA, and other processors that compatible with the instruction set
- Modern processors have many addition to the original x86 but still fully backward compatible with it.

Chronology (only Intel)

- 1978: 8086, 8088
  - 16-bit CPU with 20-bit address
- 1982: 80186, 80188
- 1982: 80286
  - 16-bit CPU with 24-bit address
- 1985: i386 (80386)
  - 32-bit CPU with 32-bit address, renamed to IA-32
- 1989: i486
  - Pipelining with integrated FPU(floating point unit)

Chronology (cont.)

- 1993: Pentium, Pentium MMX
  - 64-bit data bus, superscalar (2 ALUs)
- 1995: Pentium Pro
  - 36-bit address bus (Physical Address Extension, PAE), RISC core
- 1997: Pentium II, Pentium III
  - SSE
- 2000: Pentium IV
  - Deeply pipelining, SSE2, Hyperthreading
- 2003: Pentium M

Chronology (cont.)

- 2003: Athlon64 (from AMD)
  - First 64-bit CPU in x86 family
- 2004: Pentium 4 Prescott
  - Very deep pipeline (20 stages), SSE3, 64-bit
- 2006: Core 2
  - Multicore, SSE4
- 2008: Atom
  - Deep pipeline, very low power

Chronology

- 2008-2009: Intel Nehalem
  - 44-bit address, SSE4a, HyperTransport3, L3-cache on dye
- 2010: Intel Sandy Bridge
  - SSE5

Basic properties

- CISC design
- Primarily two-address with limitation
- Little endian
- Allow unaligned memory-address access for 16 and 32 bits
- Instruction set is mostly inherited from the previous generation Intel’s CPU (8085)

Current implementation

- Since Pentium Pro, the core unit is RISC
- An extra decoding unit asynchronously splits most x86 instructions into smaller pieces, which then are executed by many execution units
- The extra decoding unit operates like an emulator

Operation mode

- Many extensions have been introduced since the first generation of x86
- The original x86 architecture has many limitations and cannot support modern architecture such as protection and virtual memory
- How to support new requirements while maintain backward compatibility?

Operation mode

- x86 CPUs have many operation modes
  - Real mode
  - 16-bit Protected mode
  - 32-bit Protected mode
    - Virtual-8086 mode
  - Long mode (64-bit Protected mode)
- Different modes have different limitations
- After power up, CPU operates in Real mode.
- Switching between modes occurs by software (OS)

Real mode

- Addressing is fully compatible with the original x86 CPU
- Can access memory up to 1MB
- Uses segmentation
- Although the memory reference is limited to the original 16-bit CPU, data processing can utilize 32-bit instructions

Segmentation in Real mode

- The original x86 is 16 bits which has 16-bit registers.
- 64 kB memory was nearly not enough
- To extend memory size, Intel decided to use two 16-bit number to point to a physical address
- That two 16-bit number is referred to in the form of **Segment:Offset**
- The combination of segment and offset generates 20-bit address (1 MB)

Calculating address from Segment:Offset

- The segment must be stored in one of the “segment” registers
  - CS, DS, SS, ES
- Given a 16-bit selector and a 16-bit offset, the 20-bit address is computed as follows
  - Multiply the selector by 16
    - This simply transforms XXXX into XXXX0 (in base-16)
  - Add the offset

Example

- Given the segment and offset as 10DE:12A3, what is the physical address?
  - Physical = Segment * 16_{10} + offset
  - = 10DE0 + 12A3
  - Physical = 12083

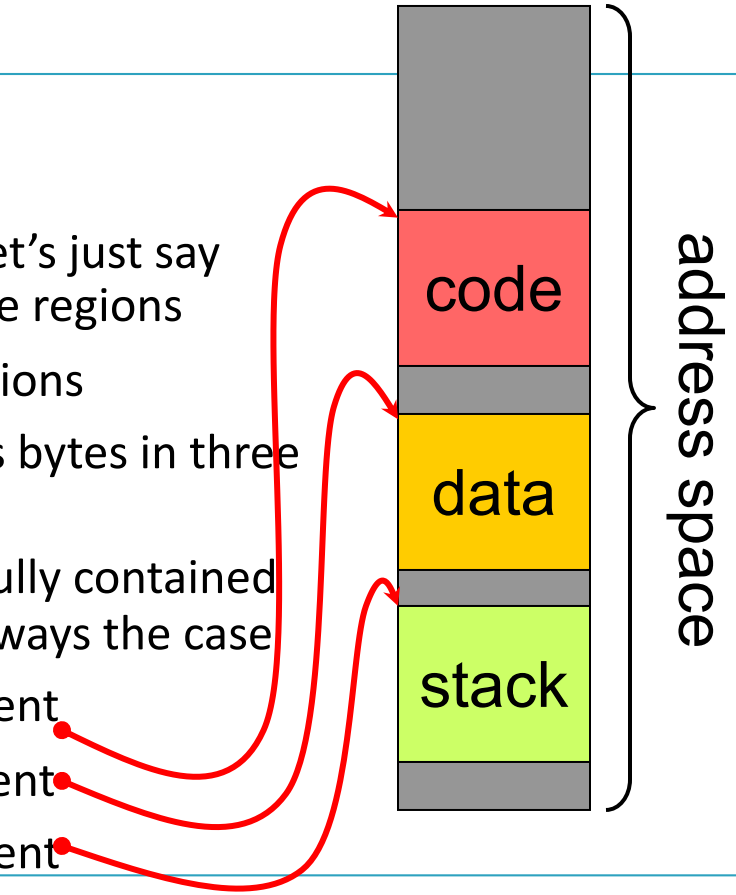
Example

- Consider the byte at address 13DFE with the segment value 10DE. What is its offset?

  - 13DDE = 10DE * 16_{10} + offset
  - offset = 13DFE - 10DE0
  - offset = 301E (a 16-bit quantity)

Code, Data, Stack

- Although we'll discuss these at length later, let's just say right for now that the address space has three regions
- A program constantly references all three regions
- Therefore, the program constantly references bytes in three different segments
  - For now let's assume that each region is fully contained in a single segment, which is in fact not always the case
- CS: points to the beginning of the code segment
- DS: points to the beginning of the data segment
- SS: points to the beginning of the stack segment



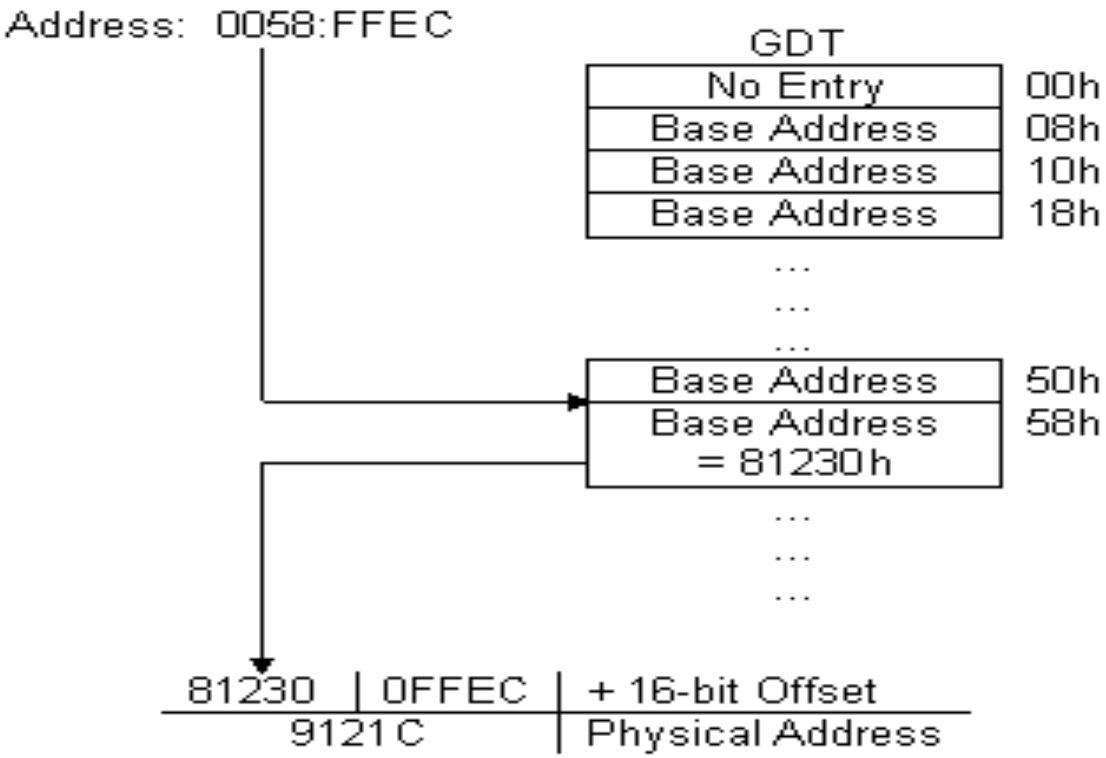
The trouble with segments

- It is well-known that programming with segmented architectures is really a pain
  - You constantly have to make sure segment registers are set up correctly
- What happens if you have data/code that's more than 64K?
  - You must then switch back and forth between selector values, which can be really awkward
- Something that can cause complexity also is that two different (selector, offset) pairs can reference the same address
  - Example: (a,b) and (a-1, b+16)
  - Concrete example: A000:5677 and A111:4567

Protected mode

- Introducing the term “logical address space” to x86
- Each position of memory has two address types:
  - Logical address – seen and used by program (also programmer)
  - Physical address – The actual address generated by memory management unit (embedded within the CPU) from a given virtual address
- Size of logical space can be different from those of physical space

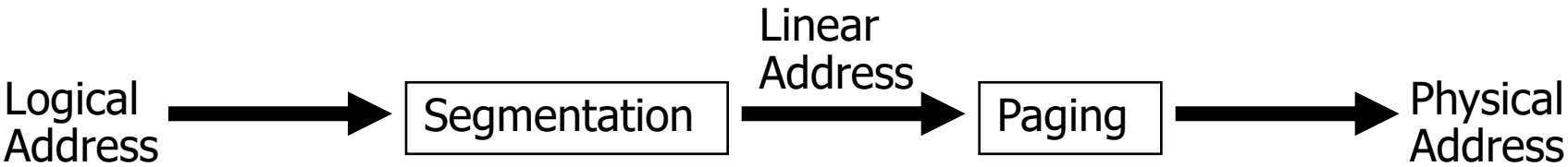
Logical -> Physical address



Difference of 286 and 386 address translation

- 286

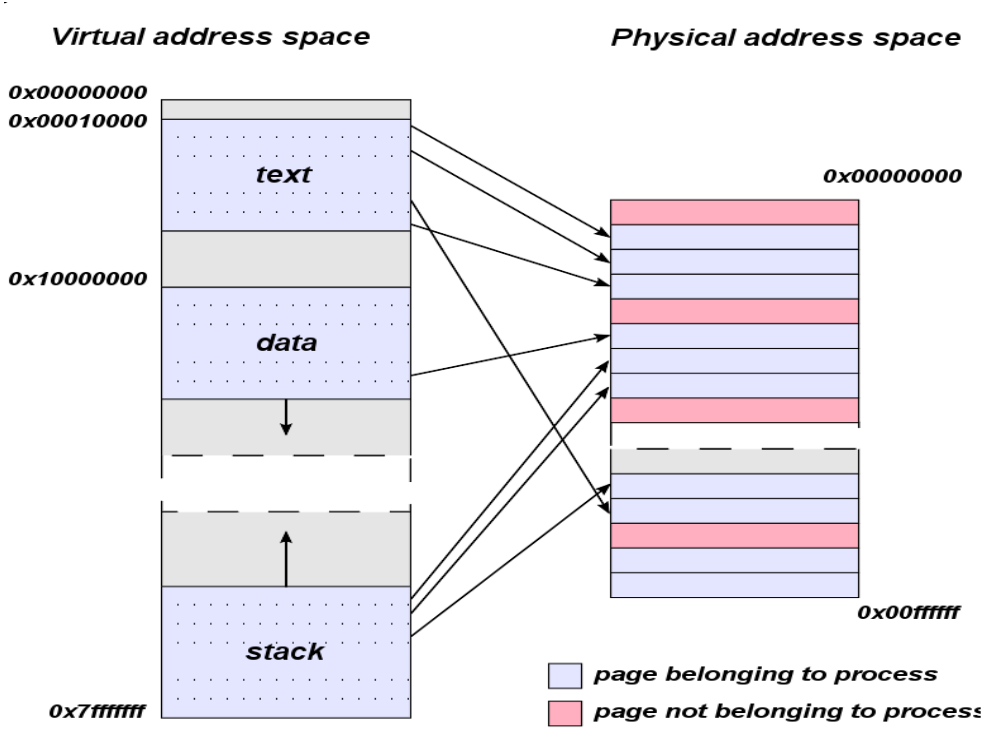
- 386



Virtual memory

- Paging in i386 introducing the world of virtual memory to x86
- A memory page can be absent from the physical memory
- If program refers to an absent page, paging unit generates a special interrupt (page fault) then OS can manage to fill the page with the required data

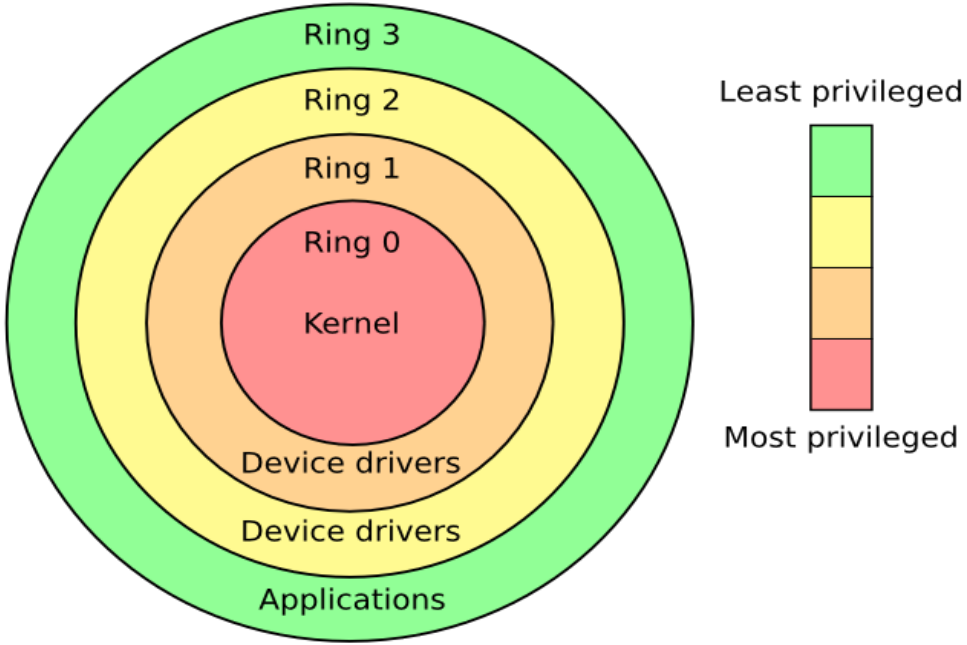
Virtual and physical space



Process privilege

- In protected mode, a process (or a task) has privilege (aka. Priority) level.
- There are 4 privilege levels (or rings) numbers from 0 to 3 (ring 0 is the highest privilege and ring 3 is the lowest)
- Usually, OS runs in ring 0 and all user applications operate in ring 3

Privileged ring



User applications cannot:

- Segment arithmetic
- Privileged instructions
- Direct hardware access
- Writing to a code segment
- Executing data
- Overlapping segments
- Use of BIOS functions, due to the BIOS interrupts being reserved
- **However, most legacy programs violate these rules!!!! To solve this problem, Virtual-8086 was introduced in i386**

Virtual 8086 mode

- As the segmentation in protected mode is far different from that in real mode, all legacy applications cannot run in protected mode.
- At the time of designing 80386, these old programs were still popular though the protected mode in 80286 had been around for sometime.
- Virtual-8086 mode is actually a sub-operation in 32-bit protected mode

Difference between protected and virtual-8086 modes

- Virtual-8086 mode is activated per process not for the whole system. Therefore, CPU is still in 32-bit protected mode and Virtual-8086 application is running in ring 3.
- In Virtual-8086, CPU uses real-mode segmentation instead of normal protected-mode segmentation
- Paging is still active so virtual memory and protection still exist.
- I/O operation is controlled by I/O permission map to allow/disallow the I/O change

Long mode

- Normally, applications cannot use 64-bit features such as 64-bit registers. Long mode enables these features.
- There is no virtual-8086 and segmentation in Long mode. However, paging is still active.
- Processor should enter 32-bit protected mode before starting Long mode.
- In Long mode, a task can be classified to either of two sub-mode:
  - 64-bit mode – For new 64-bit applications (old applications require re-compilation)
  - Compatibility mode – For 32-bit applications

x86 Registers

- The original 8086 has 14 registers. All of them are 16-bit wide.
  - 4 General purpose register (GPR): AX, BX, CX, DX
  - 2 Index register: SI, DI
  - 2 Pointer register: BP, SP
  - 4 Segment register: CS, DS, SS, ES
  - Flag
  - Instruction Pointer
- Each GPR can be accessed as two separate bytes (i.e., BX's high byte can be accessed as BH and low byte as BL)

x86 Registers (cont.)

- In 32-bit platforms, all GPR, flag, index, and pointer registers are expanded to 32-bit. (Except the segment registers)
- All 32-bit registers are represented by prefixing “E” to the 16-bit register name (e.g., to access 32-bit AX, we use EAX)
- Two new segment registers (FS and GS) were added (They are still 16-bit)

x86 Registers (cont.)

- In 64-bit platforms (also in 64-bit Long mode), all 32-bit registers are expanded to 64-bit in a similar way that 32-bit expansion did before but with the prefix “R” (e.g., AX become RAX)
- 8 additional general registers were added (R8, R9, .., R15)

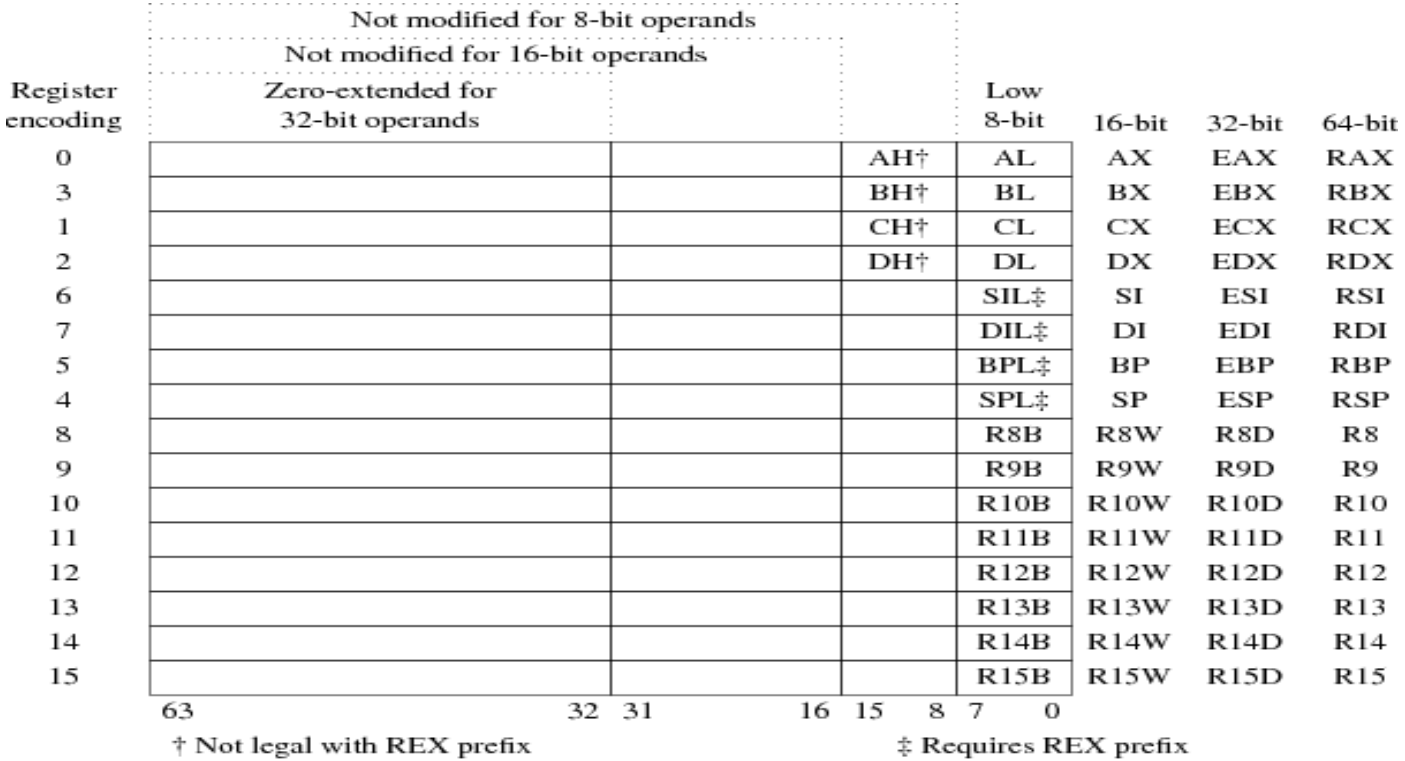
Floating-point Registers

- Floating-point unit has its own register groups separate from all operation of the other units
  - 8 80-bit floating-point registers: all are named ST
  - 3 16-bit word registers: Control word, Status word, Tag word
- Operation on these registers as well as the floating point functions require another set of instructions because this unit was originally a separate processor

Special registers

- Apart from all registers mentioned before, x86 also has various special/miscellaneous registers. They are mainly used by the OS
  - 5 Control registers: CR0 through 4
  - 6 Debug registers: DR0 through 3, plus 6 and 7
  - 4 Test registers: TR7 through 7
  - Descriptor registers: GDTR, LDTR, IDTR
  - Test register: TR

x86 Registers



MMX

- MMX is a SIMD instruction set
- MMX adds 8 new “registers” named MM0 to MM7. They are **not** actual registers. The are just aliases to the existing floating-point registers
  - Change MMx also change ST
- Each of MM registers is 64-bit. Therefore, the highest 16-bit in 80-bit floating point register is unused and filled with all 1.
- Each MM register can contain one 64-bit integer, two 32-bit integer, four 16-bit integer, or eight 8-bit integer.
- A MMX instruction calculates the results of these packed data simultaneously

3DNow

- Introduced by AMD in 1997
- Very similar architecture as MMX
- Each 64-bit register, can contain two single-precision floating point data. Hence, the operations can be performed only with real numbers

Streaming SIMD Extensions (SSE)

- SSE discarded all legacy connections to the FPU registers
- 8 new actual registers were added named XMM0 – XMM7 (in AMD64 the number of register has been increased from 8 to 16)
- Like 3DNow, SSE supports only floating-point operations
- Each XMM has 128 bits.
  - SSE1 each register can pack four single-precision floating-point data
  - SSE2 and later each register can pack four single-precision or two double-precision floating-point data
- SSE3 and SSE4 have same register structure with additional instructions

Physical Address Extension (PAE)

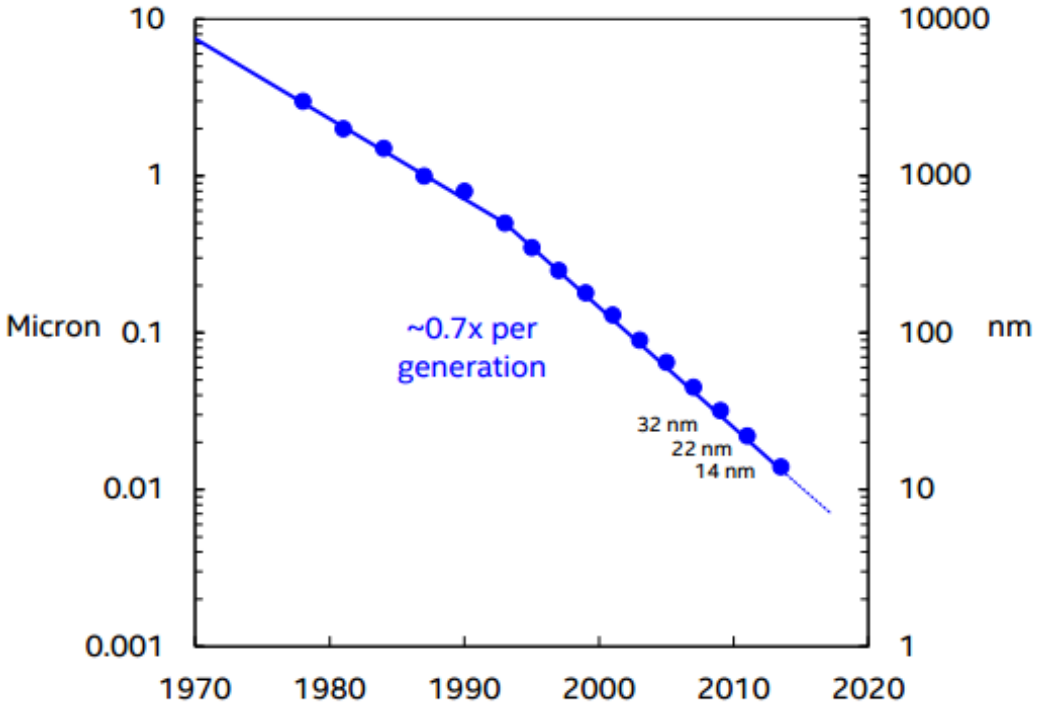
- Valid only in 32-bit protected mode
- The size of physical address is extended by 4 bit. Therefore, we can have maximum 64GB of physical memory
- The extension occurs in paging unit; hence, logical address still has 32 bits
  - Means that the size of memory that a program can access at a time is still 4GB

Data: Mark Bohr, Intel Sr. Fellow, IDF 2014

Intel Scaling Trend

22 nm | 14 nm | 10 nm | 7 nm

Manufacturing | Development | Research



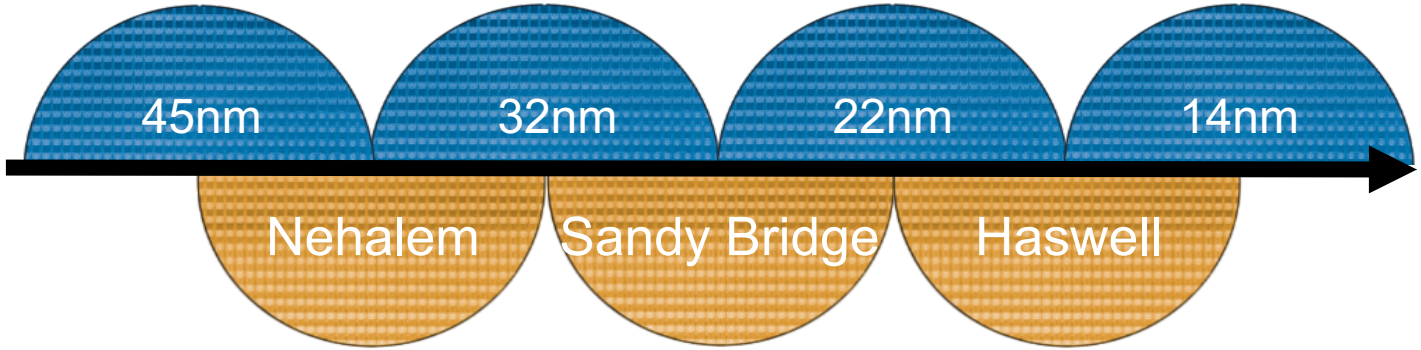
Scaled transistors provide:

- Higher performance
- Lower power
- Lower cost per transistor

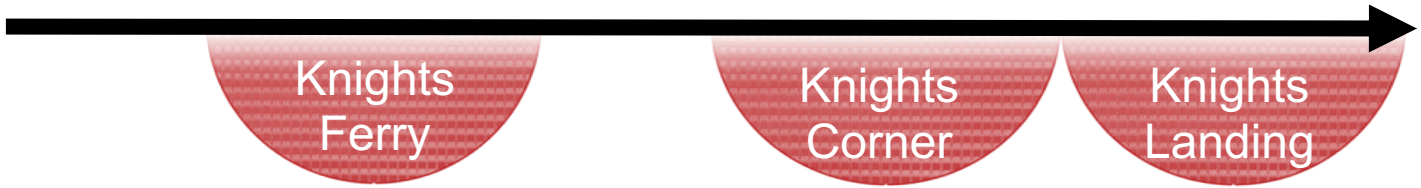
Moore's Law continues!

Tick-Tock Model (now defunct)

Scalable Performance
Energy Efficient
Microarchitecture



Highly Parallel
Energy Efficient
Architecture



NetBurst

- First released in 2000
- Hyper Pipelined Technology
  - 20-stage pipeline (compared to 10 in PIII) except for Prescott
  - Prescott has 31 stages!!!
  - Introducing of Hyper threading
  - Deep pipeline has high penalty in branching
- Rapid Execution Engine
  - ALU run at twice speed of other units
  - Speedup integer operations
  - Difficult to optimize operations among units
- Execution Trace Cache
  - When execute a previously visited instruction, CPU can use the previously decoded micro-ops instead of fetching and decoding the instruction again

Core Architecture

- First released in 2006
- Redesigned from ground up with the philosophy of Pentium M
- Only 14-stage pipeline but with wider execution unit.
- Micro-op Fusion
  - Combine two x86 instructions together and execute them simultaneously
- Multiple core CPUS with linked L1 cache and shared L2 cache
- Lower power consumption
  - 20% more performance for Merom at the same power level (compared to Dual Core)
  - 40% more performance for Conroe at 40% less power (compared to Pentium D)
  - 80% more performance for Woodcrest at 35% less power (compared to the original dual-core Xeon)

Nehalem

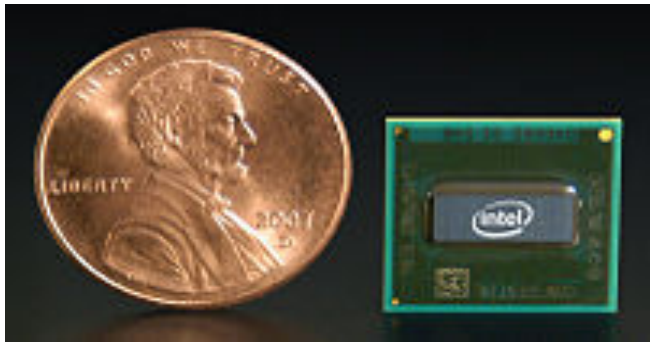
- Released 2008
- 2, 4, or 8 cores
  - 731 million transistors for the quad core variant
- 45 nm manufacturing process
- Integrated memory controller supporting DDR3 RAM and between 1 and 6 memory channels
- Integrated graphics processor (IGP) located off-die, but in the same CPU package
- A new point-to-point processor interconnect, the Intel QuickPath Interconnect, replacing the legacy front side bus
- Simultaneous multithreading, which enables two threads per core.
- Native (monolithic, i.e. all processor cores on a single die) quad and octo (8) core processors
- Linked L1 and L2 cache. Shared 8MB L3 cache
- 33% more in-flight micro-ops than Conroe
- 2nd level branch predictor and 2nd level Translation Lookaside Buffer
- Modular blocks of components such as cores that can be added and subtracted for varying market segments

Sandy Bridge

- Released in 2010
- 4 GHz clock speed.
- 4 to 8 out-of-order cores.
- Without SSE: 8 DP GFLOPS/core (2 DP FP/clock), 32-64 DP GFLOPS/processor.
- With SSE: 28 DP GFLOPS/core (7 DP FP/clock), 112-224 DP GFLOPS/processor.
- 32 KB L1 cache/core, (3 clocks).
- 512 KB L2 cache/core, (9 clocks).
- 2-3 MB L3 cache/core (8-24 MB total) (33 clocks), most likely pooled and dynamically allocated among the cores.
- 64 bytes cache line width.
- 256 bytes/cycle Ring bus bandwidth. The ring bus connects the cores.
- 0-512 MB GDDR / fast DRAM.
- 64 GB/s GDDR / fast DRAM memory bandwidth.
- 17 GB/s memory bandwidth per QuickPath link with 50 ns latency.

Atom

- Aims for ultra-low power mobile devices.
- No instruction re-ordering
- No instruction fusing
- Simpler micro-ops decoding than Nehalem (Less performance but smaller unit)
- Single core with hyper-threading



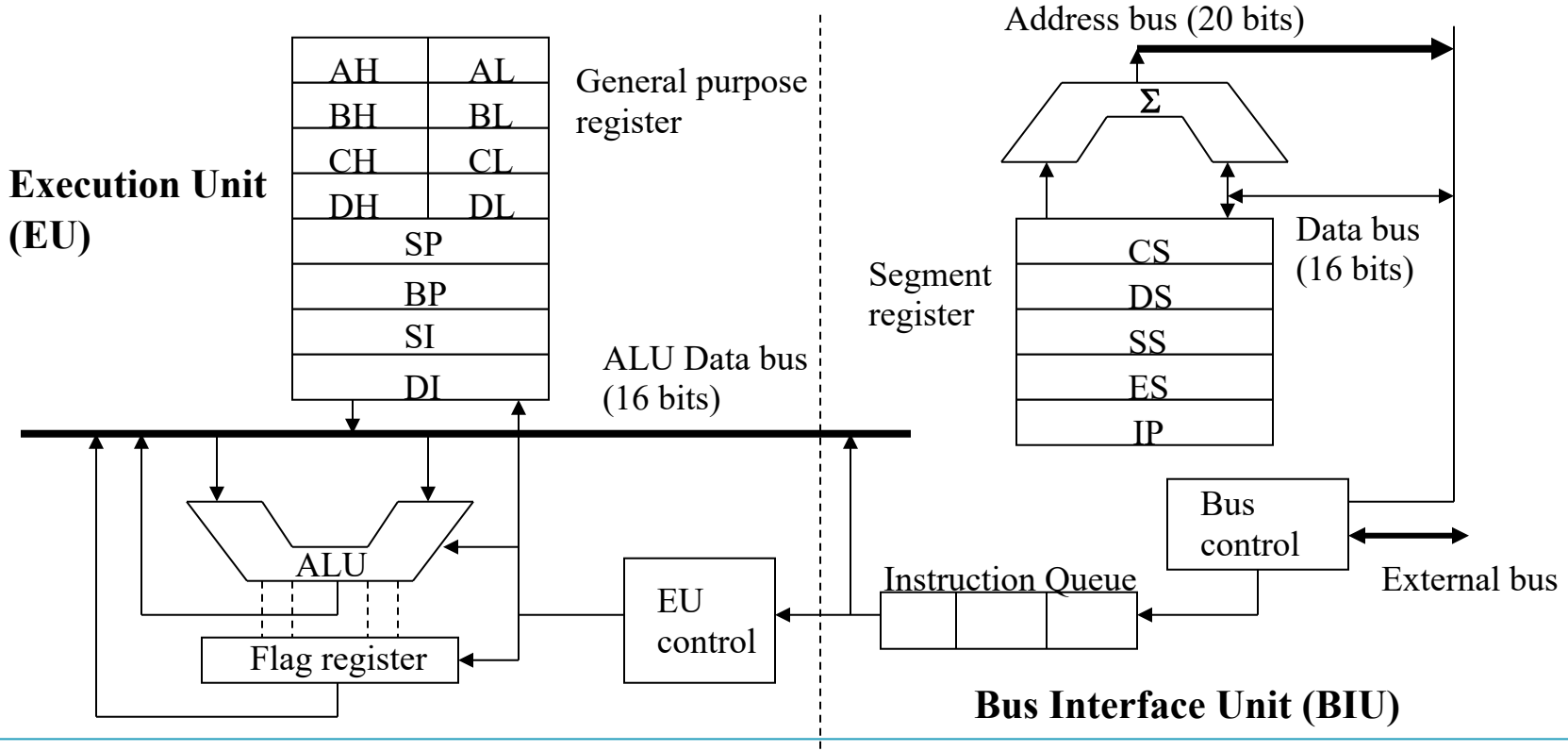
Haswell

- Introduced 2013
- 14 stage pipeline
- 4 cores, SMT machine
- In order issue, Out of Order execution, In order commit.
- Wider data paths and extra Store AGU to provide more bandwidth in AVX2 computations
- LLC/Ring is the point of coherence and distributed arbitration of requests.
- Intel TSX
  - Added support for Restricted Transaction Memory
- Integrated Graphics and Improved Power Management
  - Power Efficiency is a huge emphasis

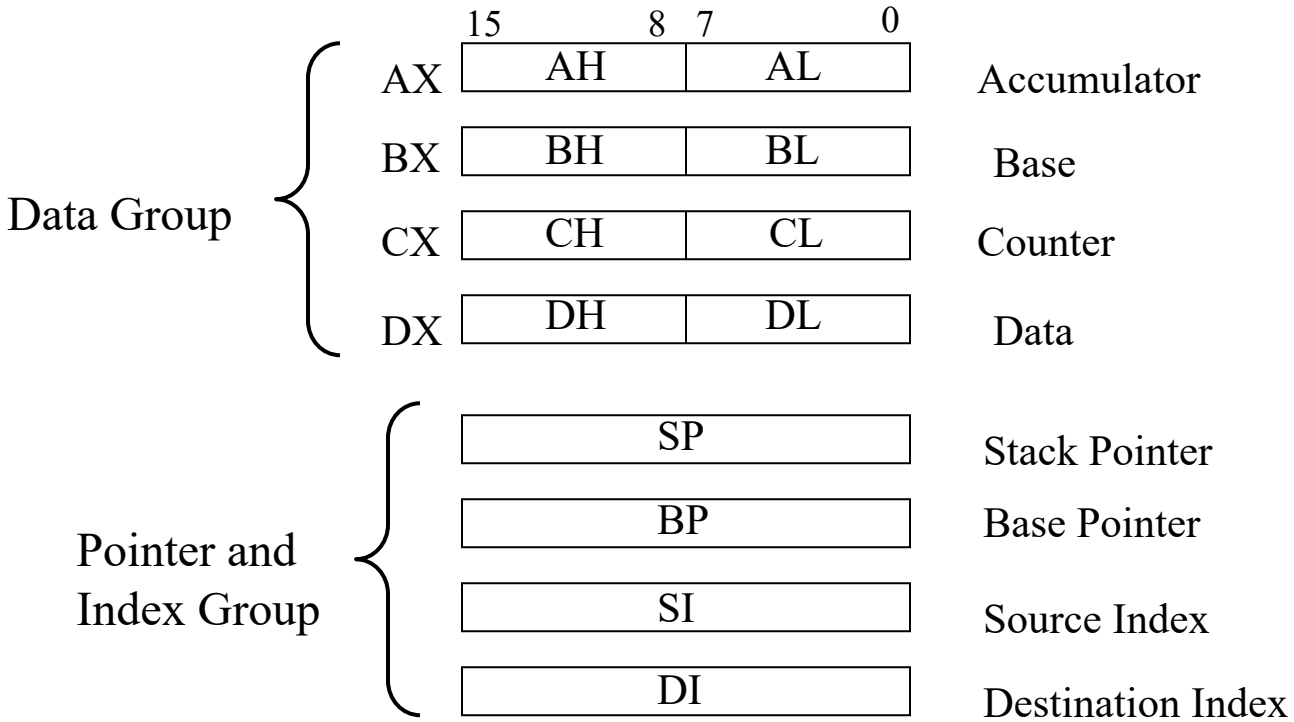
Skylake

- 6th generation Core uArch
- Released 2015
- Improved performance, particularly on mobile
- Better graphics support
- Power efficiency (4.5W at the low end)
- Better OS support (especially Win 10)

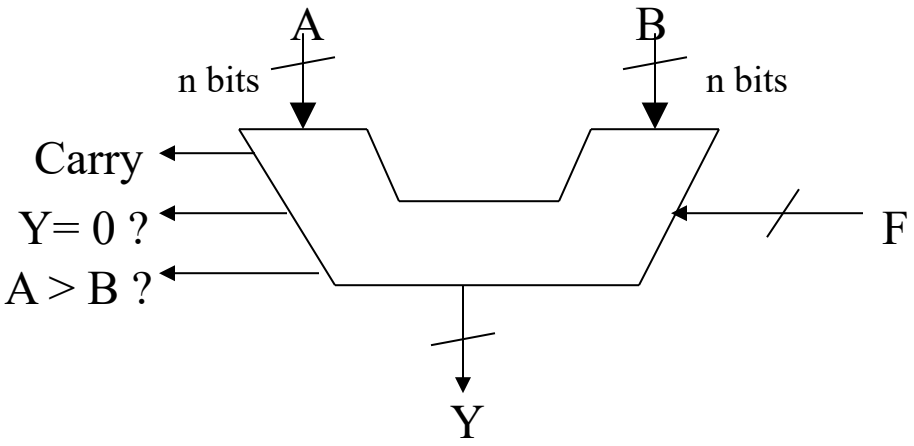
Organization of 8086



General Purpose Registers



Arithmetic Logic Unit (ALU)

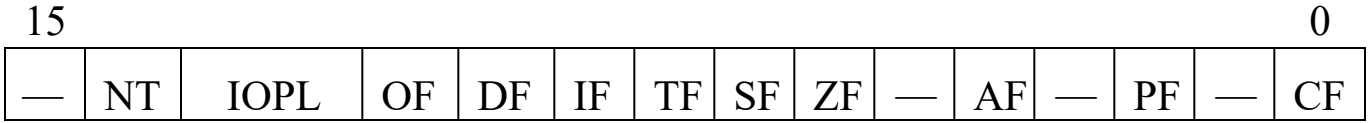


F	Y
0 0 0	A + B
0 0 1	A - B
0 1 0	A - 1
0 1 1	A ***and*** B
1 0 0	A ***or*** B
1 0 1	***not*** A
• • •	• • •

- Signal F controls which function will be conducted by ALU.
- Signal F is generated according to the current instruction.
- Basic arithmetic operations: *addition, subtraction, etc.*
- Basic logic operations: *and, or, xor, shifting, etc.*

Flag Register

❑ Flag register contains information reflecting the current status of a microprocessor. It also contains information which controls the operation of the microprocessor.



➢ Control Flags

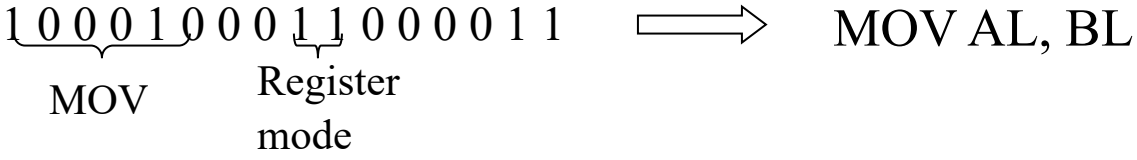
IF: Interrupt enable flag
DF: Direction flag
TF: Trap flag

➢ Status Flags

CF: Carry flag
PF: Parity flag
AF: Auxiliary carry flag
ZF: Zero flag
SF: Sign flag
OF: Overflow flag
NT: Nested task flag
IOPL: Input/output privilege level

Instruction Machine Codes

- Instruction machine codes are binary numbers
  - ***For Example:***



- Machine code structure

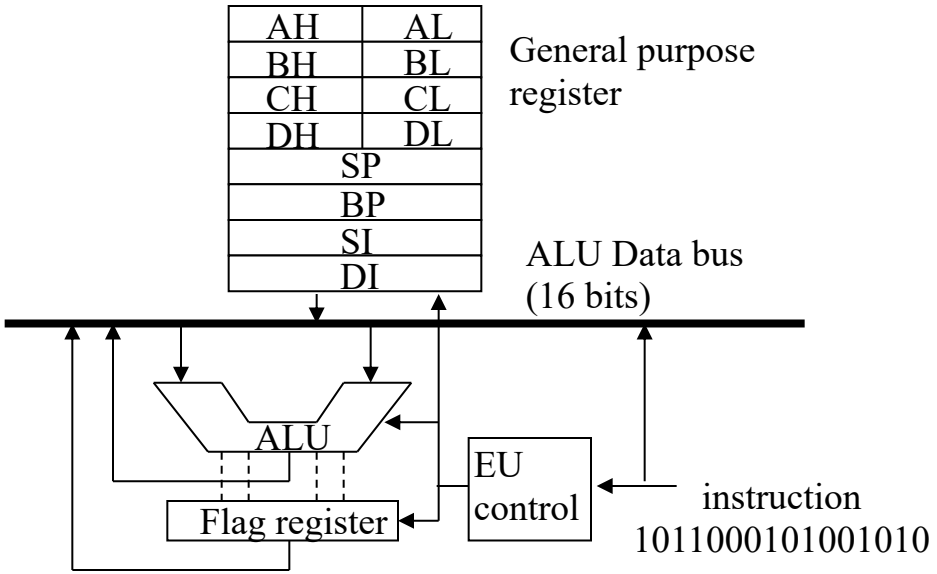
Opcode	Mode	Operand1	Operand2

  - Some instructions do not have operands, or have only one operand
  - Opcode tells what operation is to be performed.

    (EU control logic generates ALU control signals according to Opcode)
  - Mode indicates the type of a instruction: *Register type, or Memory type*
  - Operands tell what data should be used in the operation. Operands can be addresses telling where to get data (or where to store results)

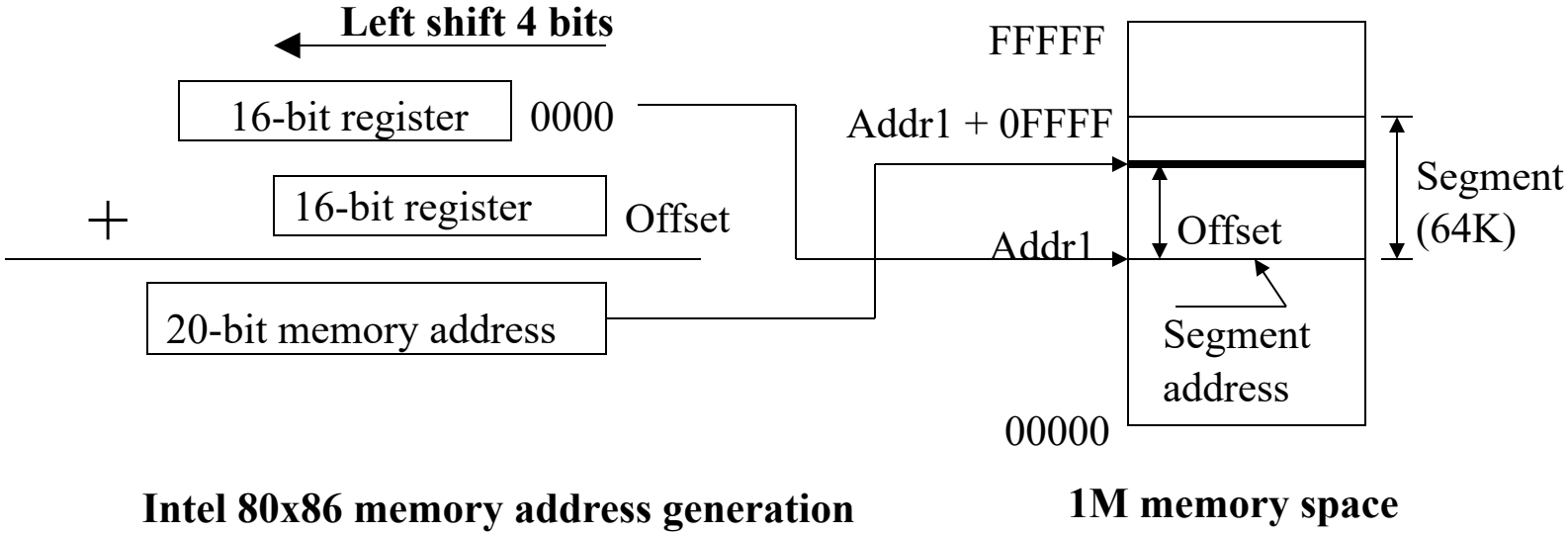
EU Operation

1. Fetch an instruction from instruction queue
2. According to the instruction, EU control logic generates control signals.
   (*This process is also referred to as instruction decoding)*
3. Depending on the control signal, EU performs one of the following operations:
   - An arithmetic operation
   - A logic operation
   - Storing data into a register
   - Moving data from a register
   - Changing flag register



Generating Memory Addresses

- How can a 16-bit microprocessor generate 20-bit memory addresses?

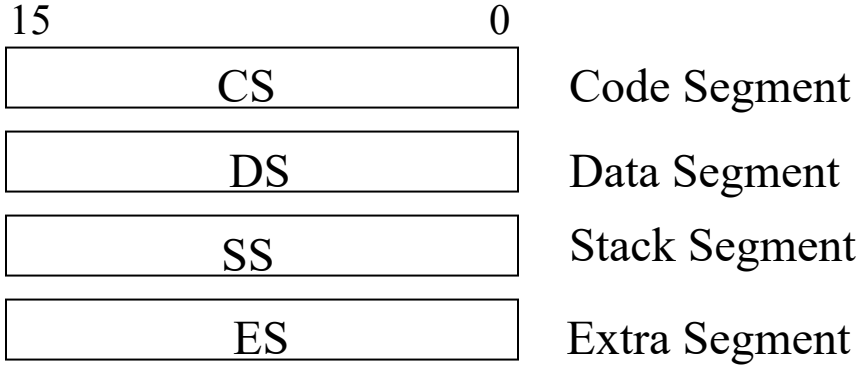


Intel 80x86 memory address generation

1M memory space

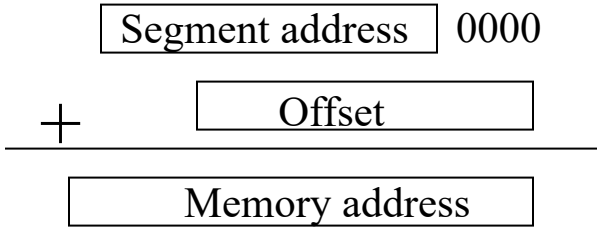
Memory Segmentation

- A segment is a 64KB block of memory starting from any 16-byte boundary
  - For example: 00000, 00010, 00020, 20000, 8CE90, and E0840 are all valid segment addresses
  - The requirement of starting from 16-byte boundary is due to the 4-bit left shifting
- Segment registers in BIU



Memory Address Calculation

- Segment addresses must be stored in segment registers
- Offset is derived from the combination of pointer registers, the Instruction Pointer (IP), and immediate values
- Examples



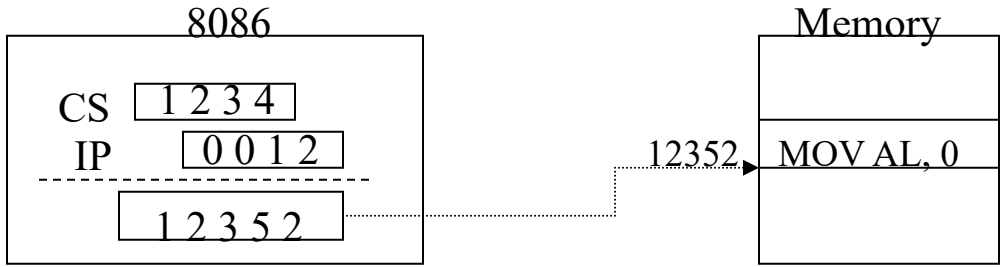
CS	3	4	8	A	0
IP +		4	2	1	4
Instruction address	3	8	A	B	4

SS	5	0	0	0	0
SP +		F	F	E	0
Stack address	5	F	F	E	0

DS	1	2	3	4	0
DI +		0	0	2	2
Data address	1	2	3	6	2

Fetching Instructions

- Where to fetch the next instruction?



- Update IP
  - After an instruction is fetched, Register IP is updated as follows:

    IP = IP + Length of the fetched instruction

  - For Example: the length of **MOV AL, 0** is 2 bytes. After fetching this instruction, the IP is updated to 0014

Accessing Data Memory

- There is a number of methods to generate the memory address when accessing data memory. These methods are referred to as **Addressing Modes**
- Examples:
  - *Direct addressing:* **MOV AL, [0300H]**

DS	1	2	3	4	0	*(assume DS=1234H)*
		0	3	0	0	
Memory address	1	2	6	4	0	

  - *Register indirect addressing:* **MOV AL, [SI]**

DS	1	2	3	4	0	*(assume DS=1234H)*
		0	3	1	0	*(assume SI=0310H)*
Memory address	1	2	6	5	0	

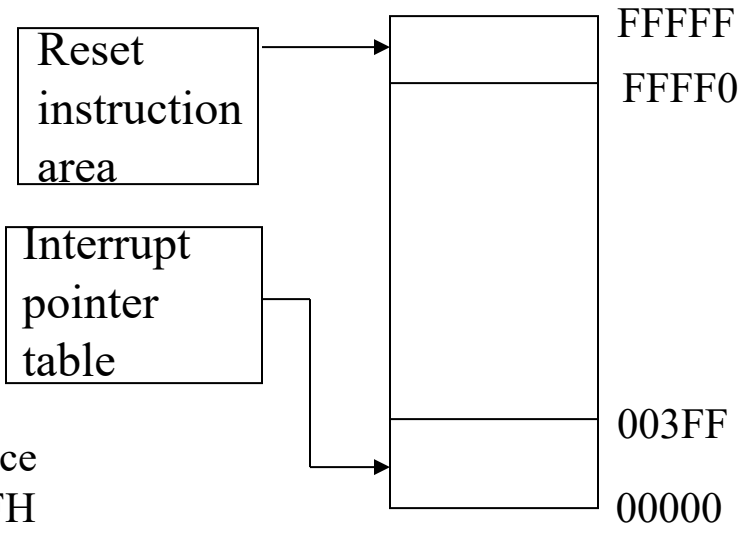
Reserved Memory Locations

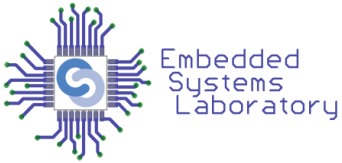
- Some memory locations are reserved for special purposes. Programs should not be loaded in these areas

  - Locations from FFFF0H to FFFFFH are used for system reset code

  - Locations from 00000H to 003FFH are used for the interrupt pointer table
    - — It has 256 table entries
    - — Each table entry is 4 bytes

    $256 \times 4 = 1024$ = memory addressing space From 00000H to 003FFH



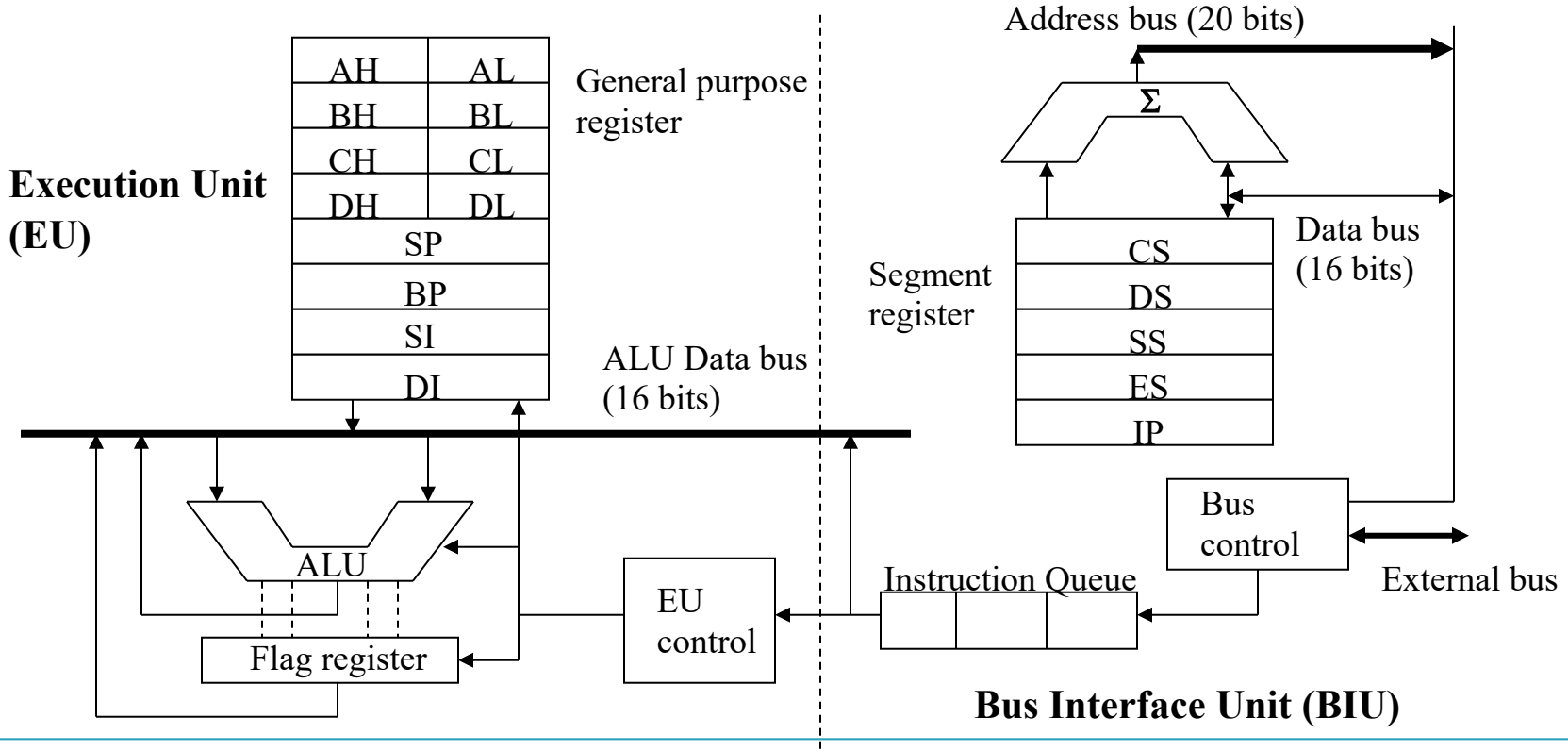


PROIECTAREA CU MICROPROCESOARE

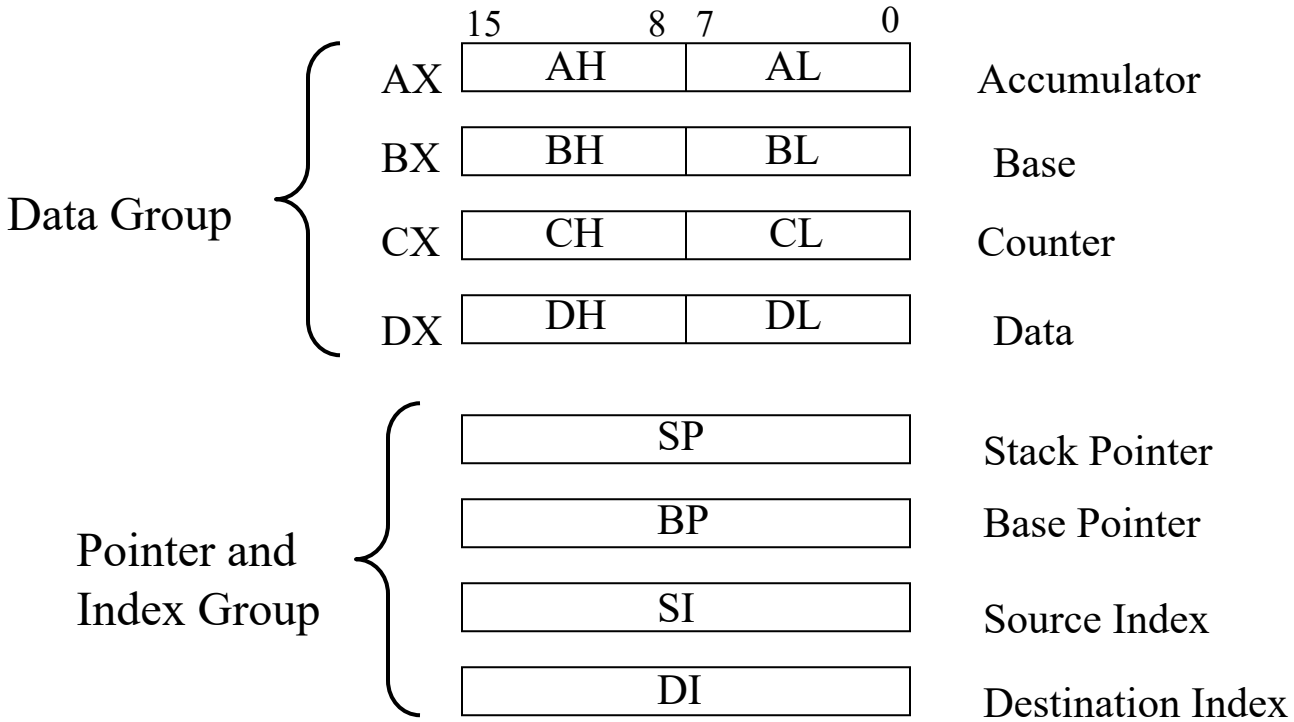
ARHITECTURA X86

Facultatea de Automatică și Calculatoare
Universitatea Politehnica București

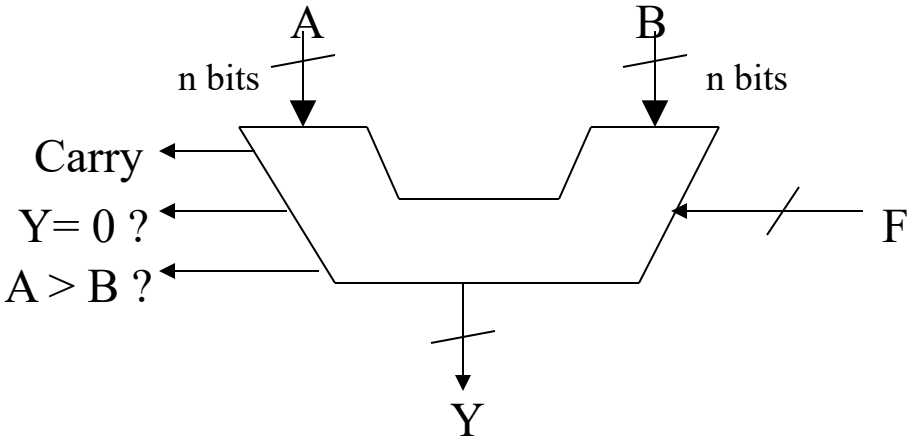
Organization of 8086



General Purpose Registers



Arithmetic Logic Unit (ALU)

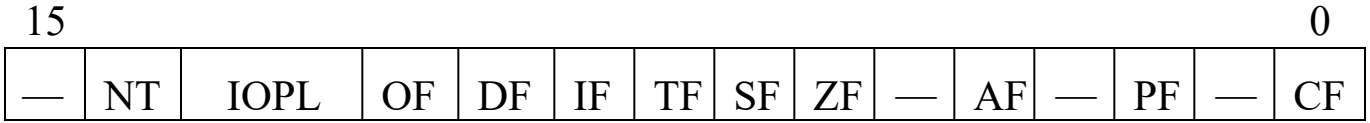


F	Y
0 0 0	A + B
0 0 1	A - B
0 1 0	A - 1
0 1 1	A ***and*** B
1 0 0	A ***or*** B
1 0 1	***not*** A
• • •	• • •

- Signal F controls which function will be conducted by ALU.
- Signal F is generated according to the current instruction.
- Basic arithmetic operations: *addition, subtraction, etc.*
- Basic logic operations: *and, or, xor, shifting, etc.*

Flag Register

❑ Flag register contains information reflecting the current status of a microprocessor. It also contains information which controls the operation of the microprocessor.



➢ Control Flags

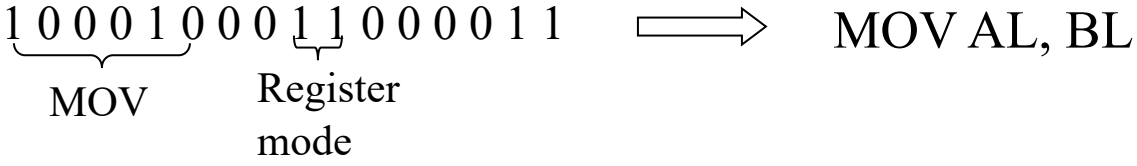
IF: Interrupt enable flag
DF: Direction flag
TF: Trap flag

➢ Status Flags

CF: Carry flag
PF: Parity flag
AF: Auxiliary carry flag
ZF: Zero flag
SF: Sign flag
OF: Overflow flag
NT: Nested task flag
IOPL: Input/output privilege level

Instruction Machine Codes

- Instruction machine codes are binary numbers
  - ***For Example:***



- Machine code structure

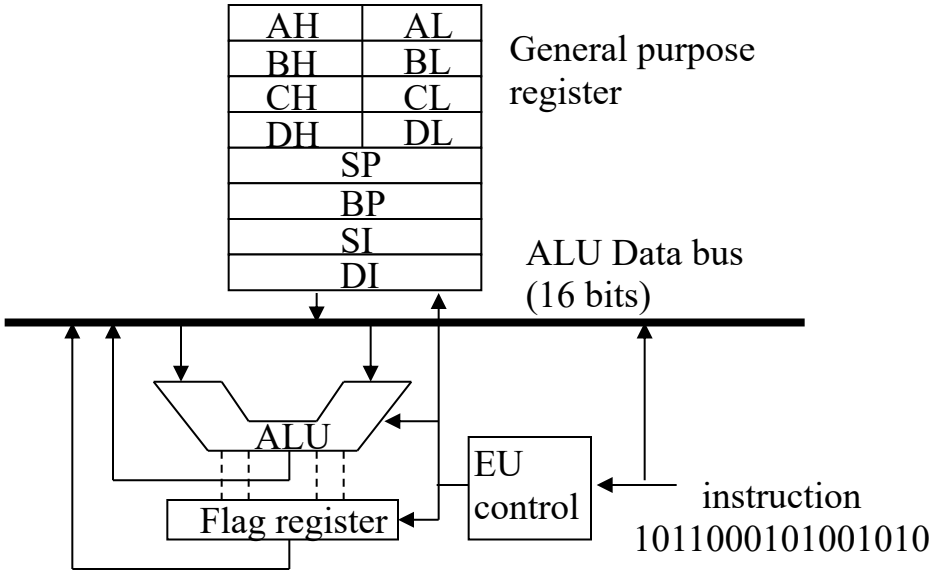
Opcode	Mode	Operand1	Operand2

  - Some instructions do not have operands, or have only one operand
  - Opcode tells what operation is to be performed.

    (EU control logic generates ALU control signals according to Opcode)
  - Mode indicates the type of a instruction: *Register type, or Memory type*
  - Operands tell what data should be used in the operation. Operands can be addresses telling where to get data (or where to store results)

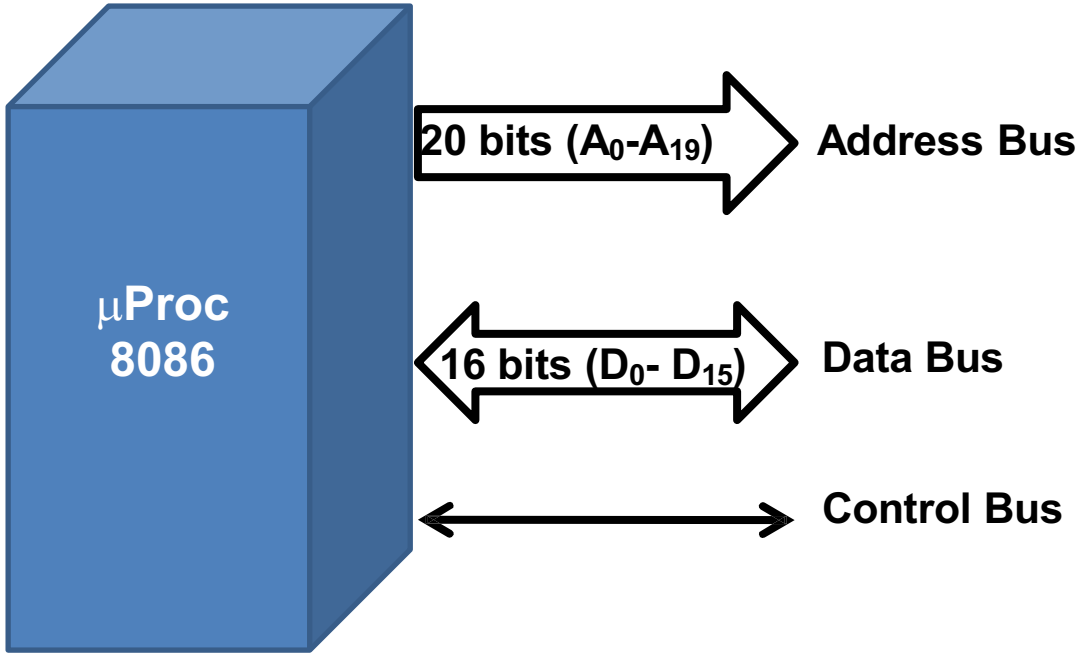
EU Operation

1. Fetch an instruction from instruction queue
2. According to the instruction, EU control logic generates control signals.
   (*This process is also referred to as instruction decoding)*
3. Depending on the control signal, EU performs one of the following operations:
   - An arithmetic operation
   - A logic operation
   - Storing data into a register
   - Moving data from a register
   - Changing flag register



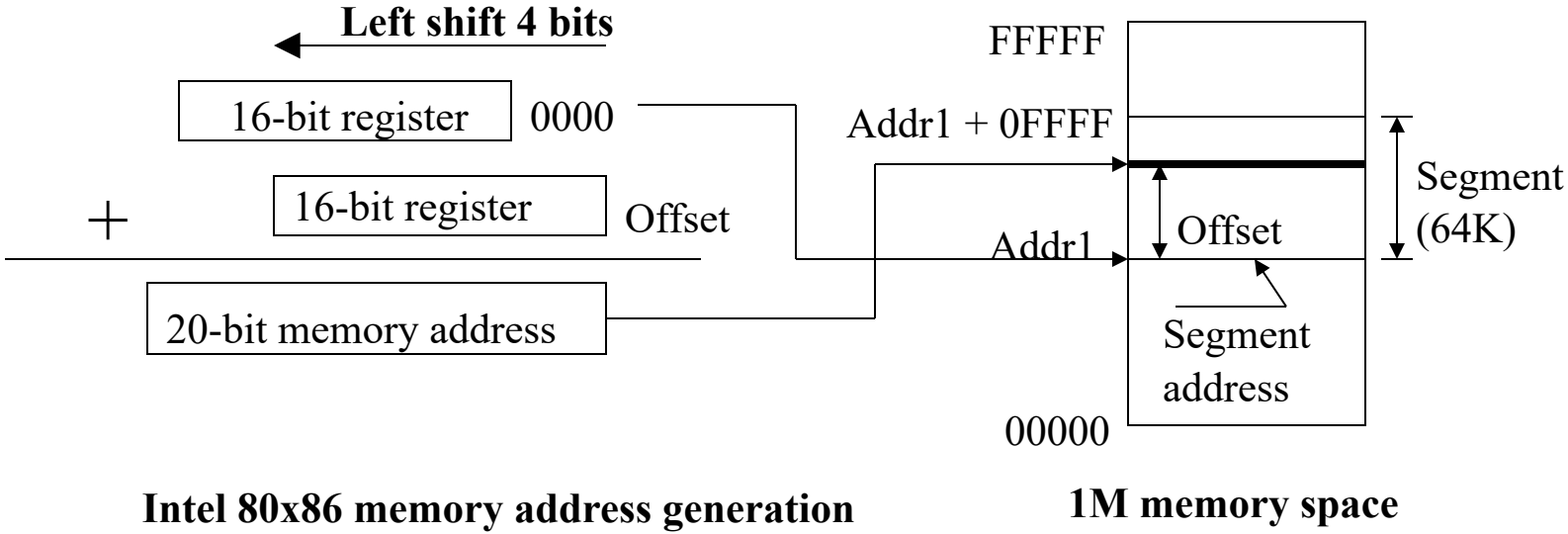
Pointers and Index Registers:

8086-based Systems can access ***2^{20}= 1M*** *memory locations at most*



Generating Memory Addresses

- How can a 16-bit microprocessor generate 20-bit memory addresses?

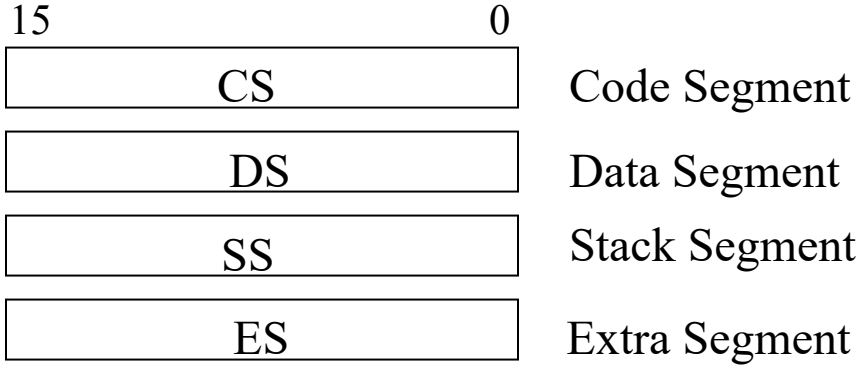


Intel 80x86 memory address generation

1M memory space

Memory Segmentation

- A segment is a 64KB block of memory starting from any 16-byte boundary
  - For example: 00000, 00010, 00020, 20000, 8CE90, and E0840 are all valid segment addresses
  - The requirement of starting from 16-byte boundary is due to the 4-bit left shifting
- Segment registers in BIU



Segmentation

CS = 1000H
DS = 2000H
SS = 3000H
What will be the actual addresses in memory?

Data Segment and Code Segment can have a complete overlapping. In addition Stack Segment and Extra Segment can have an overlapping.

Code Segment:	20-bit **start** address	= CS x10h +0000H = 10000h
	20-bit **end** address	= CS x10H +FFFFH = 1FFFFH
Data Segment :	20-bit **start** address	= DS x10h +0000H = 20000H
	20-bit **end** address	= DS x10H +FFFFH = 2FFFFH
Stack Segment :	20-bit **start** address	= SS x10h +0000H = 30000H
	20-bit **end** address	= SS x10H +FFFFH = 3FFFFH

The trouble with segments

It is well-known that programming with segmented architectures is really a pain

- In the 8086 you constantly have to make sure segment registers are set up correctly
- What happens if you have data/code that's more than 64KiB?
- You must then switch back and forth between selector values, which can be really awkward
- Something that can cause complexity also is that two different (selector, offset) pairs can reference the same address
- Example: (a,b) and (a-1, b+16)

There is an interesting on-line article on the topic:

http://world.std.com/~swmcd/steven/rants/pc.html

Why did segmentation survive?

If you code and your data are <64KiB, segments are great

- Otherwise, they are a pain
- Given the horror of segmented programming, one may wonder how come it stuck?
- From the linked article: "*Under normal circumstances, a design so twisted and flawed as the 8086 would have simply been ignored by the market and faded away*."

But in 1980, Intel was lucky that IBM picked it for the PC!

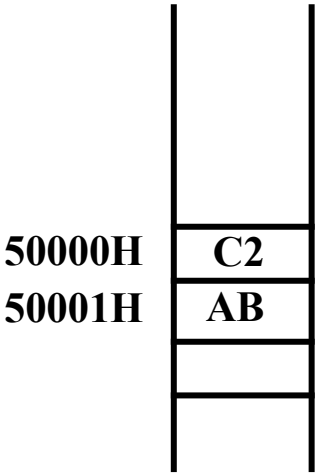
- Not to criticize IBM or anything, but they were also the reason why we got stuck with FORTRAN for so many years :/
- Big companies making "wrong" decisions has impact

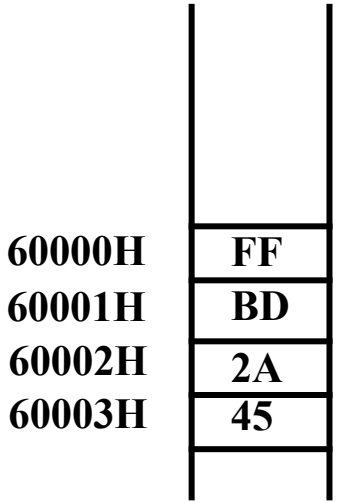
Memory storage

Lower byte of word is stored at lower address

The word **ABC2H** stored in the memory starting at 20-bit address 50000H

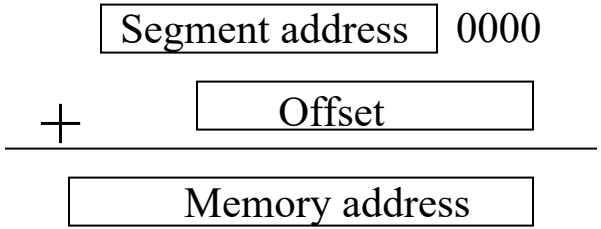
The double word **452ABDFF** stored in the memory starting at 20-bit address 60000H





Memory Address Calculation

- Segment addresses must be stored in segment registers
- Offset is derived from the combination of pointer registers, the Instruction Pointer (IP), and immediate values
- Examples



CS	3	4	8	A	0
IP +		4	2	1	4
Instruction address	3	8	A	B	4

SS	5	0	0	0	0
SP +		F	F	E	0
Stack address	5	F	F	E	0

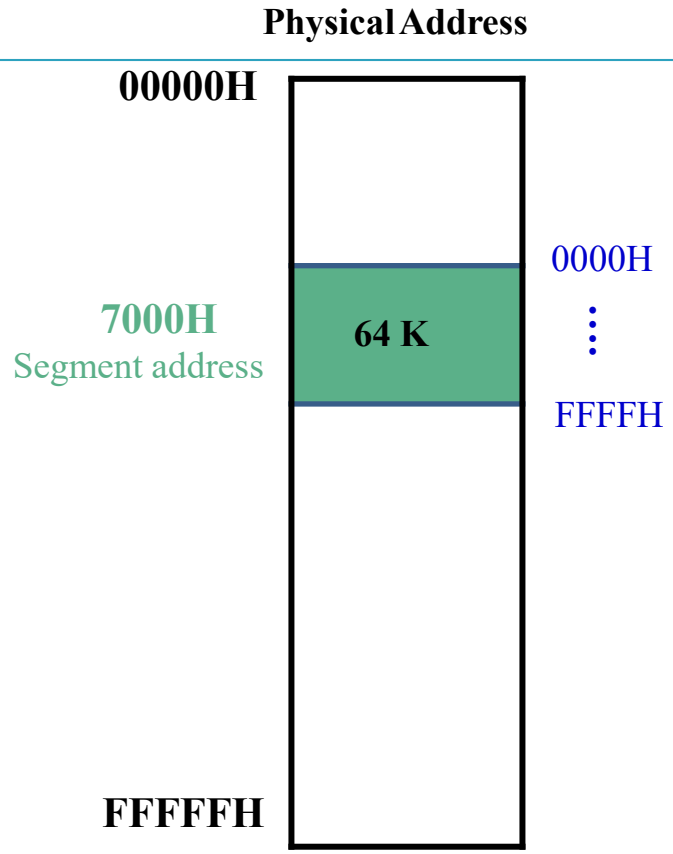
DS	1	2	3	4	0
DI +		0	0	2	2
Data address	1	2	3	6	2

The IP Register

The Instruction Pointer register (IP) contains the offset address of the next sequential instruction to be executed. Thus, the IP register cannot be directly modified.

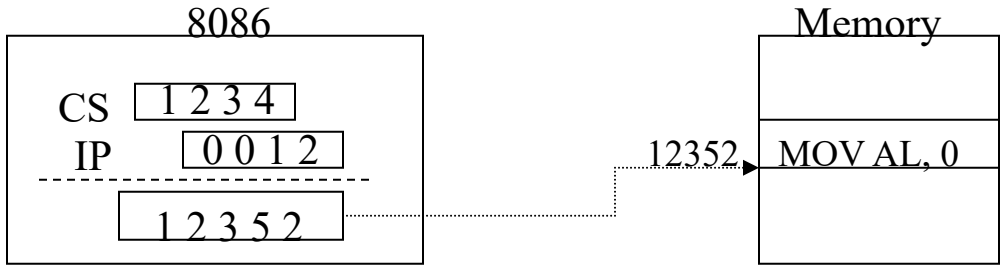
These register descriptions have slowly been introducing us to a new way of addressing memory, called:
segment-offset addressing.

The segment register is used to point to the beginning of any one of the 64K sixteen-byte boundaries



Fetching Instructions

- Where to fetch the next instruction?



- Update IP
  - After an instruction is fetched, Register IP is updated as follows:

    IP = IP + Length of the fetched instruction

  - For Example: the length of **MOV AL, 0** is 2 bytes. After fetching this instruction, the IP is updated to 0014

Accessing Data Memory

- There is a number of methods to generate the memory address when accessing data memory. These methods are referred to as **Addressing Modes**
- Examples:
  - *Direct addressing:* **MOV AL, [0300H]**

DS	1	2	3	4	0	*(assume DS=1234H)*
		0	3	0	0	
Memory address	1	2	6	4	0	

  - *Register indirect addressing:* **MOV AL, [SI]**

DS	1	2	3	4	0	*(assume DS=1234H)*
		0	3	1	0	*(assume SI=0310H)*
Memory address	1	2	6	5	0	

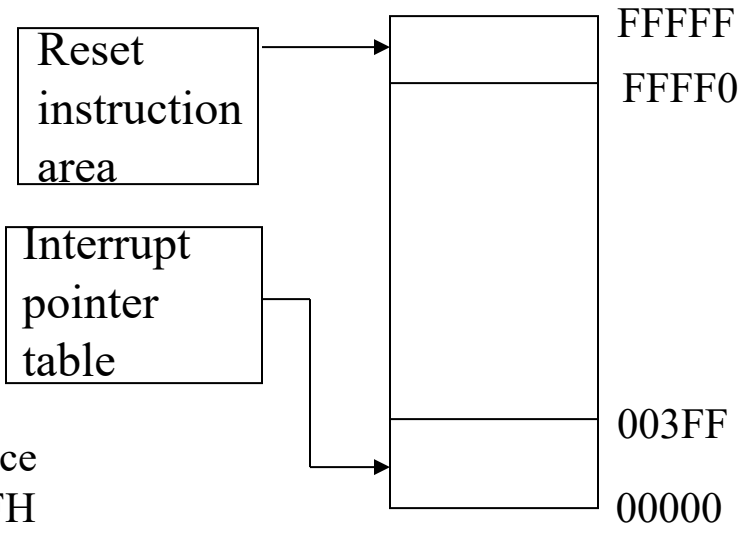
Reserved Memory Locations

- Some memory locations are reserved for special purposes. Programs should not be loaded in these areas

  - Locations from FFFF0H to FFFFFH are used for system reset code

  - Locations from 00000H to 003FFH are used for the interrupt pointer table
    - — It has 256 table entries
    - — Each table entry is 4 bytes

    $256 \times 4 = 1024$ = memory addressing space From 00000H to 003FFH



Interrupts

- An interrupt is an event that occurs while the processor is executing a program
- The interrupt temporarily suspends execution of the program and switch the processor to executing a special routine (interrupt service routine)
- When the execution of interrupt service routine is complete, the processor resumes the execution of the original program
- Interrupt classification

Hardware Interrupts	Software Interrupts
— Caused by activating the processor's interrupt control signals (NMI, INTR)	— Caused by the execution of an INT instruction — Caused by an event which is generated by the execution of a program, such as division by zero

- 8088 can have 256 interrupts

Minimum and Maximum Operation modes

- Intel 8088 (8086) has two operation modes:

Minimum Mode	***Maximum Mode***
— 8088 generates control signals for memory and I/O operations	— It needs 8288 bus controller to generate control signals for memory and I/O operations
— Some functions are not available in minimum mode	— It allows the use of 8087 coprocessor; it also provides other functions
— Compatible with 8085-based systems	

8086 Microprocessor

Microprocessor

Program controlled semiconductor device (IC) which fetches (from memory), decodes and executes instructions.

It is used as CPU (Central Processing Unit) in computers.

Microprocessor

Fifth Generation **Pentium**

Fourth Generation
During 1980s
Low power version of HMOS technology (HCMOS)
32 bit processors
Physical memory space 2^{24} bytes = 16 Mb
Virtual memory space 2^{40} bytes = 1 Tb
Floating point hardware
Supports increased number of addressing modes

Intel 80386

Third Generation
During 1978
HMOS technology ⇒ Faster speed, Higher packing density
16 bit processors ⇒ 40/ 48/ 64 pins
Easier to program
Dynamically relatable programs
Processor has multiply/ divide arithmetic hardware
More powerful interrupt handling capabilities
Flexible I/O port addressing

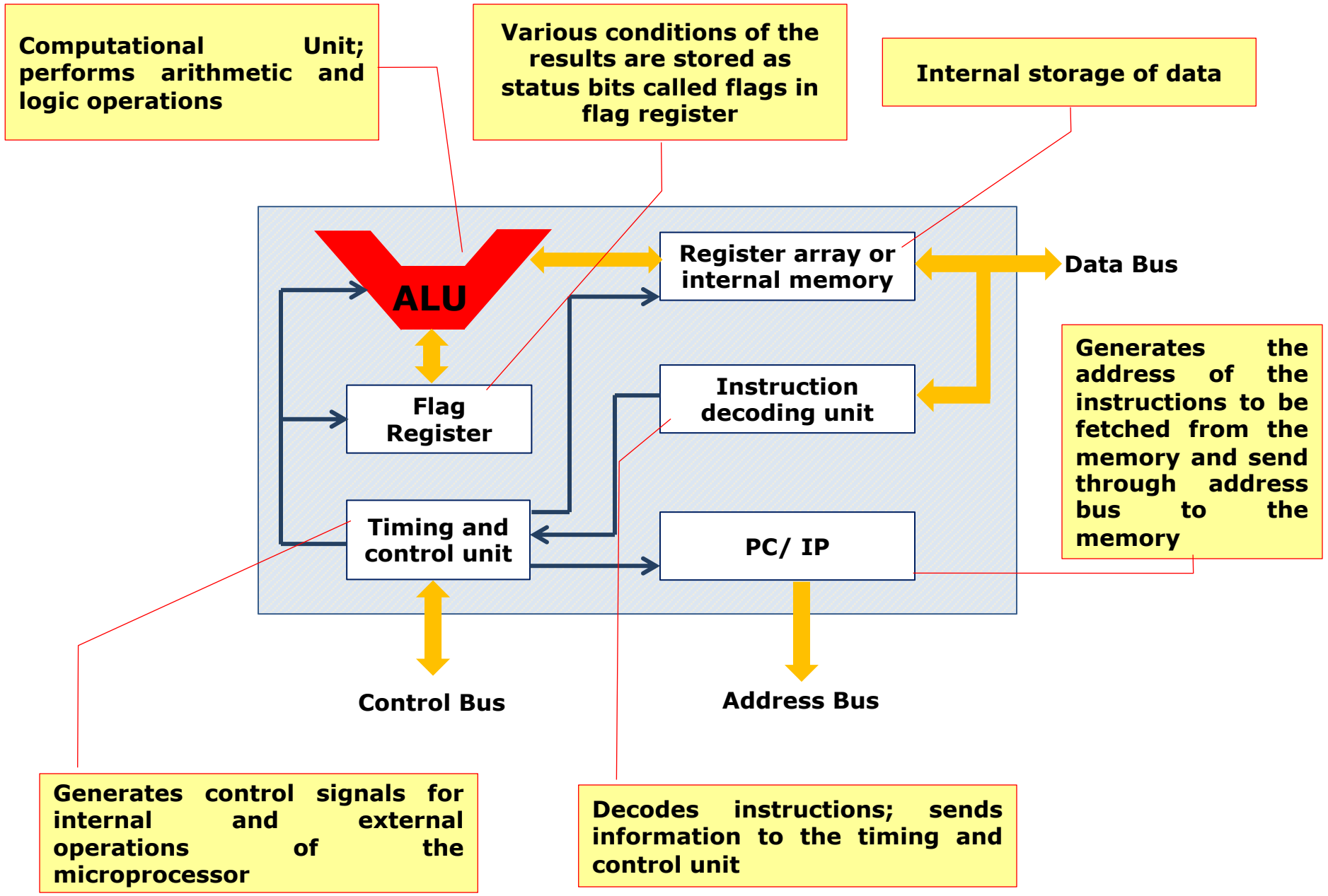
Intel 8086 (16 bit processor)

Second Generation
During 1973
NMOS technology ⇒ Faster speed, Higher density, Compatible with TTL
4 / 8/ 16 bit processors ⇒ 40 pins
Ability to address large memory spaces and I/O ports
Greater number of levels of subroutine nesting
Better interrupt handling capabilities

Intel 8085 (8 bit processor)

First Generation
Between 1971 – 1973
PMOS technology, non compatible with TTL
4 bit processors ⇒ 16 pins
8 and 16 bit processors ⇒ 40 pins
Due to limitations of pins, signals are multiplexed

Functional blocks



8086 - Overview

First 16- bit processor released by INTEL in the year 1978

Originally HMOS, now manufactured using HMOS III technique

Approximately 29, 000 transistors, 40 pin DIP, 5V supply

Does not have internal clock; external asymmetric clock source with 33% duty cycle

20-bit address to access memory ⇒ can address up to 2^{20} = 1 megabytes of memory space.

Addressable memory space is organized in to two banks of 512 kb each; Even (or lower) bank and Odd (or higher) bank. Address line A_0 is used to select even bank and control signal $\overline{BHE}$ is used to access odd bank

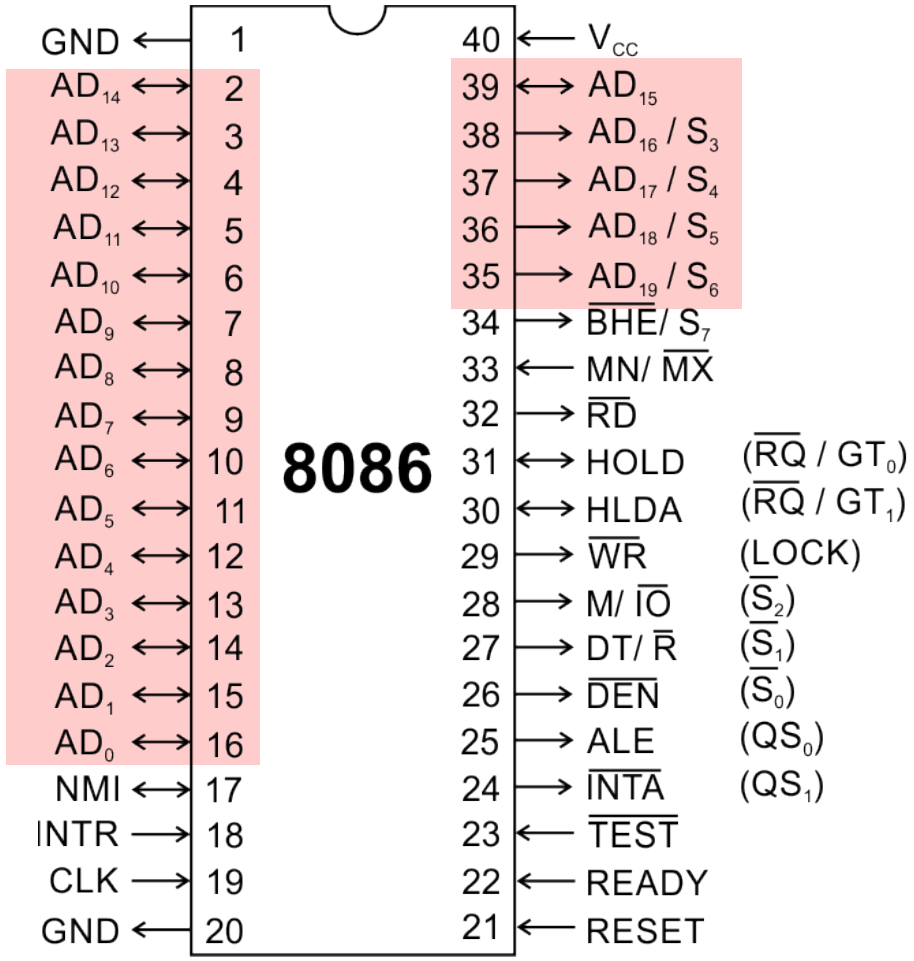
Uses a separate 16 bit address for I/O mapped devices ⇒ can generate 2^{16} = 64 k addresses.

Operates in two modes: minimum mode and maximum mode, decided by the signal at MN and $\overline{MX}$ pins.

Pins and signals

Pins and Signals

Common signals



AD_0-AD_{15} (Bidirectional)

Address/Data bus

Low order address bus; these are multiplexed with data.

When AD lines are used to transmit memory address the symbol A is used instead of AD, for example A_0-A_{15}.

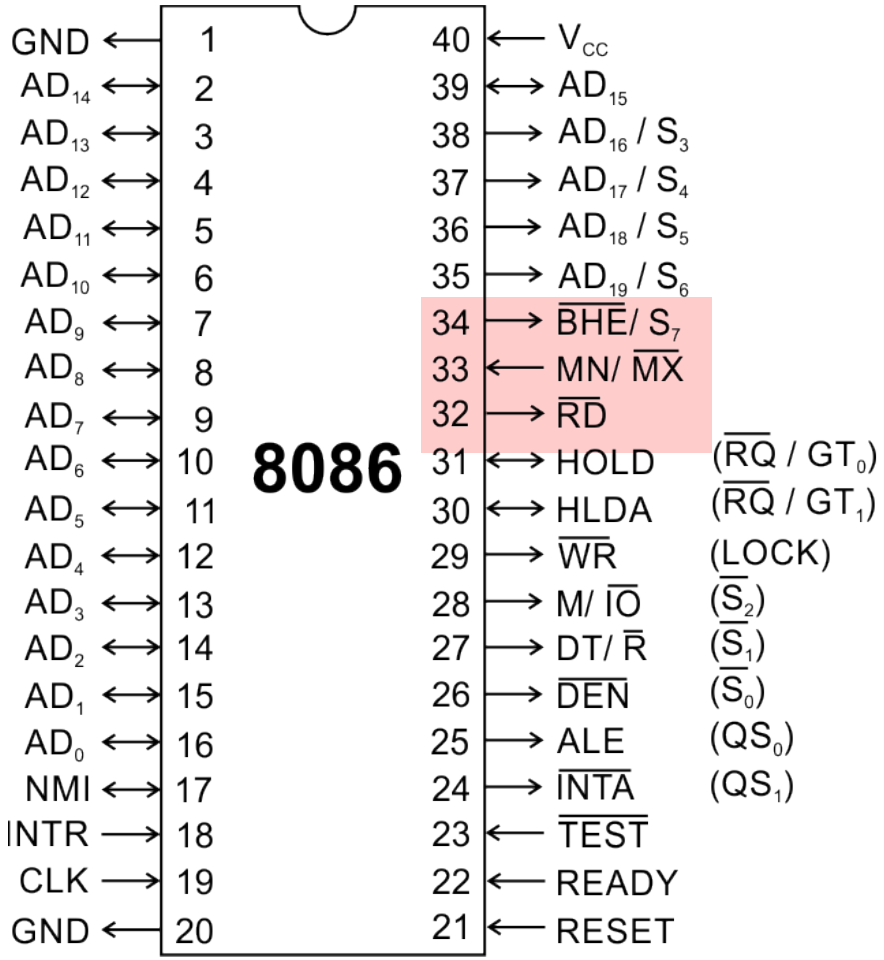
When data are transmitted over AD lines the symbol D is used in place of AD, for example D_0-D_7, D_8-D_{15} or D_0-D_{15}.

A_{16}/S_3, A_{17}/S_4, A_{18}/S_5, A_{19}/S_6

High order address bus. These are multiplexed with status signals

Pins and Signals

Common signals



BHE (Active Low)/S_7 (Output)

Bus High Enable/Status

It is used to enable data onto the most significant half of data bus, D_8-D_{15}. 8-bit device connected to upper half of the data bus use BHE (Active Low) signal. It is multiplexed with status signal S_7.

MN/ MX

MINIMUM / MAXIMUM

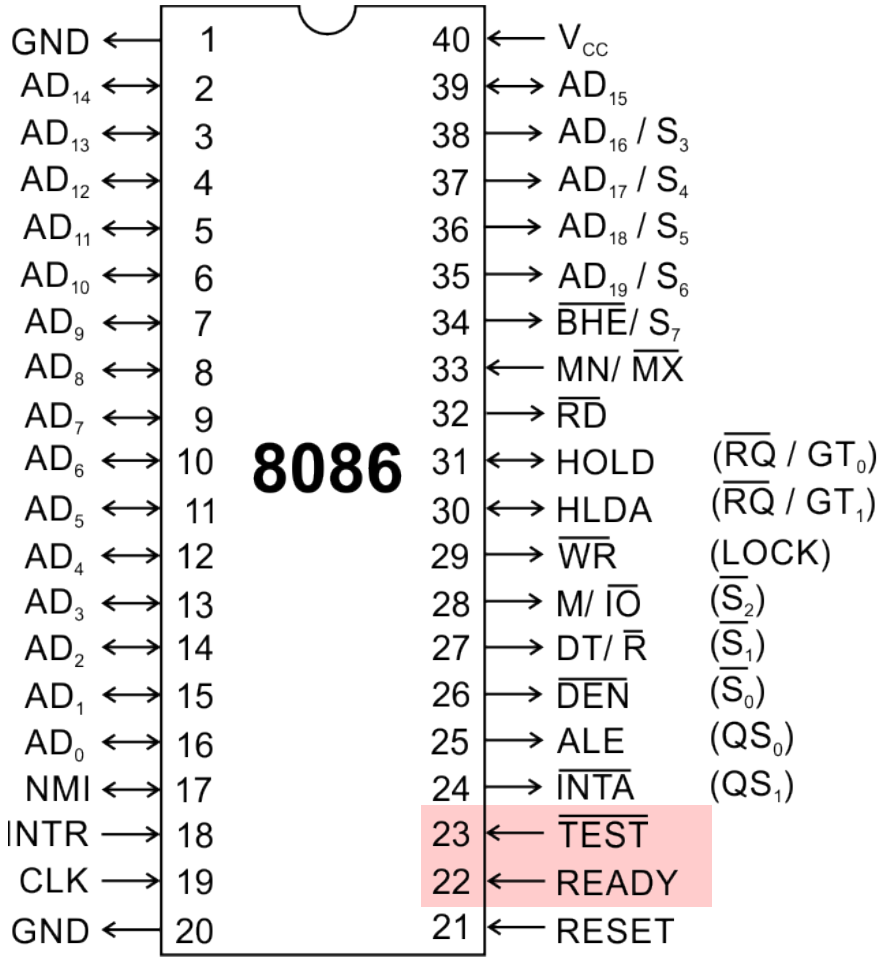
This pin signal indicates what mode the processor is to operate in.

RD (Read) (Active Low)

The signal is used for read operation.
It is an output signal.
It is active when low.

Pins and Signals

Common signals



TEST

$\overline{\text{TEST}}$ input is tested by the 'WAIT' instruction.

8086 will enter a wait state after execution of the WAIT instruction and will resume execution only when the $\overline{\text{TEST}}$ is made low by an active hardware.

This is used to synchronize an external activity to the processor internal operation.

READY

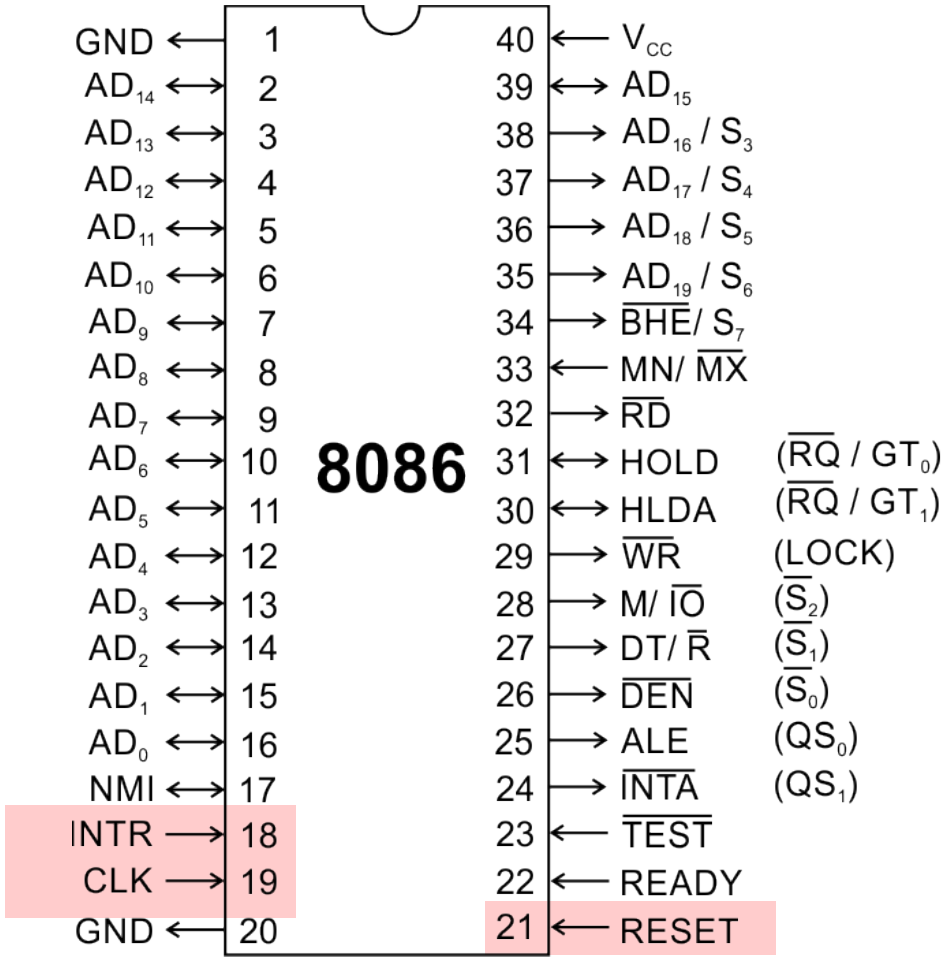
This is the acknowledgement from the slow device or memory that they have completed the data transfer.

The signal made available by the devices is synchronized by the 8284A clock generator to provide ready input to the 8086.

The signal is active high.

Pins and Signals

Common signals



RESET (Input)

Causes the processor to immediately terminate its present activity.

The signal must be active HIGH for at least four clock cycles.

CLK

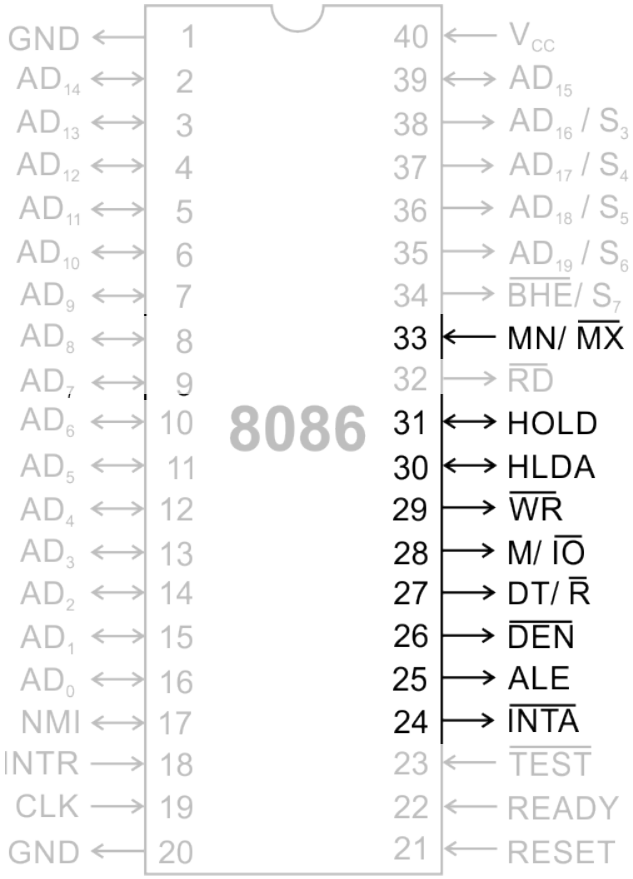
The clock input provides the basic timing for processor operation and bus control activity. Its an asymmetric square wave with 33% duty cycle.

INTR Interrupt Request

This is a triggered input. This is sampled during the last clock cycles of each instruction to determine the availability of the request. If any interrupt request is pending, the processor enters the interrupt acknowledge cycle.

This signal is active high and internally synchronized.

Min/ Max Pins



The 8086 microprocessor can work in two modes of operations : Minimum mode and Maximum mode.

In the minimum mode of operation the microprocessor do not associate with any co-processors and can not be used for multiprocessor systems.

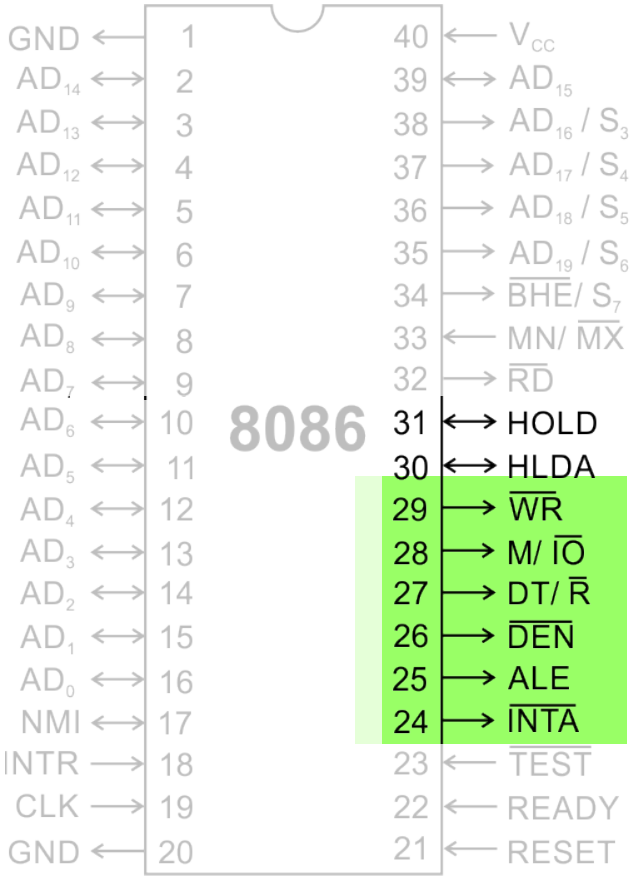
In the maximum mode the 8086 can work in multi-processor or co-processor configuration.

Minimum or maximum mode operations are decided by the pin MN/ MX(Active low).

When this pin is high 8086 operates in minimum mode otherwise it operates in Maximum mode.

Pins and Signals

Minimum mode signals



Pins 24 -31

For minimum mode operation, the MN/ $\overline{MX}$ is tied to VCC (logic high)

8086 itself generates all the bus control signals

DT/$\overline{R}$ — **(Data Transmit/ Receive) Output signal from the processor to control the direction of data flow through the data transceivers**

$\overline{DEN}$ — **(Data Enable) Output signal from the processor used as out put enable for the transceivers**

ALE — **(Address Latch Enable) Used to demultiplex the address and data lines using external latches**

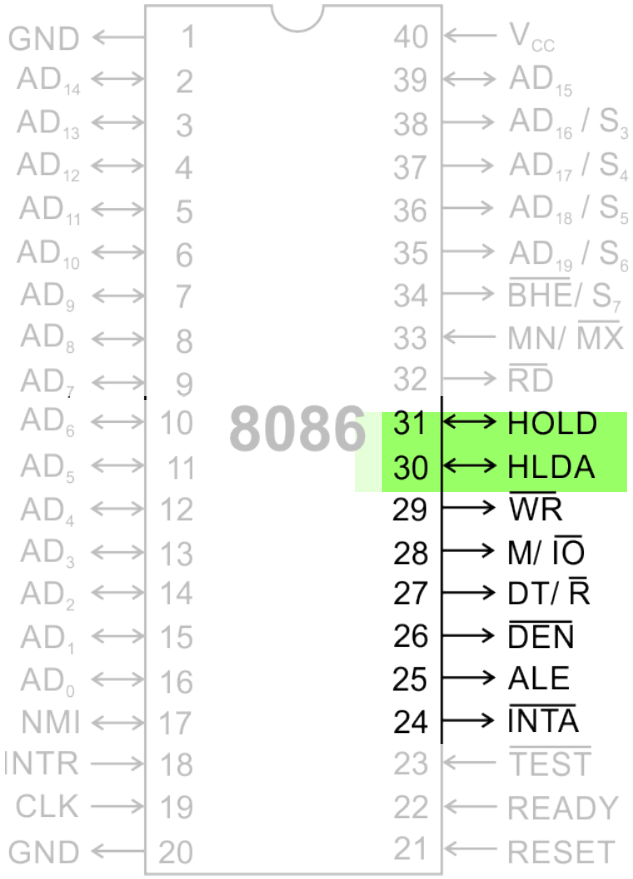
M/$\overline{IO}$ — **Used to differentiate memory access and I/O access. For memory reference instructions, it is high. For IN and OUT instructions, it is low.**

$\overline{WR}$ — **Write control signal; asserted low Whenever processor writes data to memory or I/O port**

$\overline{INTA}$ — **(Interrupt Acknowledge) When the interrupt request is accepted by the processor, the output is low on this line.**

Pins and Signals

Minimum mode signals



Pins 24 -31

For minimum mode operation, the MN/ $\overline{MX}$ is tied to VCC (logic high)

8086 itself generates all the bus control signals

HOLD — **Input signal to the processor form the bus masters as a request to grant the control of the bus.**

Usually used by the DMA controller to get the control of the bus.

HLDA — **(Hold Acknowledge) Acknowledge signal by the processor to the bus master requesting the control of the bus through HOLD.**

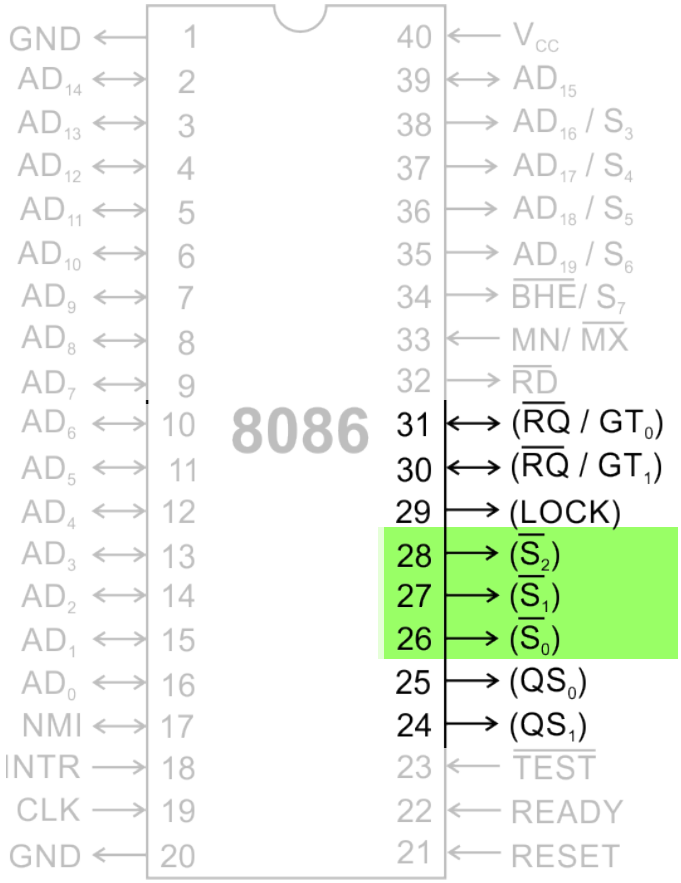
The acknowledge is asserted high, when the processor accepts HOLD.

Pins and Signals

Maximum mode signals

During maximum mode operation, the MN/ $\overline{MX}$ is grounded (logic low)

Pins 24 -31 are reassigned



$\overline{S_0}$, $\overline{S_1}$, $\overline{S_2}$ **Status signals; used by the 8086 bus controller to generate bus timing and control signals. These are decoded as shown.**

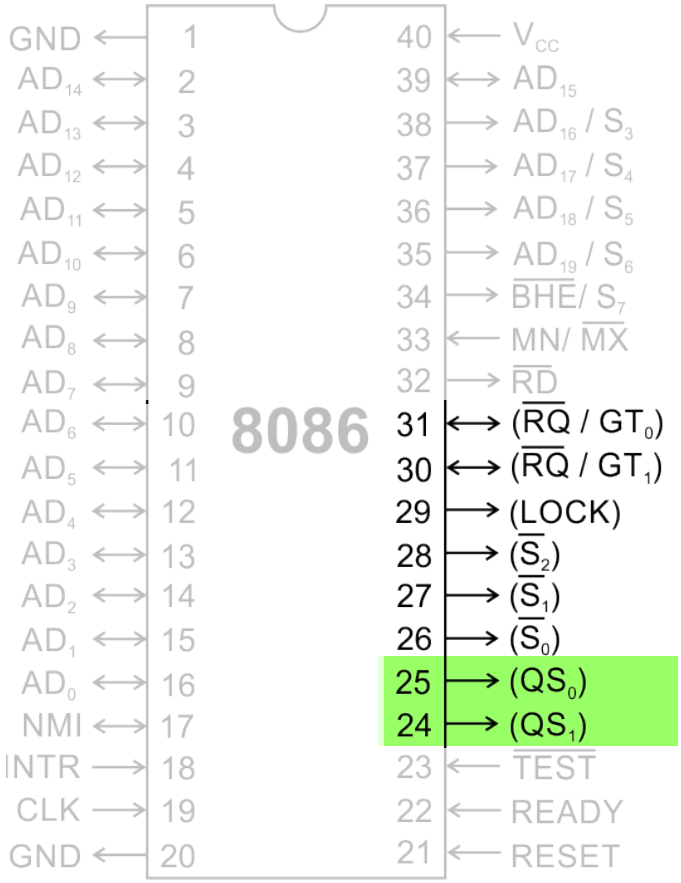
Status Signal			Machine Cycle
$\overline{S}_2$	$\overline{S}_1$	$\overline{S}_0$	
0	0	0	Interrupt acknowledge
0	0	1	Read I/O port
0	1	0	Write I/O port
0	1	1	Halt
1	0	0	Code access
1	0	1	Read memory
1	1	0	Write memory
1	1	1	Passive/Inactive

Pins and Signals

Maximum mode signals

During maximum mode operation, the MN/ $\overline{MX}$ is grounded (logic low)

Pins 24 -31 are reassigned



$\overline{QS_0}$, $\overline{QS_1}$ **(Queue Status) The processor provides the status of queue in these lines.**

The queue status can be used by external device to track the internal status of the queue in 8086.

The output on QS_0 and QS_1 can be interpreted as shown in the table.

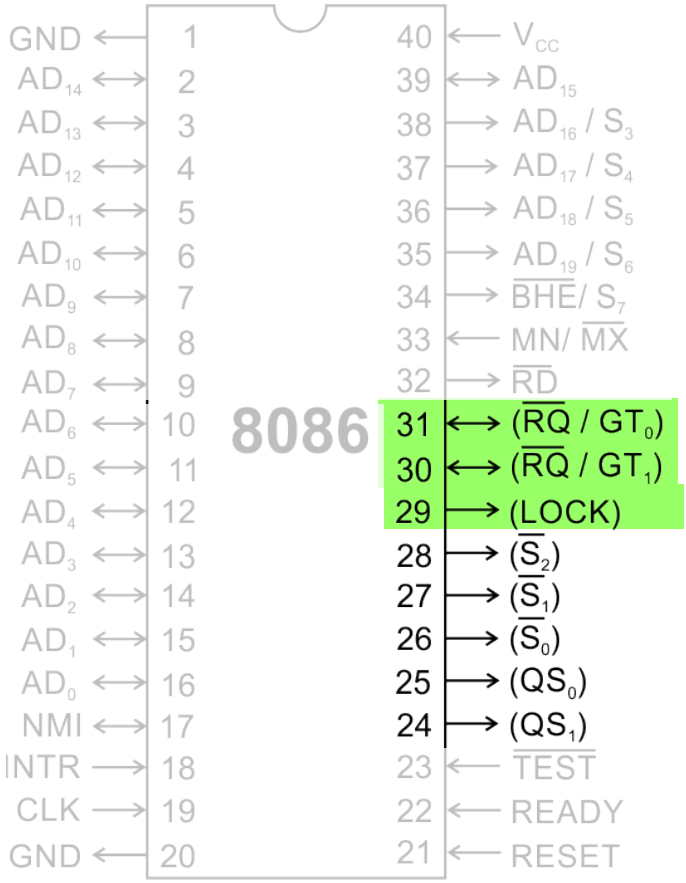
Queue status		Queue operation
QS_1	QS_0	
0	0	No operation
0	1	First byte of an opcode from queue
1	0	Empty the queue
1	1	Subsequent byte from queue

Pins and Signals

Maximum mode signals

During maximum mode operation, the MN/ $\overline{MX}$ is grounded (logic low)

Pins 24 -31 are reassigned



$\overline{RQ}/\overline{GT_0}$, $\overline{RQ}/\overline{GT_1}$

(Bus Request/ Bus Grant) These requests are used by other local bus masters to force the processor to release the local bus at the end of the processor's current bus cycle.

These pins are bidirectional.

The request on $\overline{GT_0}$ will have higher priority than $\overline{GT_1}$

$\overline{LOCK}$

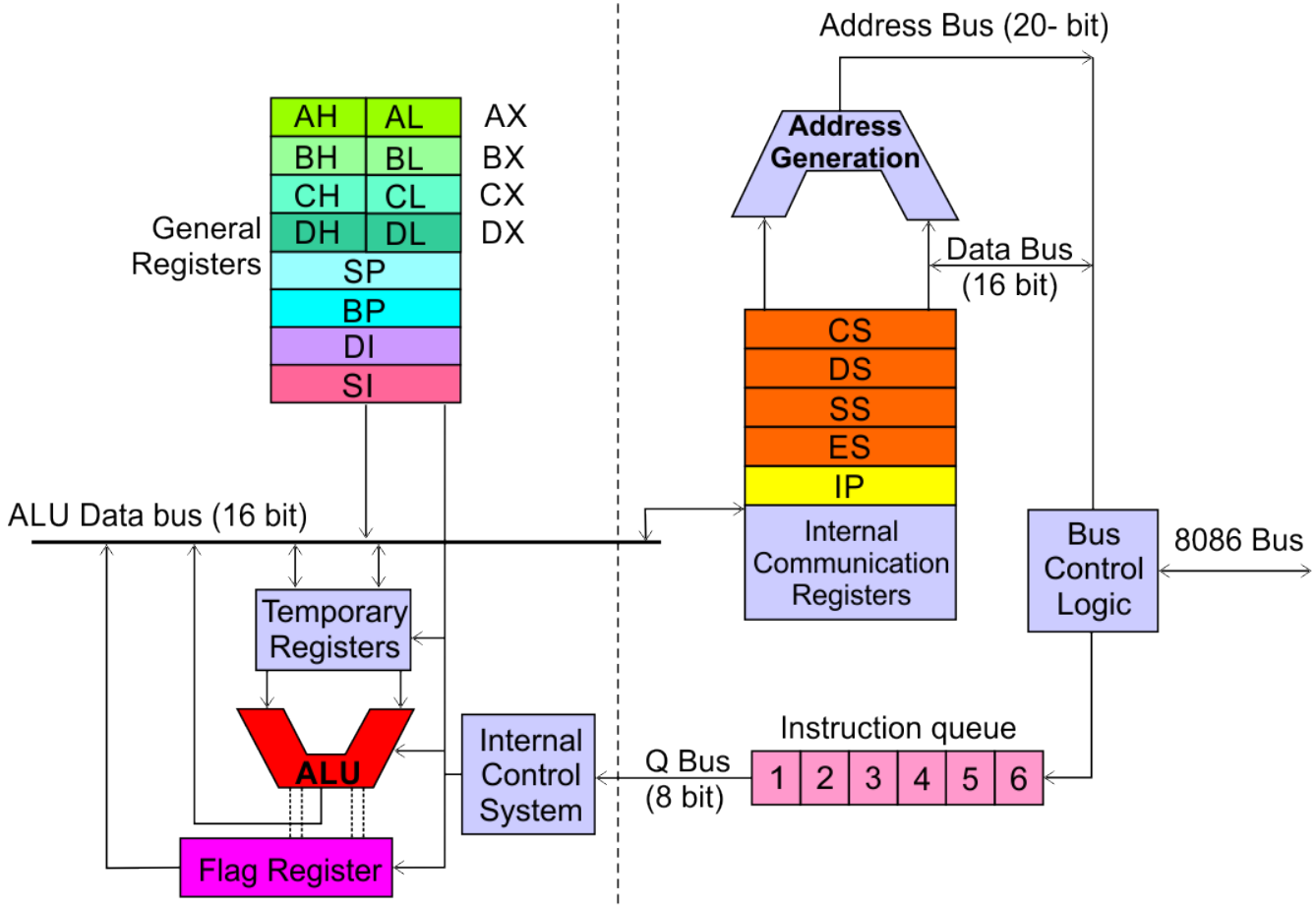
An output signal activated by the LOCK prefix instruction.

Remains active until the completion of the instruction prefixed by LOCK.

The 8086 output low on the $\overline{LOCK}$ pin while executing an instruction prefixed by LOCK to prevent other bus masters from gaining control of the system bus.

8086 Architecture

Architecture



Execution Unit (EU)

EU executes instructions that have already been fetched by the BIU.

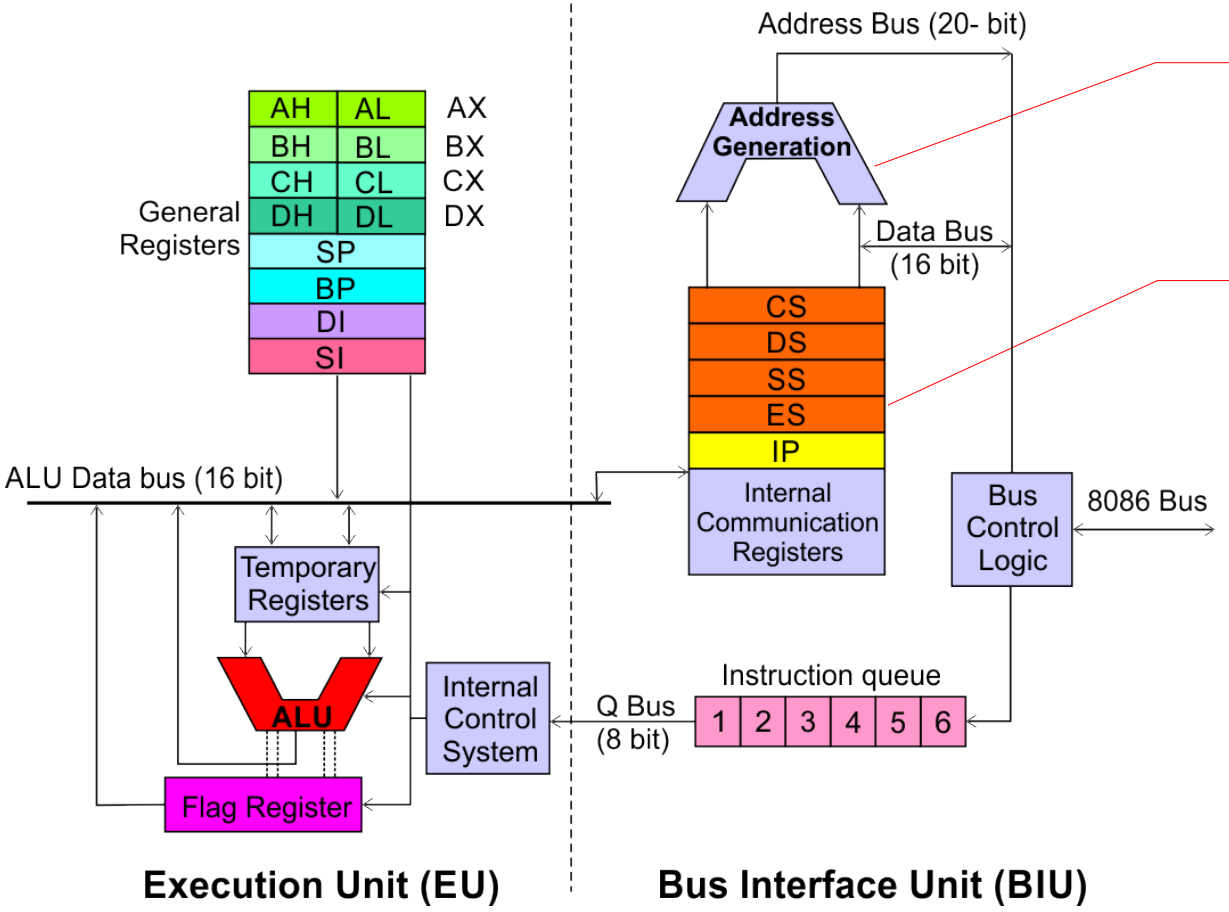
BIU and EU functions separately.

Bus Interface Unit (BIU)

BIU fetches instructions, reads data from memory and I/O ports, writes data to memory and I/ O ports.

Architecture

Bus Interface Unit (BIU)



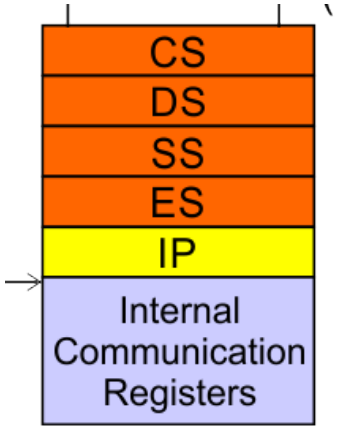
Dedicated Adder to generate 20 bit address

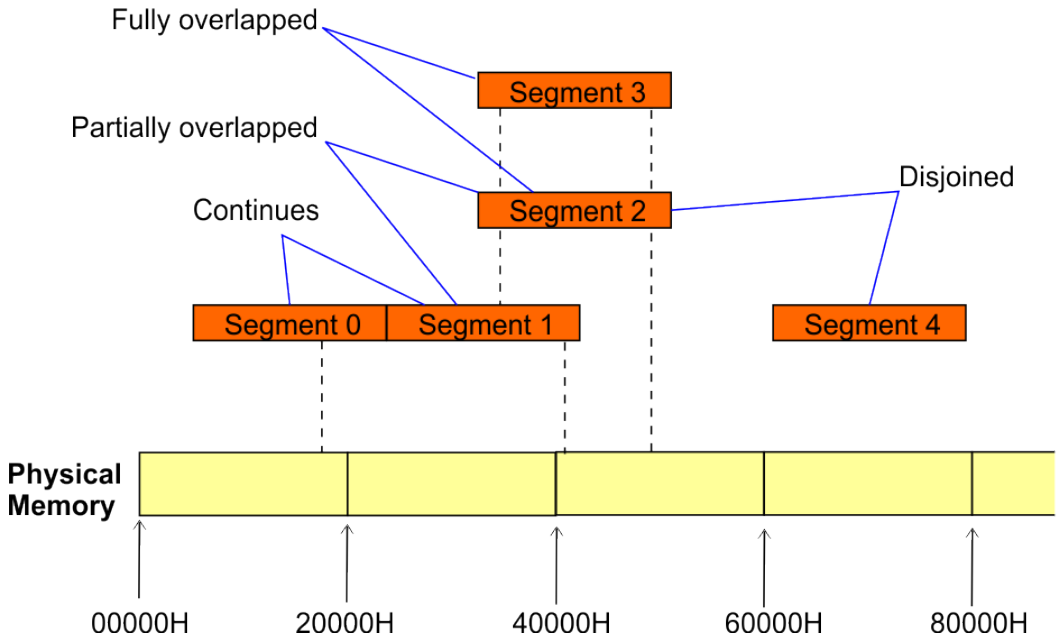
Four 16-bit segment registers

Code Segment (CS)
Data Segment (DS)
Stack Segment (SS)
Extra Segment (ES)

Bus Interface Unit (BIU)

Segment Registers





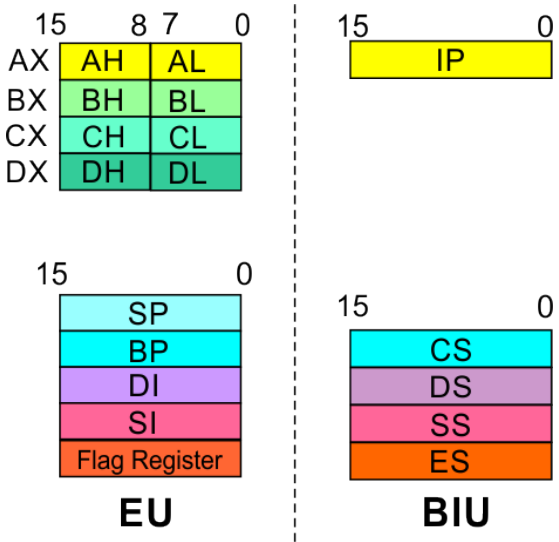
- **8086's 1-megabyte memory is divided into segments of up to 64K bytes each.**
- **The 8086 can directly address four segments (256 K bytes within the 1 M byte of memory) at a particular time.**
- **Programs obtain access to code and data in the segments by changing the segment register content to point to the desired segments.**

Bus Interface Unit (BIU)

Segment Registers

Code Segment Register

- 16-bit
- CS contains the base or start of the current code segment; IP contains the distance or offset from this address to the next instruction byte to be fetched.
- BIU computes the 20-bit physical address by logically shifting the contents of CS 4-bits to the left and then adding the 16-bit contents of IP.
- That is, all instructions of a program are relative to the contents of the CS register multiplied by 16 and then offset is added provided by the IP.

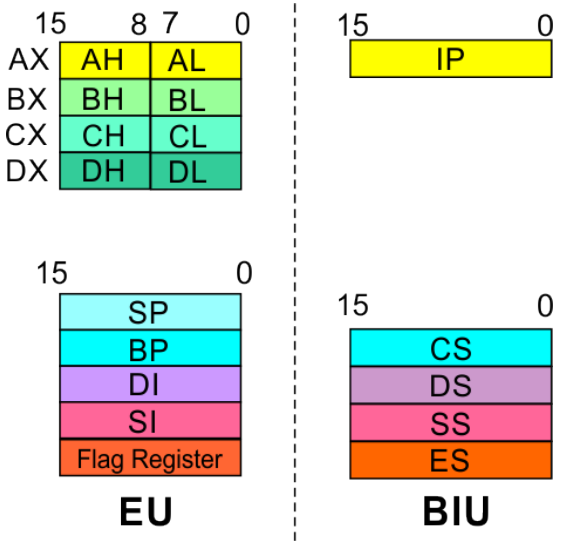


Bus Interface Unit (BIU)

Segment Registers

Data Segment Register

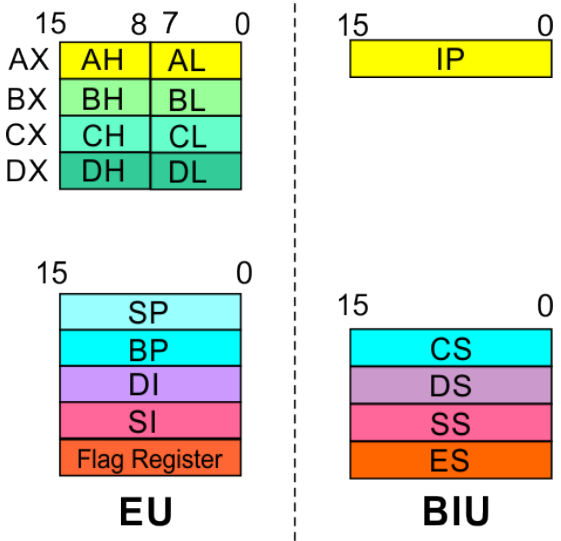
- 16-bit
- Points to the current data segment; operands for most instructions are fetched from this segment.
- The 16-bit contents of the Source Index (SI) or Destination Index (DI) or a 16-bit displacement are used as offset for computing the 20-bit physical address.



Segment Registers

Stack Segment Register

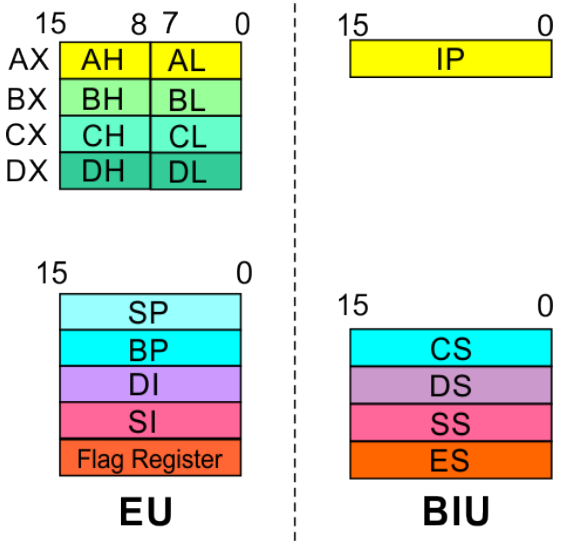
- 16-bit
- Points to the current stack.
- The 20-bit physical stack address is calculated from the Stack Segment (SS) and the Stack Pointer (SP) for stack instructions such as **PUSH** and **POP**.
- In based addressing mode, the 20-bit physical stack address is calculated from the Stack segment (SS) and the Base Pointer (BP).



Segment Registers

Extra Segment Register

- 16-bit
- Points to the extra segment in which data (in excess of 64K pointed to by the DS) is stored.
- String instructions use the ES and DI to determine the 20-bit physical address for the destination.

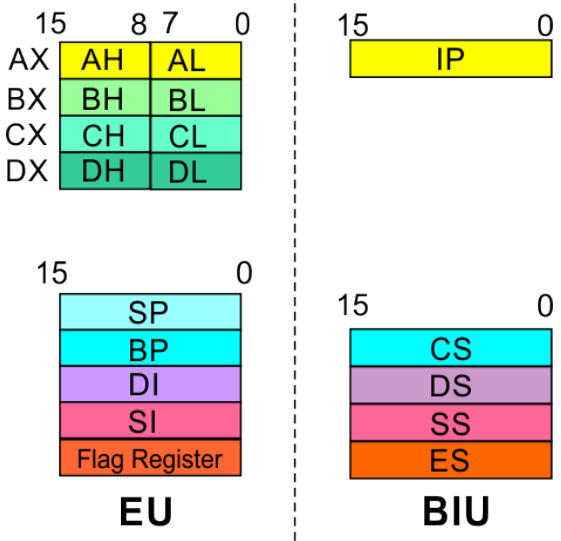


Bus Interface Unit (BIU)

Segment Registers

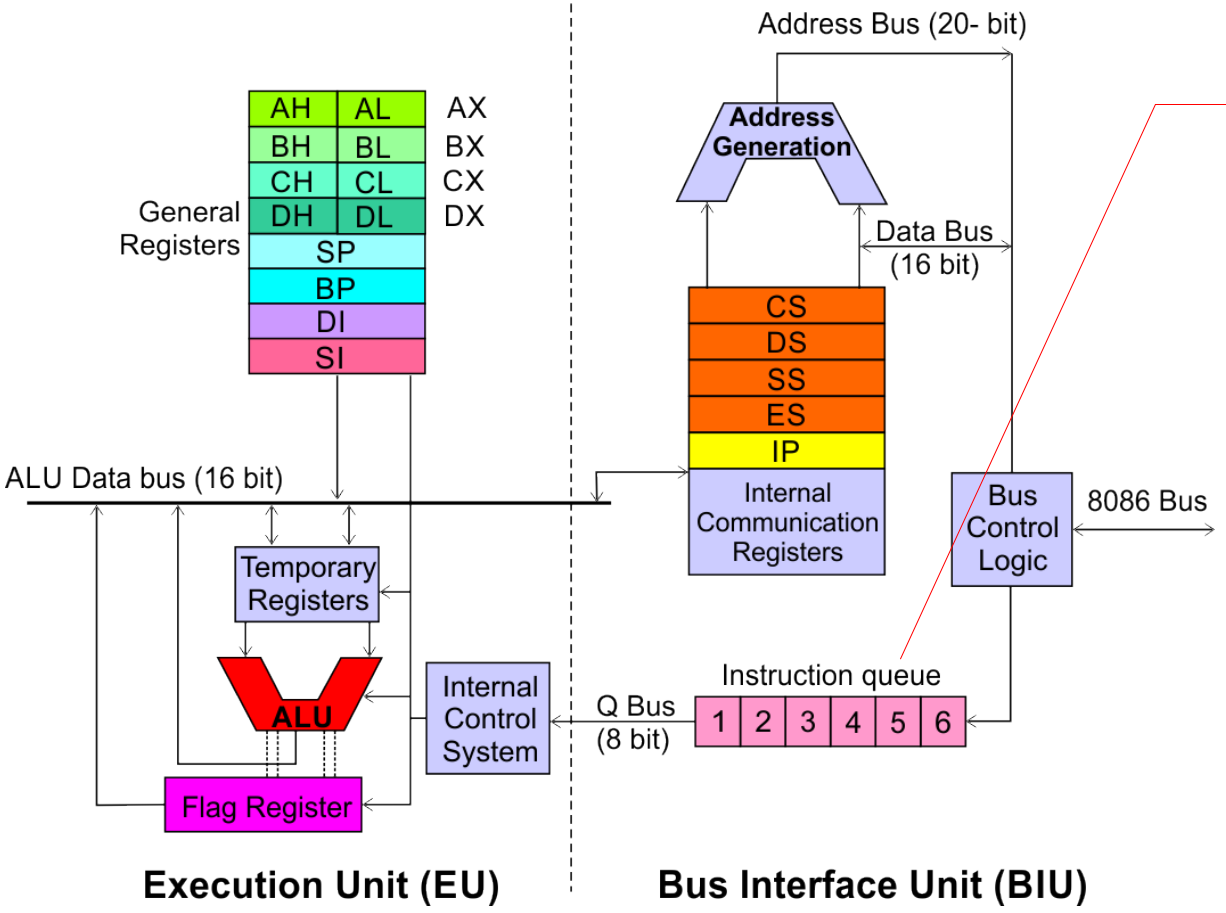
Instruction Pointer

- 16-bit
- Always points to the next instruction to be executed within the currently executing code segment.
- So, this register contains the 16-bit offset address pointing to the next instruction code within the 64Kb of the code segment area.
- Its content is automatically incremented as the execution of the next instruction takes place.



Architecture

Bus Interface Unit (BIU)



Instruction queue

- **A group of First-In-First-Out (FIFO) in which up to 6 bytes of instruction code are pre fetched from the memory ahead of time.**
- **This is done in order to speed up the execution by overlapping instruction fetch with execution.**
- **This mechanism is known as pipelining.**

Architecture

Execution Unit (EU)

EU decodes and executes instructions.

A decoder in the EU control system translates instructions.

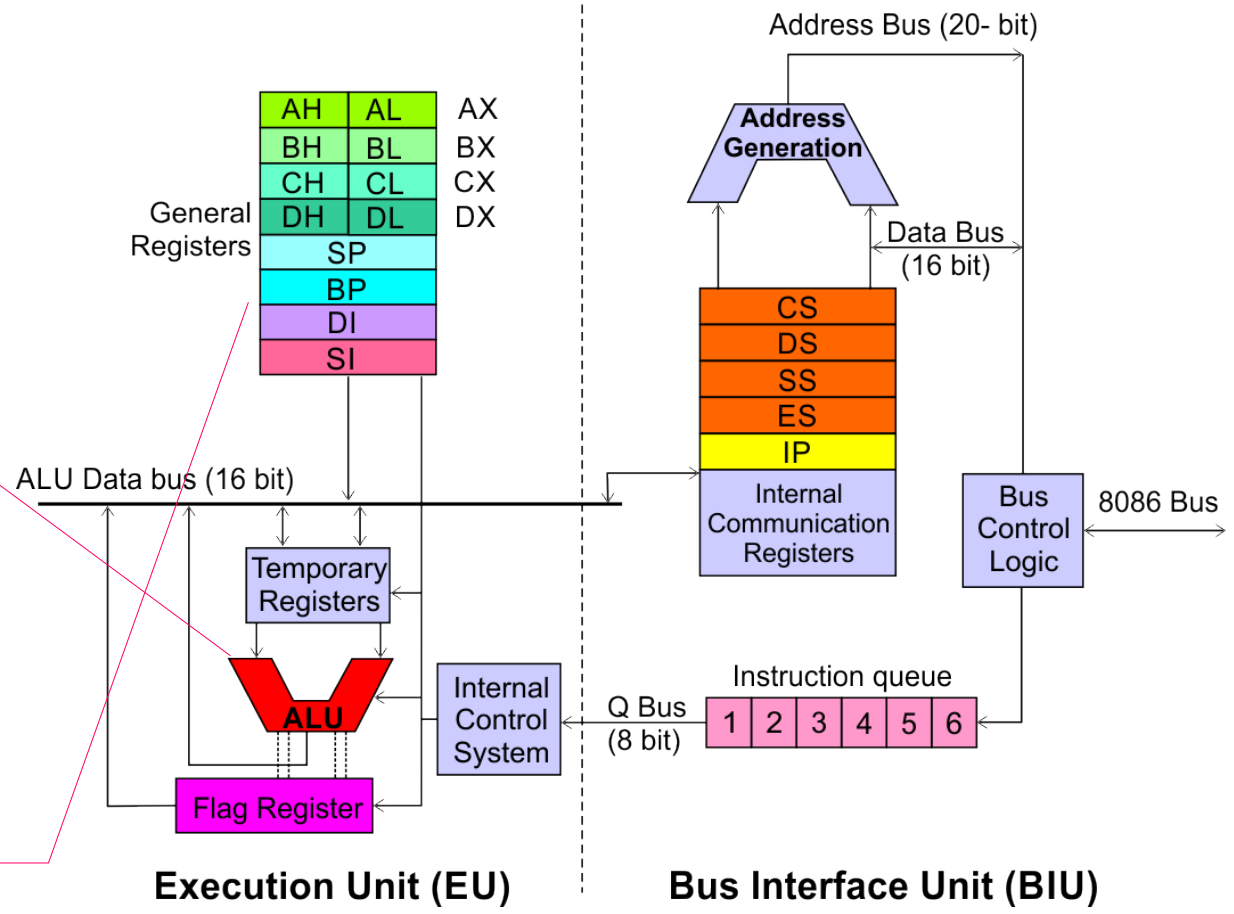
16-bit ALU for performing arithmetic and logic operation

Four general purpose registers(AX, BX, CX, DX);

Pointer registers (Stack Pointer, Base Pointer);

and

Index registers (Source Index, Destination Index) each of 16-bits



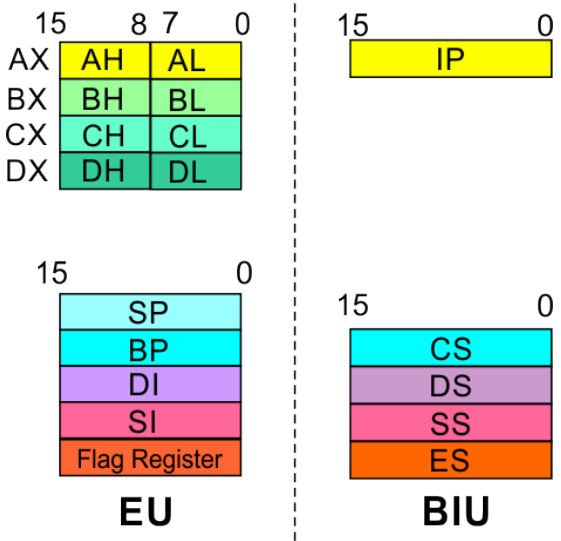
Some of the 16 bit registers can be used as two 8 bit registers as :

AX can be used as AH and AL
BX can be used as BH and BL
CX can be used as CH and CL
DX can be used as DH and DL

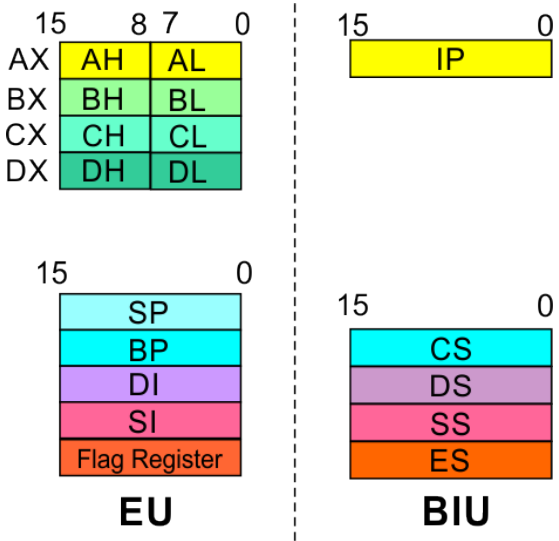
EU Registers

Accumulator Register (AX)

- **Consists of two 8-bit registers AL and AH, which can be combined together and used as a 16-bit register AX.**
- **AL in this case contains the low order byte of the word, and AH contains the high-order byte.**
- **The I/O instructions use the AX or AL for inputting / outputting 16 or 8 bit data to or from an I/O port.**
- **Multiplication and Division instructions also use the AX or AL.**



EU Registers



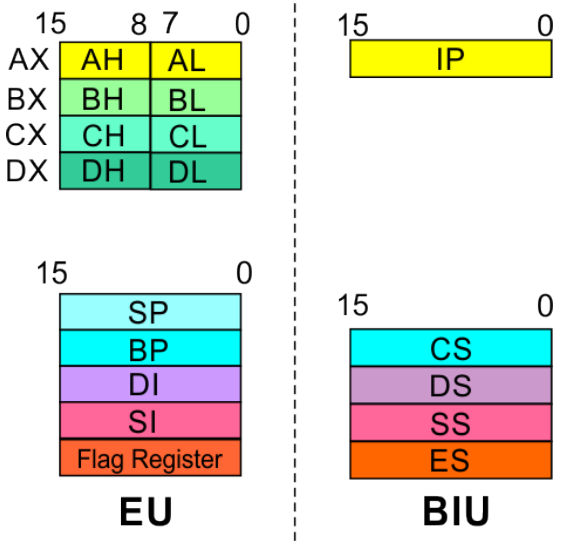
Base Register (BX)

- Consists of two 8-bit registers BL and BH, which can be combined together and used as a 16-bit register BX.
- BL in this case contains the low-order byte of the word, and BH contains the high-order byte.
- This is the only general purpose register whose contents can be used for addressing the 8086 memory.
- All memory references utilizing this register content for addressing use DS as the default segment register.

EU Registers

Counter Register (CX)

- Consists of two 8-bit registers CL and CH, which can be combined together and used as a 16-bit register CX.
- When combined, CL register contains the low order byte of the word, and CH contains the high-order byte.
- Instructions such as SHIFT, ROTATE and LOOP use the contents of CX as a counter.



Example:

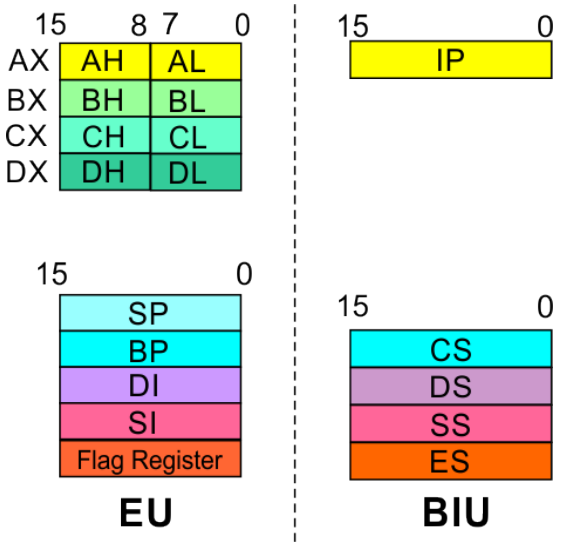
The instruction LOOP START automatically decrements CX by 1 without affecting flags and will check if [CX] = 0.

If it is zero, 8086 executes the next instruction; otherwise the 8086 branches to the label START.

EU Registers

Data Register (DX)

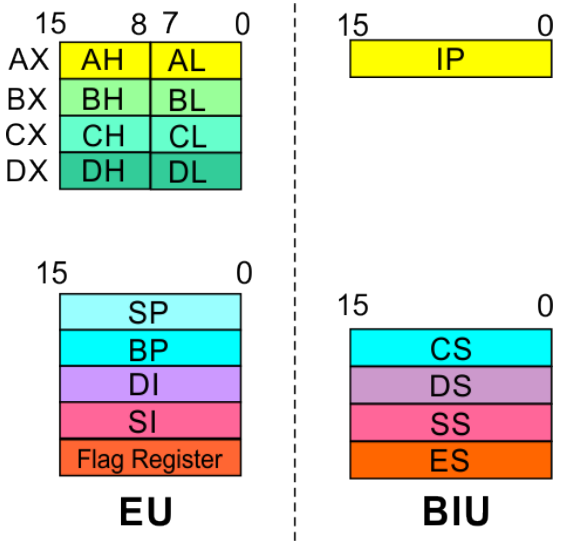
- Consists of two 8-bit registers DL and DH, which can be combined together and used as a 16-bit register DX.
- When combined, DL register contains the low order byte of the word, and DH contains the high-order byte.
- Used to hold the high 16-bit result (data) in 16 X 16 multiplication or the high 16-bit dividend (data) before a 32 ÷ 16 division and the 16-bit reminder after division.



EU Registers

Stack Pointer (SP) and Base Pointer (BP)

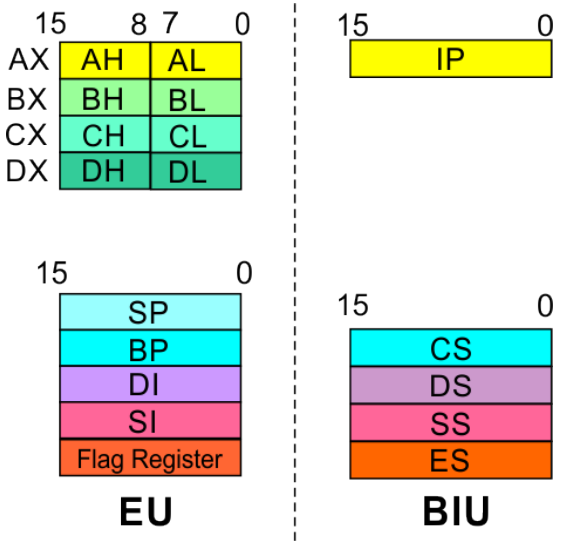
- **SP and BP are used to access data in the stack segment.**
- **SP is used as an offset from the current SS during execution of instructions that involve the stack segment in the external memory.**
- **SP contents are automatically updated (incremented/ decremented) due to execution of a POP or PUSH instruction.**
- **BP contains an offset address in the current SS, which is used by instructions utilizing the based addressing mode.**



EU Registers

Source Index (SI) and Destination Index (DI)

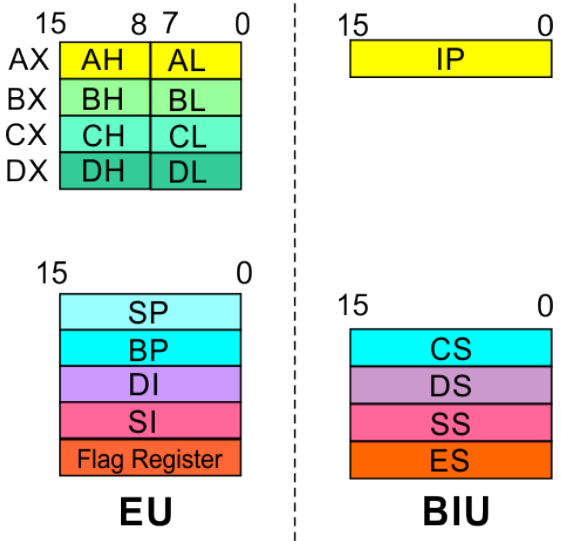
- **Used in indexed addressing.**
- **Instructions that process data strings use the SI and DI registers together with DS and ES respectively in order to distinguish between the source and destination addresses.**



EU Registers

Source Index (SI) and Destination Index (DI)

- **Used in indexed addressing.**
- **Instructions that process data strings use the SI and DI registers together with DS and ES respectively in order to distinguish between the source and destination addresses.**



Flag Register

Auxiliary Carry Flag

This is set, if there is a carry from the lowest nibble, i.e, bit three during addition, or borrow for the lowest nibble, i.e, bit three, during subtraction.

Carry Flag

This flag is set, when there is a carry out of MSB in case of addition or a borrow in case of subtraction.

Sign Flag

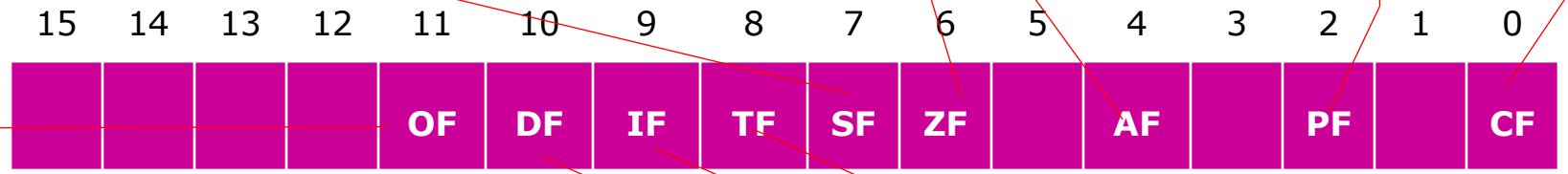
This flag is set, when the result of any computation is negative

Zero Flag

This flag is set, if the result of the computation or comparison performed by an instruction is zero

Parity Flag

This flag is set to 1, if the lower byte of the result contains even number of 1's ; for odd number of 1's set to zero.



Over flow Flag

This flag is set, if an overflow occurs, i.e, if the result of a signed operation is large enough to accommodate in a destination register. The result is of more than 7-bits in size in case of 8-bit signed operation and more than 15-bits in size in case of 16-bit sign operations, then the overflow will be set.

Tarp Flag

If this flag is set, the processor enters the single step execution mode by generating internal interrupts after the execution of each instruction

Direction Flag

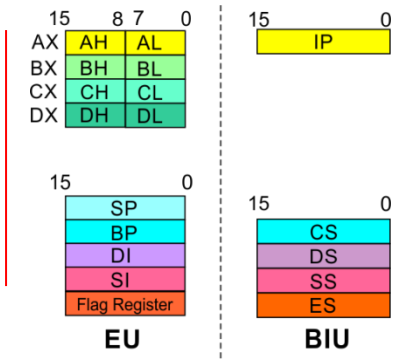
This is used by string manipulation instructions. If this flag bit is '0', the string is processed beginning from the lowest address to the highest address, i.e., auto incrementing mode. Otherwise, the string is processed from the highest address towards the lowest address, i.e., auto incrementing mode.

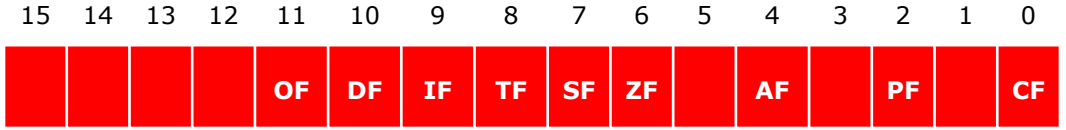
Interrupt Flag

Causes the 8086 to recognize external mask interrupts; clearing IF disables these interrupts.

Architecture

8086 registers categorized into 4 groups





Sl.No.	Type	Register width	Name of register
1	General purpose register	16 bit	AX, BX, CX, DX
		8 bit	AL, AH, BL, BH, CL, CH, DL, DH
2	Pointer register	16 bit	SP, BP
3	Index register	16 bit	SI, DI
4	Instruction Pointer	16 bit	IP
5	Segment register	16 bit	CS, DS, SS, ES
6	Flag (PSW)	16 bit	Flag register

Registers and Special Functions

Register	Name of the Register	Special Function
AX	16-bit Accumulator	Stores the 16-bit results of arithmetic and logic operations
AL	8-bit Accumulator	Stores the 8-bit results of arithmetic and logic operations
BX	Base register	Used to hold base value in base addressing mode to access memory data
CX	Count Register	Used to hold the count value in SHIFT, ROTATE and LOOP instructions
DX	Data Register	Used to hold data for multiplication and division operations
SP	Stack Pointer	Used to hold the offset address of top stack memory
BP	Base Pointer	Used to hold the base value in base addressing using SS register to access data from stack memory
SI	Source Index	Used to hold index value of source operand (data) for string instructions
DI	Data Index	Used to hold the index value of destination operand (data) for string operations

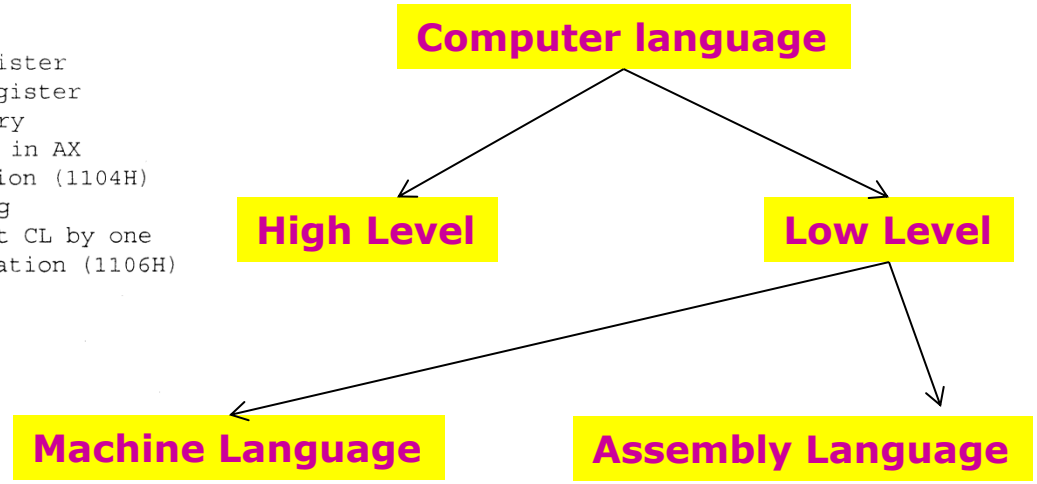
Addressing Modes & Instruction set

Introduction

```
;PROGRAM TO ADD TWO 16-BIT DATA (METHOD-1)

DATA SEGMENT              ;Assembler directive

        ORG 1104H         ;Assembler directive
        SUM DW 0          ;Assembler directive
        CARRY DB 0        ;Assembler directive

DATA ENDS                 ;Assembler directive

CODE SEGMENT              ;Assembler directive

       ASSUME CS:CODE     ;Assembler directive
       ASSUME DS:DATA     ;Assembler directive
        ORG 1000H         ;Assembler directive

        MOV AX,205AH      ;Load the first data in AX register
        MOV BX,40EDH      ;Load the second data in BX register
        MOV CL,00H        ;Clear the CL register for carry
        ADD AX,BX         ;Add the two data, sum will be in AX
        MOV SUM,AX        ;Store the sum in memory location (1104H)
        JNC AHEAD         ;Check the status of carry flag
        INC CL            ;If carry flag is set,increment CL by one
AHEAD:  MOV CARRY,CL      ;Store the carry in memory location (1106H)
        HLT

CODE ENDS                 ;Assembler directive
END                       ;Assembler directive
```

Program
A set of instructions written to solve a problem.

Instruction
Directions which a microprocessor follows to execute a task or part of a task.



- **Binary bits**

- **English Alphabets**
- **'Mnemonics'**
- **Assembler**
  Mnemonics → Machine Language

8086 Addressing Modes

Addressing Modes

- Every instruction of a program has to operate on a data.
- The different ways in which a source operand is denoted in an instruction are known as addressing modes.

1. Register Addressing 2. Immediate Addressing	Group I : Addressing modes for register and immediate data
3. Direct Addressing 4. Register Indirect Addressing 5. Based Addressing 6. Indexed Addressing 7. Based Index Addressing 8. String Addressing	Group II : Addressing modes for memory data
9. Direct I/O port Addressing 10. Indirect I/O port Addressing	Group III : Addressing modes for I/O ports
11. Relative Addressing	Group IV : Relative Addressing mode
12. Implied Addressing	Group V : Implied Addressing mode

Addressing Modes

Group I : Addressing modes for register and immediate data

1. **Register Addressing**
2. **Immediate Addressing**
3. **Direct Addressing**
4. **Register Indirect Addressing**
5. **Based Addressing**
6. **Indexed Addressing**
7. **Based Index Addressing**
8. **String Addressing**
9. **Direct I/O port Addressing**
10. **Indirect I/O port Addressing**
11. **Relative Addressing**
12. **Implied Addressing**

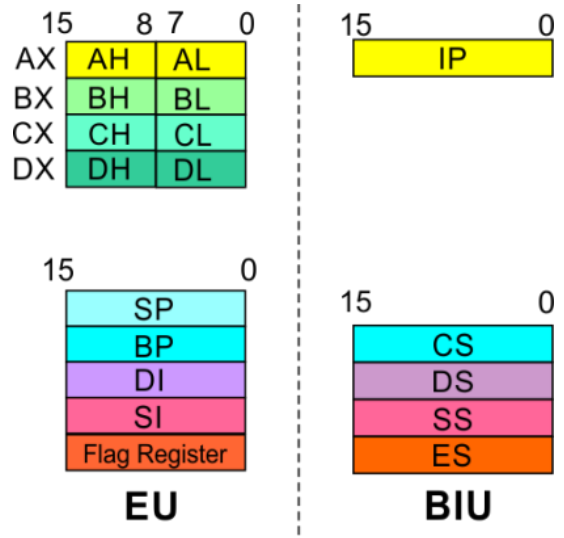
The instruction will specify the name of the register which holds the data to be operated by the instruction.

Example:

MOV CL, DH

The content of 8-bit register DH is moved to another 8-bit register CL

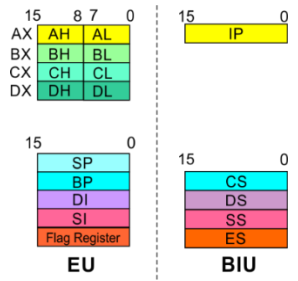
(CL) ← (DH)



Addressing Modes

Group I : Addressing modes for register and immediate data

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing



In immediate addressing mode, an 8-bit or 16-bit data is specified as part of the instruction

Example:

MOV DL, 08H

The 8-bit data (08_H) given in the instruction is moved to DL

(DL) ← 08_H

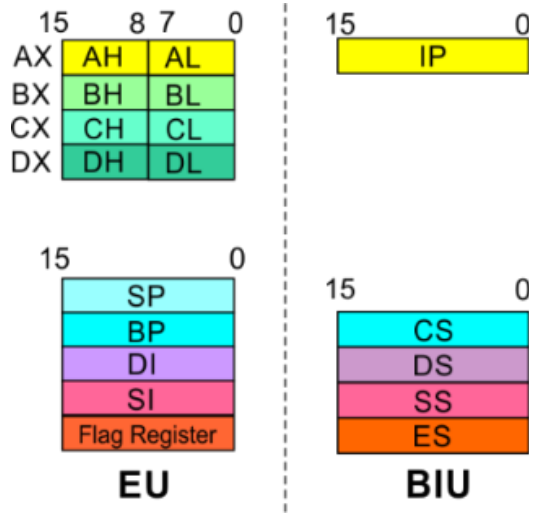
MOV AX, 0A9FH

The 16-bit data ($0A9F_H$) given in the instruction is moved to AX register

(AX) ← $0A9F_H$

Addressing Modes : Memory Access

- **20 Address lines ⇒ 8086 can address up to 2^{20} = 1M bytes of memory**
- **However, the largest register is only 16 bits**
- **Physical Address will have to be calculated**
  Physical Address : Actual address of a byte in memory. i.e. the value which goes out onto the address bus.
- **Memory Address represented in the form –**
  Seg : Offset (Eg - 89AB:F012)
- **Each time the processor wants to access memory, it takes the contents of a segment register, shifts it one hexadecimal place to the left (same as multiplying by 16_{10}), then add the required offset to form the 20- bit address**

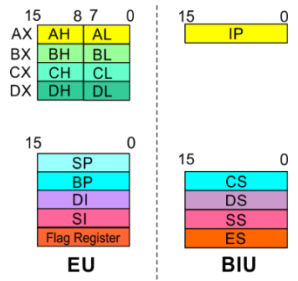


16 bytes of contiguous memory

89AB : F012 → 89AB → 89AB0 (Paragraph to byte → 89AB x 10 = 89AB0)
F012 → 0F012 (Offset is already in byte unit)
+ -------
98AC2 (The absolute address)

Addressing Modes

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing



Here, the effective address of the memory location at which the data operand is stored is given in the instruction.

The effective address is just a 16-bit number written directly in the instruction.

Example:

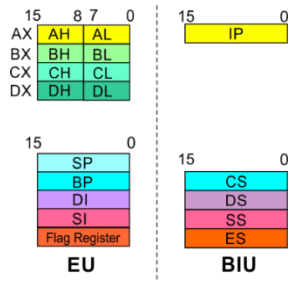
```
MOV   BX, [1354H]
MOV   BL, [0400H]
```

The square brackets around the 1354_H denotes the contents of the memory location. When executed, this instruction will copy the contents of the memory location into BX register.

This addressing mode is called direct because the displacement of the operand from the segment base is specified directly in the instruction.

Addressing Modes

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing



In Register indirect addressing, name of the register which holds the effective address (EA) will be specified in the instruction.

Registers used to hold EA are any of the following registers:

BX, BP, DI and SI.

Content of the DS register is used for base address calculation.

Example:

MOV CX, [BX]

Note : Register/ memory enclosed in brackets refer to content of register/ memory

Operations:

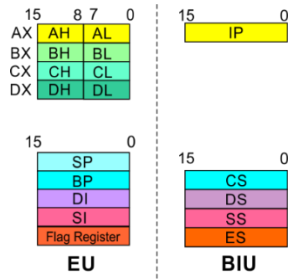
EA = (BX)
BA = (DS) x 16_{10}
MA = BA + EA

(CX) ← (MA) or,

(CL) ← (MA)
(CH) ← (MA +1)

Addressing Modes

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing



In Based Addressing, BX or BP is used to hold the base value for effective address and a signed 8-bit or unsigned 16-bit displacement will be specified in the instruction.

In case of 8-bit displacement, it is sign extended to 16-bit before adding to the base value.

When BX holds the base value of EA, 20-bit physical address is calculated from BX and DS.

When BP holds the base value of EA, BP and SS is used.

Example:

MOV AX, [BX + 08H]

Operations:

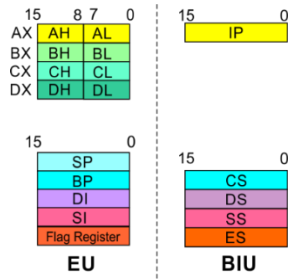
$0008_H \leftarrow 08_H$ (Sign extended)
EA = (BX) + 0008_H
BA = (DS) x 16_{10}
MA = BA + EA

(AX) ← (MA) or,

(AL) ← (MA)
(AH) ← (MA + 1)

Addressing Modes

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing



SI or DI register is used to hold an index value for memory data and a signed 8-bit or unsigned 16-bit displacement will be specified in the instruction.

Displacement is added to the index value in SI or DI register to obtain the EA.

In case of 8-bit displacement, it is sign extended to 16-bit before adding to the base value.

Example:

MOV CX, [SI + 0A2H]

Operations:

$FFA2_H \leftarrow A2_H$ (Sign extended)

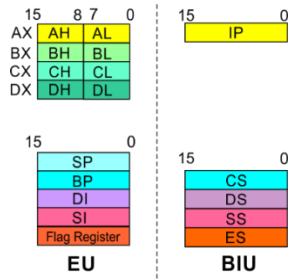
$EA = (SI) + FFA2_H$
$BA = (DS) \times 16_{10}$
$MA = BA + EA$

$(CX) \leftarrow (MA)$ or,

$(CL) \leftarrow (MA)$
$(CH) \leftarrow (MA + 1)$

Addressing Modes

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing



In Based Index Addressing, the effective address is computed from the sum of a base register (BX or BP), an index register (SI or DI) and a displacement.

Example:

MOV DX, [BX + SI + 0AH]

Operations:

$000A_H \leftarrow 0A_H$ (Sign extended)

$EA = (BX) + (SI) + 000A_H$
$BA = (DS) \times 16_{10}$
$MA = BA + EA$

$(DX) \leftarrow (MA)$ or,

$(DL) \leftarrow (MA)$
$(DH) \leftarrow (MA + 1)$

Addressing Modes

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing

Note : Effective address of the Extra segment register

Employed in string operations to operate on string data.

The effective address (EA) of source data is stored in SI register and the EA of destination is stored in DI register.

Segment register for calculating base address of source data is DS and that of the destination data is ES

Example: MOVS BYTE

Operations:

Calculation of source memory location:

EA = (SI) BA = (DS) x 16_{10} MA = BA + EA

Calculation of destination memory location:

EA_E = (DI) BA_E = (ES) x 16_{10} $MA_E = BA_E + EA_E$

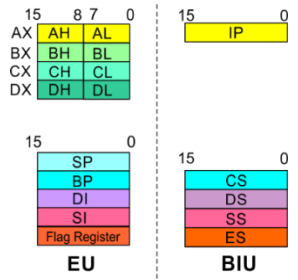
(MAE) ← (MA)

If DF = 1, then (SI) ← (SI) – 1 and (DI) = (DI) - 1
If DF = 0, then (SI) ← (SI) +1 and (DI) = (DI) + 1

Addressing Modes

Group III : Addressing modes for I/O ports

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing



These addressing modes are used to access data from standard I/O mapped devices or ports.

In direct port addressing mode, an 8-bit port address is directly specified in the instruction.

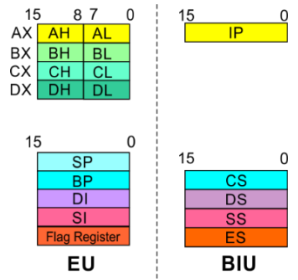
Example: **IN AL, [09H]**

Operations: $PORT_{addr} = 09_H$
$(AL) \leftarrow (PORT)$

Content of port with address 09_H is moved to AL register

Addressing Modes

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing



In this addressing mode, the effective address of a program instruction is specified relative to Instruction Pointer (IP) by an 8-bit signed displacement.

Example: JZ 0AH

Operations:

$000A_H \leftarrow 0A_H$ **(sign extend)**

If ZF = 1, then

$EA = (IP) + 000A_H$
$BA = (CS) \times 16_{10}$
$MA = BA + EA$

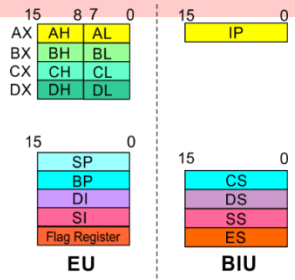
If ZF = 1, then the program control jumps to new address calculated above.

If ZF = 0, then next instruction of the program is executed.

Addressing Modes

Group IV : Implied Addressing mode

1. Register Addressing
2. Immediate Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based Index Addressing
8. String Addressing
9. Direct I/O port Addressing
10. Indirect I/O port Addressing
11. Relative Addressing
12. Implied Addressing



Instructions using this mode have no operands. The instruction itself will specify the data to be operated by the instruction.

Example: **CLC**

This clears the carry flag to zero.

The Intel Microprocessors

8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro Processor, Pentium II, Pentium 4, and Core2 with 64-bit Extensions

Architecture, Programming, and Interfacing

EIGHTH EDITION

Barry B. Brey

PEARSON

Chapter 9: 8086/8088 Hardware Specifications

Introduction

- In this chapter, the pin functions of the 8086 microprocessor are detailed and information is provided on the following hardware topics: clock generation, bus buffering, bus latching, timing, wait states, and minimum mode operation versus maximum mode operation.
- These simple microprocessors are explained as an introduction to the Intel microprocessor family.

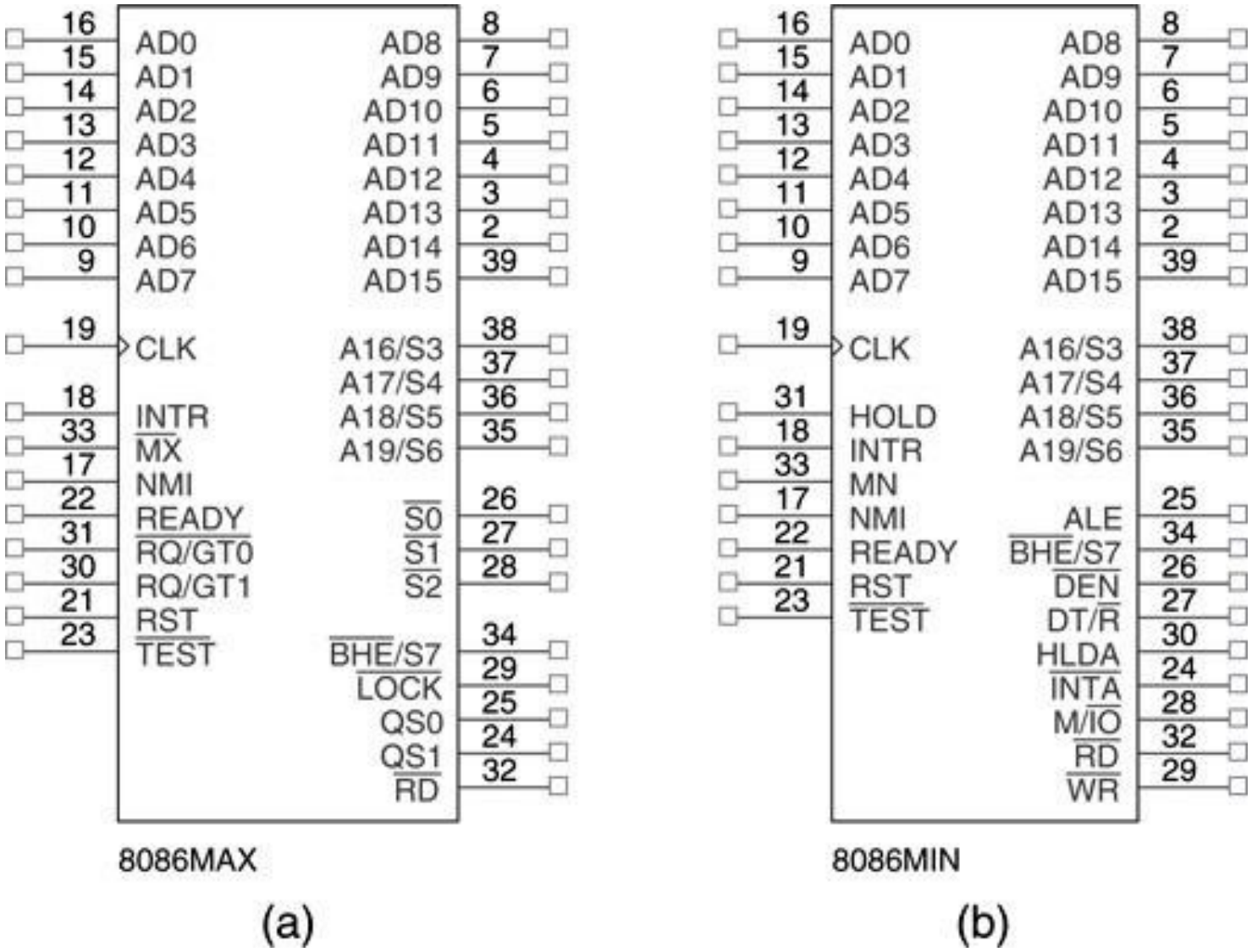
PIN-OUTS AND THE PIN FUNCTIONS

- In this section, we explain the function and the multiple functions of each of the microprocessor's pins.
- In addition, we discuss the DC characteristics to provide a basis for understanding the later sections on buffering and latching.

The Pin-Out

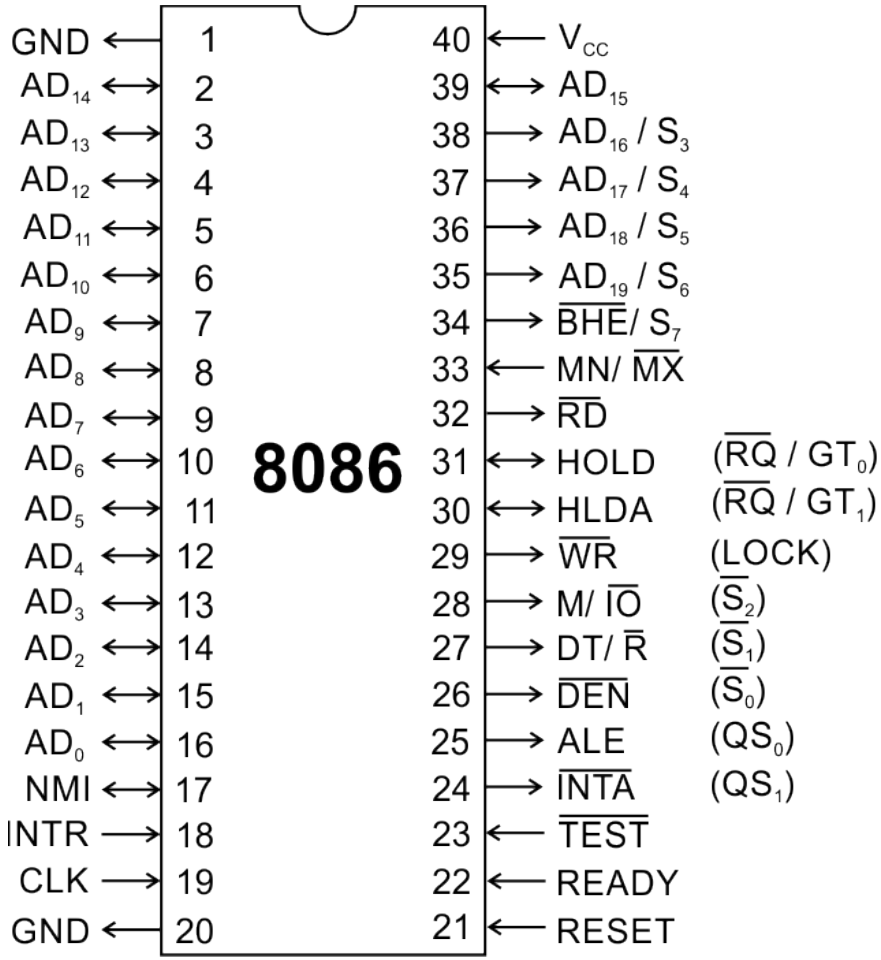
- Figure 1 illustrates pin-outs of 8086.
  - packaged in 40-pin **dual in-line** package (DIP)
- 8086 is a 16-bit microprocessor with a 16-bit data bus;
  - 8086 has pin connections AD_0–AD_{15}

Figure 1 (a) The pin-out of the 8086 in maximum mode; (b) the pin-out of the 8086 in minimum mode.



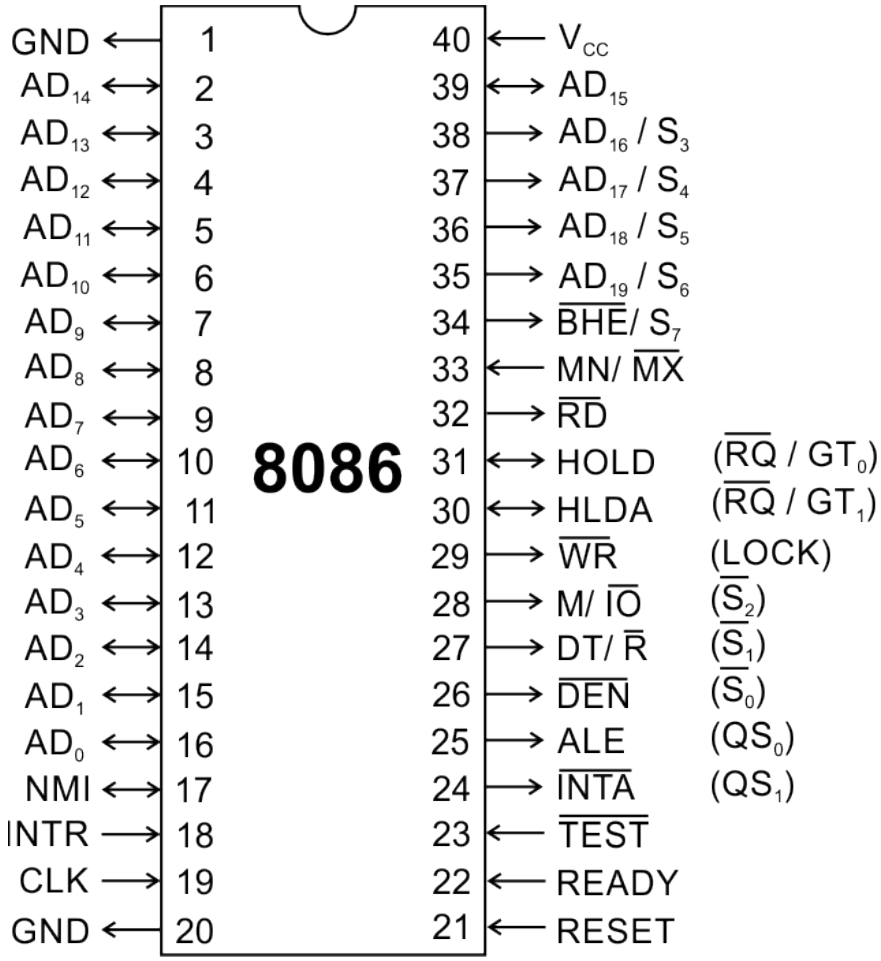
Power Supply Requirements

- Both microprocessors require +5.0 V with a supply voltage tolerance of ±10 percent.
  - 8086 uses a maximum supply current of 360 mA
- Both microprocessors operate in ambient temperatures of between 0ºC and 80ºC.
- 80C86 is a CMOS version that requires only 10 mA of power supply current.
  - and function in temperature extremes of –40°C through +200° C



Pin Connections AD_{15} - AD_8

- 8086 **address/data bus** lines compose upper multiplexed address/data bus on the 8086.
- These lines contain address bits A_{15}–A_8 whenever ALE is a logic 1, and data bus connections D_{15}–D_8 when ALE is a logic 0.
- These pins enter a high-impedance state when a hold acknowledge occurs.



Pin Connections A_{19}/S_6 - A_{16}/S_3

- **Address/status bus** bits are multiplexed to provide address signals A_{19}–A_{16} and status bits S_6–S_3.
  - high-impedance state during hold acknowledge
  - status bit S_6 is always logic 0,
  - bit S_5 indicates the condition of the IF flag bit
- S_4 and S_3 show which segment is accessed during the current bus cycle.
  - these status bits can address four separate 1M byte memory banks by decoding as A_{21} and A_{20}

Pin Connections $\overline{RD}$

- When **read signal** is logic 0, the data bus is receptive to data from memory or I/O devices
  - pin floats high-impedance state during a hold acknowledge

Ready

- Inserts wait states into the timing.
  - if placed at a logic 0, the microprocessor enters into wait states and remains idle
  - if logic 1, no effect on the operation

Pin Connections INTR

- **Interrupt request** is used to request a hardware interrupt.
  - If INTR is held high when IF = 1, 8086 enters an interrupt acknowledge cycle after the current instruction has completed execution

NMI

- The **non-maskable interrupt** input is similar to INTR.
  - does not check IF flag bit for logic 1
  - if activated, uses interrupt vector 2

Pin Connections $\overline{\text{TEST}}$

- The $\overline{\textbf{Test}}$ pin is an input that is tested by the WAIT instruction.
- If $\overline{\textbf{TEST}}$ is a logic 0, the WAIT instruction functions as an NOP.
- If $\overline{\textbf{TEST}}$ is a logic 1, the WAIT instruction waits for **TEST** to become a logic 0.
- The $\overline{\textbf{TEST}}$ pin is most often connected to the 8087 numeric coprocessor.

Pin Connections RESET

- Causes the microprocessor to reset itself if held high a minimum of four clocking periods.
  - when 8086 is reset, it executes instructions at memory location FFFF0H
  - also disables future interrupts by clearing IF flag

CLK

- The **clock** pin provides the basic timing signal.
  - must have a duty cycle of 33 % (high for one third of clocking period, low for two thirds) to provide proper internal timing

Pin Connections VCC

- This **power supply** input provides a +5.0 V, ±10 % signal to the microprocessor.

GND

- The **ground** connection is the return for the power supply.
  - 8086 microprocessors have two pins labeled GND—both must be connected to ground for proper operation

Pin Connections MN/$\overline{MX}$

- **Minimum/maximum** mode pin selects either minimum or maximum mode operation.
  - if minimum mode selected, the MN/$\overline{MX}$ pin must be connected directly to +5.0 V

BHE S_7

- The **bus high enable** pin is used in 8086 to enable the most-significant data bus bits (D_{15}–D_8) during a read or a write operation.
- The state of S_7 is always a logic 1.

Minimum Mode Pins

- Minimum mode operation is obtained by connecting the MN/MX pin directly to +5.0 V.
  - do not connect to +5.0 V through a pull-up register; it will not function correctly

M/$\overline{IO}$

- The **M/$\overline{IO}$** (8086) pin selects memory or I/O.
  - indicates the address bus contains either a memory address or an I/O port address.
  - high-impedance state during hold acknowledge

Minimum Mode Pins

$\overline{\text{WR}}$

- **Write line** indicates 8086 is outputting data to a memory or I/O device.
  - during the time $\overline{\text{WR}}$ is a logic 0, the data bus contains valid data for memory or I/O
  - high-impedance during a hold acknowledge

$\overline{\text{INTA}}$

- The **interrupt acknowledge** signal is a response to the INTR input pin.
  - normally used to gate the interrupt vector number onto the data bus in response to an interrupt

Minimum Mode Pins

ALE

- **Address latch enable** shows the 8086 address/data bus contains an address.
  - can be a memory address or an I/O port number
  - ALE signal doesn't float during hold acknowledge

DT/$\overline{R}$

- The **data transmit/receive** signal shows that the microprocessor data bus is transmitting (**DT/$\overline{R}$ = 1**) or receiving (**DT/$\overline{R}$ = 0**) data.
  - used to enable external data bus buffers

Minimum Mode Pins

DEN

- **Data bus enable** activates external data bus buffers.

HOLD

- **Hold input** requests a direct memory access (DMA).
  - if HOLD signal is a logic 1, the microprocessor stops executing software and places address, data, and control bus at high-impedance
  - if a logic 0, software executes normally

Minimum Mode Pins

HLDA

- **Hold acknowledge** indicates the 8086 has entered the hold state.

$\overline{\mathbf{SS0}}$

- The $\overline{SS0}$ status line is equivalent to the S_0 pin in maximum mode operation.
- Signal is combined with **IO/$\overline{\mathbf{M}}$** and **DT/$\overline{\mathbf{R}}$** to decode the function of the current bus cycle.

Maximum Mode Pins

- In order to achieve maximum mode for use with external coprocessors, connect the MN/$\overline{\text{MX}}$ pin to ground.

$\overline{\textbf{S2}}$, $\overline{\textbf{S1}}$, and $\overline{\textbf{S0}}$

- **Status bits** indicate function of the current bus cycle.
  - normally decoded by the 8288 bus controller

Maximum Mode Pins $\overline{\text{RQ}}$/$\overline{\text{GT1}}$

- The **request/grant** pins request direct memory accesses (DMA) during maximum mode operation.
  - bidirectional; used to request and grant a DMA operation

$\overline{\text{LOCK}}$

- The **lock** output is used to lock peripherals off the system. This pin is activated by using the LOCK: prefix on any instruction.

Maximum Mode Pins QS_1 and QS_0

- The **queue status** bits show the status of the internal instruction queue.
  - provided for access by the 8087 coprocessor

Queue status		Queue operation
QS_1	QS_0	
0	0	No operation
0	1	First byte of an opcode from queue
1	0	Empty the queue
1	1	Subsequent byte from queue

BUS BUFFERING AND LATCHING

- Before 8086 can be used with memory or I/O interfaces, their multiplexed buses must be demultiplexed.
- This section provides detail required to demultiplex the buses and illustrates how the buses are buffered for very large systems.
  - because the maximum fan-out is 10, the system must be buffered if it contains more than 10 other components

Demultiplexing the Buses

- The address/data bus of the 8086 is multiplexed (shared) to reduce the number of pins required for the integrated circuit
  - the hardware designer must extract or demultiplex information from these pins
- Memory & I/O require the address remain valid and stable throughout a read/write cycle.
- If buses are multiplexed, the address changes at the memory and I/O, causing them to read or write data in the wrong locations

- All computer systems have three buses:
  - an address bus that provides memory and I/O with the memory address or the I/O port number
  - a data bus that transfers data between the microprocessor and the memory and I/O
  - a control bus that provides control signals to the memory and I/O
- These buses must be present in order to interface to memory and I/O.

Demultiplexing the 8086

- Figure 2 illustrates components required to demultiplex 8086 buses.
  - three 74LS373 or 74LS573 transparent latches are used to demultiplex the address/data bus connections AD_{15}–AD_0
  - and address/status connections A_{19}/S_6–A_{16}/S_3
- The latches, which are like wires whenever the Address Latch Enable pin (ALE) becomes a logic 1, pass the inputs to the outputs.

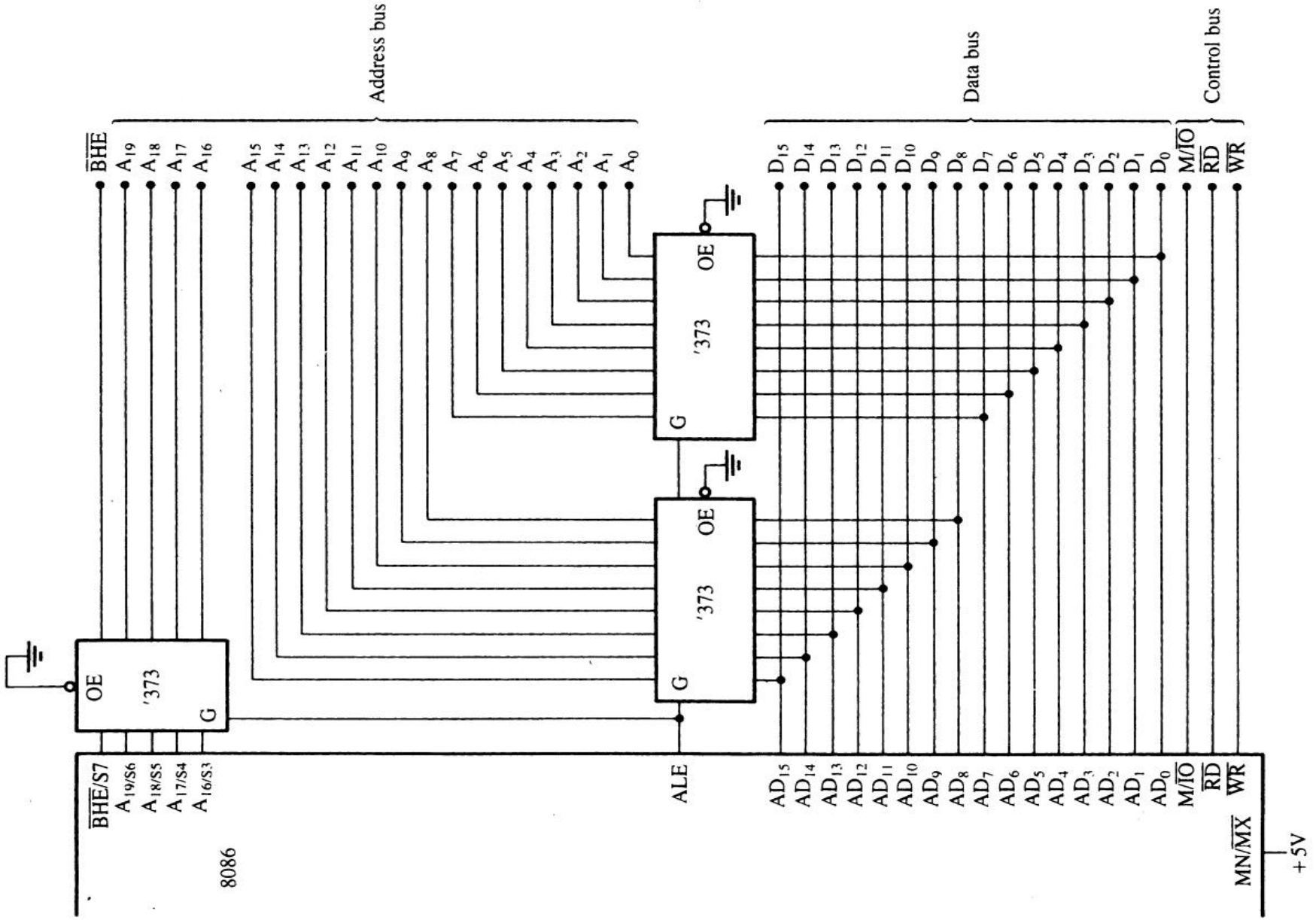
- After a short time, ALE returns to logic 0 causing the latches to remember inputs at the time of the change to a logic 0.
- This yields a separate address bus with connections A_{19}–A_0.
  - these allow 8086 to address 1Mb of memory
- The separate data bus allows it to be connected to any 8-bit peripheral device or memory component.

Demultiplexing the 8086

- Fig 2 illustrates a demultiplexed 8086 with all three buses:

- address (A_{19}–A_0 and $\overline{BHE}$)
- data (D_{15}–D_0),
- control (M/$\overline{IO}$,$\overline{RD}$, and $\overline{WR}$)
- Here, the memory and I/O system see the 8086 as a device with:
  - a 20-bit address bus;16-bit data bus
  - and a three-line control bus

Figure 2 The 8086 microprocessor shown with a demultiplexed address bus. This is the model used to build many 8086-based systems.



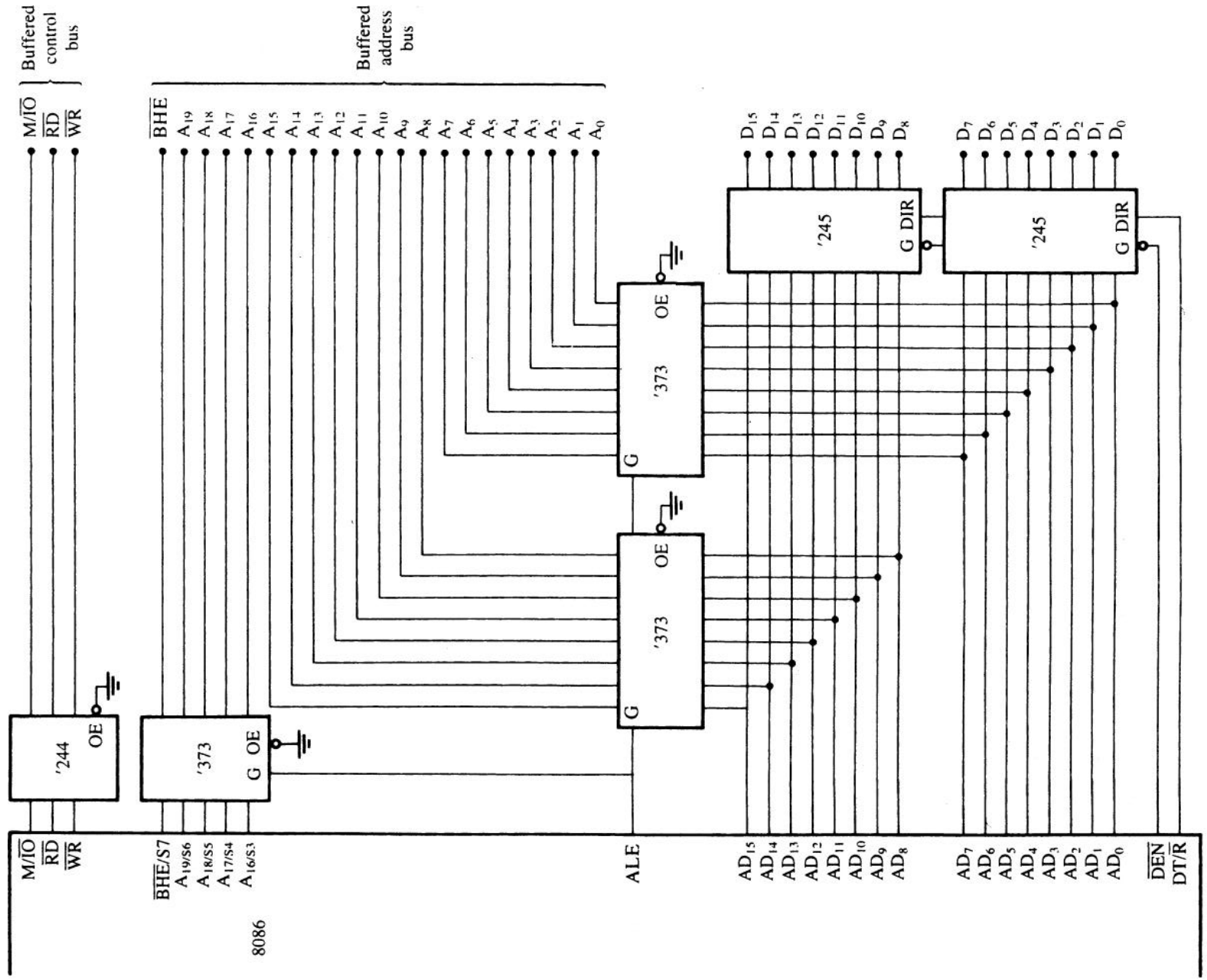
The Buffered System

- If more than 10 unit loads are attached to any bus pin, the entire system must be buffered.
- Buffer output currents have been increased so that more TTL unit loads may be driven.
- A fully buffered signal will introduce a timing delay to the system.
- No difficulty unless memory or I/O devices are used which function at near maximum bus speed.

The Fully Buffered 8086

- Figure 3 illustrates a fully buffered 8086.
  - a fully buffered 8086 system requires one 74LS244, two 74LS245s, and three 74LS373s
- Direction of the 74LS245 is controlled by the DT/$\overline{R}$ signal.
  - enabled and disabled by the $\overline{DEN}$ signal
- It also has a $\overline{BHE}$ signal that is buffered for memory-bank selection.

Figure 3 A fully buffered 8086 microprocessor.



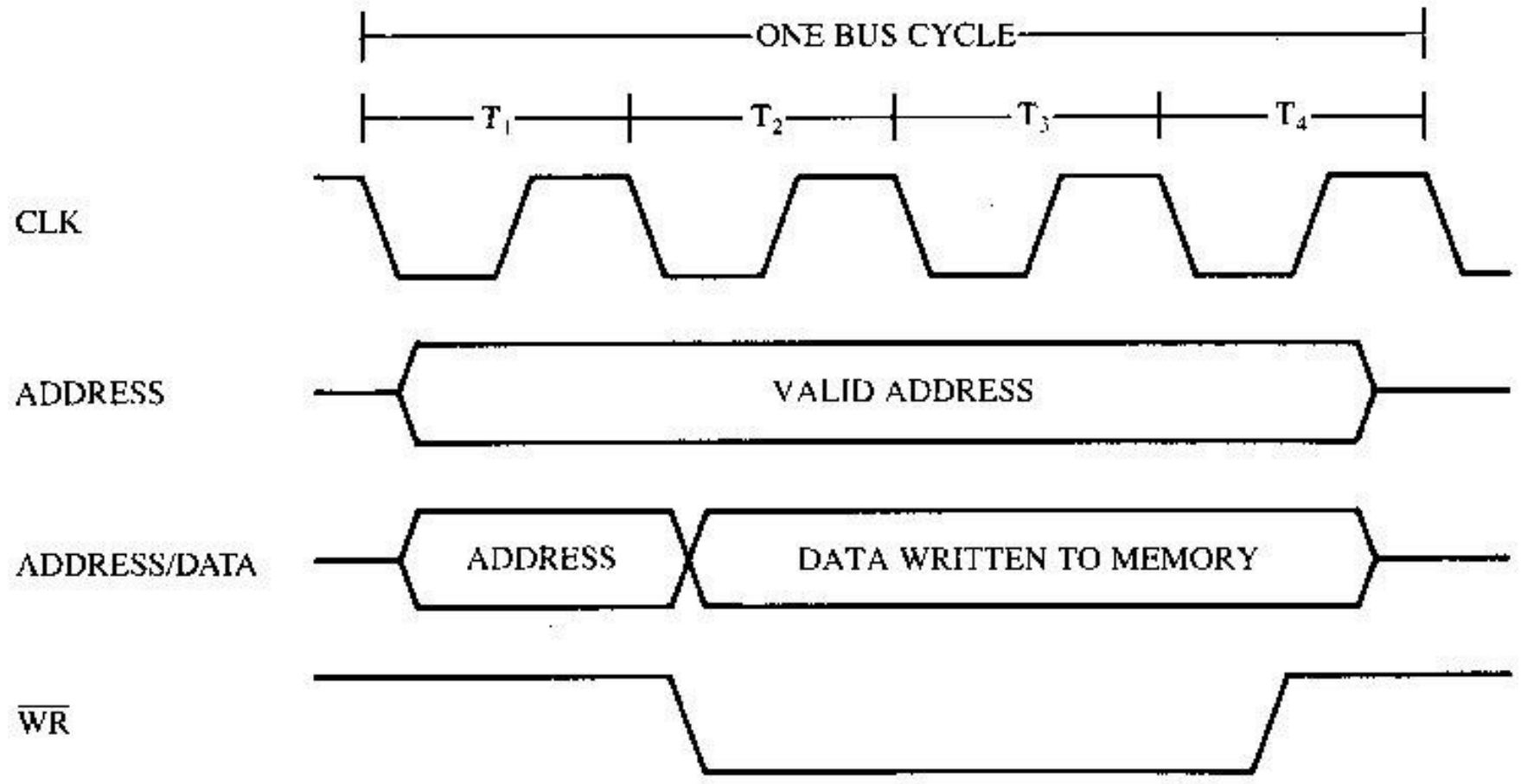
BUS TIMING

- It is essential to understand system bus timing before choosing memory or I/O devices for interfacing to 8086 microprocessors.
- This section provides insight into operation of the bus signals and the basic read/write timing of the 8086.

Basic Bus Operation

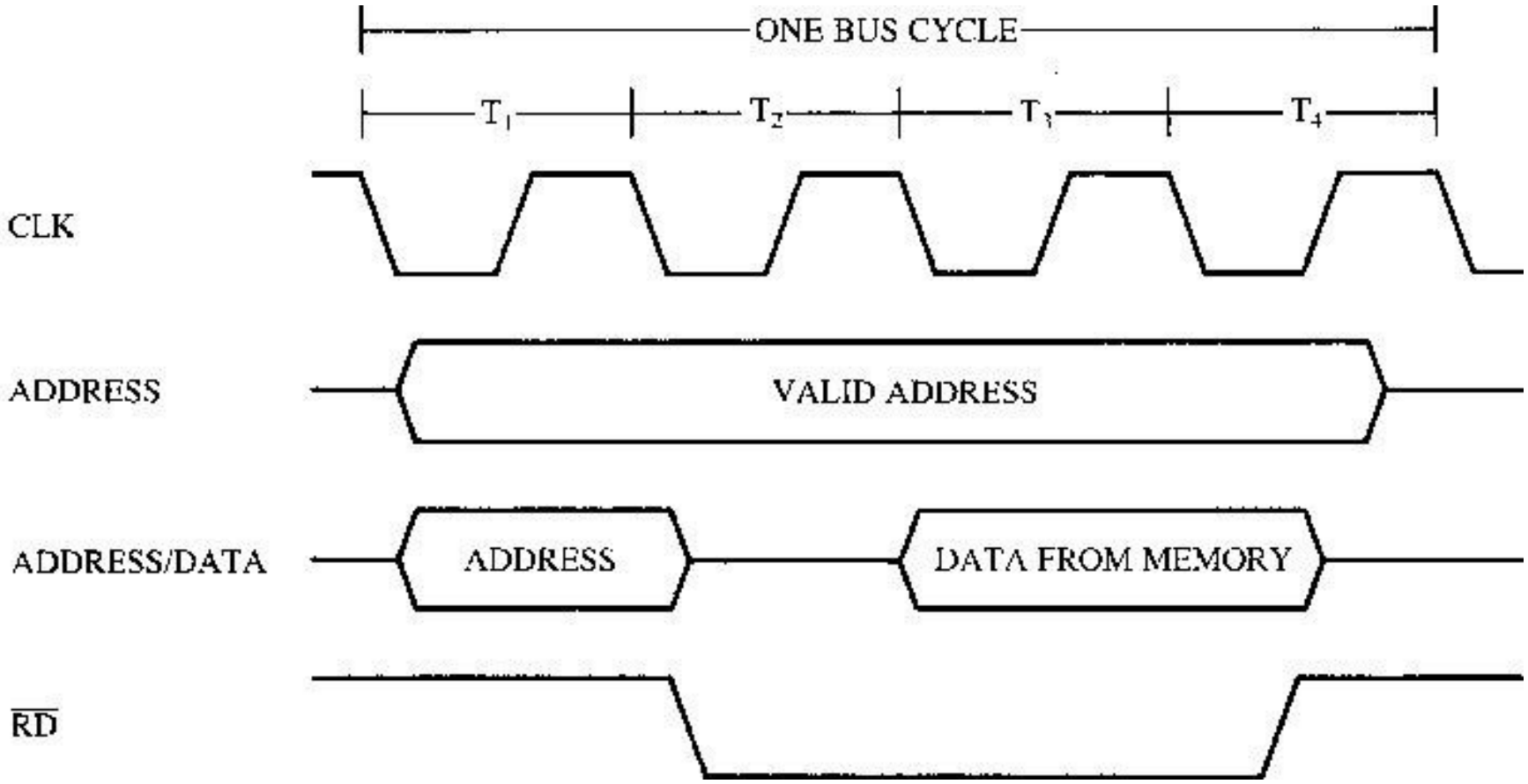
- The three buses of 8086 function the same way as any other microprocessor.
- If data are written to memory the processor:
  - outputs the memory address on the address bus
  - outputs the data to be written on the data bus
  - issues a write ($\overline{WR}$) to memory
  - and $\overline{IO}$/M = 0 for peripheral and $\overline{IO}$/M = 1 for writing in a memory
- See simplified timing for write in Fig 4.

Figure 4 Simplified 8086 write bus cycle.



- If data are read from the memory the microprocessor:
  - outputs the memory address on the address bus
  - issues a read memory signal ($\overline{RD}$)
  - and accepts the data via the data bus
- See simplified timing for read in Fig 5.

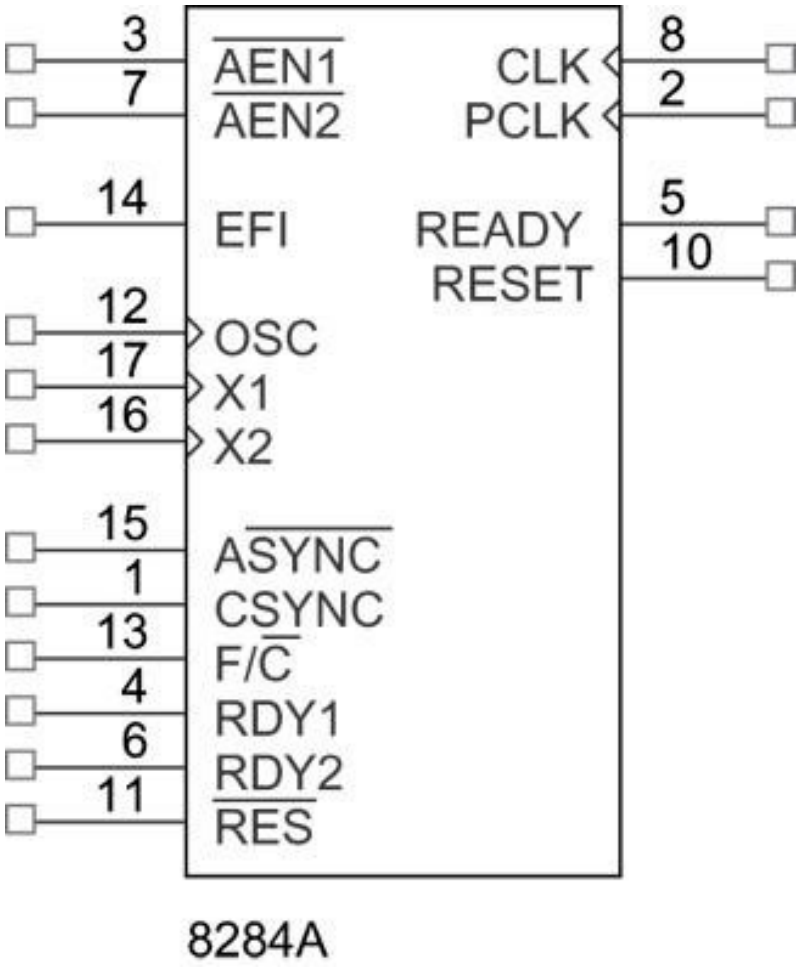
Figure 5 Simplified 8086 read bus cycle.



CLOCK GENERATOR (8284A)

- This section describes the 8484A clock generator and the RESET signal.
  - also introduces the READY signal for 8086
- With no clock generator, many circuits would be required to generate the clock (CLK).
- 8284A provides the following basic functions:
  - clock generation; RESET & READY synch;
  - TTL-level peripheral clock signal
- Figure 6 shows pin-outs of the 8284A

Figure 6 The pin-out of the 8284A clock generator.



8284A *Pin Functions*

- 8284A is an 18-pin integrated circuit designed specifically for use 8086.

$\overline{\text{AEN1}}$ and $\overline{\text{AEN2}}$

- The **address enable** pins are provided to qualify bus ready signals, RDY1 and RDY2.
  - used to cause wait states
- Wait states are generated by the READY pin of 8086 controlled by these two inputs.

Pin Functions **RDY_1** and **RDY_2**

- The **bus ready** inputs are provided, in conjunction with the $\overline{AEN1}$ & $\overline{AEN2}$ pins, to cause wait states in 8086.

ASYNC

- The **ready synchronization** selection input selects either one or two stages of synchronization for the RDY_1 and RDY_2 inputs.

Pin Functions READY

- **8284 Ready** is an output pin that connects to the 8086 READY input.
  - synchronized with the RDY_1 and RDY_2 inputs

X_1 and X_2

- The **crystal oscillator** pins connect to an external crystal used as the timing source for the clock generator and all its functions

Pin Functions F/$\overline{C}$

- The **frequency/crystal** select input chooses the clocking source for the 8284A.
  - if held high, an external clock is provided to the EFI input pin
  - if held low, the internal crystal oscillator provides the timing signal
- The external frequency input is used when the **F/C** pin is pulled high.
- EFI supplies timing when the **F/$\overline{C}$** pin is high.

Pin Functions CLK

- The **clock output** pin provides the CLK input signal to 8086 and other components.
  - output signal is one third of the crystal or EFI input frequency
  - 33% duty cycle required by the 8086

PCLK

- The **peripheral clock** signal is one sixth the crystal or EFI input frequency.
  - PCLK output provides a clock signal to the peripheral equipment in the system

Pin Functions OSC

- **Oscillator output** is a TTL-level signal at the same frequency as crystal or EFI input.
  - OSC output provides EFI input to other 8284A clock generators in multiple-processor systems

$\overline{\text{RES}}$

- **Reset input** is an active-low input to 8284A.
  - often connected to an RC network that provides power-on resetting

Pin Functions RESET

- **Reset output** is connected to the 8086 RESET input pin.

CSYNCH

- The **clock synchronization** pin is used when the EFI input provides synchronization in systems with multiple processors.
  - if internal crystal oscillator is used, this pin must be grounded

Pin Functions GND

- The **ground pin** connects to ground.

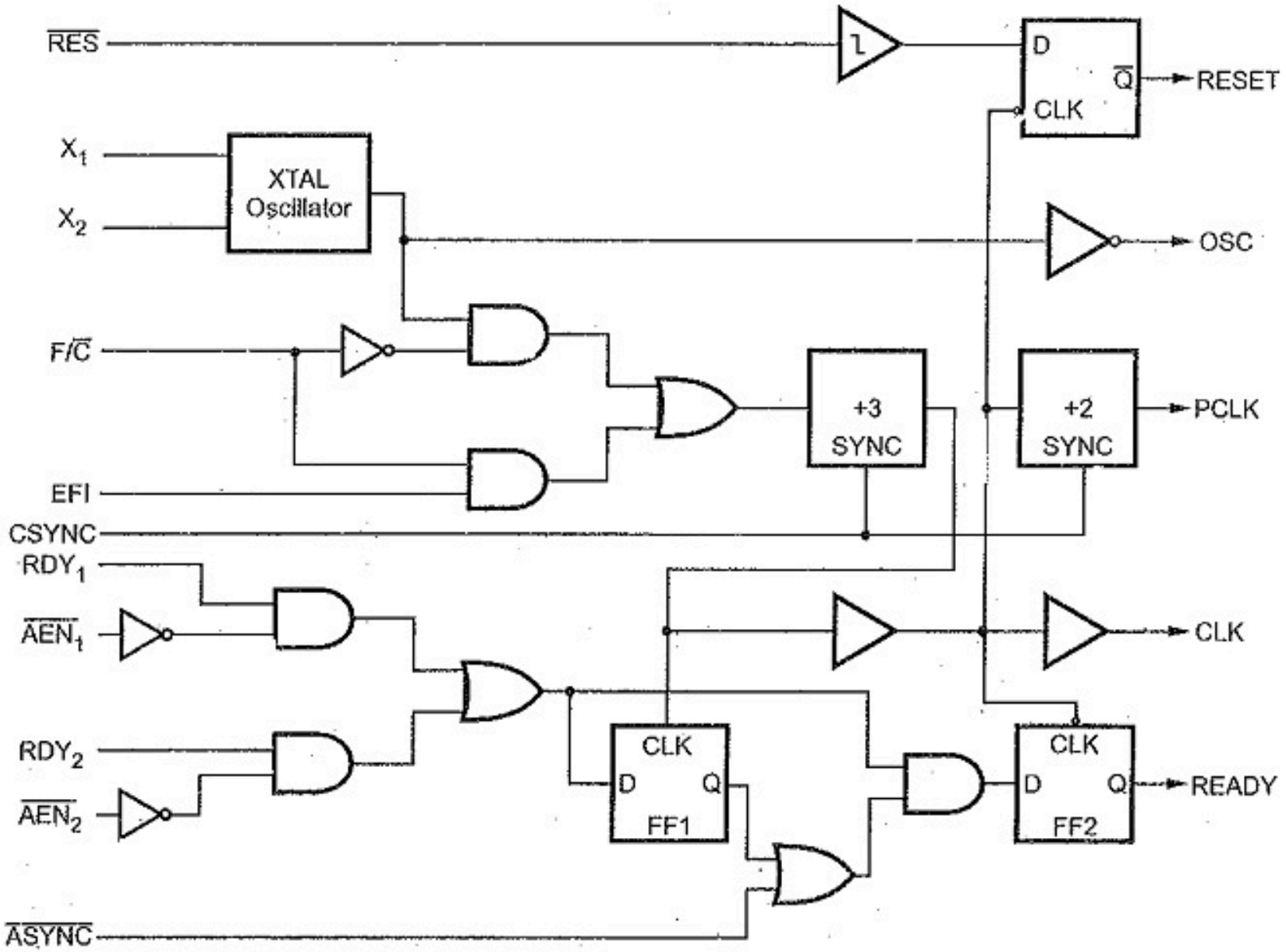
VCC

- This **power supply** pin connects to +5.0 V with a tolerance of ±10%.

Operation of the 8284A

- The 8284A is a relatively easy component to understand.
- Figure 7 illustrates the internal timing diagram of the 8284A clock generator.
- The top half of the logic diagram represents the clock and synchronization section of the 8284A clock generator.

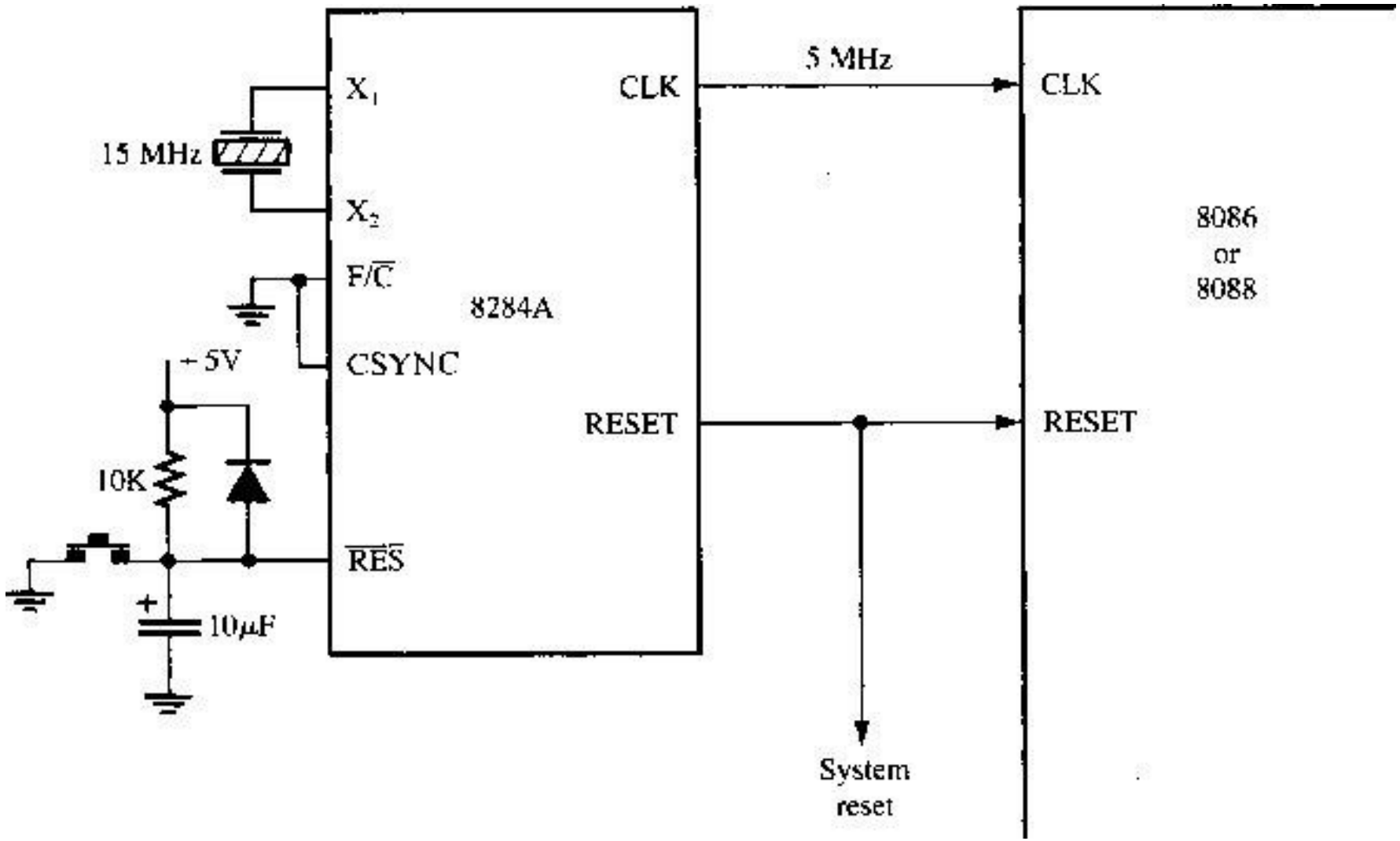
Figure 7 The internal block diagram of the 8284A clock generator.



Operation of the Clock Section

- Crystal oscillator has two inputs: X_1 and X_2.
  - if a crystal is attached to X_1 and X_2, the oscillator generates a square-wave signal at the same frequency as the crystal
- The square-wave is fed to an AND gate & an inverting buffer to provide an OSC output.
- The OSC signal is sometimes used as an EFI input to other 8284A circuits in a system.
- Figure 8 shows how an 8284A is connected to the 8086/8088.

Figure 8 The clock generator (8284A) and the 8086 microprocessor illustrating the connection for the clock and reset signals. A 15 MHz crystal provides the 5 MHz clock for the microprocessor.



Operation of the Reset Section

- The reset section of 8284A consists of a Schmitt trigger buffer and a D-type flip-flop.
  - the D-type flip-flop ensures timing requirements of 8086/8088 RESET input are met
- This circuit applies the RESET signal on the negative edge (1-to-0 transition) of each clock.
- 8086 microprocessors sample RESET at the positive edge (0-to-1 transition) clocks.
  - thus, this circuit meets 8086 timing requirements

RDY and the 8284A

- RDY is the synchronized ready input to the 8284A clock generator.
- Internal 8284A circuitry guarantees the accuracy of the READY synchronization.

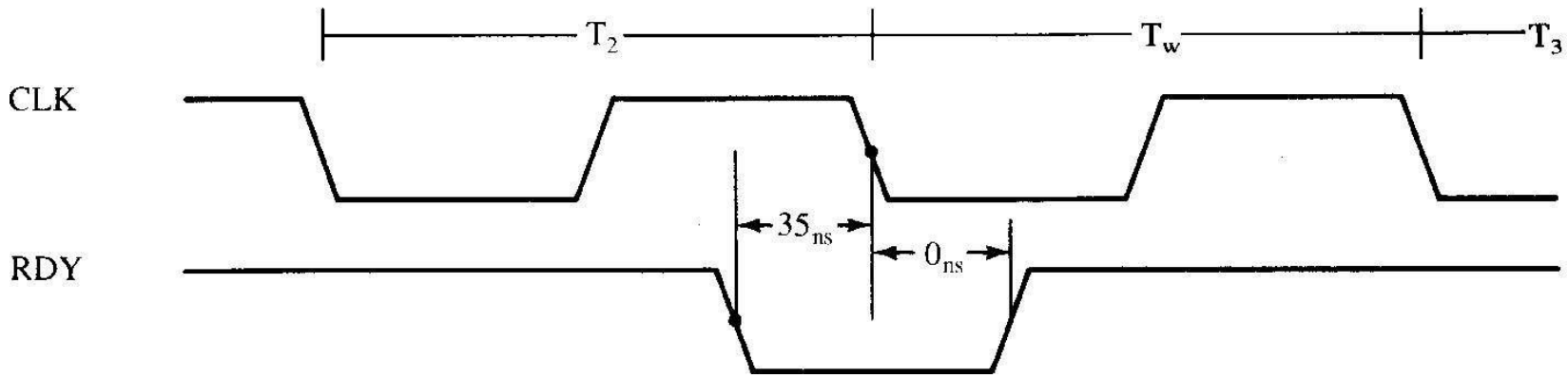
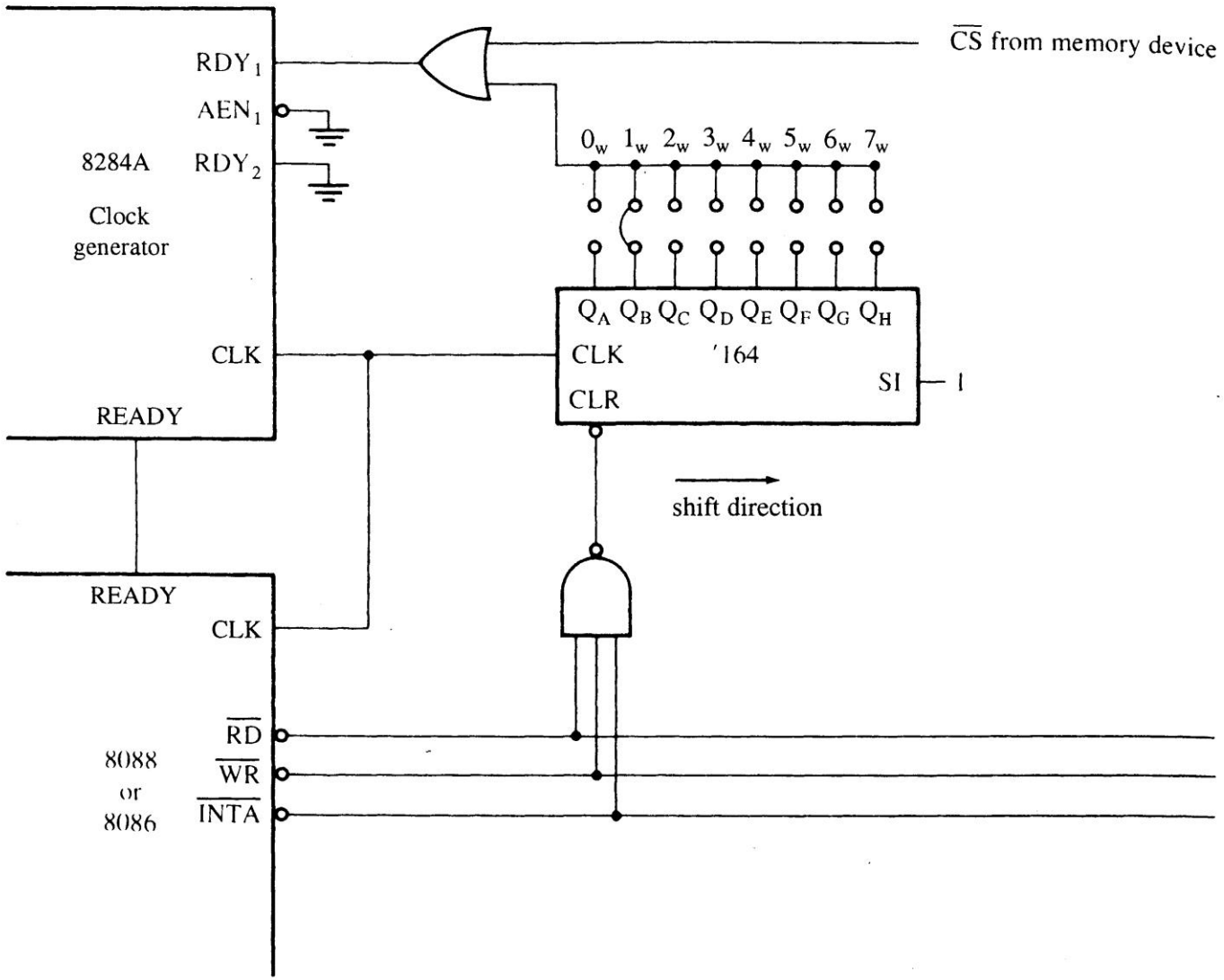


Figure 9 8284A RDY input timing.

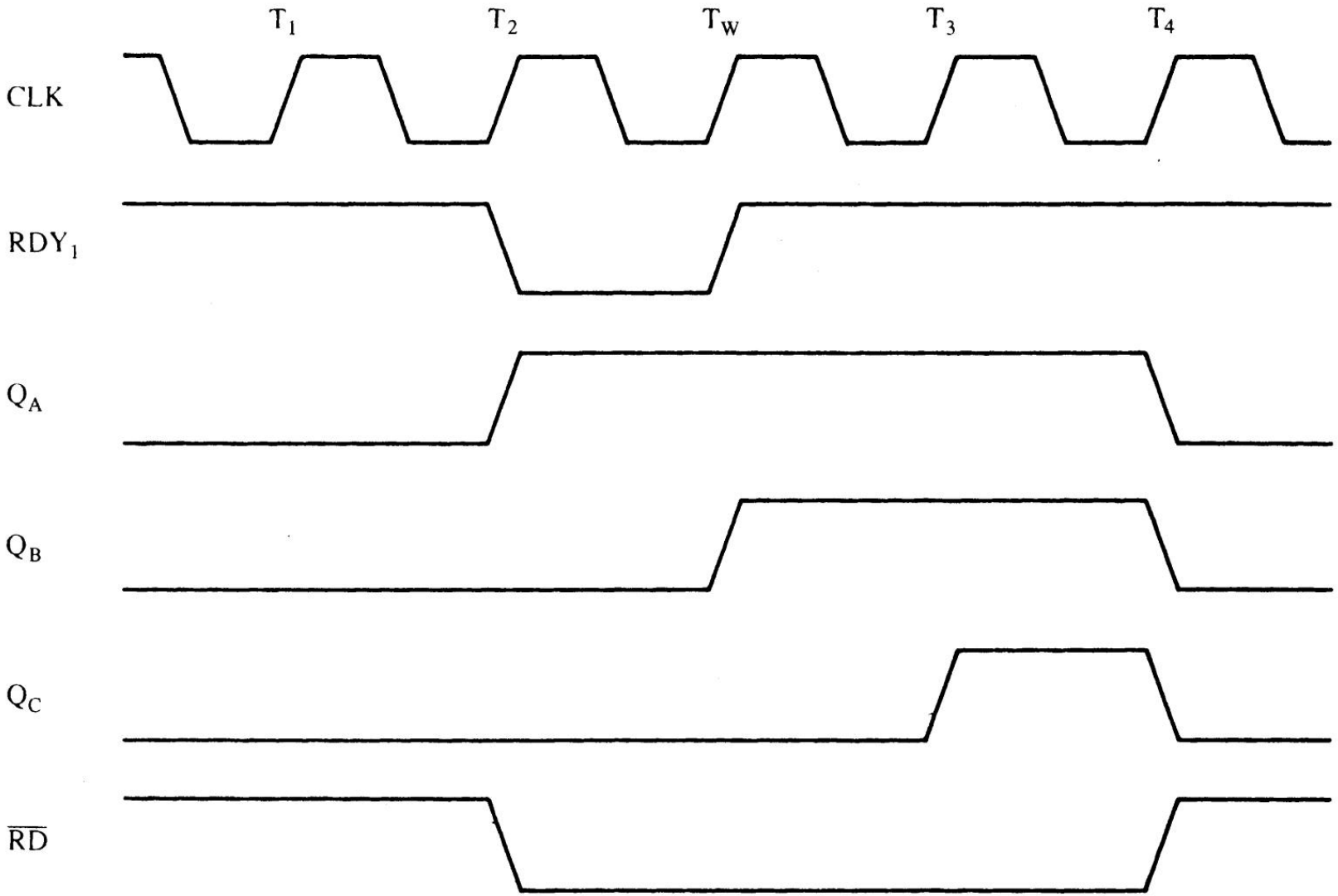
- Fig 7 depicts internal structure of 8284A.
  - the bottom half is the READY synch circuitry
- Fig 10 shows a circuit to introduce almost any number of wait states to 8086/8088.
- An 8-bit serial shift register (74LS164) shifts a logic 0 for one or more clock periods from one of its Q outputs through to the RDY1 input of the 8284A.
- With appropriate strapping, this circuit can provide various numbers of wait states.

Figure 10 A circuit that will cause between 0 and 7 wait states.



- Note in Fig 10 that this circuit is enabled only for devices that need insertion of waits.
  - if the selection signal is a logic 0, the device is selected and this circuit generates a wait state
- Figure 11 shows timing of this shift register wait state generator when wired to insert one wait state.
- The timing diagram also illustrates the internal contents of the shift register's flip-flops
  - to present a more detailed view of its operation
- In this example, one wait state is generated.

Figure 11 Wait state generation timing of the circuit of Figure 10



MINIMUM VS MAXIMUM MODE

- Minimum mode is obtained by connecting the mode selection MN/$\overline{MX}$ pin to +5.0 V,
  - maximum mode selected by grounding the pin
- The mode of operation provided by minimum mode is similar to that of the 8085A
  - the most recent Intel 8-bit microprocessor
- Maximum mode is designed to be used whenever a coprocessor exists in a system.
  - maximum mode was dropped with 80286

Minimum Mode Operation

- Least expensive way to operate 8086.
  - because all control signals for the memory & I/O are generated by the microprocessor
- Control signals are identical to Intel 8085A.
- The minimum mode allows 8085A 8-bit peripherals to be used with the 8086 without any special considerations.

Time in µP

- T-state is the smallest unit of time in a µp
- 1 clock cycle = 1 T-state
- In 8086, 1 machine cycle = 4 T-states
- 1 machine cycle (or bus cycle) is the time required to
  - T1 – send out an address – on address bus
  - T2 – send out a signal (read/ write) – on control bus
  - T3 – read/ write data on that location – on data bus
  - T4 – release all buses
- 1 instruction cycle = *n* machine cycles (depends on the instruction)

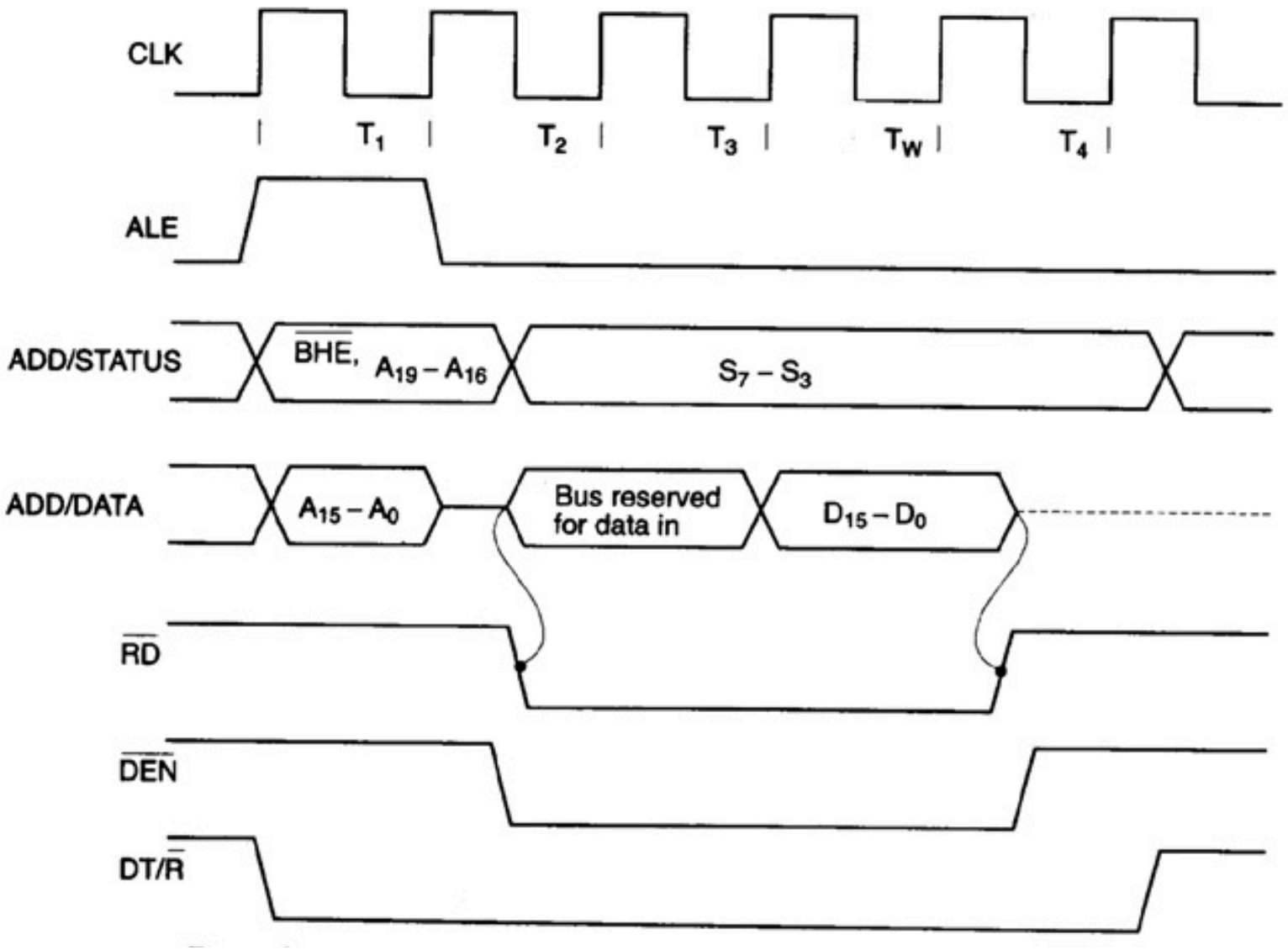
Timing in General

- 8086 use memory and I/O in periods called bus cycles.
- Each cycle equals four system-clocking periods (T states).
  - newer microprocessors divide the bus cycle into as few as two clocking periods
- If the clock is operated at 5 MHz, one 8086 bus cycle is complete in 800 ns.
  - basic operating frequency for these processors

- During the first clocking period in a bus cycle, called T1, many things happen:
  - the address of the memory or I/O location is sent out via the address bus and the address/data bus connections.
- During T1, control signals are also output.
  - indicating whether the address bus contains a memory address or an I/O device (port) number
- During T2, the processor issues the RD or $\overline{WR}$ signal, DEN, and in the case of a write, the data to be written appears on the data bus.

- These events cause the memory or I/O device to begin to perform a read or a write.
- READY is sampled at the end of T_2.
  - if low at this time, T_3 becomes a wait state (T_w)
  - this clocking period is provided to allow the memory time to access data
- If a read bus cycle, the data bus is sampled at the end of T_3.
- Illustrated in Figure 12.

Figure 12 Minimum mode 8086 bus timing for a read operation.



- In T_4, all bus signals are deactivated in preparation for the next bus cycle
- Data bus connections are sampled for data read from memory or I/O

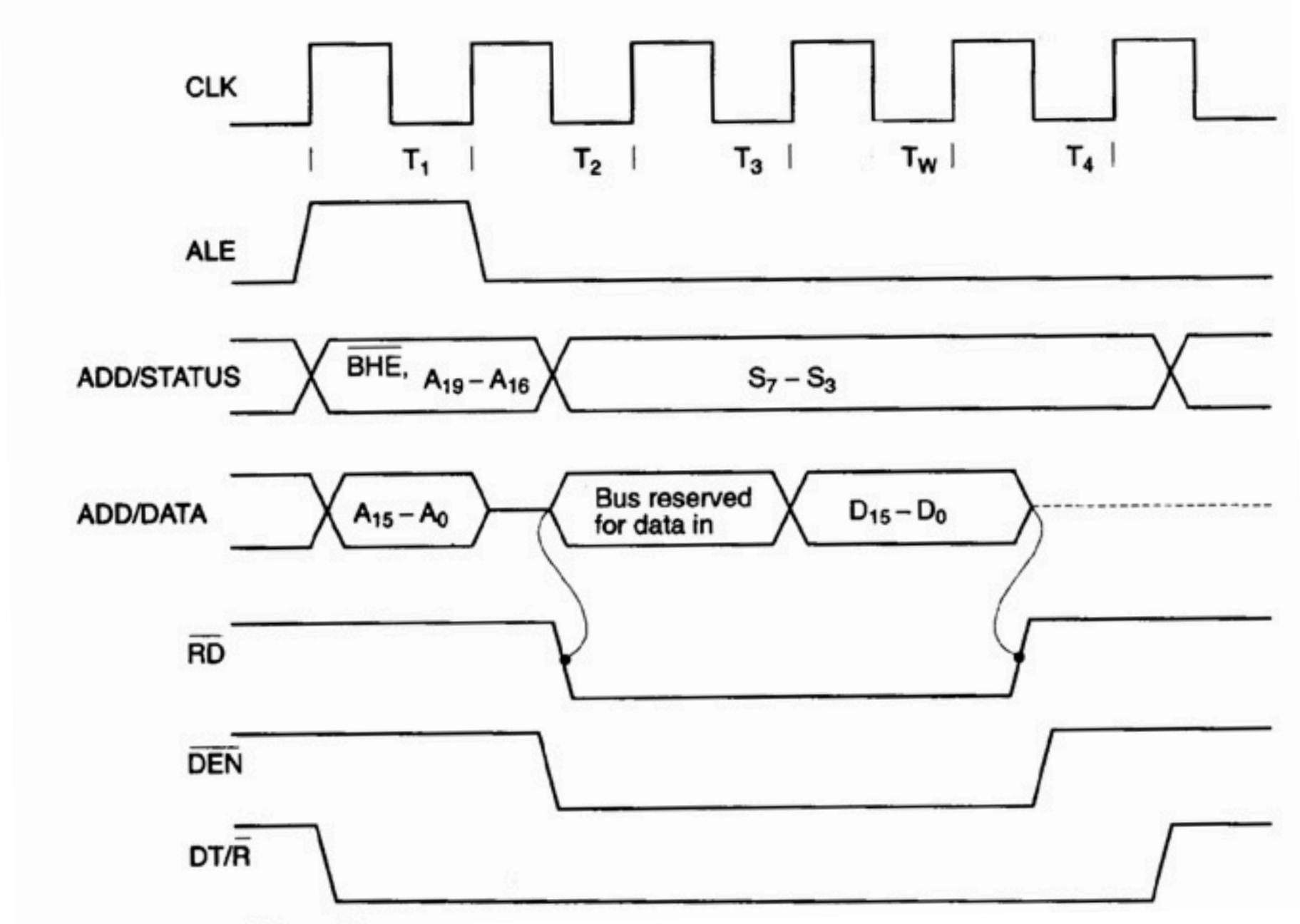
Read Timing

- Figure 12 depicts 8086 read timing.
- Important item in the read timing diagram is time allowed for memory & I/O to read data.
- Memory is chosen by its access time.
  - the fixed amount of time the microprocessor allows it to access data for the read operation
- It is extremely important that memory chosen complies with the limitations of the system.

Write Timing

- Figure 13 illustrates 8086 write-timing.
- The $\overline{RD}$ strobe is replaced by the $\overline{WR}$ strobe,
  - the data bus contains information *for* the memory rather than information *from* the memory,
  - DT/$\overline{R}$ remains a logic 1 instead of a logic 0 throughout the bus cycle
- When interfacing some devices, timing may be critical between when $\overline{WR}$ becomes logic 1 and the data are removed from the data bus.

Figure 13 Minimum mode 8086 write bus timing.



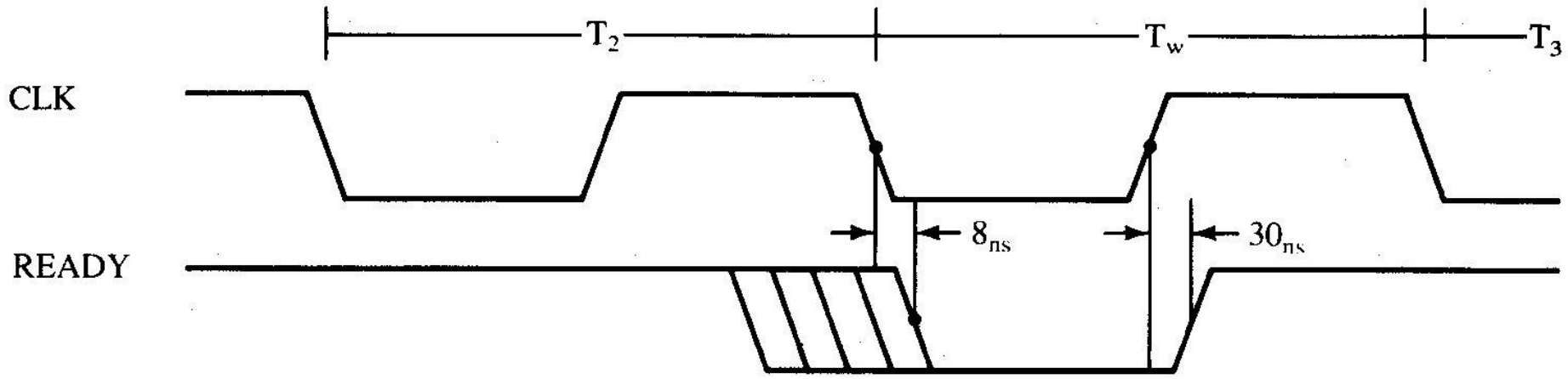
READY AND THE WAIT STATE

- The READY input causes wait states for slower memory and I/O components.
  - a wait state (T_w) is an extra clocking period between T_2 and T_3 to lengthen bus cycle
  - on one wait state, memory access time of 460 ns, is lengthened by one clocking period (200 ns) to 660 ns, based on a 5 MHz clock

The READY Input

- The READY input is sampled at the end of T_2 and again, if applicable, in the middle of T_w.
- The READY input to 8086 has stringent timing requirements.
- Fig 14 shows READY causing one wait state (T_w), with the required setup and hold times from the system clock.

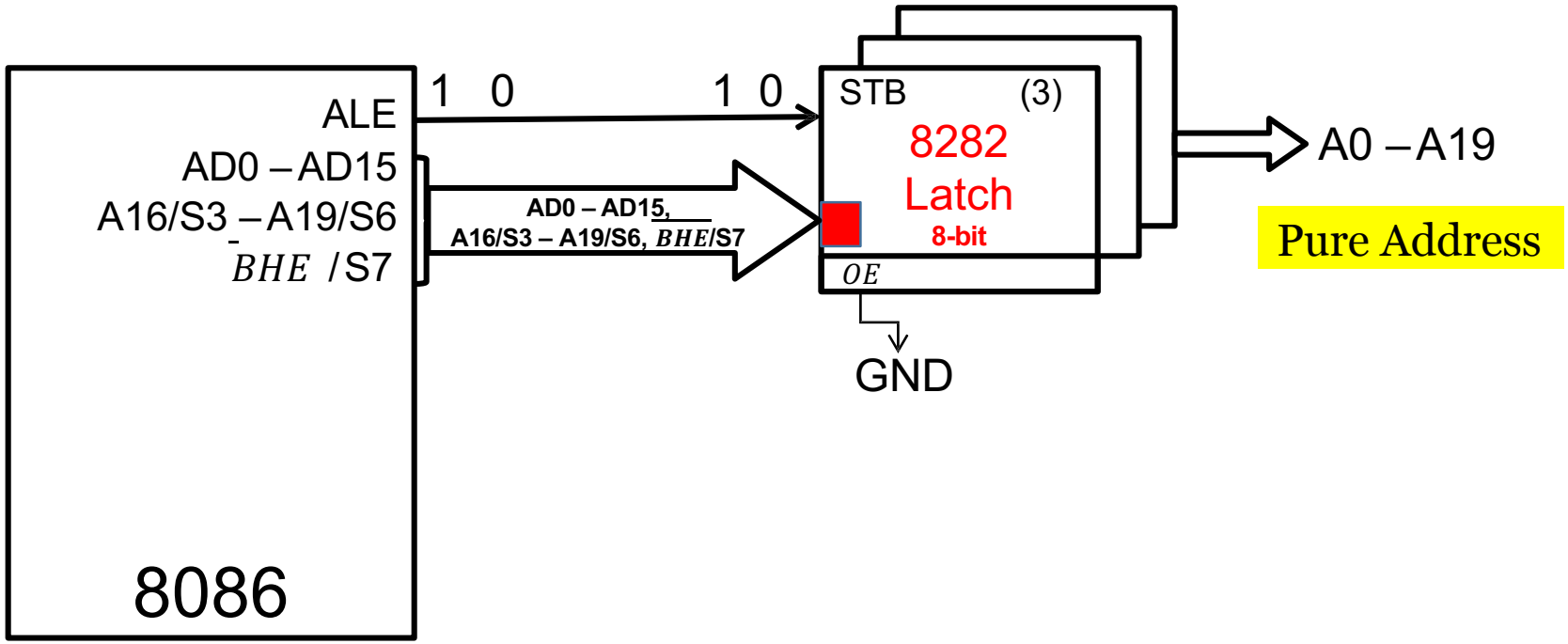
Figure 14 8086 READY input timing.

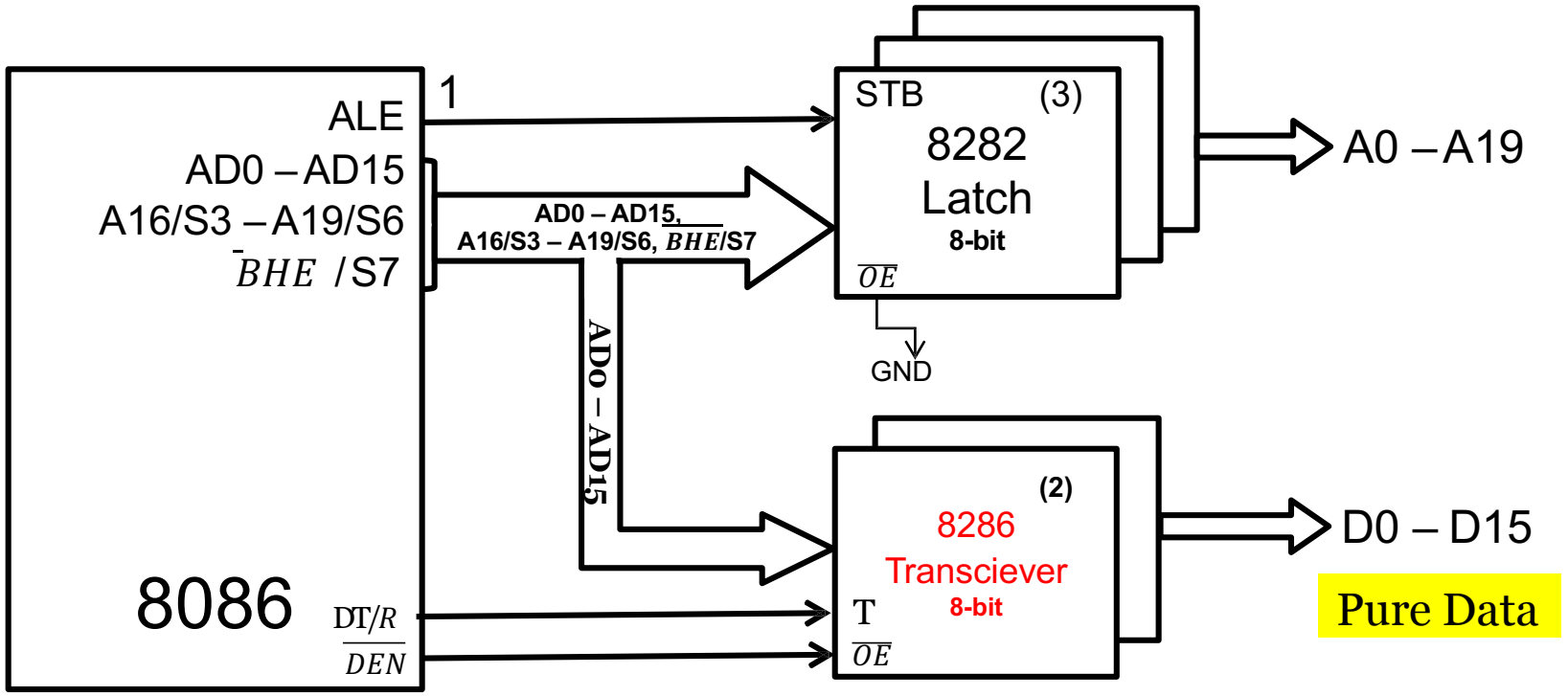


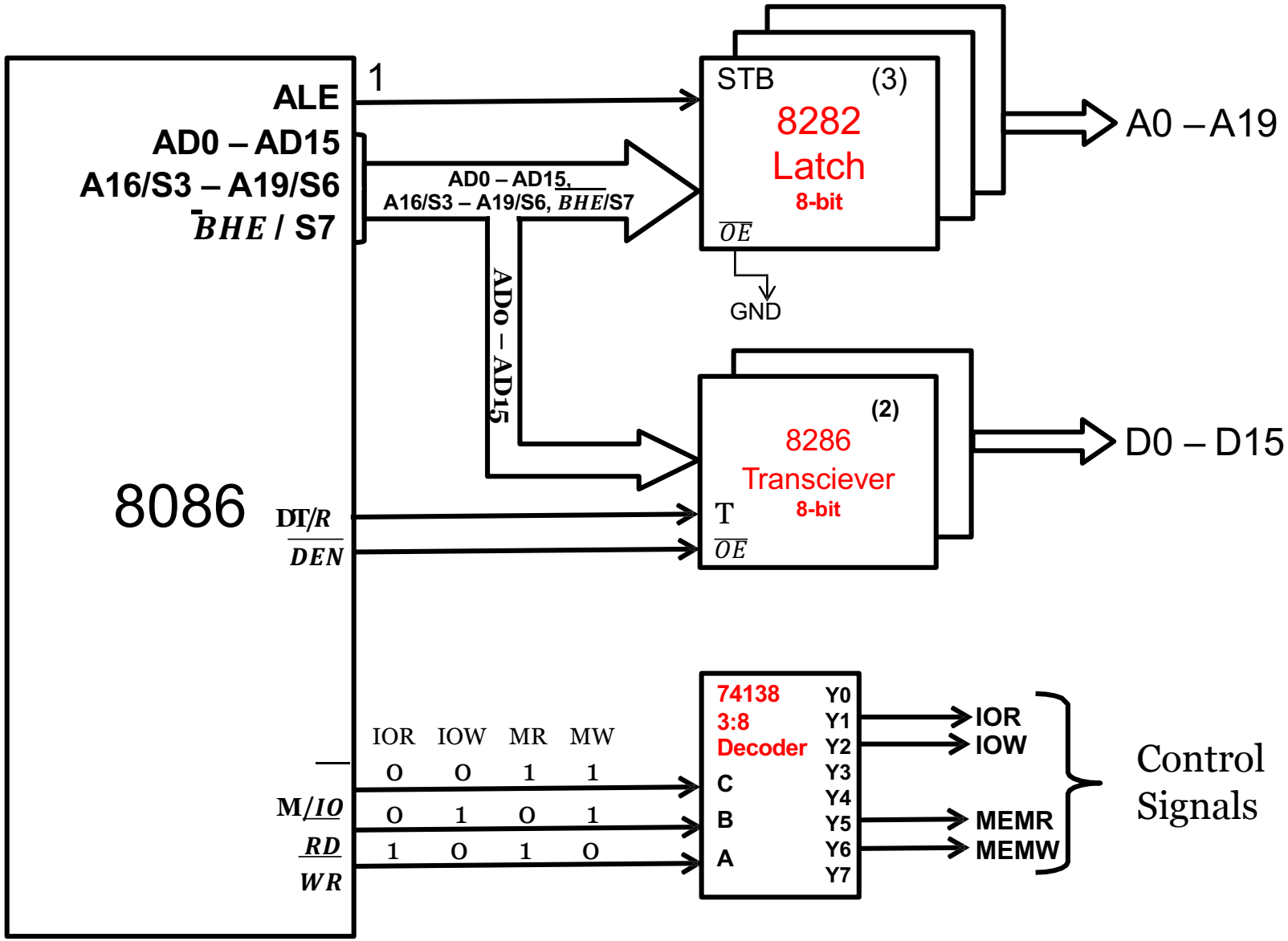
- If READY is logic 0 at the end of T_2, T_3 is delayed and T_w inserted between T_2 and T_3.
- READY is next sampled at the middle of T_w to determine if the next state is T_w or T_3.

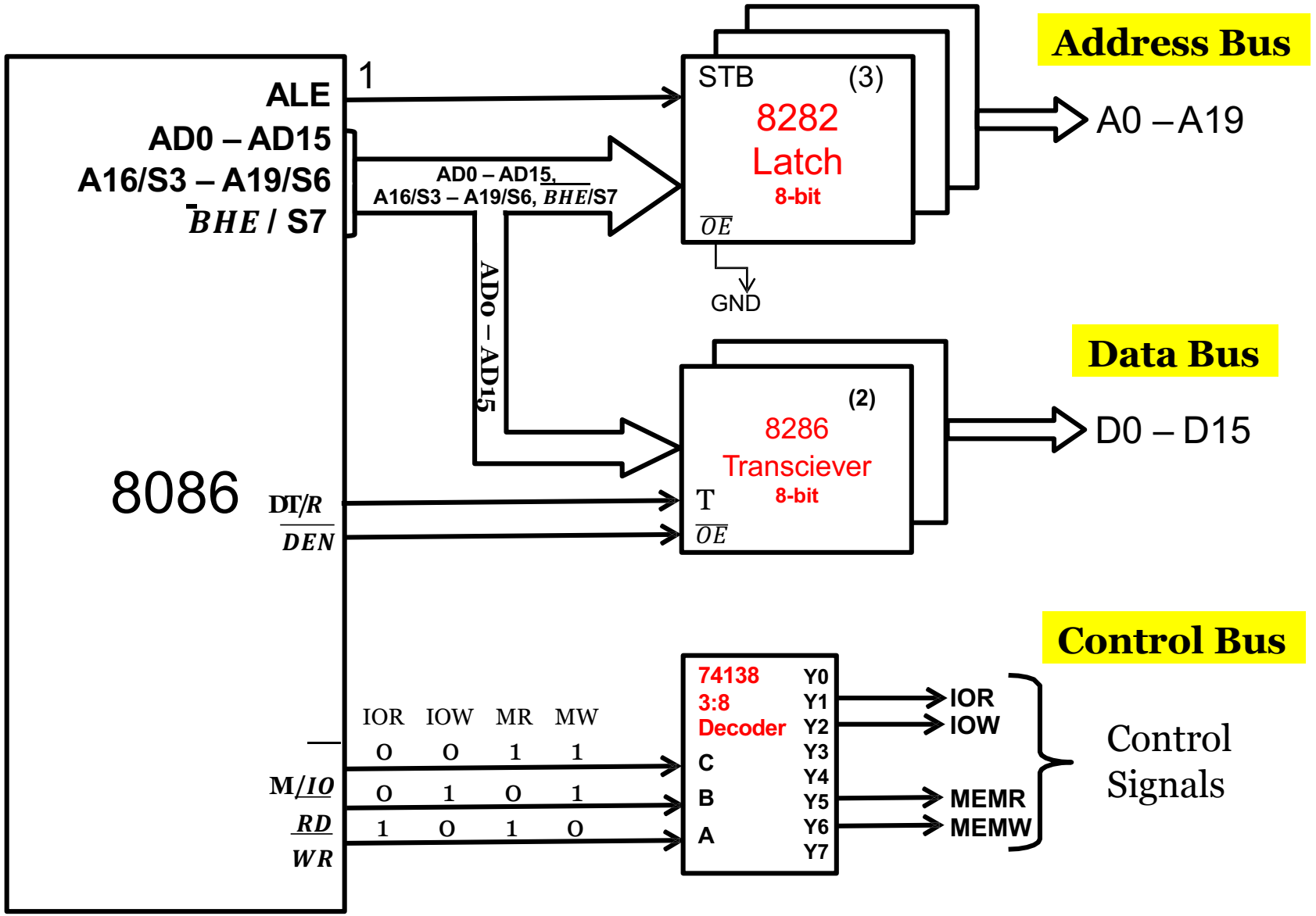
- The 8086 ALU sets the pins A0 – A19 to match the 20-bit address of the location it wants to access
- This value is not stored on these pins for a long time
- This value gets rewritten as D0 – D15 and status lines within the next fraction of a second
- Where is A0 – A19 after that?
- In the address bus!
- How to get pure address from AD0 – AD15? (Demultiplex)

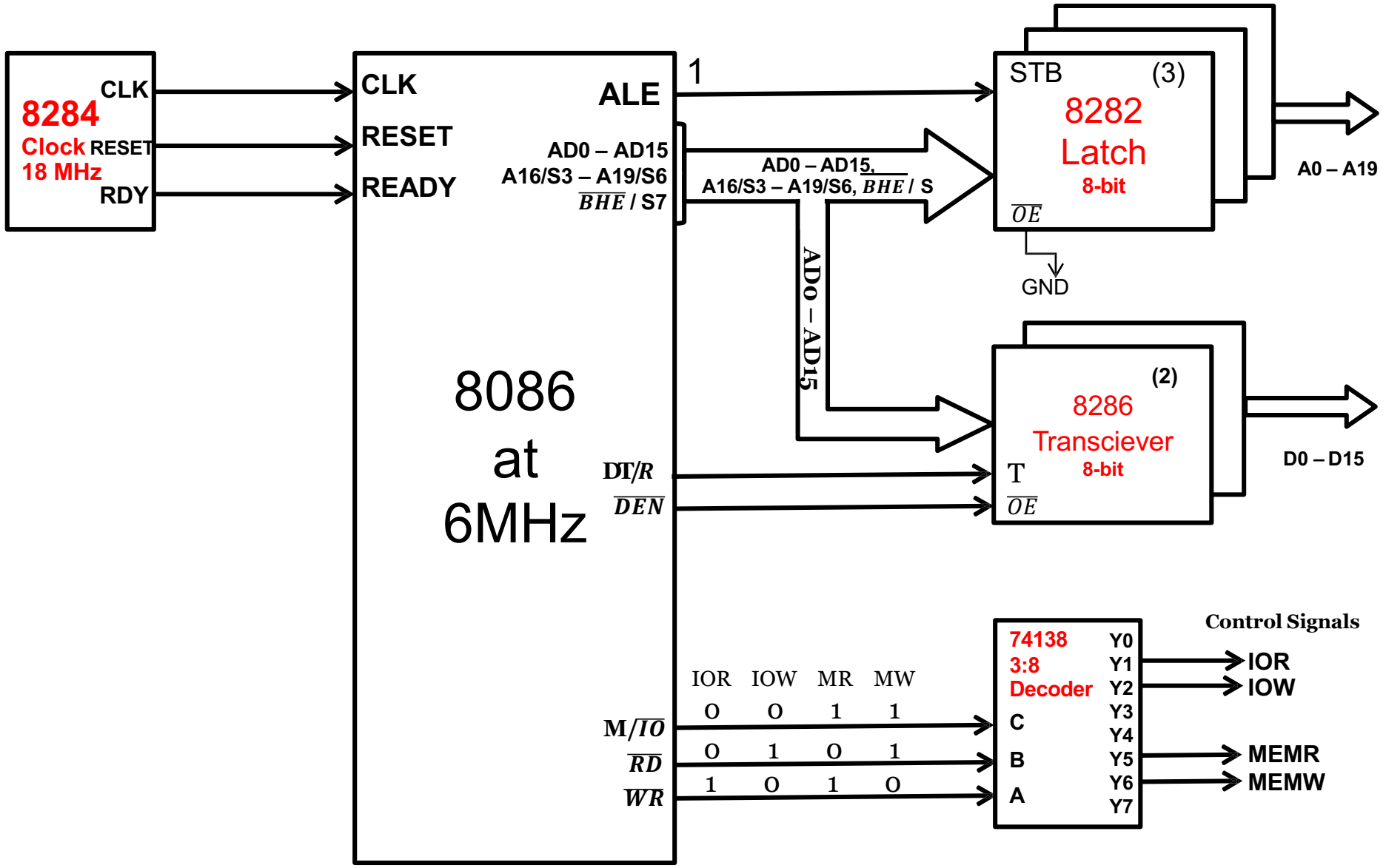
- By using a LATCH
- to save only the address from these pins, and remain cut-off when data is on them.

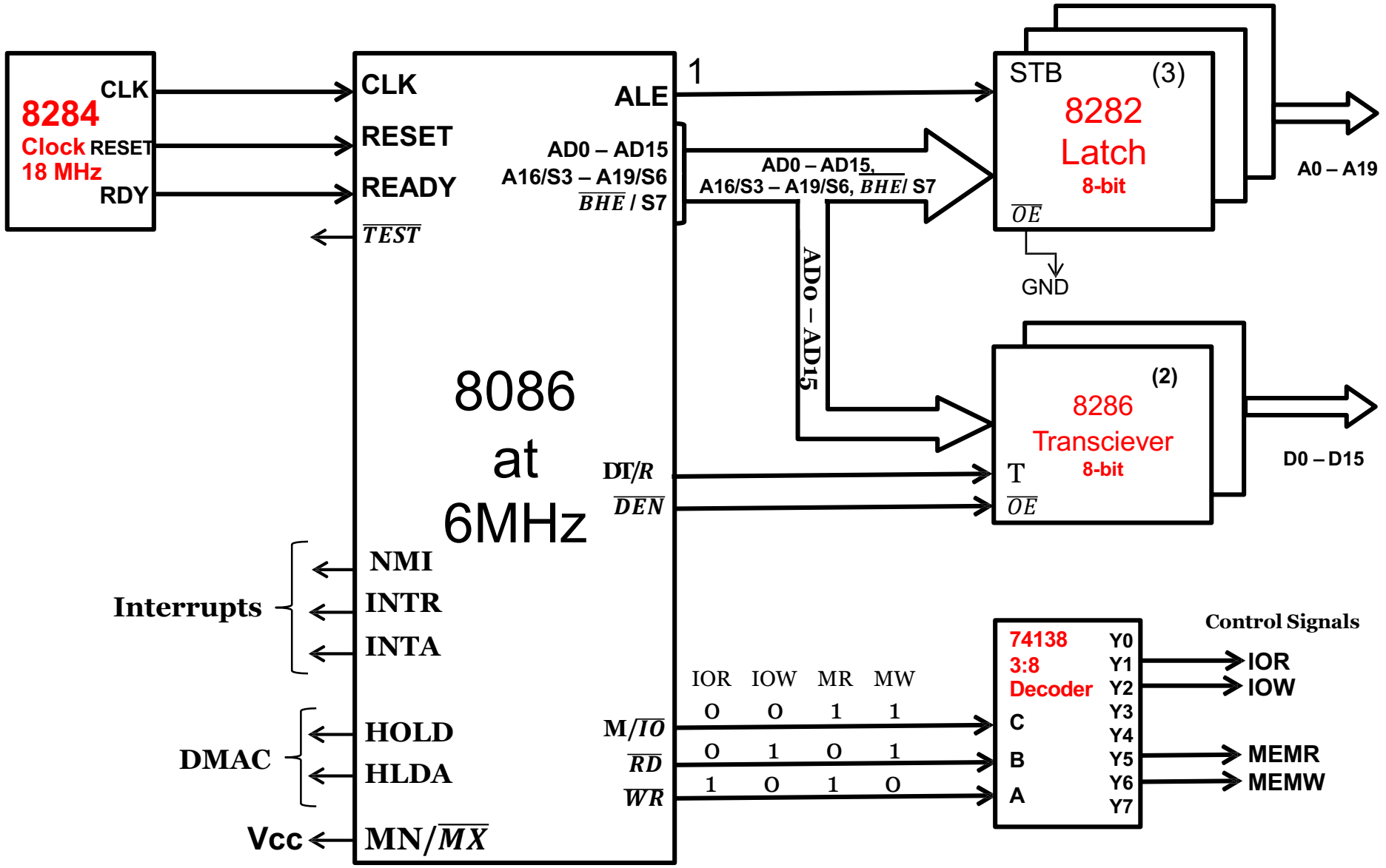

ALE
AD0 – AD15
A16/S3 – A19/S6
BHE /S7
8086
1 0
1 0
AD0 – AD15,
A16/S3 – A19/S6, BHE/S7
STB
(3)
8282
Latch
8-bit
OE
GND
A0 – A19
Pure Address


ALE
AD0 – AD15
A16/S3 – A19/S6
$\overline{BHE}$ /S7
8086
DT/$\overline{R}$
$\overline{DEN}$
1
AD0 – AD15, A16/S3 – A19/S6, $\overline{BHE}$/S7
AD0 – AD15
STB
(3)
8282
Latch
8-bit
$\overline{OE}$
GND
A0 – A19
(2)
8286
Transciever
8-bit
T
$\overline{OE}$
D0 – D15
Pure Data

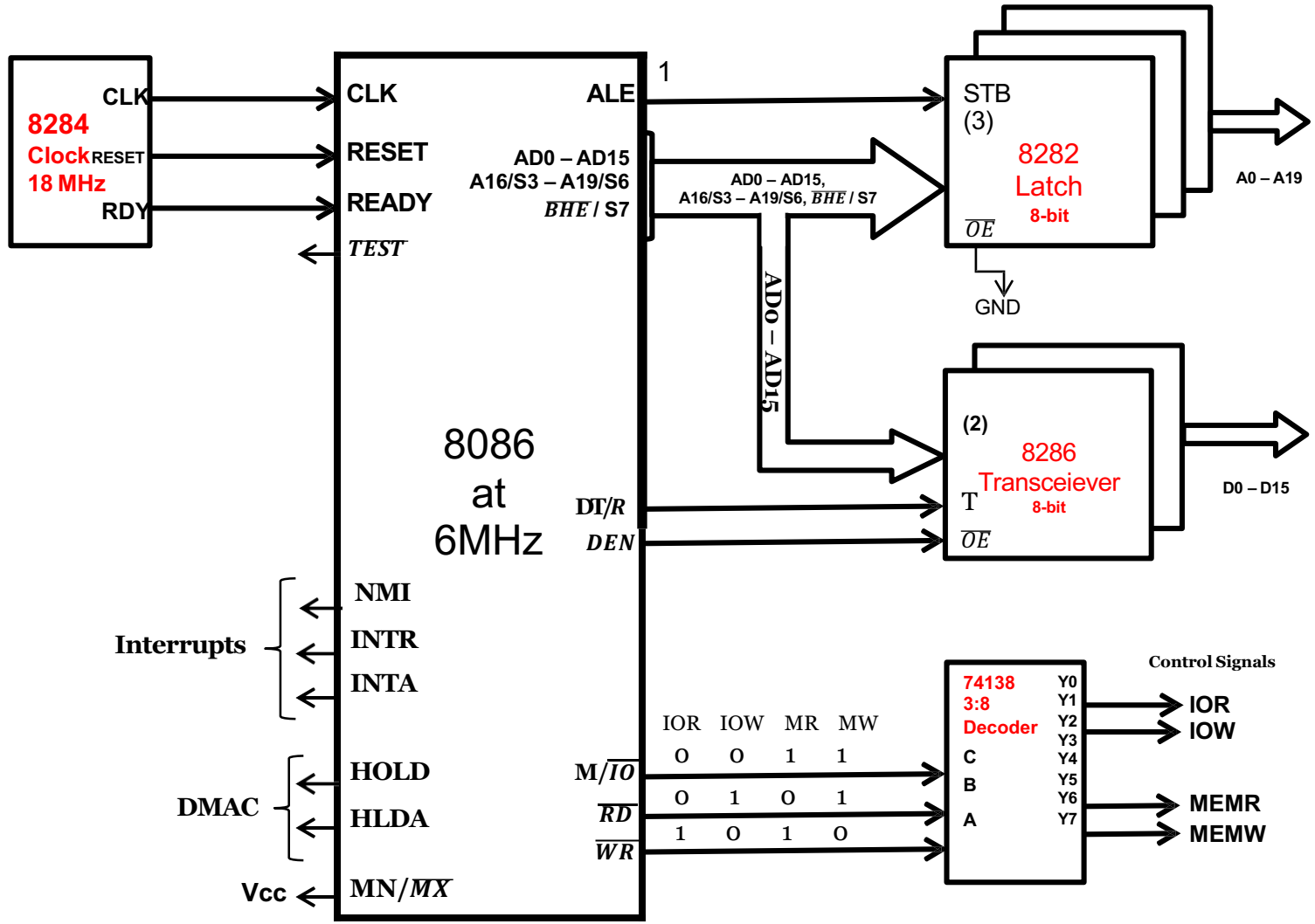

ALE
AD0 – AD15
A16/S3 – A19/S6
BHE / S7
8086
DT/R
DEN
M/IO
RD
WR
1
AD0 – AD15, A16/S3 – A19/S6, BHE/S7
AD0 – AD15
STB
(3)
8282
Latch
8-bit
OE
GND
A0 – A19
(2)
8286
Transciever
8-bit
T
OE
D0 – D15
IOR IOW MR MW
0 0 1 1
0 1 0 1
1 0 1 0
74138
3:8
Decoder
C
B
A
Y0
Y1
Y2
Y3
Y4
Y5
Y6
Y7
IOR
IOW
MEMR
MEMW
Control Signals


Address Bus
ALE
AD0 – AD15
A16/S3 – A19/S6
BHE / S7
1
STB
(3)
8282
Latch
8-bit
OE
GND
A0 – A19
AD0 – AD15, A16/S3 – A19/S6, BHE/S7
AD0 – AD15
Data Bus
(2)
8286
Transciever
8-bit
T
OE
D0 – D15
8086
DT/R
DEN
Control Bus
IOR IOW MR MW
0 0 1 1
0 1 0 1
1 0 1 0
M/IO
RD
WR
74138
3:8
Decoder
C
B
A
Y0
Y1
Y2
Y3
Y4
Y5
Y6
Y7
IOR
IOW
MEMR
MEMW
Control Signals


8284
Clock
18 MHz
CLK
RESET
RDY
CLK
RESET
READY
ALE
AD0 – AD15
A16/S3 – A19/S6
$\overline{BHE}$ / S7
1
8086
at
6MHz
DT/$\overline{R}$
$\overline{DEN}$
M/$\overline{IO}$
$\overline{RD}$
$\overline{WR}$
STB
(3)
8282
Latch
8-bit
$\overline{OE}$
GND
A0 – A19
AD0 – AD15,
A16/S3 – A19/S6, $\overline{BHE}$ / S
AD0 – AD15
(2)
8286
Transciever
8-bit
T
$\overline{OE}$
D0 – D15
IOR IOW MR MW
0 0 1 1
0 1 0 1
1 0 1 0
74138
3:8
Decoder
C
B
A
Y0
Y1
Y2
Y3
Y4
Y5
Y6
Y7
Control Signals
IOR
IOW
MEMR
MEMW


8284
Clock
18 MHz
CLK
RESET
RDY
CLK
RESET
READY
TEST
8086
at
6MHz
ALE
AD0 – AD15
A16/S3 – A19/S6
BHE / S7
DT/R
DEN
M/IO
RD
WR
NMI
INTR
INTA
HOLD
HLDA
MN/MX
Interrupts
DMAC
Vcc
1
STB
(3)
8282
Latch
8-bit
OE
GND
A0 – A19
AD0 – AD15,
A16/S3 – A19/S6, BHE/ S7
AD0 – AD15
(2)
8286
Transciever
8-bit
T
OE
D0 – D15
IOR IOW MR MW
0 0 1 1
0 1 0 1
1 0 1 0
74138
3:8
Decoder
C
B
A
Y0
Y1
Y2
Y3
Y4
Y5
Y6
Y7
Control Signals
IOR
IOW
MEMR
MEMW

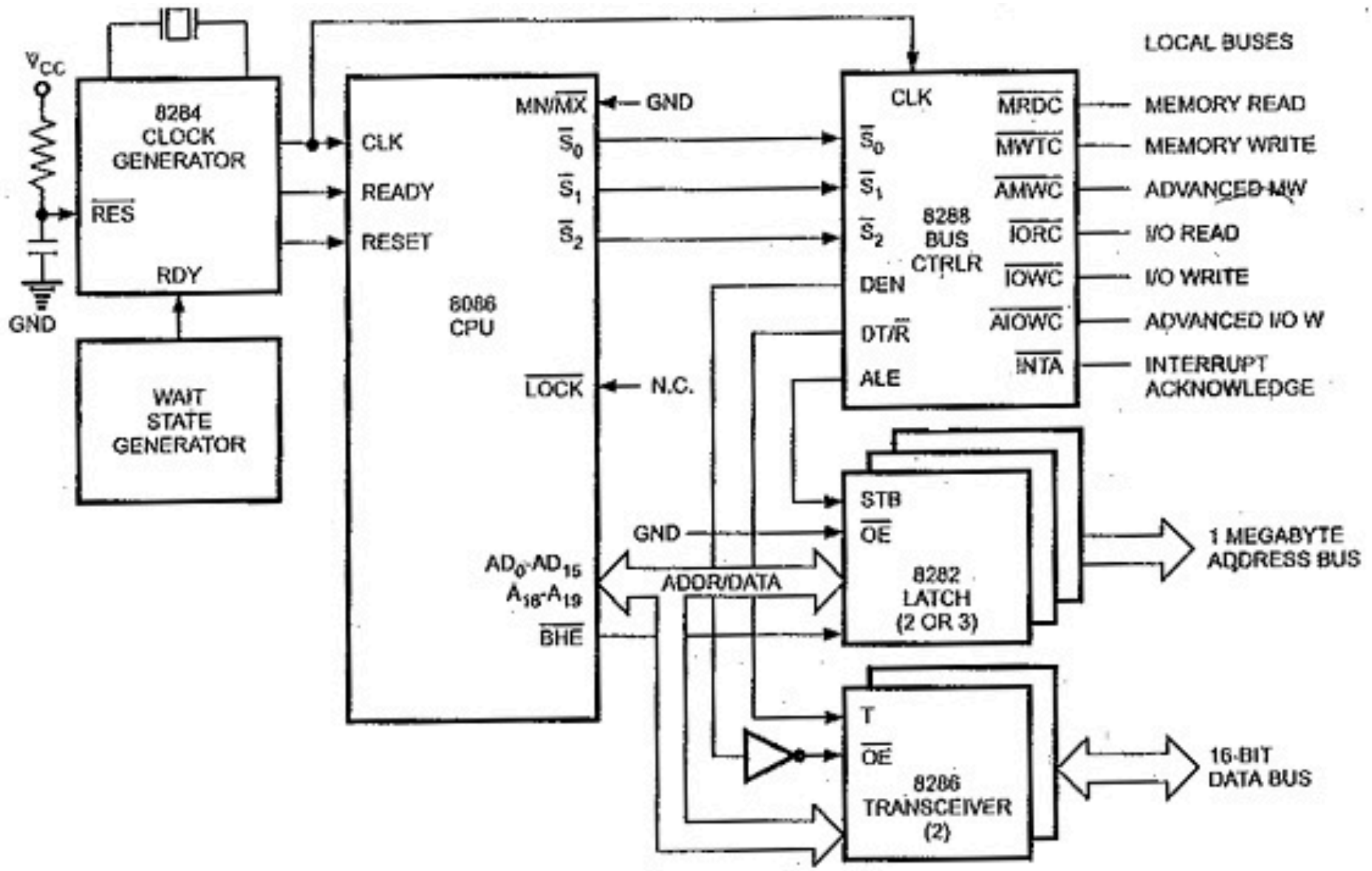
8086 in Minimum Mode!



Maximum Mode Operation

- Differs from minimum mode in that some control signals must be externally generated.
  - requires addition of the 8288 bus controller
- There are not enough pins on the 8086 for bus control during maximum mode
  - new pins and features replaced some of them
- Maximum mode used only when the system contains external coprocessors such as 8087.

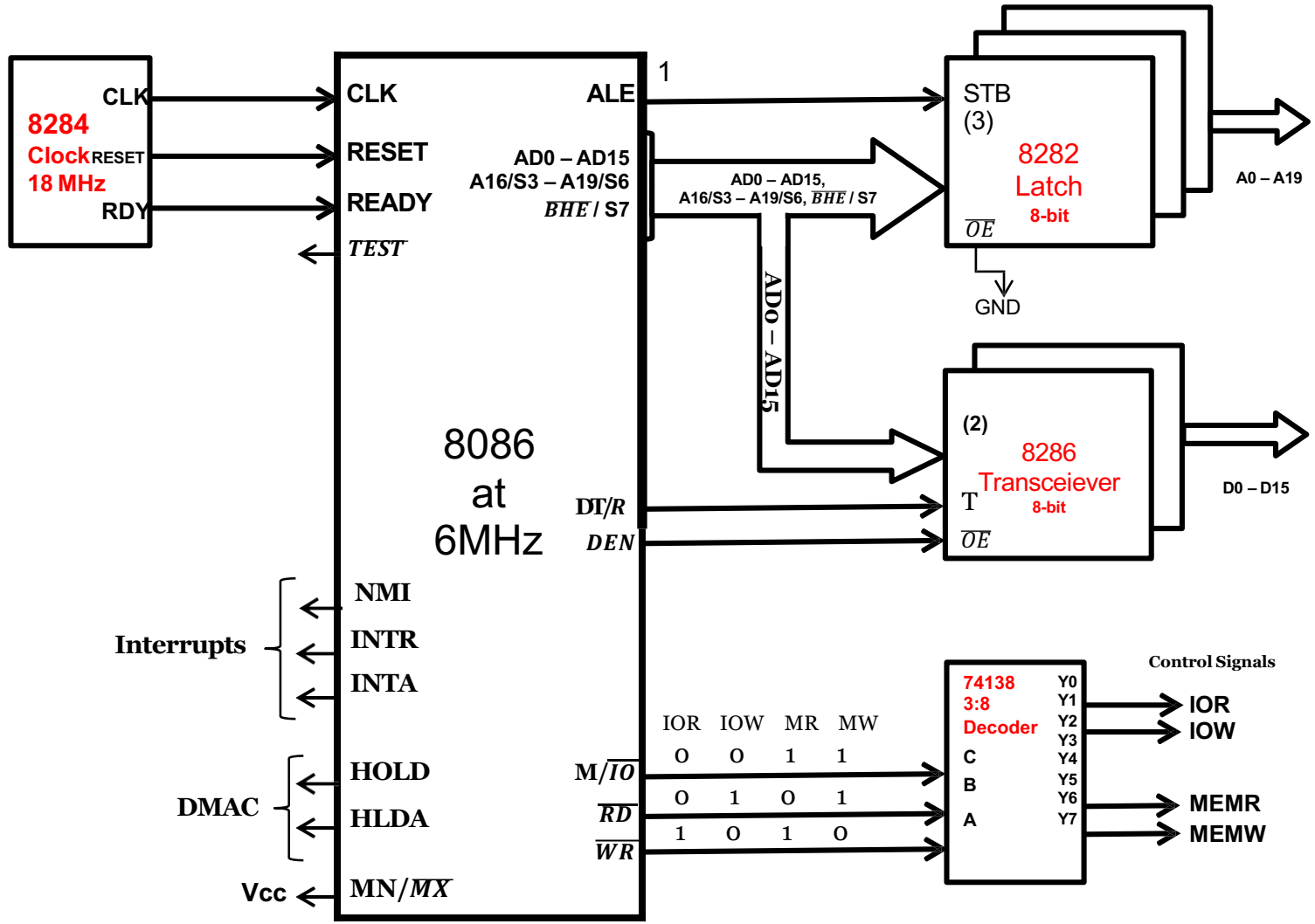
Typical Maximum Mode Configuration



That's all Folks!

8086 in Maximum Mode

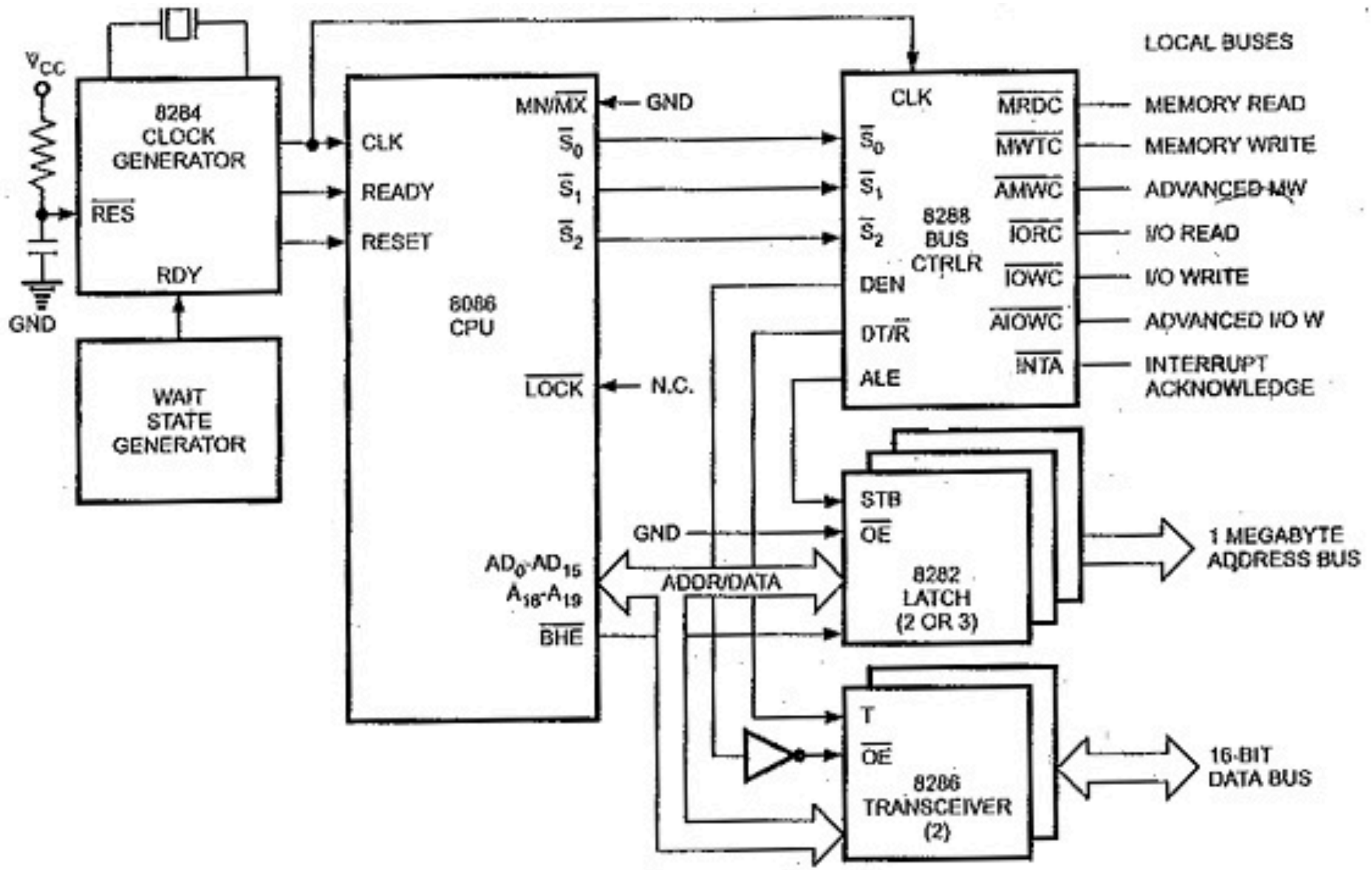
8086 in Minimum Mode!



Maximum Mode Operation

- Differs from minimum mode in that some control signals must be externally generated.
  - requires addition of the 8288 bus controller
- There are not enough pins on the 8086 for bus control during maximum mode
  - new pins and features replaced some of them
- Maximum mode used only when the system contains external coprocessors such as 8087.

Typical Maximum Mode Configuration



The 8288 Bus Controller

- Provides the signals eliminated from the 8086/8088 by the maximum mode operation.

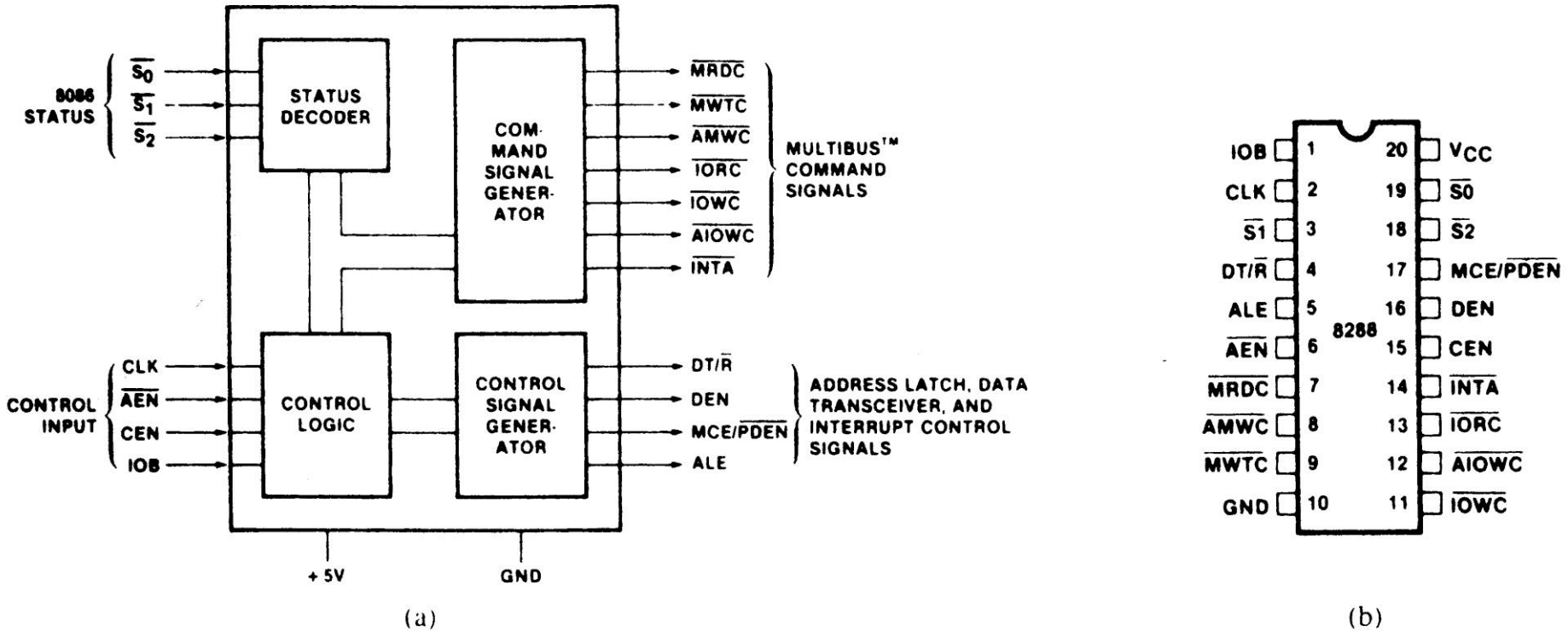
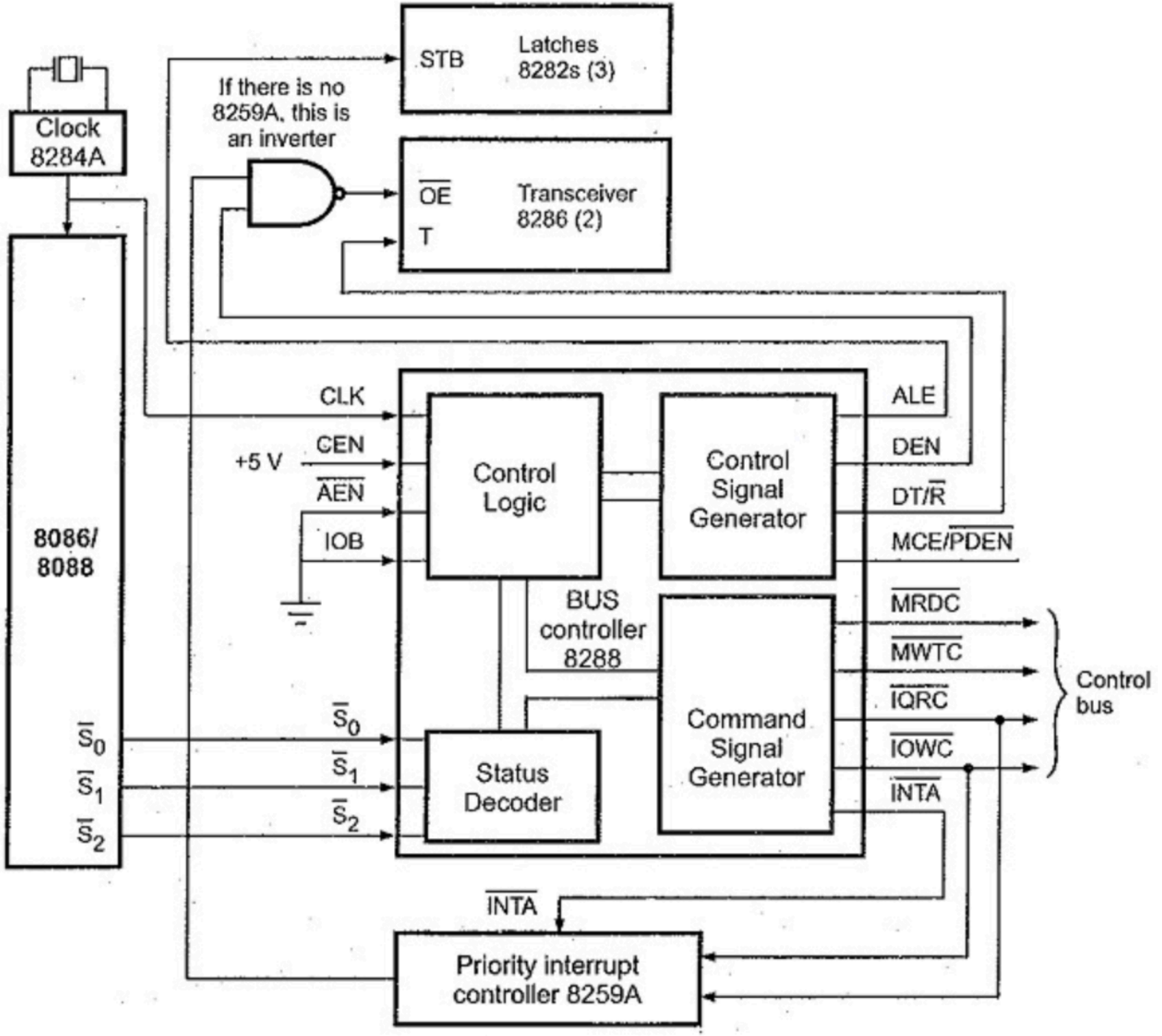


Figure 1. The 8288 bus controller; (a) block diagram and (b) pin-out.

8288 Internal Architecture



8288 Bus Controller *Pin Functions*

S2, S1, and S0

- **Status inputs** are connected to the status output pins on 8086
  - three signals decoded to generate timing signals

$\overline{S}_2$	$\overline{S}_1$	$\overline{S}_0$	Machine cycle
0	0	0	Interrupt Acknowledge
0	0	1	I/O Read
0	1	0	I/O Write
0	1	1	Halt

$\overline{S}_2$	$\overline{S}_1$	$\overline{S}_0$	Machine cycle
1	0	0	Instruction fetch
1	0	1	Memory read
1	1	0	Memory write
1	1	1	Inactive-Passive

CLK

- The **clock** input provides internal timing.
  - must be connected to the CLK output pin of the 8284A clock generator

8288 Bus Controller *Pin Functions*

ALE

- The **address latch enable** output is used to demultiplex the address/data bus.

DEN

- The **data bus enable** pin controls the bidirectional data bus buffers in the system.

DT/$\overline{R}$

- **Data transmit/receive** signal output to control direction of the bidirectional data bus buffers.

8288 Bus Controller *Pin Functions*

$\overline{\text{AEN}}$

- The **address enable** input causes the 8288 to enable the memory control signals.

CEN

- The **control enable** input enables the command output pins on the 8288.

IOB

- The **I/O bus mode** input selects either I/O bus mode or system bus mode operation.

8288 Bus Controller *Pin Functions*

$\overline{\text{AIOWC}}$

- **Advanced I/O write** is a command output to an advanced I/O write control signal.

$\overline{\text{IORC}}$

- The **I/O read command** output provides I/O with its read control signal.

$\overline{\text{IOWC}}$

- The **I/O write command** output provides I/O with its main write signal.

8288 *Pin Functions*

$\overline{\text{AMWT}}$

- **Advanced memory write** control pin provides memory with an early/advanced write signal.

$\overline{\text{MWTC}}$

- The **memory write** control pin provides memory with its normal write control signal.

$\overline{\text{MRDC}}$

- The **memory read** control pin provides memory with a read control signal.

8288 Bus Controller *Pin Functions*

$\overline{\text{INTA}}$

- The **interrupt acknowledge** output acknowledges an interrupt request input applied to the INTR pin.

MCE/$\overline{\text{PDEN}}$

- The **master cascade/peripheral data** output selects cascade operation for an interrupt controller if IOB is grounded, and enables the I/O bus transceivers if IOB is tied high.

Bus Timings for Maximum Mode Configuration of 8086

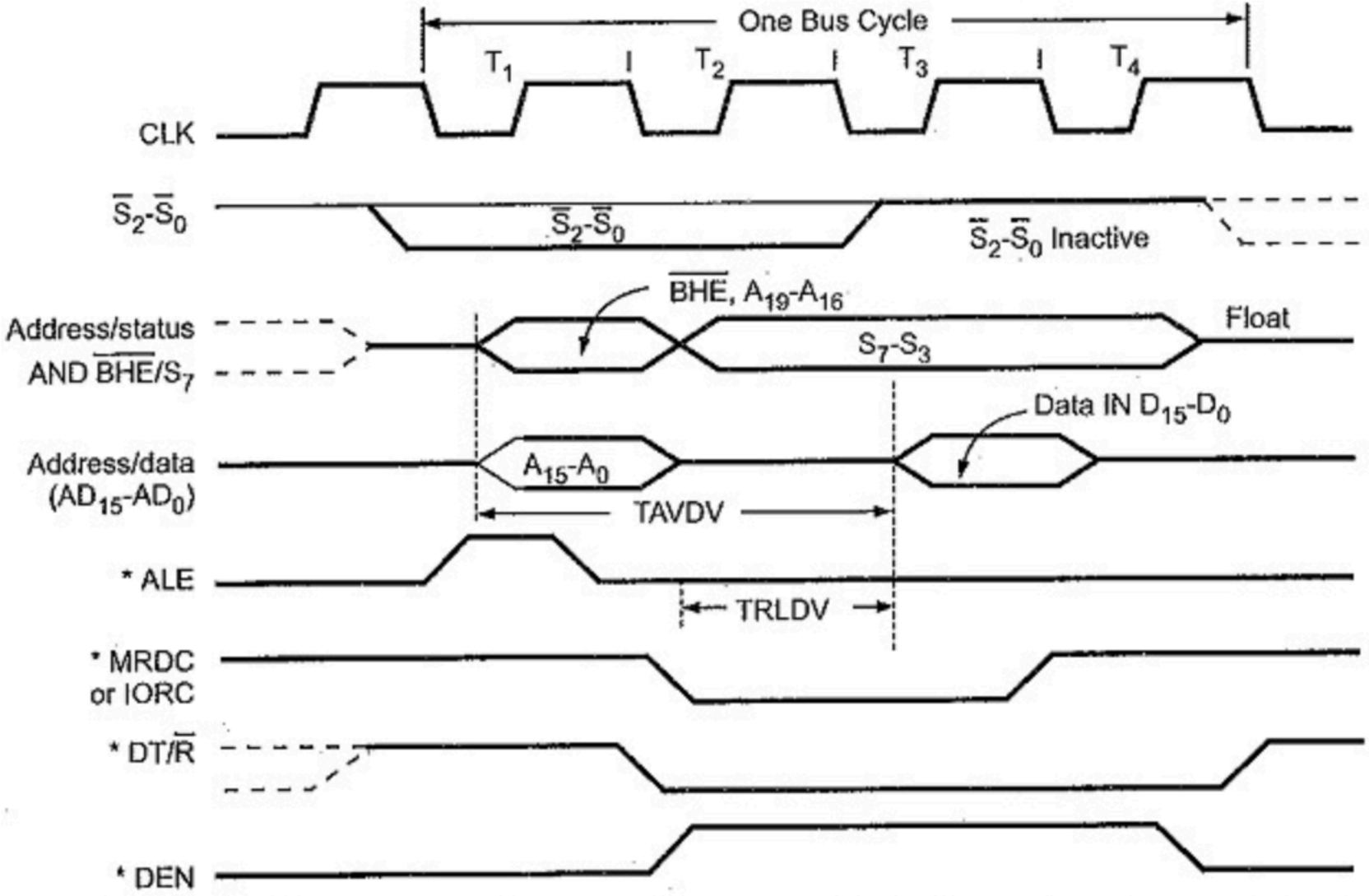
1. S_0,S_1,S_2 are set at the beginning of bus cycle. On detecting the change on passive state $S_0 = S_1 = S_2 = 1$, the 8288 bus controller will output a pulse on its ALE and apply a required signal to its DT/R pin during T_1.

2. In T_2, 8288 will set DEN = 1 thus enabling transceiver. For an input, 8288 it will activate MRDC or IORC. These signals are activated until T_4. For an output, the AMWC or AIOWC is activated from T_2 to T_4 and MWTC or IOWC is activated from T_3 to T_4.

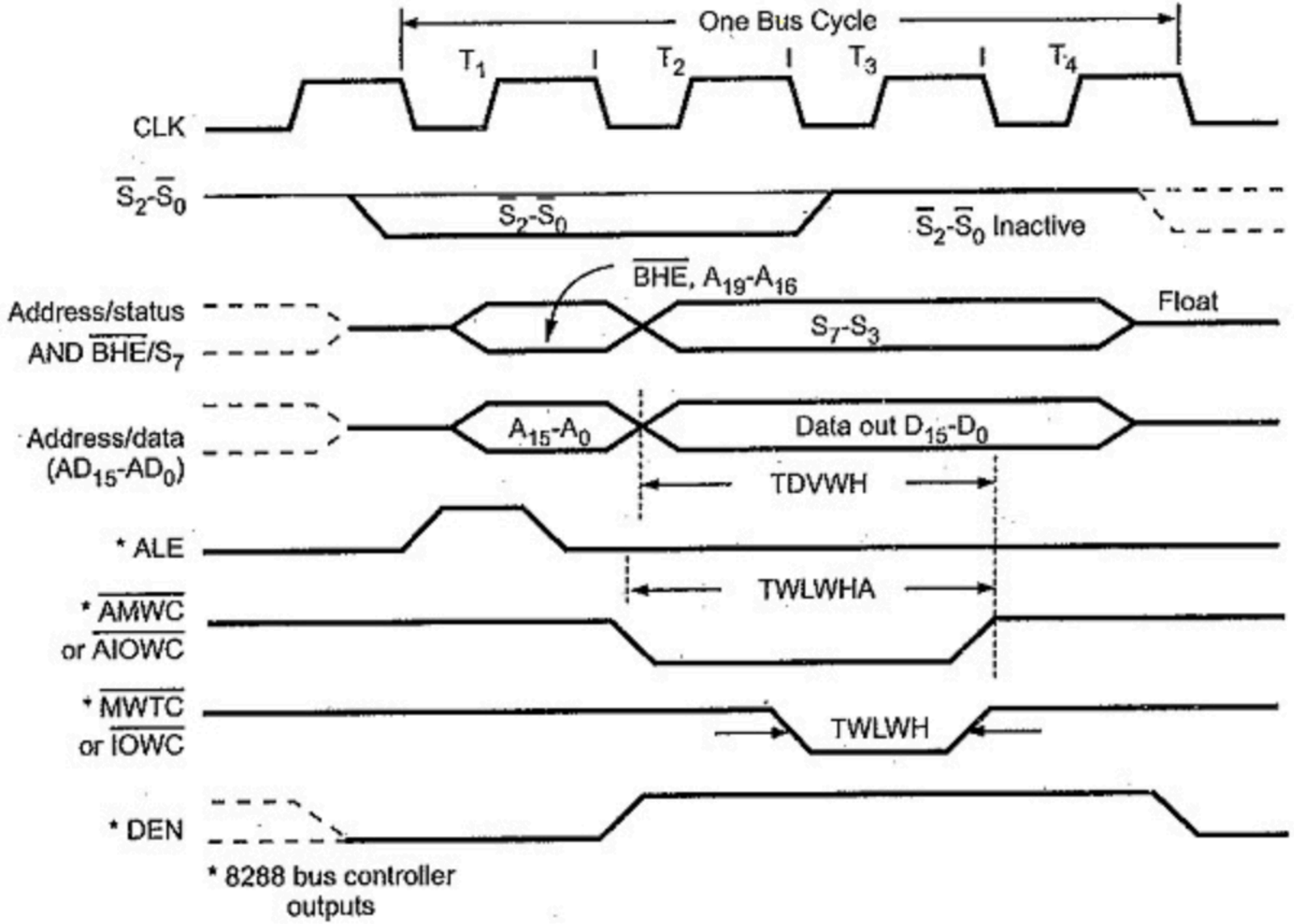
3. The status bits S_0 to S_2 remain active until T_3 and become passive during T_3 and T_4.

4. If READY input is not activated before T3, wait state will be inserted between T_3 and T_4.

Maximum Mode Read



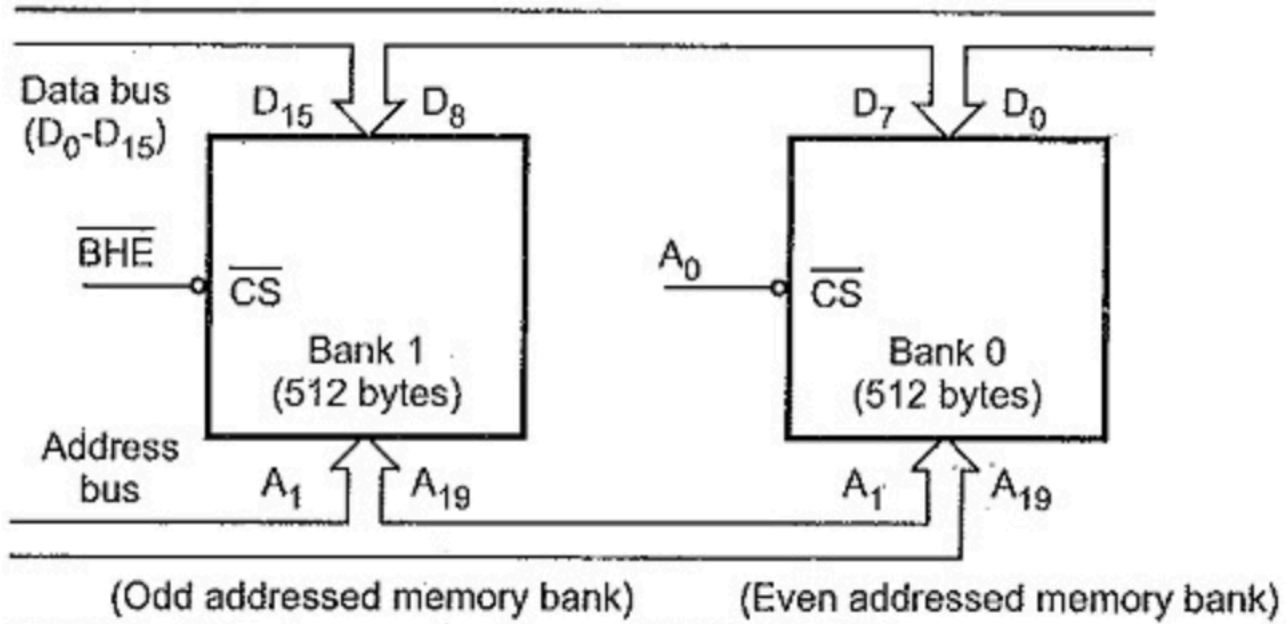
Maximum Mode Write



Memory Addressing Modes of 8086

- Most of the memory ICs are byte oriented
- The 8086 is a 16-bit microprocessor
- In addition to byte, word (16-bit) has to be stored in the memory
- To implement this, the entire memory is divided into two memory banks : $bank_0$ and bank1.
- $Bank_0$ is selected only when A_0 is zero and $Bank_1$ is selected only when BHE is zero.
- A_0 is zero for all even addresses, so $Bank_0$ is usually referred as **even addressed memory** bank. BHE is used to access higher order memory bank, referred to as **odd addressed memory** bank.

No.	Operation	$\overline{BHE}$	A_0	Data Lines Used
1.	Read/Write a byte at an even address	1	0	D_7 - D_0
2.	Read/Write a byte at an odd address	0	1	D_{15} - D_8
3.	Read/Write a word at an even address	0	0	D_{15} - D_0
4.	Read/Write a word at an odd address	0	1	D_{15}-D_0 in first operation byte from odd bank is transferred.
		1	0	D_7-D_0 in second operation byte from even bank is transferred.



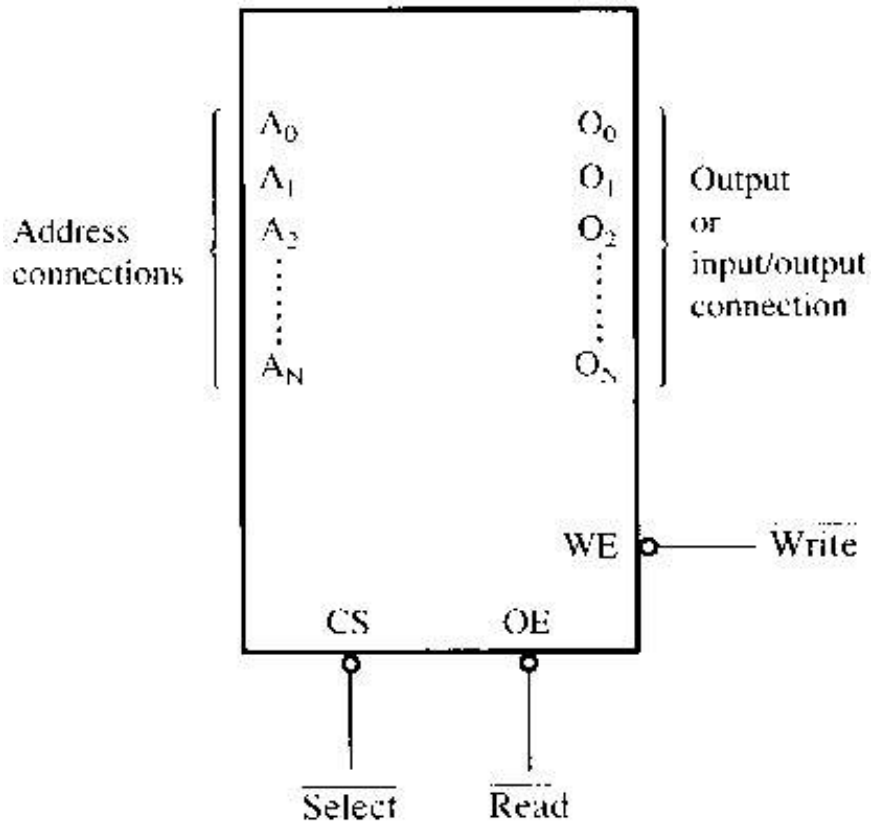
Memory system interfacing

- Every microprocessor-based system has a memory system
- Read-Only Memory (ROM) and Random Access Memory (RAM)
- ROMs / PROMs / EPROMs are mapped to cover the CPU's reset address, since these are non-volatile
- When the 8086 is reset; the next instruction is fetched from memory location FFFF0H. So in the 8086 systems, the location FFFF0H must be ROM location.
- It is important in any memory interface that one block of memory must not be allowed to overlap another memory block.
- In order to connect a memory device to the microprocessor, it is necessary to decode the address from the microprocessor to access each memory IC independently.

MEMORY DEVICES

- Before attempting to interface memory to the microprocessor, it is essential to understand the operation of memory components.
- In this section, we explain functions of the four common types of memory:
  - Read-Only Memory (ROM)
  - Flash memory (EEPROM)
  - Static Random Access Memory (SRAM)
  - Dynamic Random Access Memory (DRAM)

Memory Pin Connections



- address inputs
- data outputs or input/outputs
- some type of selection input
- at least one control input to select a read or write operation

A pseudomemory component illustrating the address, data, and control connections.

Address Connections

- Memory devices have address inputs to select a memory location within the device.
- Almost always labeled from A_0, the least significant address input, to A_n
  - where subscript n can be any value
  - always labeled as one less than total number of address pins
- A memory device with 10 address pins has its address pins labeled from A_0 to A_9.

- The number of address pins on a memory device is determined by the number of memory locations found within it.
- Today, common memory devices have between 1K (1024) to 1G (1,073,741,824) memory locations.
  - with 4G and larger devices on the horizon
- A 1K memory device has 10 address pins.
  - therefore, 10 address inputs are required to select any of its 1024 memory locations

- It takes a 10-bit binary number to select any single location on a 1024-location device.
  - 1024 different combinations
  - if a device has 11 address connections, it has 2048 (2K) internal memory locations
- The number of memory locations can be extrapolated from the number of pins.

Data Connections

- All memory devices have a set of data outputs or input/outputs.
  - today, many devices have bidirectional common I/O pins
  - data connections are points at which data are entered for storage or extracted for reading
- Data pins on memory devices are labeled D_0 through D_7 for an 8-bit-wide memory device.

- An 8-bit-wide memory device is often called a **byte-wide** memory.
  - most devices are currently 8 bits wide,
  - some are 16 bits, 4 bits, or just 1 bit wide
- Catalog listings of memory devices often refer to memory locations times bits per location.
  - a memory device with 1K memory locations and 8 bits in each location is often listed as a 1K $\times$ 8 by the manufacturer
- Memory devices are often classified according to total bit capacity.

Selection Connections

- Each memory device has an input that selects or enables the memory device.
  - sometimes more than one
- This type of input is most often called a **chip select** ($\overline{CS}$) **chip enable** ($\overline{CE}$) or simply **select** ($\overline{S}$) input.
- RAM memory generally has at least one or input, and ROM has at least one
- If more than one $\overline{CE}$ connection is present, all must be activated to read or write data.

Control Connections

All memory devices have some form of control input or inputs.

ROM usually has one control input, while RAM often has one or two control inputs

- Control input often found on ROM is the **output enable** or **gate** connection, which allows data flow from output data pins.
- The $\overline{OE}$ connection enables and disables a set of three-state buffers located in the device and must be active to read data.

- RAM has either one or two control inputs.
  - if one control input, it is often called R/$\overline{W}$
- If the RAM has two control inputs, they are usually labeled $\overline{WE}$ (or $\overline{W}$), and $\overline{OE}$ (or $\overline{G}$).
  - **write enable** must be active to perform memory write, and $\overline{OE}$ active to perform a memory read
  - when the two controls are present, they must never both be active at the same time
- If both inputs are inactive, data are neither written nor read.
  - the connections are at their high-impedance state

ROM Memory

- **Read-only memory** (ROM) permanently stores programs/data resident to the system.
  - and must not change when power disconnected
- Often called **nonvolatile memory**, because its contents *do not* change even if power is disconnected.
- A device we call a ROM is purchased in mass quantities from a manufacturer.
  - programmed during fabrication at the factory

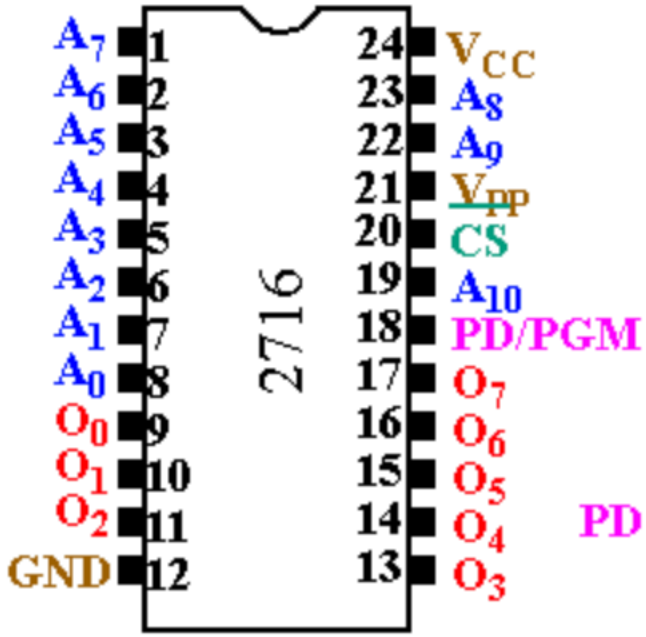
- The EPROM (**erasable programmable read-only memory**) is commonly used when software must be changed often.
  - or when low demand makes ROM uneconomical
  - for ROM to be practical at least 10,000 devices must be sold to recoup factory charges
- An EPROM is programmed in the field on a device called an EPROM programmer.
- Also erasable if exposed to high-intensity ultraviolet light.
  - depending on the type of EPROM

- PROM memory devices are also available, although they are not as common today.
- The PROM (**programmable read-only memory**) is also programmed in the field by burning open tiny NI-chrome or silicon oxide fuses.
- Once it is programmed, it cannot be erased.

- A newer type of **read-mostly memory** (RMM) is called the **flash memory**.
  - also often called an EEPROM (**electrically erasable programmable ROM**)
  - EAROM (**electrically alterable ROM**)
  - or a NOVRAM (**nonvolatile RAM**)
- Electrically erasable in the system, but they require more time to erase than normal RAM.
- The flash memory device is used to store setup information for systems such as the video card in the computer.

- Flash has all but replaced the EPROM in most computer systems for the BIOS.
  - some systems contain a password stored in the flash memory device
- Flash memory has its biggest impact in memory cards for digital cameras and memory in MP3 audio players.
- Figure 2 illustrates the 2716 EPROM, which is representative of most common EPROMs.

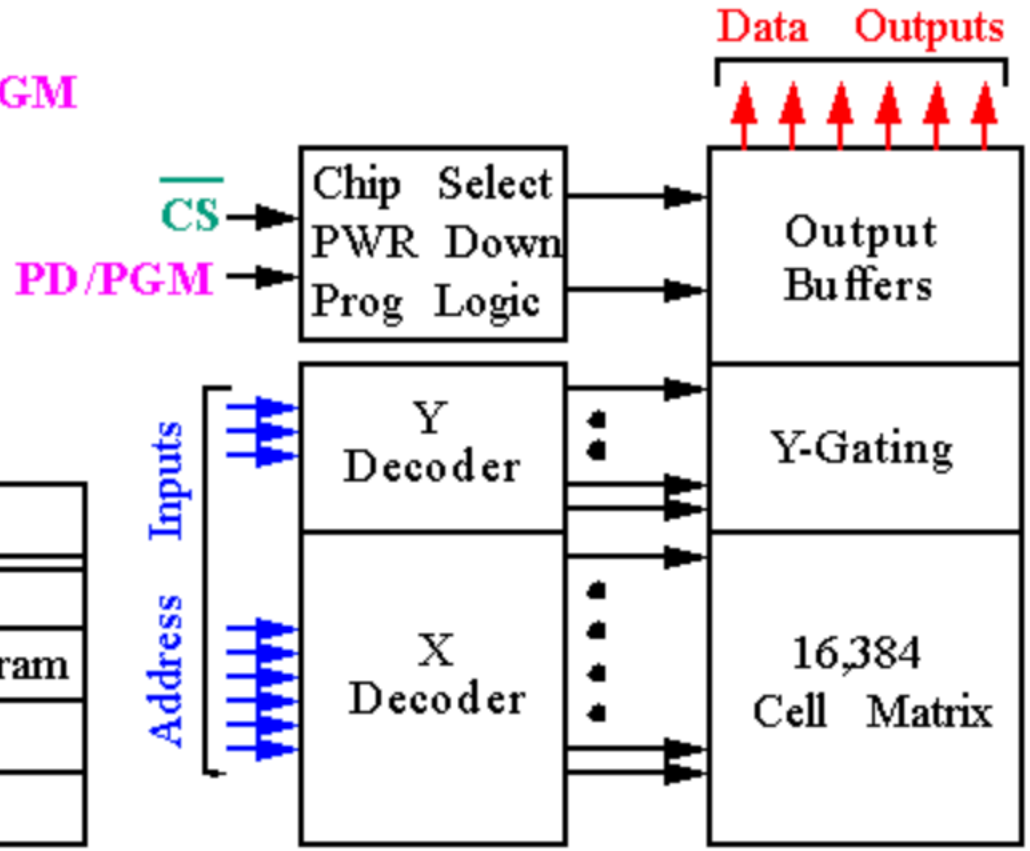
Figure 2 The pin-out of the 2716, 2K × 8 EPROM



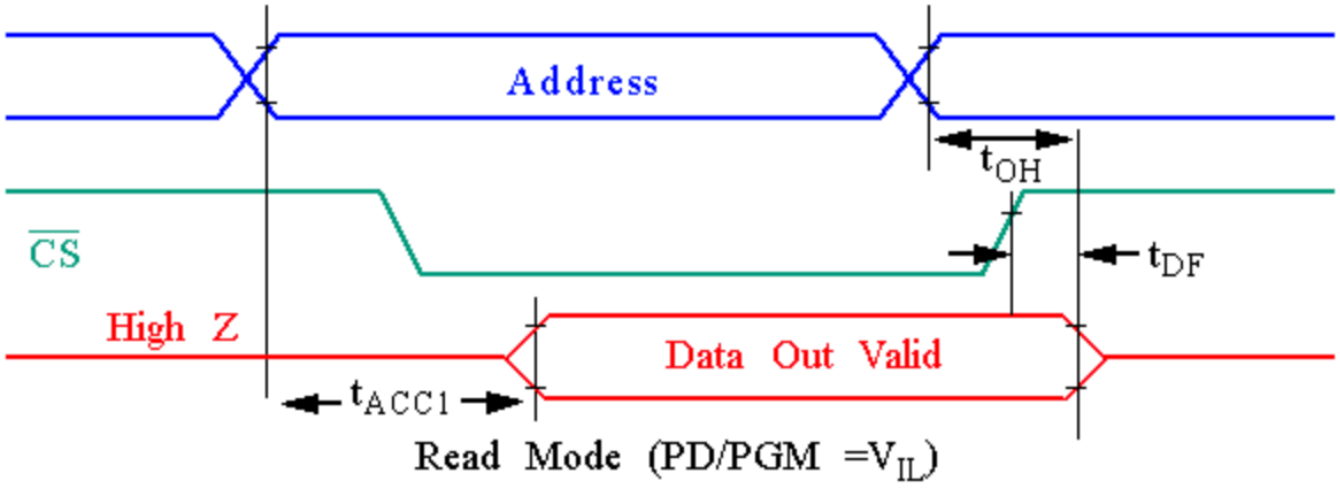
2K x 8 EPROM

Pin(s)	Function
A_0-A_{10}	Address
PD/PGM	Power down/Program
$\overline{CS}$	Chip Select
O_0-O_7	Outputs

V_{PP} is used to program the device by applying 25V and pulsing PGM while holding $\overline{CS}$ high.



2716 Timing Diagram



- *Sample of the data sheet for the 2716 A.C. Characteristics.*

Symbol	Parameter	Limits			Unit	Test Condition
		Min	Typ.	Max		
tACC1	Addr. to Output Delay		250	450	ns	PD/PGM= CS =VIL
tOH	Addr. to Output Hold	0			ns	PD/PGM= CS =VIL
tDF	Chip Deselect to Output Float	0		100	ns	PD/PGM=VIL
...	...	...	...	...	...	...

- *This EPROM requires a wait state for use with the 8086 (460ns constraint).*

- Figure 3 illustrates the timing diagram for the 2716 EPROM.
- The V_{PP} pin must be placed at a logic 1 level for data to be read from the EPROM.
- In some cases, the V_{PP} pin is in the same position as the $\overline{WE}$ pin on the SRAM.
- This will allow a single socket to hold either an EPROM or an SRAM.
  - an example is the 27256 EPROM and 62256 SRAM, both 32K $\times$ 8 devices with the same pin-out, except for V_{PP} on the EPROM and $\overline{WE}$ on the SRAM.

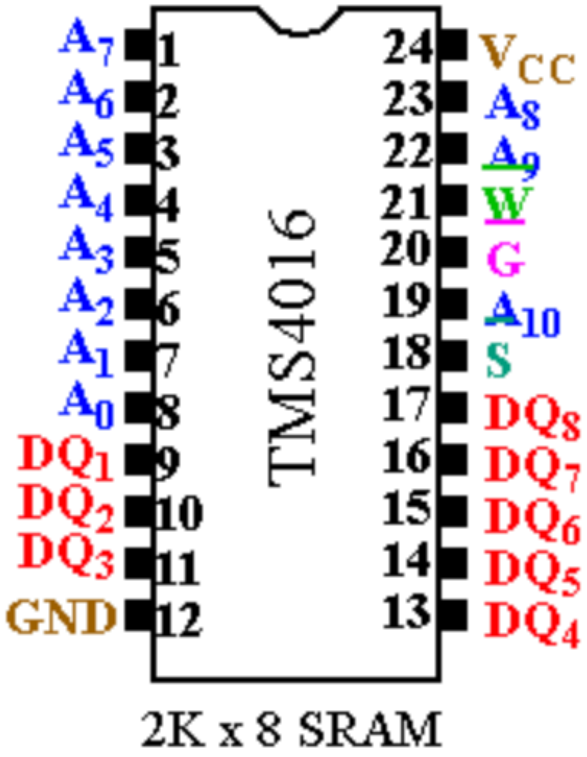
- The basic speed of this EPROM is 450 ns.
  - recall that 8086 operated with a 5 MHz clock allowed memory 460 ns to access data
- This type of component requires wait states to operate properly with 8086 because of its rather long access time.
  - if wait states are not desired, higher-speed EPROMs are available at additional cost
  - EPROM memory is available with access times of as little as 100 ns
- Obviously, wait states are required in modern microprocessors for any EPROM device.

Static RAM (SRAM) Devices

- Static RAM memory devices retain data for as long as DC power is applied.
- Because no special action is required to retain data, these devices are called **static memory**.
  - also called **volatile memory** because they will not retain data without power
- The main difference between ROM and RAM is that RAM is written under normal operation, whereas ROM is programmed outside the computer and normally is only read.

- Fig 4 illustrates the 4016 SRAM,
  - a 2K $\times$ 8 read/write memory
- This device is representative of all SRAM devices.
  - except for the number of address and data connections.
- The control inputs of this RAM are slightly different from those presented earlier.
  - however the control pins function exactly the same as those outlined previously
- Found under part numbers 2016 and 6116.

Figure 4 The pin-out of the TMS4016, 2K × 8 static RAM (SRAM). (Courtesy of Texas Instruments Incorporated.)



Pin(s)	Function
A_0-A_{10}	Address
DQ_0-DQ_7	Data In/Data Out
$\overline{S}$ ($\overline{CS}$)	Chip Select
$\overline{G}$ ($\overline{OE}$)	Read Enable
$\overline{W}$ ($\overline{WE}$)	Write Enable

- *Virtually identical to the EPROM with respect to the pinout.*
- *However, access time is faster (250ns).*
  - *See the timing diagrams and data sheets in text.*
- *SRAMs used for caches have access times as low as 10ns .*

- Figure 6 illustrates pin-outs of the 62256, 32K $\times$ 8 static RAM.
- Packaged in a 28-pin integrated circuit
- Available with access times of 120 or 150 ns.
- Other common SRAM devices are
  - 8K $\times$ 8; 128K $\times$ 8; 256K $\times$ 8
  - 512K $\times$ 8; 1M $\times$ 8
- Access times can be as low as 1.0 ns for SRAM used in computer cache memory.

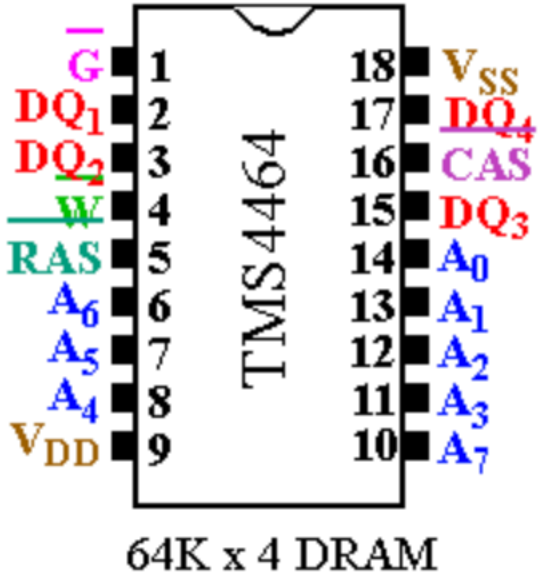
Dynamic RAM (DRAM) Memory

- Available up to 256M $\times$ 8 (2G bits).
- DRAM is essentially the same as SRAM, except that it retains data for only 2 or 4 ms
  on an integrated capacitor.
- After 2 or 4 ms, the contents of the DRAM must be completely rewritten *(refreshed)*.
  - because the capacitors, which store a logic 1 or logic 0, lose their charges

- In DRAM, the entire contents are refreshed with 256 reads in a 2- or 4-ms interval.
  - also occurs during a write, a read, or during a special refresh cycle
- DRAM requires so many address pins that manufacturers multiplexed address inputs.
- Figure 7 illustrates a 64K $\times$ 4 DRAM, the TMS4464, which stores 256K bits of data.
  - note it contains only eight address inputs where it should contain 16—the number required to address 64K memory locations

Figure 7 The pin-out of the TMS4464, 64K × 4 dynamic RAM (DRAM). (Courtesy of Texas Instruments Incorporated.)

- *TI TMS4464 DRAM (64K X 4):*



64K x 4 DRAM

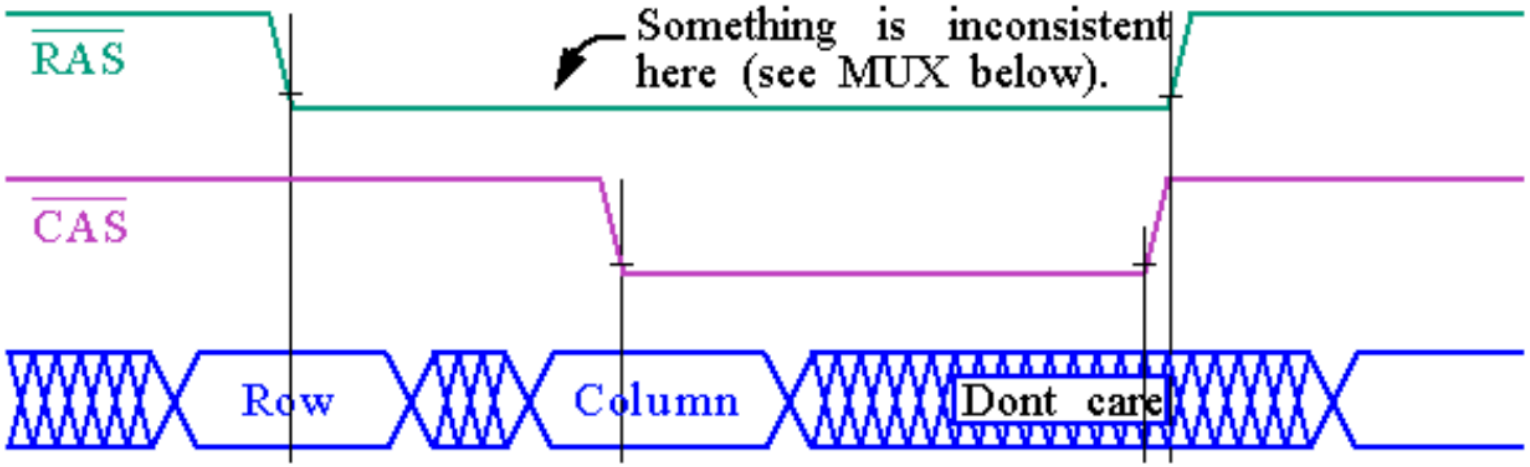
Pin(s)	Function
A_0-A_7	Address
DQ_0-DQ_4	Data In/Data Out
$\overline{RAS}$	Row Address Strobe
$\overline{CAS}$	Column Address Strobe
$\overline{G}$	Output Enable
$\overline{W}$	Write Enable

  - *The TMS4464 can store a total of 256K bits of data.*

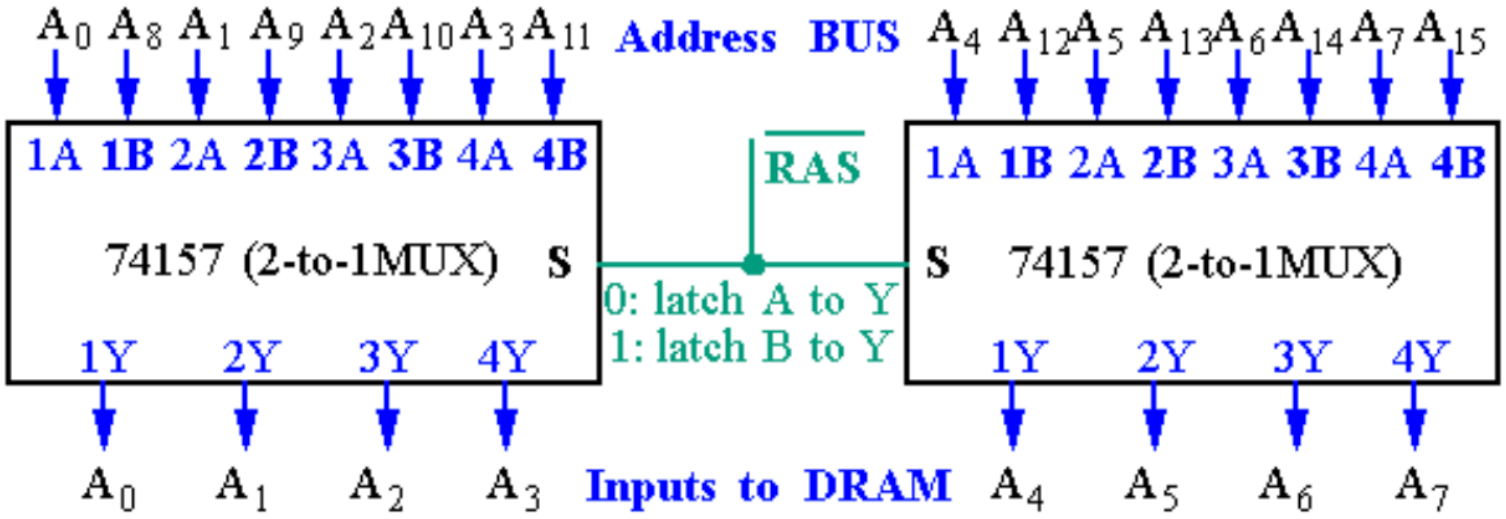
- 16 address bits can be forced into eight address pins in two 8-bit increments
- this requires two special pins: the **column address strobe** ($\overline{CAS}$) and **row address strobe** ($\overline{RAS}$)

- First, A_0–A_7 are placed on the address pins and strobed into an internal row latch by $\overline{RAS}$ as the row address.
- Next, address bits A_8-A_{15} are placed on the same eight address inputs and strobed into an internal column latch by $\overline{CAS}$ as the column address
- The 16-bit address in the internal latches addresses the contents of one of the 4-bit memory locations.
  - $\overline{CAS}$ also performs chip selection input to DRAM

Figure 8 $\overline{\text{RAS}}$, $\overline{\text{CAS}}$ and address input timing for the TMS4464 DRAM. (Courtesy of Texas Instruments Incorporated.)



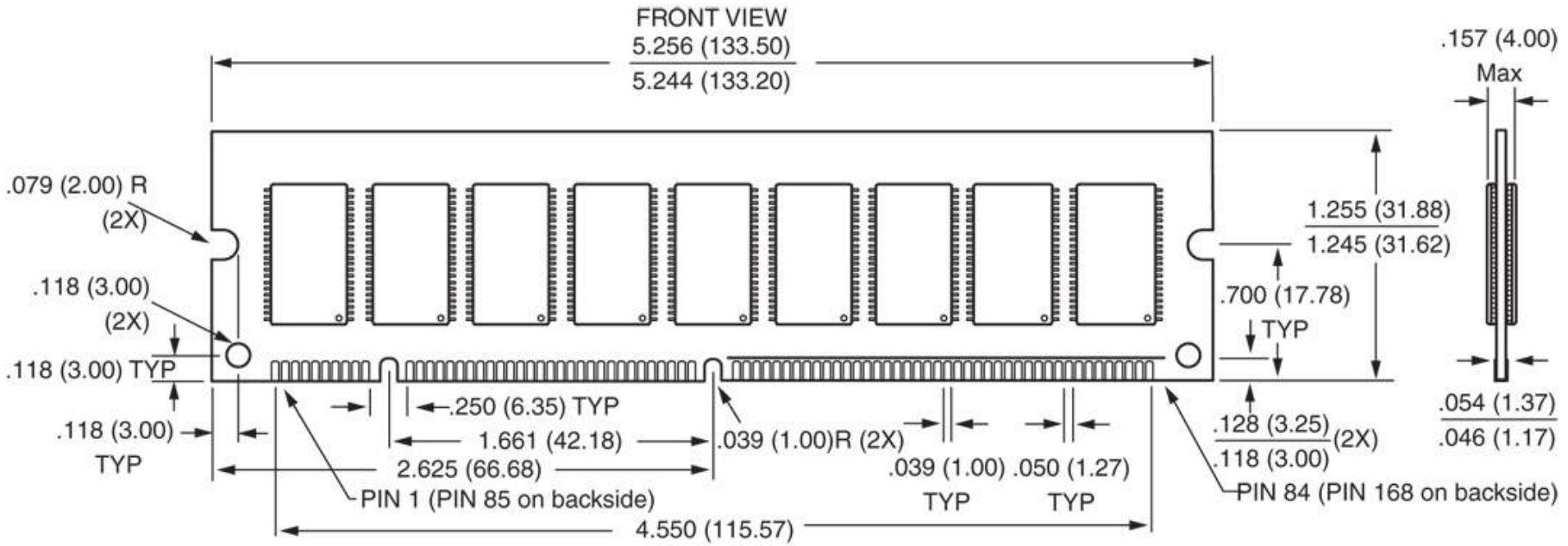
- CAS *also performs the function of the chip select input.*



–multiplexers used to strobe column and row addresses into the address pins on a pair of TMS4464 DRAMs.

–the RAS signal not only strobes the row address into the DRAMs, but it also selects which part of the address is applied to the address inputs.

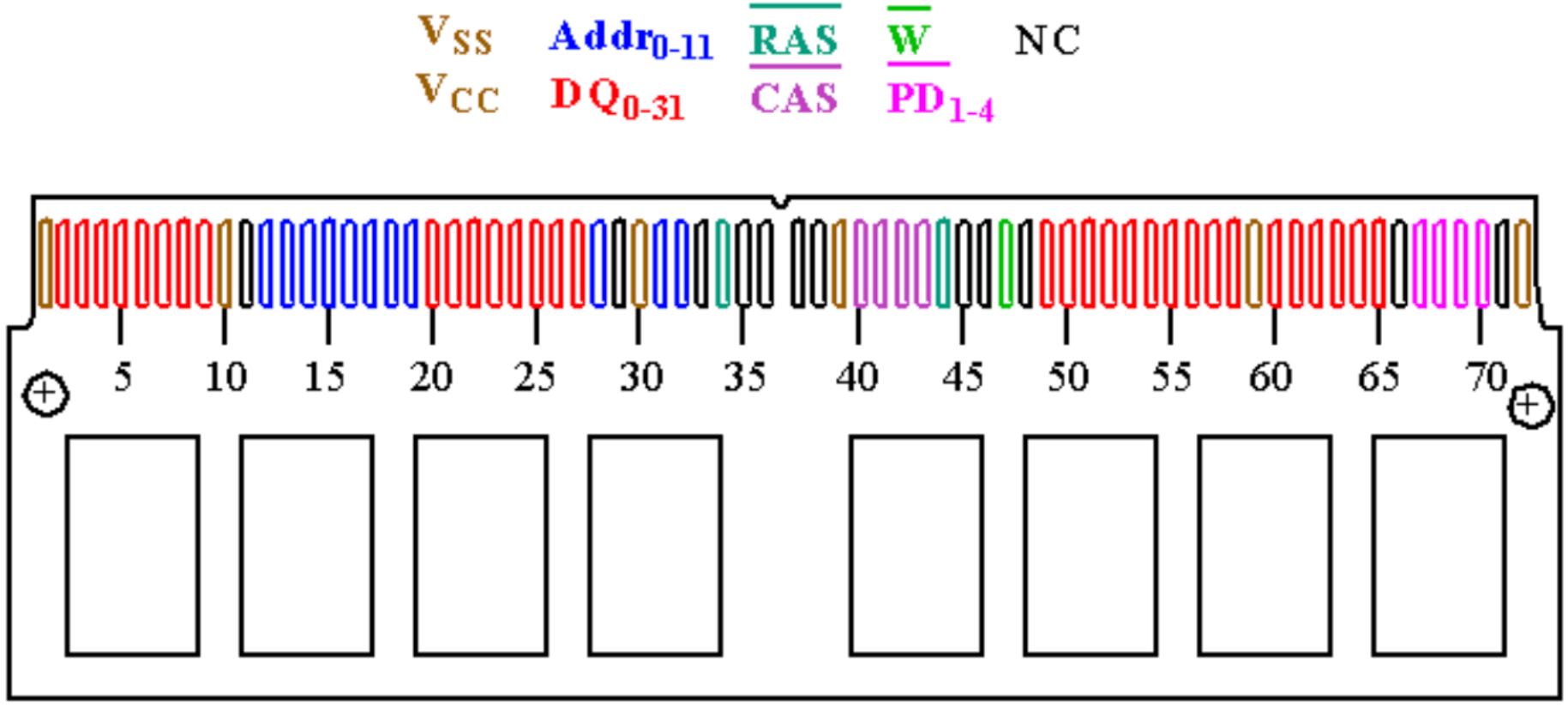
Figure 10 The 41256 dynamic RAM organized as a 256K × 1 memory device.



- the pin-out of the 41256 dynamic RAM
- this device is organized as a 256K × 1 memory
- requires as little as 70 ns to access data

- DRAM is often placed on small boards called SIMMs (Single In-Line Memory Modules).
- The 30-pin SIMM is organized most often as 1M $\times$ 8 or 1M $\times$ 9, and 4M $\times$ 8 or 4M $\times$ 9.
  - illustrated in Fig 11 is a 4M $\times$ 9
  - the ninth bit is the parity bit
- Also shown is a newer 72 pin SIMM.
- 72-pin SIMMs are often organized as 1M $\times$ 32 or 1M $\times$ 36 (with parity).
- Fig 11 illustrates a 4M $\times$ 36 SIMM, which has 16M bytes of memory

Figure 10–11 The pin-outs of the 30-pin and 72-pin SIMM. 72-pin SIMM organized as 4M × 32.



- Pentium–Pentium 4 microprocessors have a 64-bit wide data bus, which precludes use of the 8-bit-wide SIMMs described here.
  - 72-pin SIMMs are cumbersome as they must be paired to obtain a 64-bit-wide data connection
- 64-bit-wide DIMMs (Dual In-line Memory Modules) have become the standard.
- The memory on these modules is organized as 64 bits wide.
- Common sizes available are from 16M bytes (2M $\times$ 64) to 1G bytes (128M $\times$ 64).

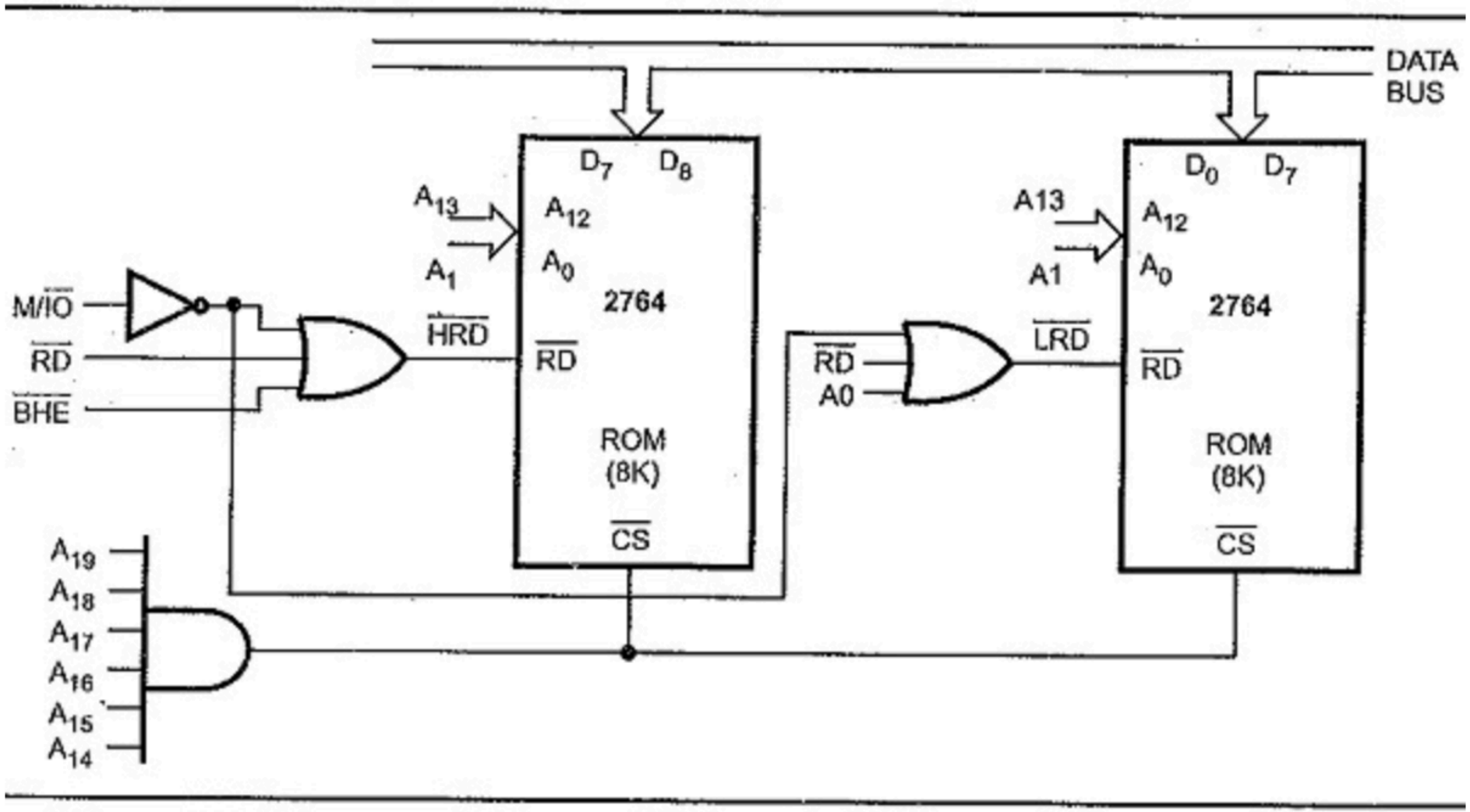
Address decoding techniques in 8086

- Absolute decoding
- Linear decoding
- Block decoding

Absolute decoding

- Memory chip is selected only for the specified logic level on the address lines
- No other logic levels can select the chip.
- Example: two 8K EPROMs (2764) are used to provide even and odd memory banks.
  - Control signals BHE and A_0 are used to enable outputs of odd and even memory banks
  - As each memory chip has 8K memory locations, thirteen address lines are required to address each locations
  - All remaining address lines are used to generate an unique chip select signal.
  - This addressing technique is normally used in large, memory systems.

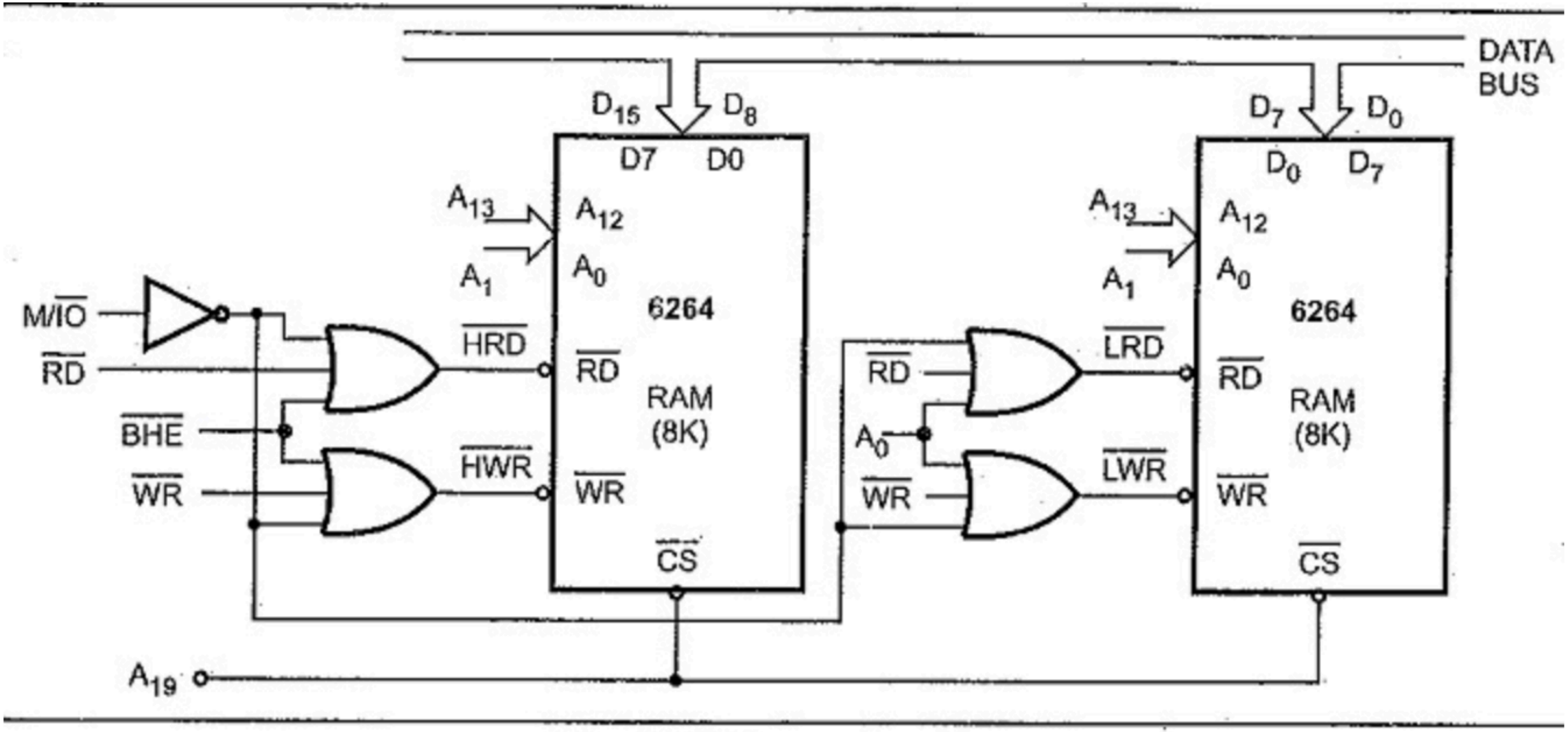
Absolute decoding example



Linear decoding

- In small systems, hardware for the decoding logic can be eliminated by using only required number of addressing lines
- This technique is referred as **Linear Decoding** or **Partial Decoding**
- Example: the addressing of 16K RAM (6264) with linear decoding
  - Control signals BHE and A_0 are used to enable odd and even memory banks, respectively.
  - The address line A_{19} is used to select the RAM chips.
  - When A_{19} is low, chip is selected, otherwise it is disabled.
  - The status of A_{14} to A_{18} does not affect the chip selection logic.
  - This gives you multiple addresses (shadow addresses)

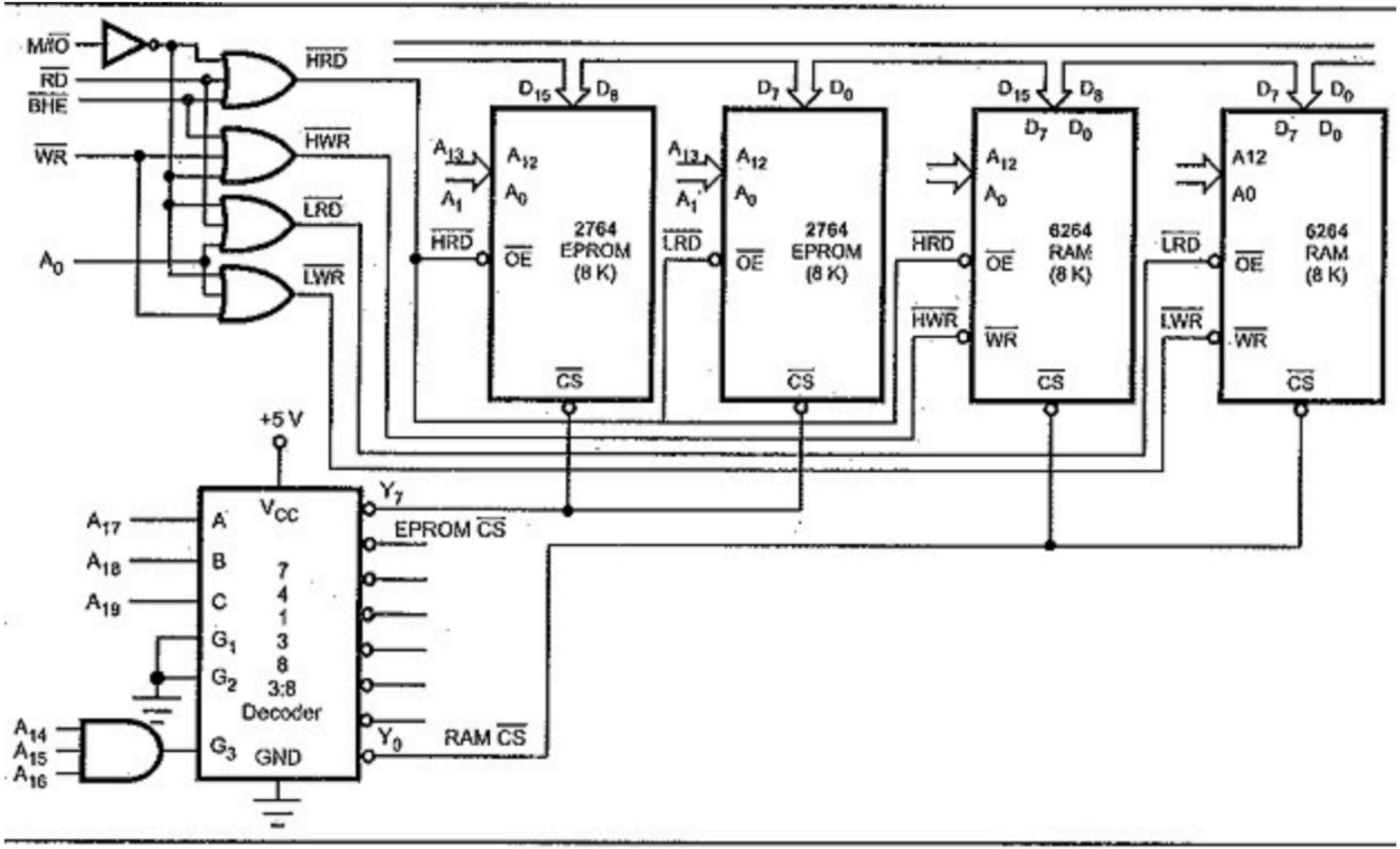
Linear decoding example



Block decoding

- Memory array often consists of several blocks of memory chips.
- Each block of memory requires a decoding circuit.
- To avoid separate decoding for each memory block a special decoder IC is used to generate chip select signal for each block.
- The example shows the block Address Decoding Techniques in 8086 Microprocessor using a 74138, 3:8 decoder.

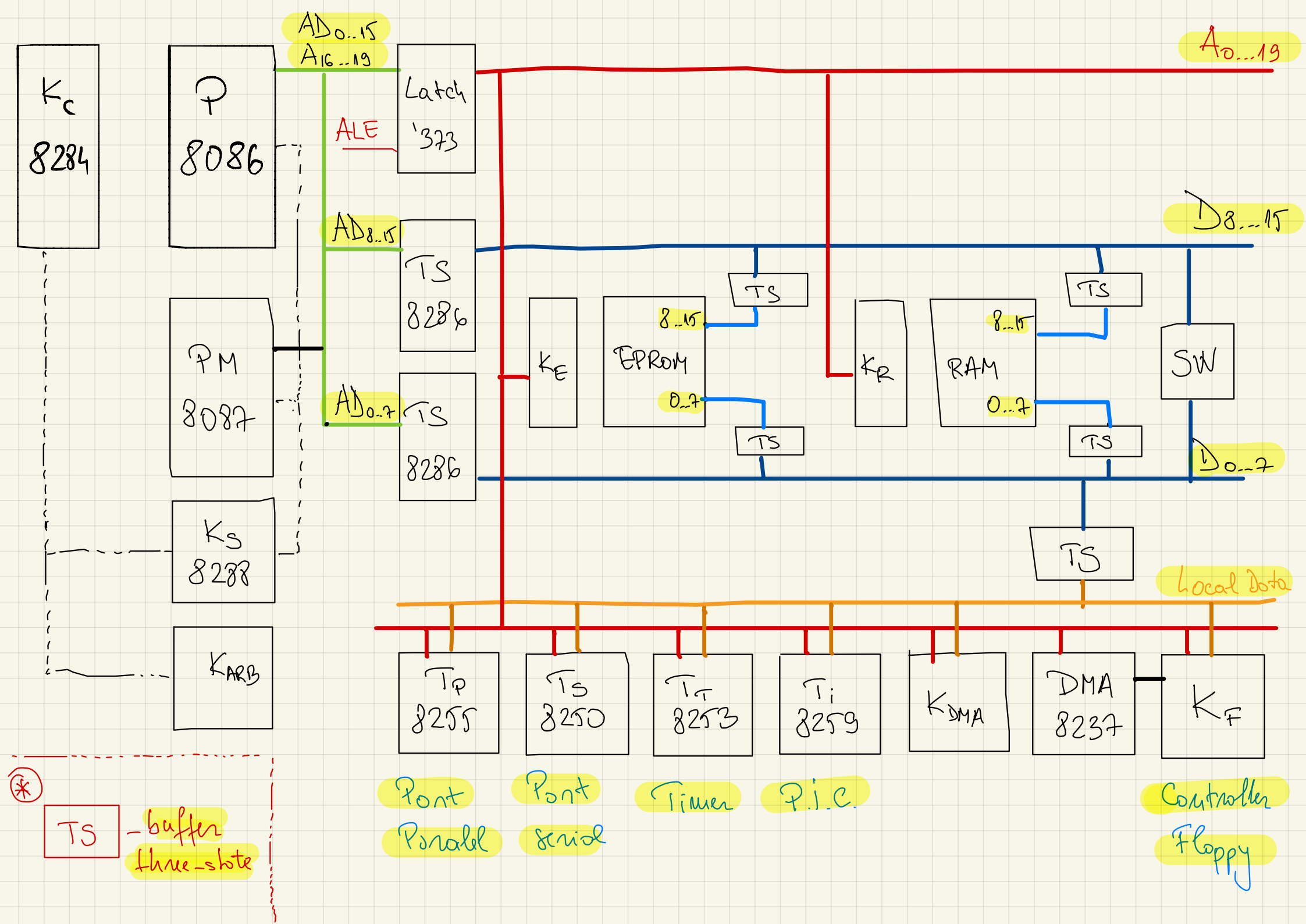
Block decoding example

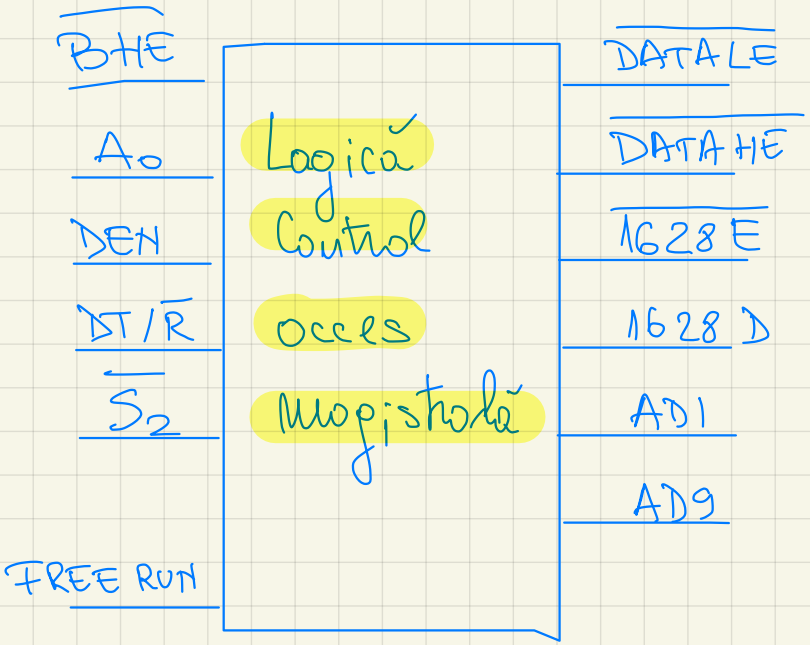


That's all Folks!

PM - Cursul 6

Notite de curs


Kc
8284
P
8086
PM
8087
KS
8288
KARB
AD0..15
A16..19
ALE
Latch
'373
AD8..15
TS
8286
AD0..7
TS
8286
A0...19
D8...15
D0...7
TS
KE
EPROM
8..15
0..7
KR
RAM
8..15
0..7
SW
Local Data
TP
8255
TS
8250
TT
8253
Ti
8259
KDMA
DMA
8237
KF
Port
Parallel
Port
seriale
Timer
P.I.C.
Controller
Floppy
(*)
TS
- buffer
three-state



$$\overline{1628E} = INTA + \bar{A}_0 + \bar{S}_2 \cdot BHE$$

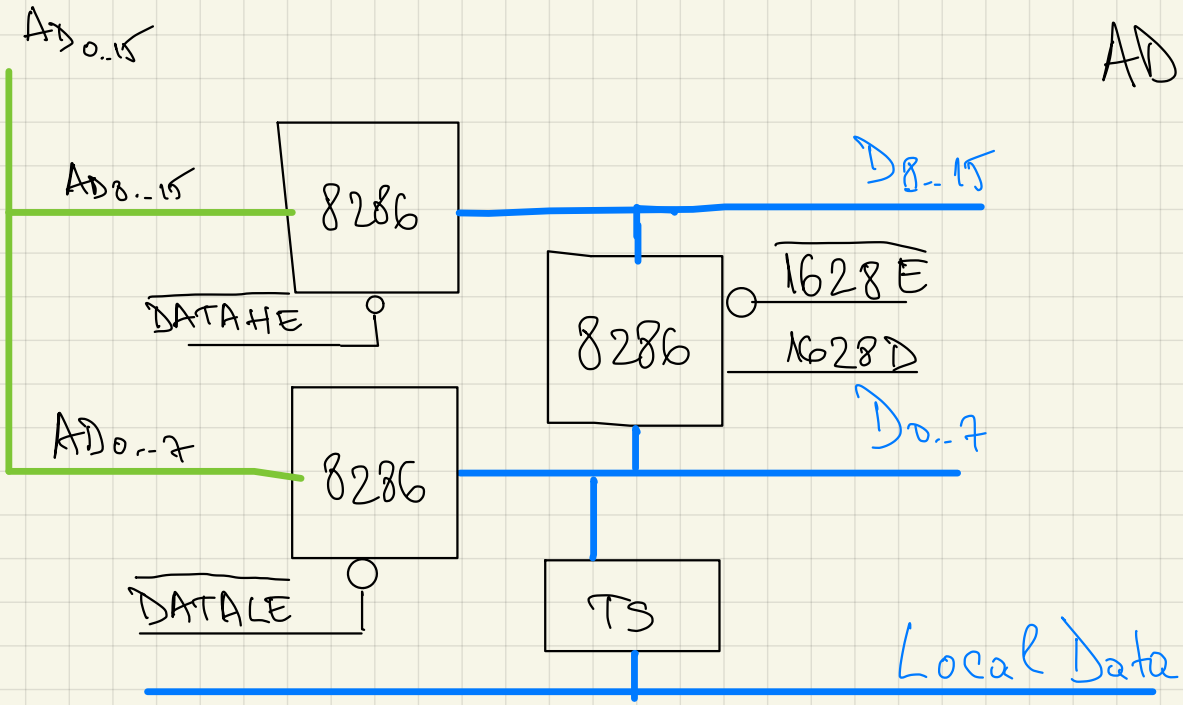
$$1628D = \overline{MEMW} + \overline{IOR}$$

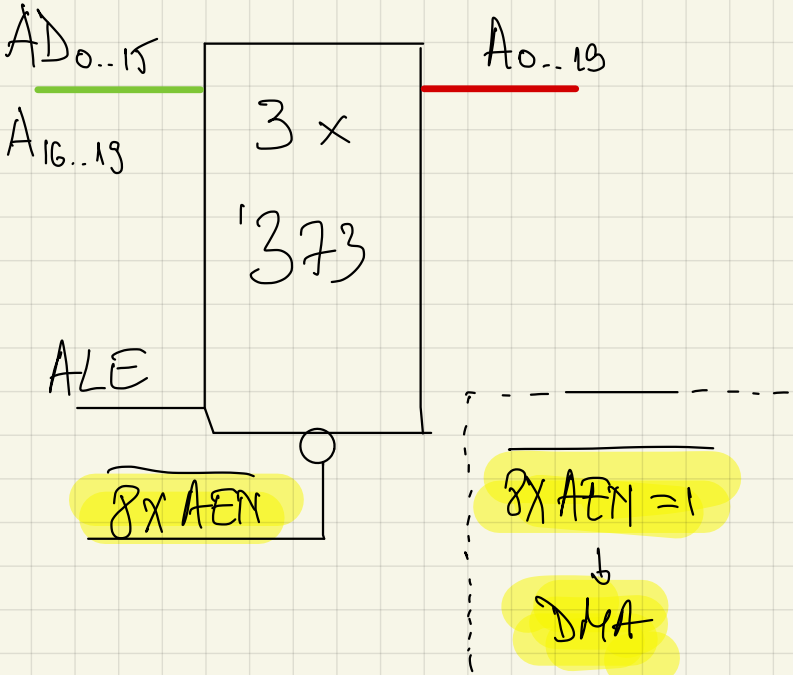
$$\overline{DATAHE} = \overline{DEN} + FREERUN$$

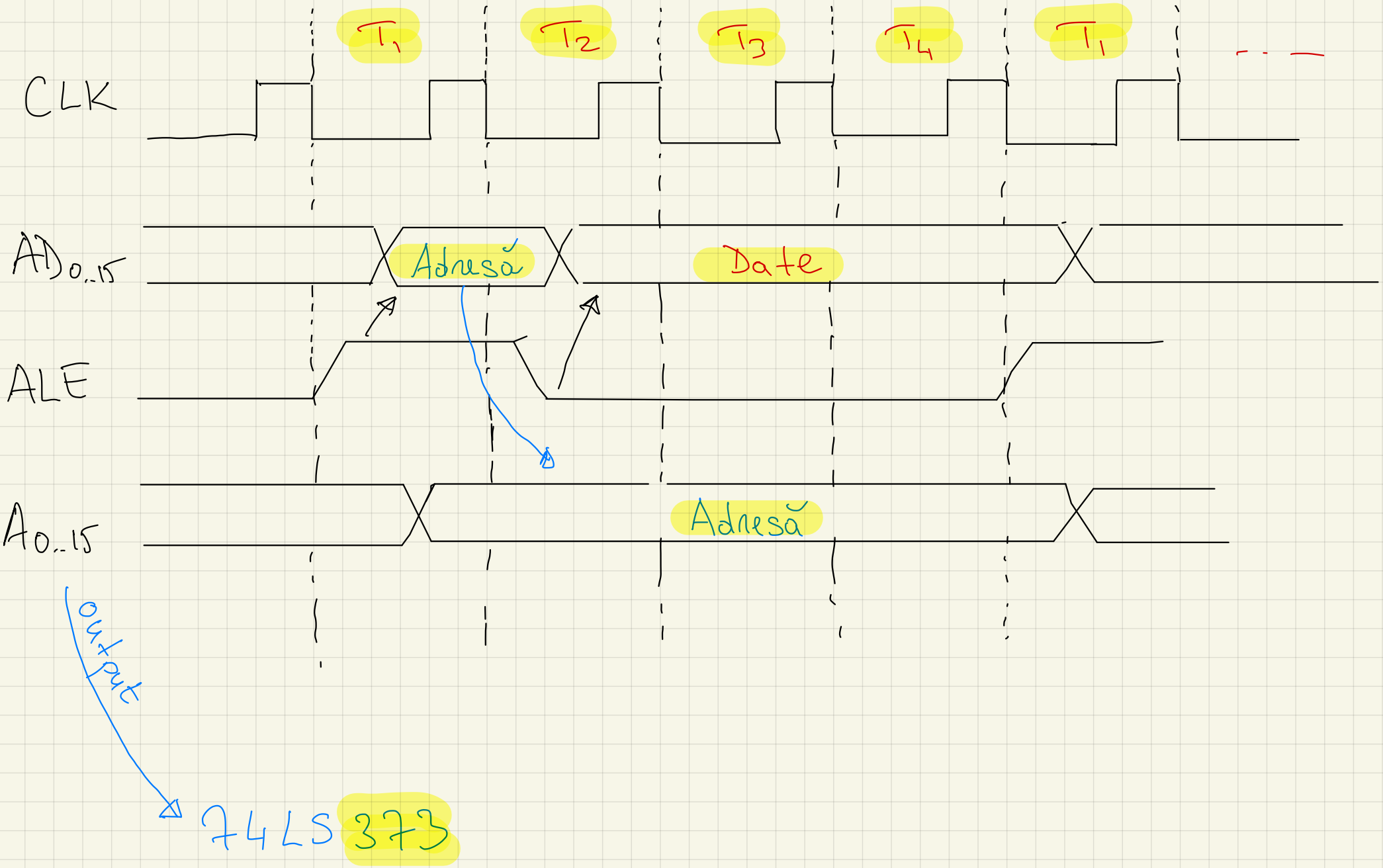
$$\overline{DATALE} = \overline{DEN} + FREERUN + 1628E$$

$$AD_1 = \overline{DEN} + \overline{FREERUN}$$

$$AD_9 = DEN + FREERUN$$



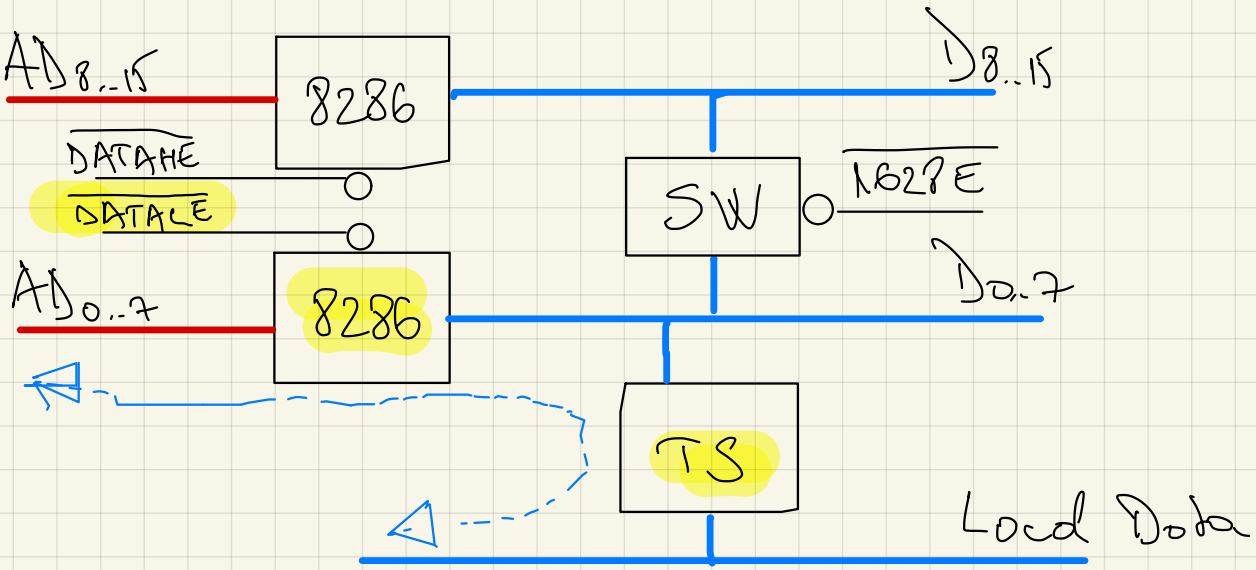



T1
T2
T3
T4
T1
CLK
AD0..15
Adresă
Date
ALE
A0..15
Adresă
output
74LS373

$\overline{BHE}$	A_0	Tip Date
0	0	Date cuvânt $D_{0..15}$ adresă pară
0	1	Date $D_{8..15}$ adresă impară
1	0	Date $D_{0..7}$ adresă pară
1	1	

1. Ciclul recunoaștere întrerupere

$\overline{INTA} = 0$ ($\overline{S_2} = X$ $\overline{BHE} = X$ $A_0 = X$)



$\overline{DATAHE} = 1$

$\overline{DATALE} = 0$

$\overline{BHE} = 1$

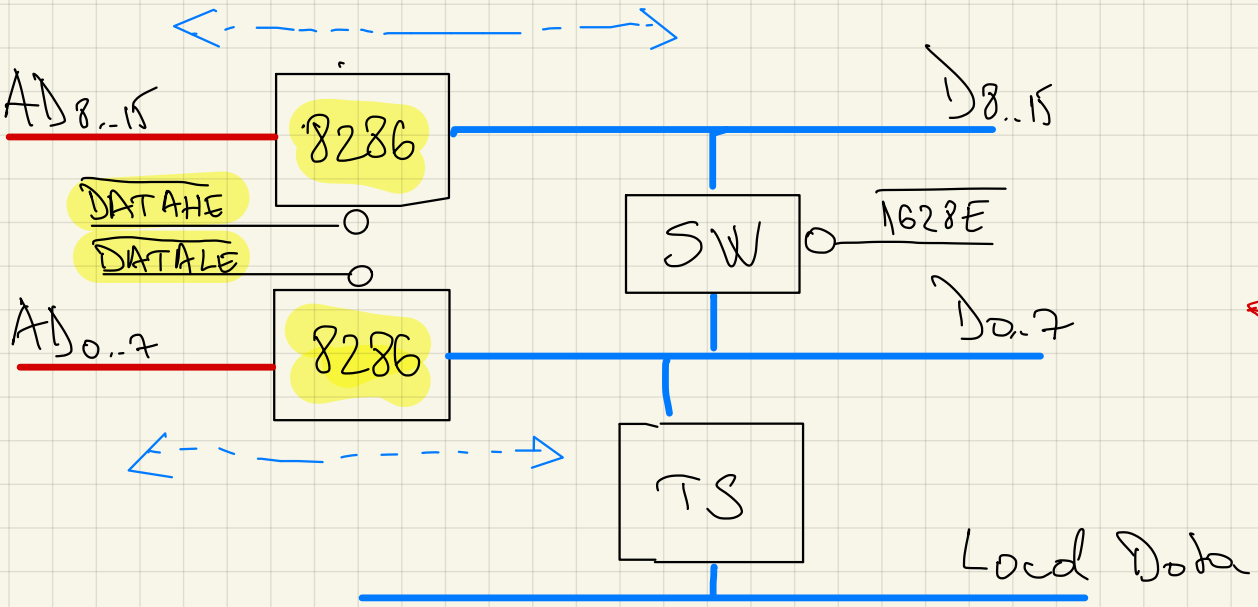
$A_0 = 0$

$\overline{1628E} = 1$

2. Ciclul transfer date

Port i/O $\overline{S_2} = 0$

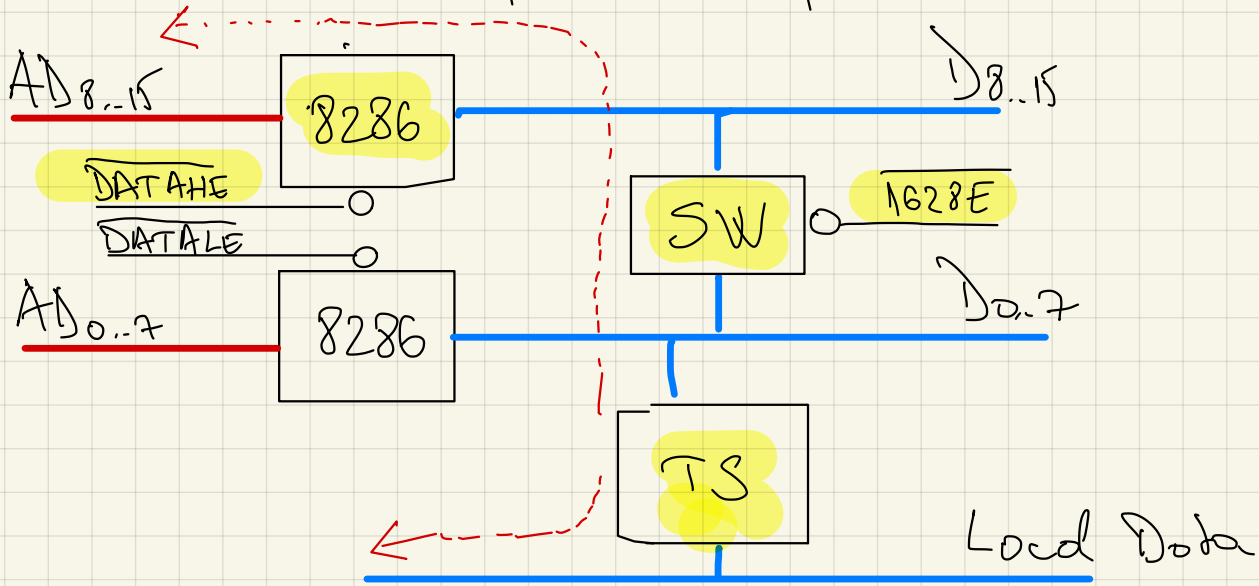
Memorie $\overline{S_2} = 1$



a) transfer i/O pe 16 biti : $\overline{DATAHE} \Rightarrow$ $\overline{DATALE} \Rightarrow$ $\overline{BHE} \Rightarrow$ $A_0 = 0$

$\overline{1628E} = 1$ (inactiv)

b) transfer date i/o pe 8 biti pe $D_{8:15}$



$\overline{DATAHE} = 0$

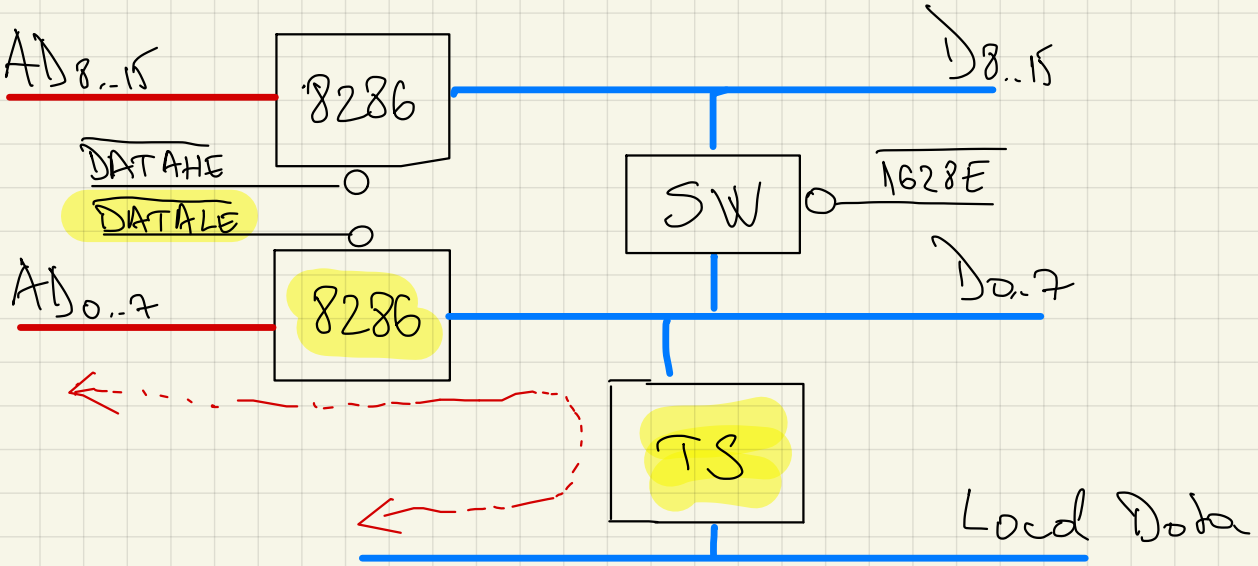
$\overline{DATALE} = 1$

$\overline{BHE} = 0$

$A_0 = 1$

$\overline{1628E} = 0$

c) transfer date i/o pe 8 biti pe $D_{0..7}$



$\overline{DATAHE} = 1$

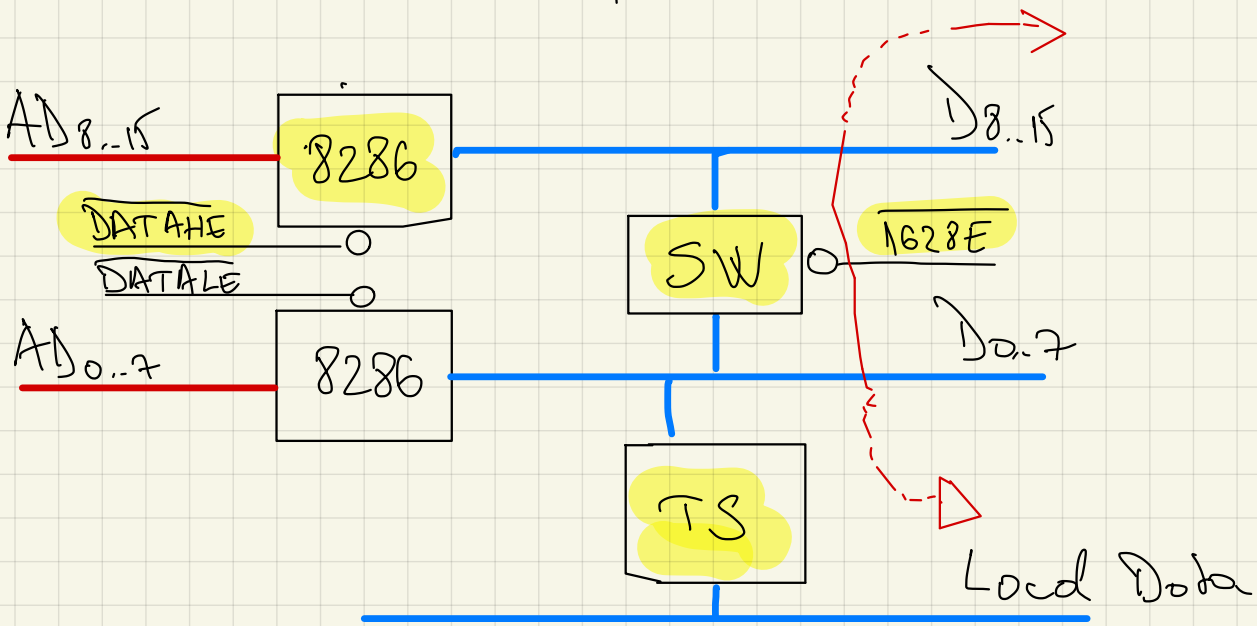
$\overline{DATALE} = 0$

$\overline{BHE} = 1$

$A_0 = 0$

$\overline{1628E} = 0$

d) transfer date i/o pe 8 biti controlat de alt Master



$\overline{DATAHE} = 0$

$\overline{DATALE} = 1$

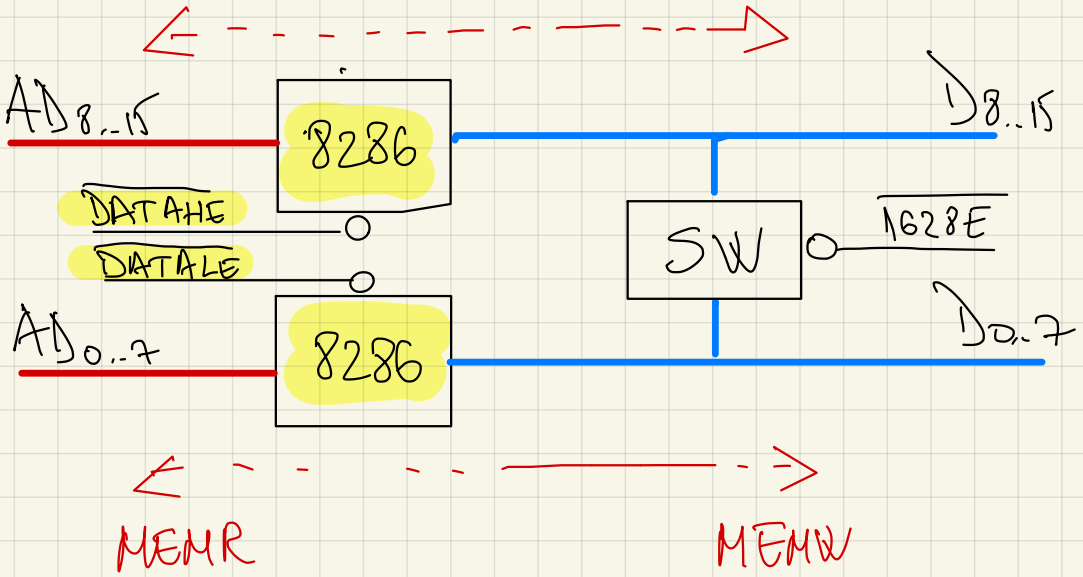
$\overline{BHE} = 1$

$A_0 = 1$

$\overline{1628E} = 0$

3. Cicli transfer cu memoria

a) transfer date cu memoria pe 16 biți



$\overline{DATAHE} = 0$

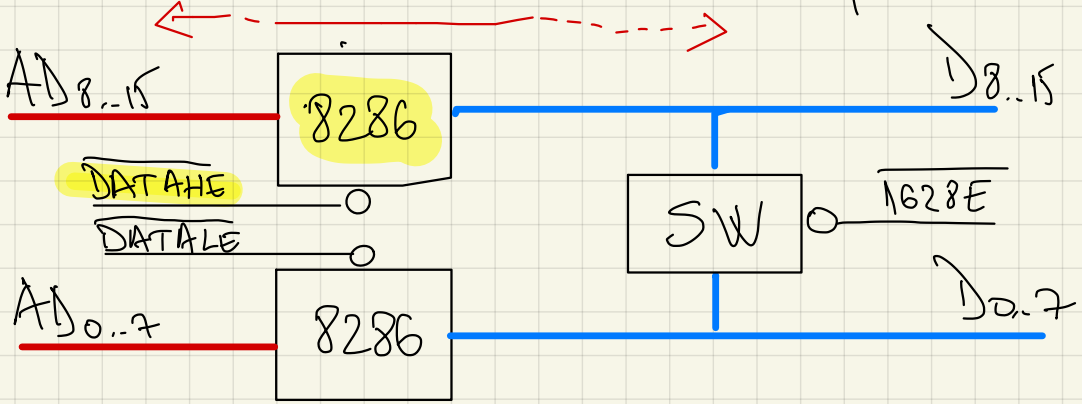
$\overline{DATALE} = 0$

$\overline{BHE} = 0$

$A_0 = 0$

$\overline{1628E} = 1$

b) transfer date cu mem. pe $D_{8..15}$



$\overline{DATAHE} = 0$

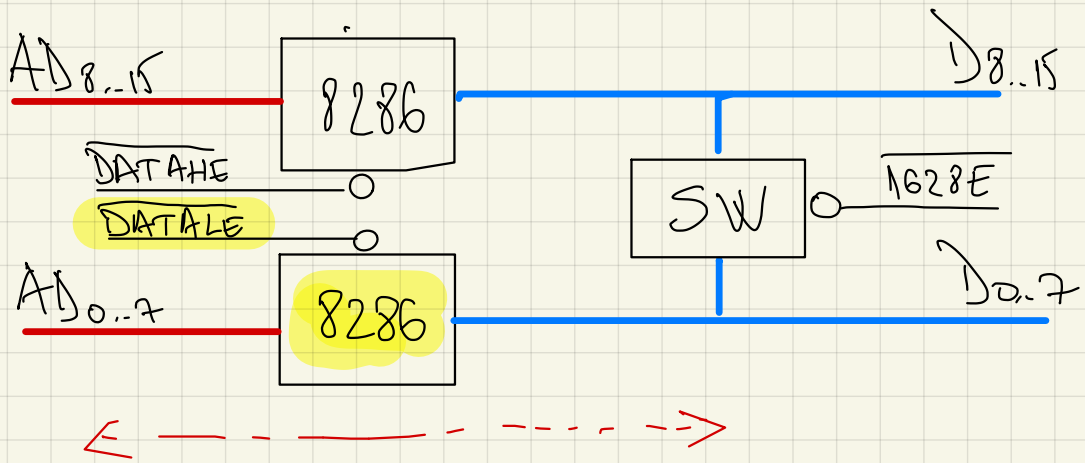
$\overline{DATALE} = 1$

$\overline{BHE} = 0$

$A_0 = 1$

$\overline{1628E} = 1$

c) transfer cu memoria pe $D_{0..7}$



$\overline{DATAHE} = 1$

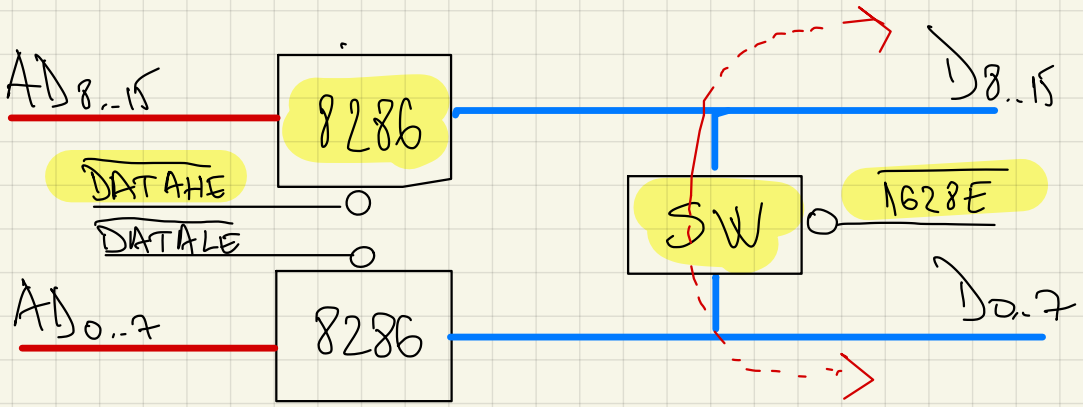
$\overline{DATALE} = 0$

$\overline{BHE} = 1$

$A_0 = 0$

$\overline{1628E} = 1$

d) transfer cu memoria controlat de alt master (pe 8 biți)



$\overline{DATAHE} = 0$

$\overline{DATALE} = 1$

$\overline{BHE} = 1$

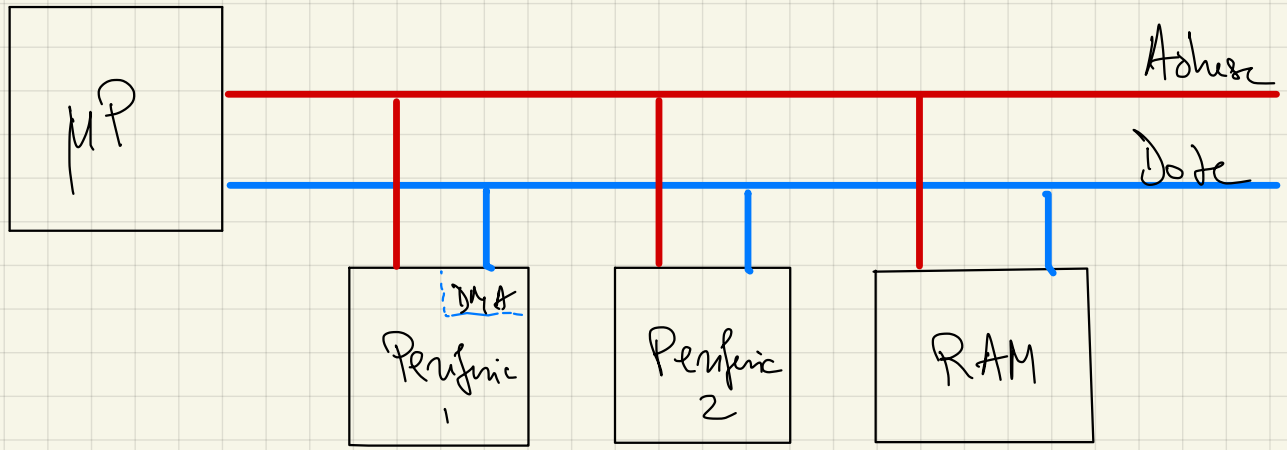
$A_0 = 1$

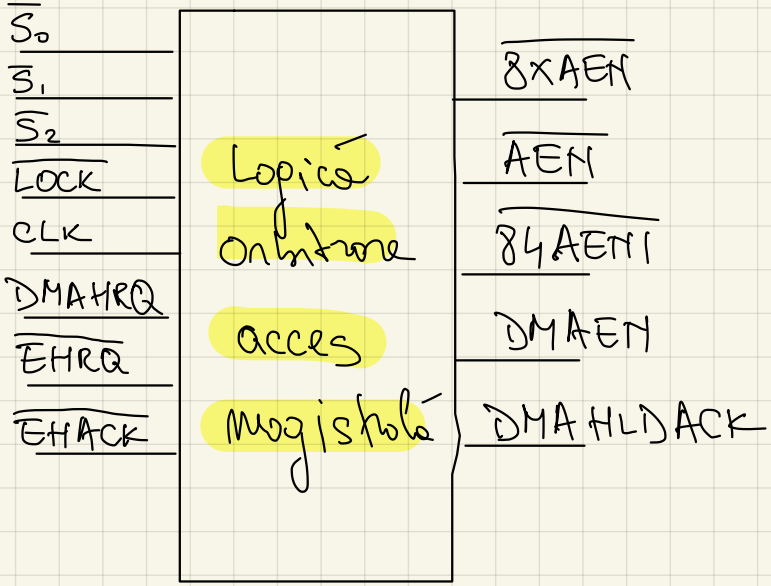
$\overline{1628E} = 0$

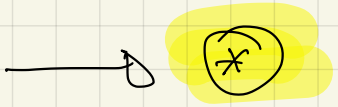
PM - Cursul 7

Notițe de curs

Logică acces între MP și DMA







Cicli mașină procesor

$\overline{S_{0..2}}$ – codifică starea curentă

$\overline{T_1}$ – starea ocupată

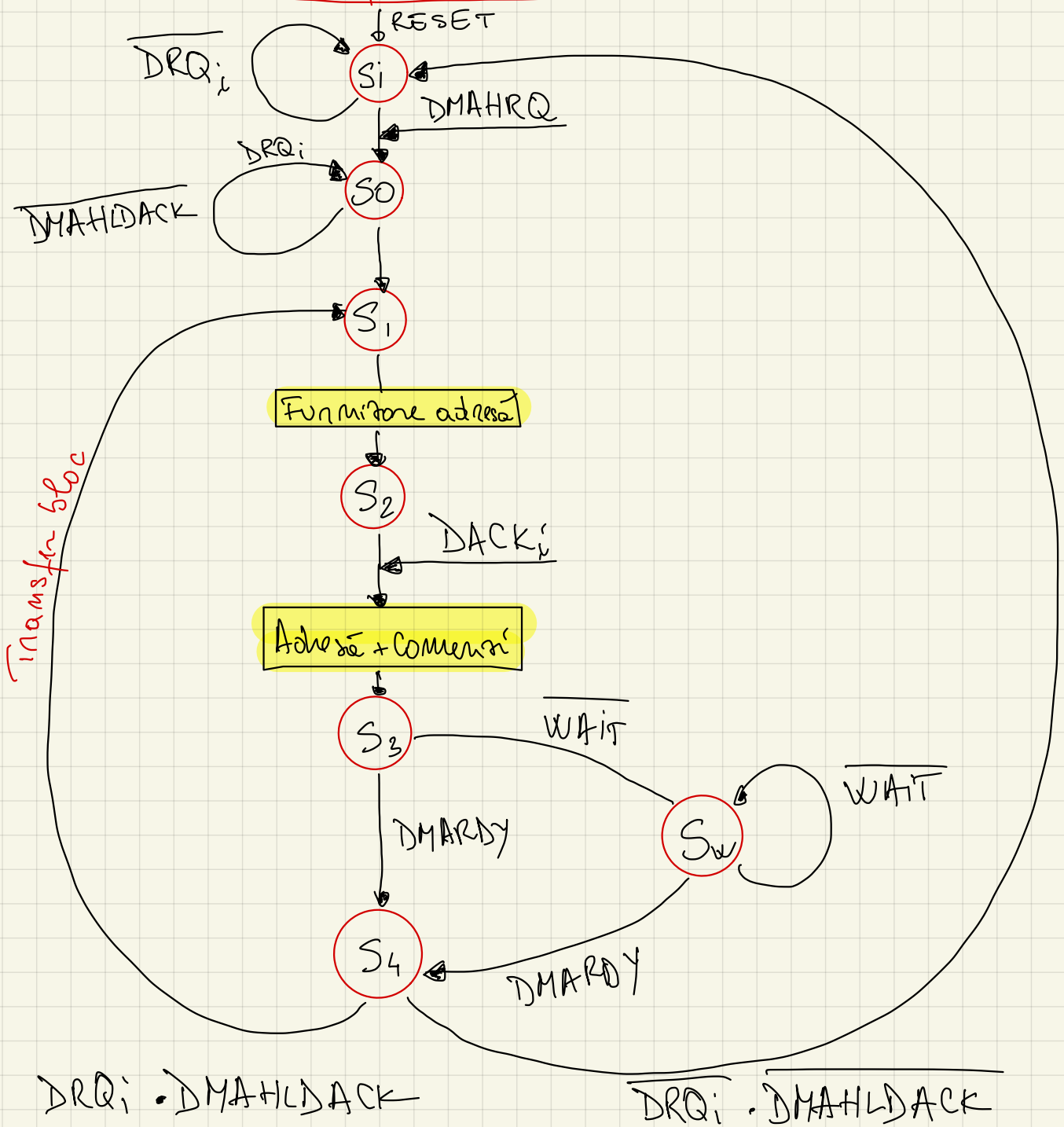
$\overline{T_2}$ – comută direcția magistralei de date

$\overline{T_3}$ – începe transfer

$\overline{T_w}$ – stare de așteptare

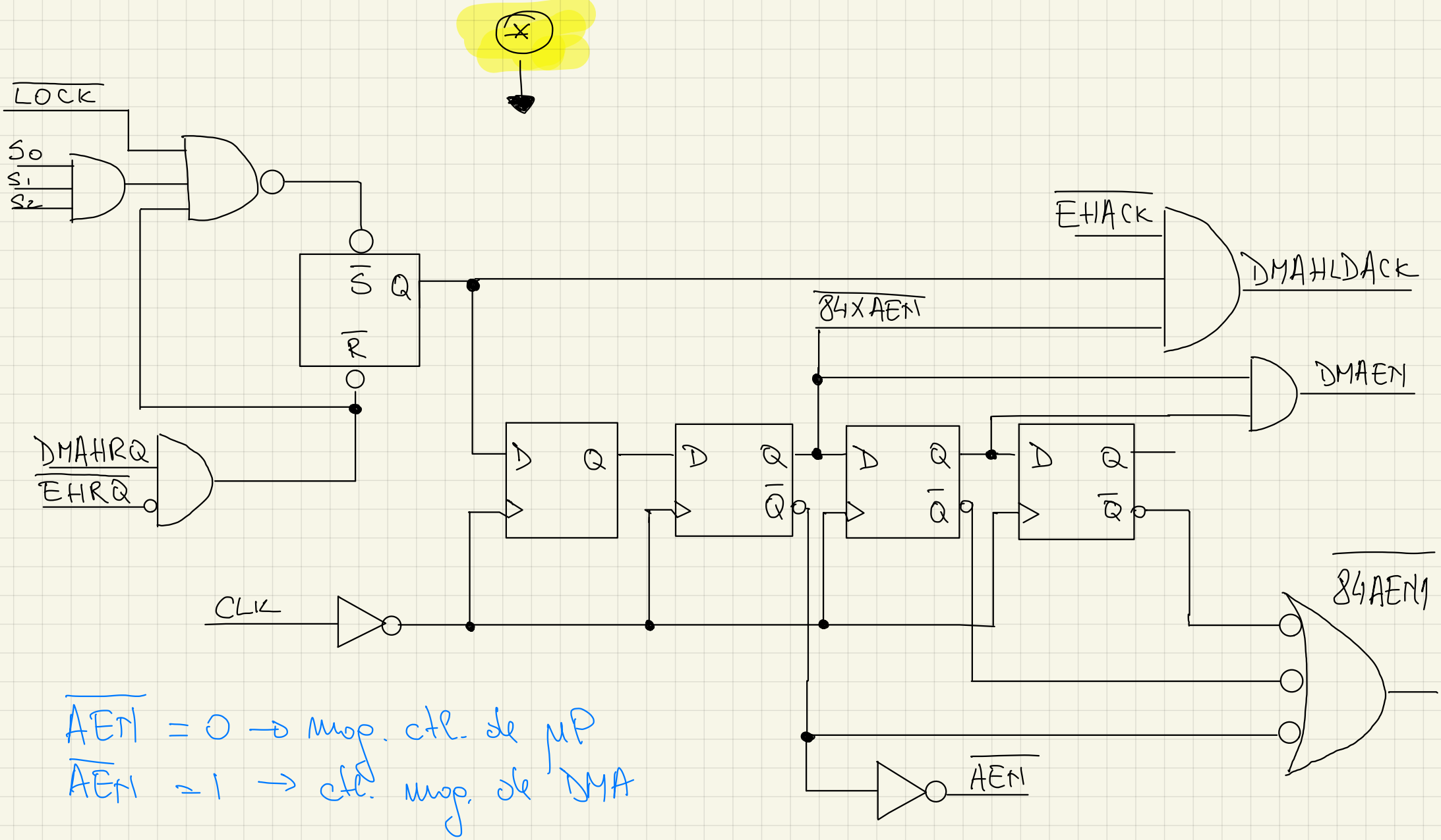
$\overline{T_4}$ – finalizare transfer

Automatul pt arbitrarea accesului DMA



MP stări pornire:

1. $\overline{S_{0..2}}$: 1, 1, 1
2. $\overline{LOCK}$: 1
3. $\overline{EHRQ}$: 1



$\overline{AEN} = 0 \rightarrow$ mop. ctl. de μP

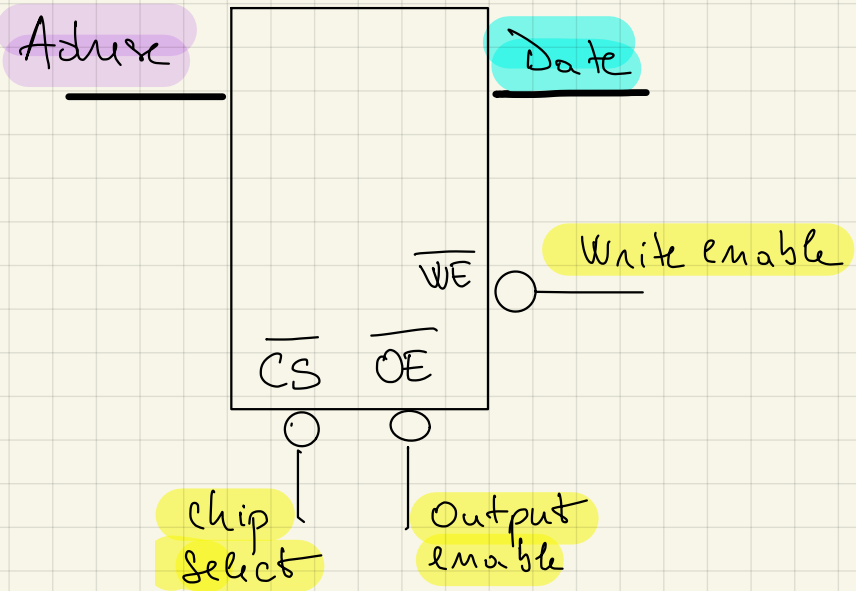
$\overline{AEN} = 1 \rightarrow$ ctl. mop. de DMA

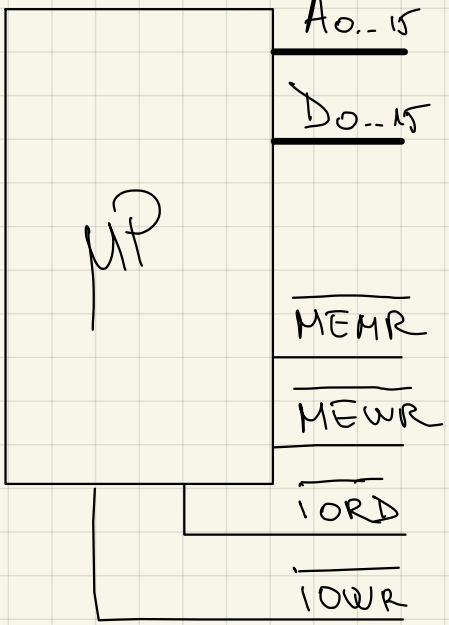
PM - Cursul 8

Notițe curs

Interfațarea memoriilor

- memorie volatilă (RAM) — SRAM, DRAM
- memorie nevolatilă (ROM) — EEPROM, Flash





2^{16} loc. mem.

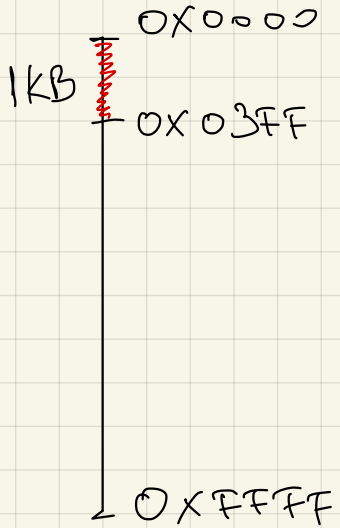
$2^{10} = 1024 = 1K$

$2^{16} = 2^{10} \cdot 2^6 = 64K$

Maximul sp. de mem este 64K x 16 biți

Conectăm un EEPROM de 1KB (1Kx8) la începutul sp. de. mem.

A_{15} A_{14} ...	... A_1	A_0	
0 0 ...	0	0	0X0000
...	0	1	0X0001
...	...	...	...
1 1 ...	1	1	0XFFFF



A_{15} -- A_{10} A_9 - - -- A_2 A_1 A_0

0 -- 0 0 --- 0 0 0

0 --- 0 1 1 1 1

0x0000 - prima adresa

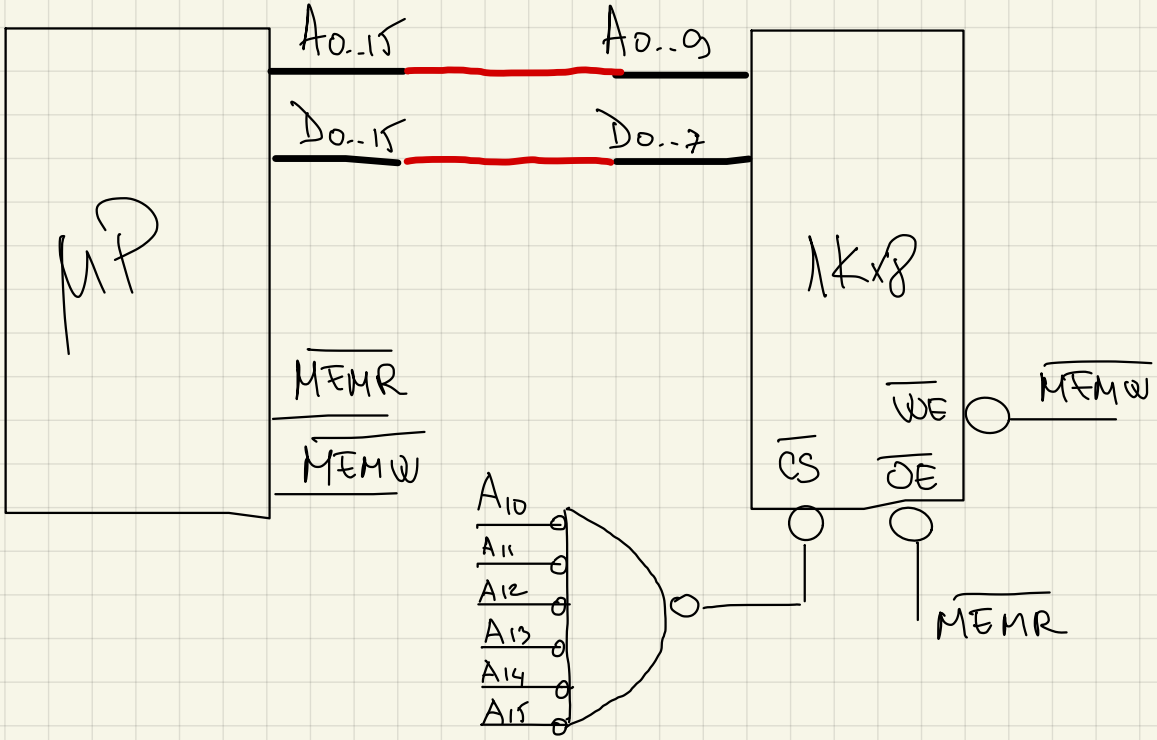
0x03FF - ultima adresa

A_{15} A_{14} A_{13} A_{12} A_{11} A_{10} A_9 A_8 A_7 ... A_0

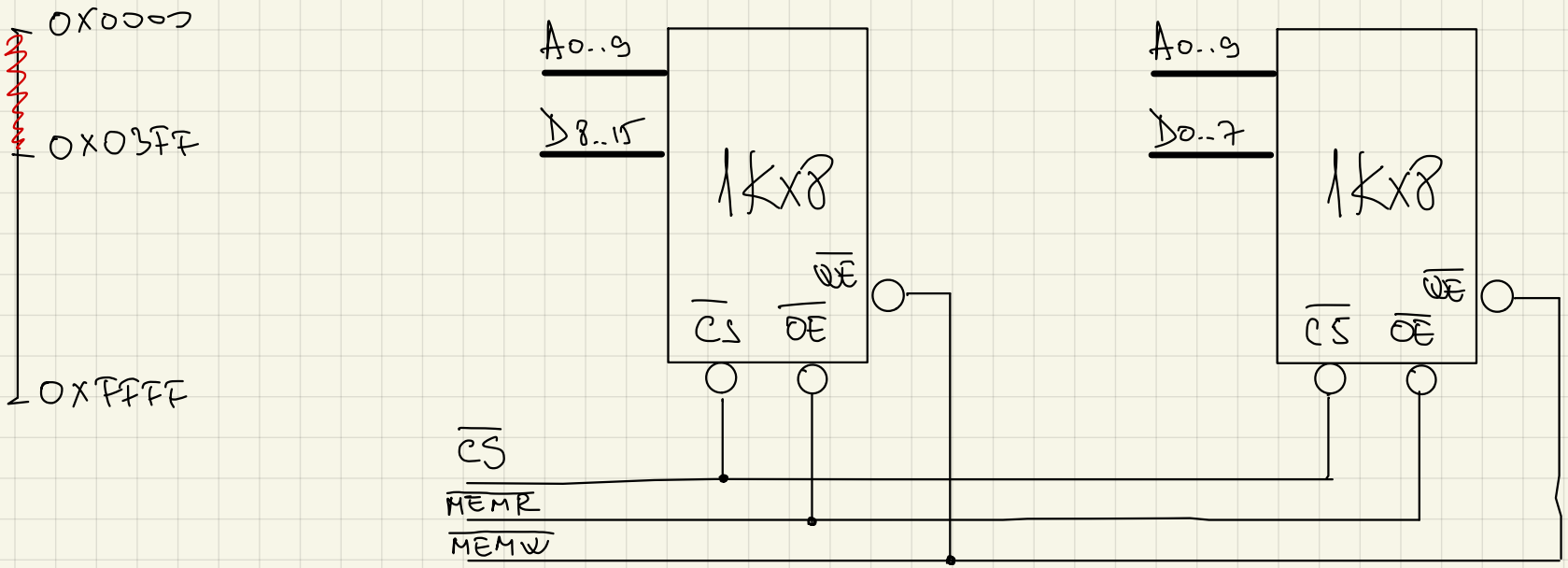
0 0 0 0 0 0 1 1 1 ---- 1

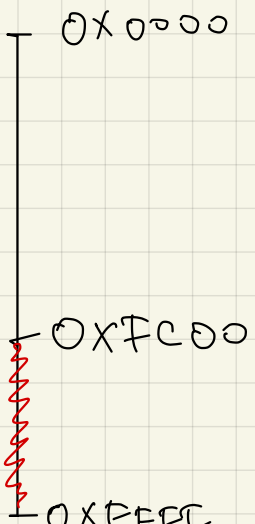
0 3 F F

$$CS = \overline{A_{10}} \cdot \overline{A_{11}} \cdot \overline{A_{12}} \cdot \overline{A_{13}} \cdot \overline{A_{14}} \cdot \overline{A_{15}}$$

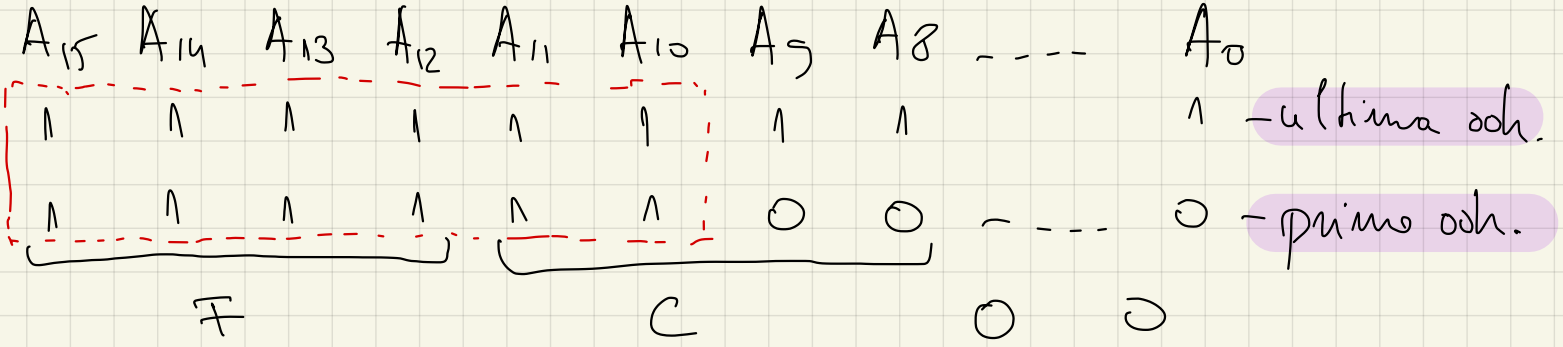


1K x 16 biti → 2 x (1K x 8) la începutul sp. de adrese





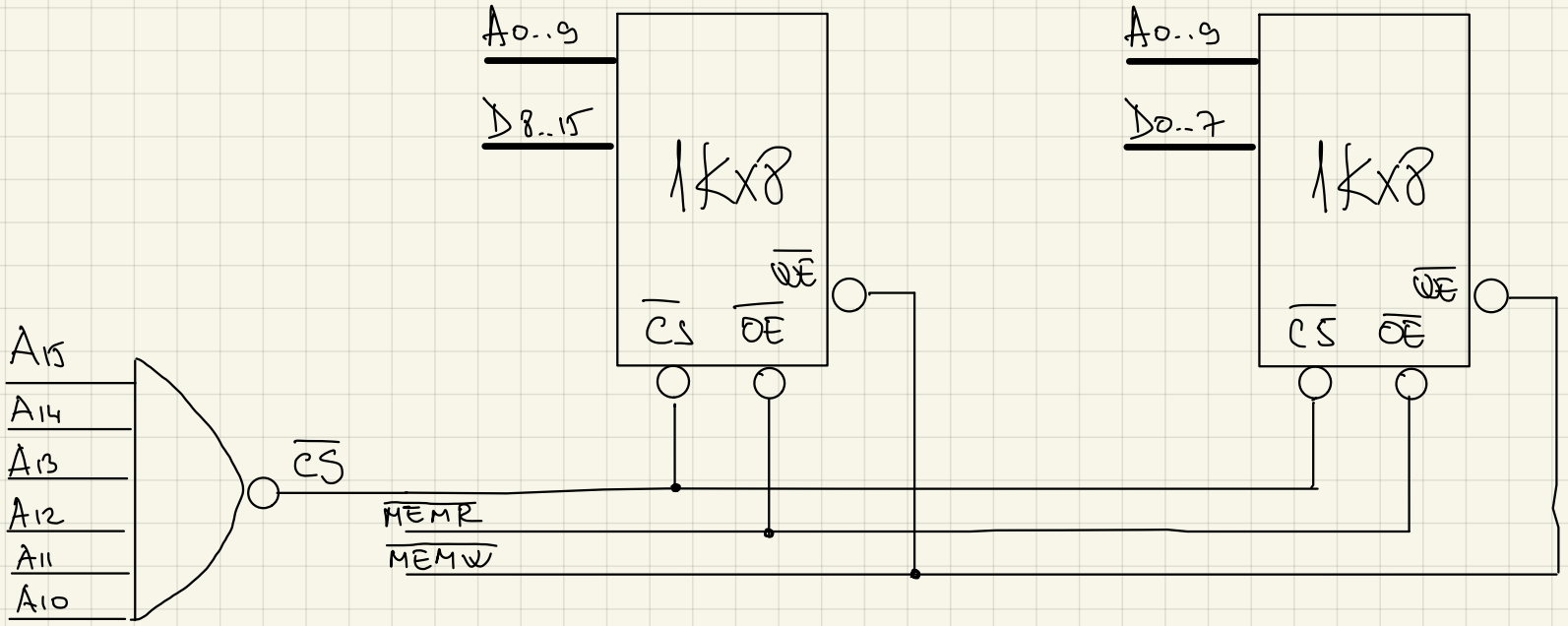
1K x 16 la sfârșitul spațiului de memorie



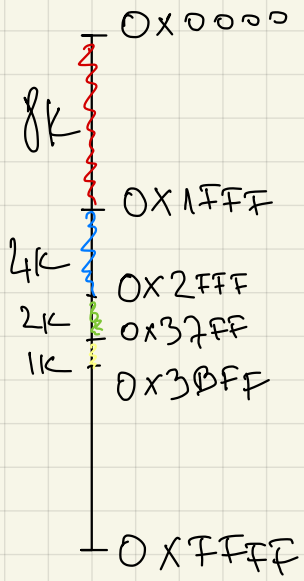
$CS = A_{15} \cdot A_{14} \cdot A_{13} \cdot A_{12} \cdot A_{11} \cdot A_{10}$

1Kx16 la sfârșitul spațiului de memorie

Schema de conectare:



15K x 16 - avem chips de 8K, 4K, 2K, 1K x 8 biti, mapare la inceputul spatiului de memorie



$8K \rightarrow 2^{13} = \underbrace{2^{10}}_{1K} \cdot \underbrace{2^{3}}_{8} \rightarrow A_{0..12}$

$4K \rightarrow 2^{12} = 2^{10} \cdot 2^{2} \rightarrow A_{0..11}$

$2K \rightarrow 2^{11} = 2^{10} \cdot 2 \rightarrow A_{0..10}$

$1K \rightarrow 2^{10} \rightarrow A_{0..9}$

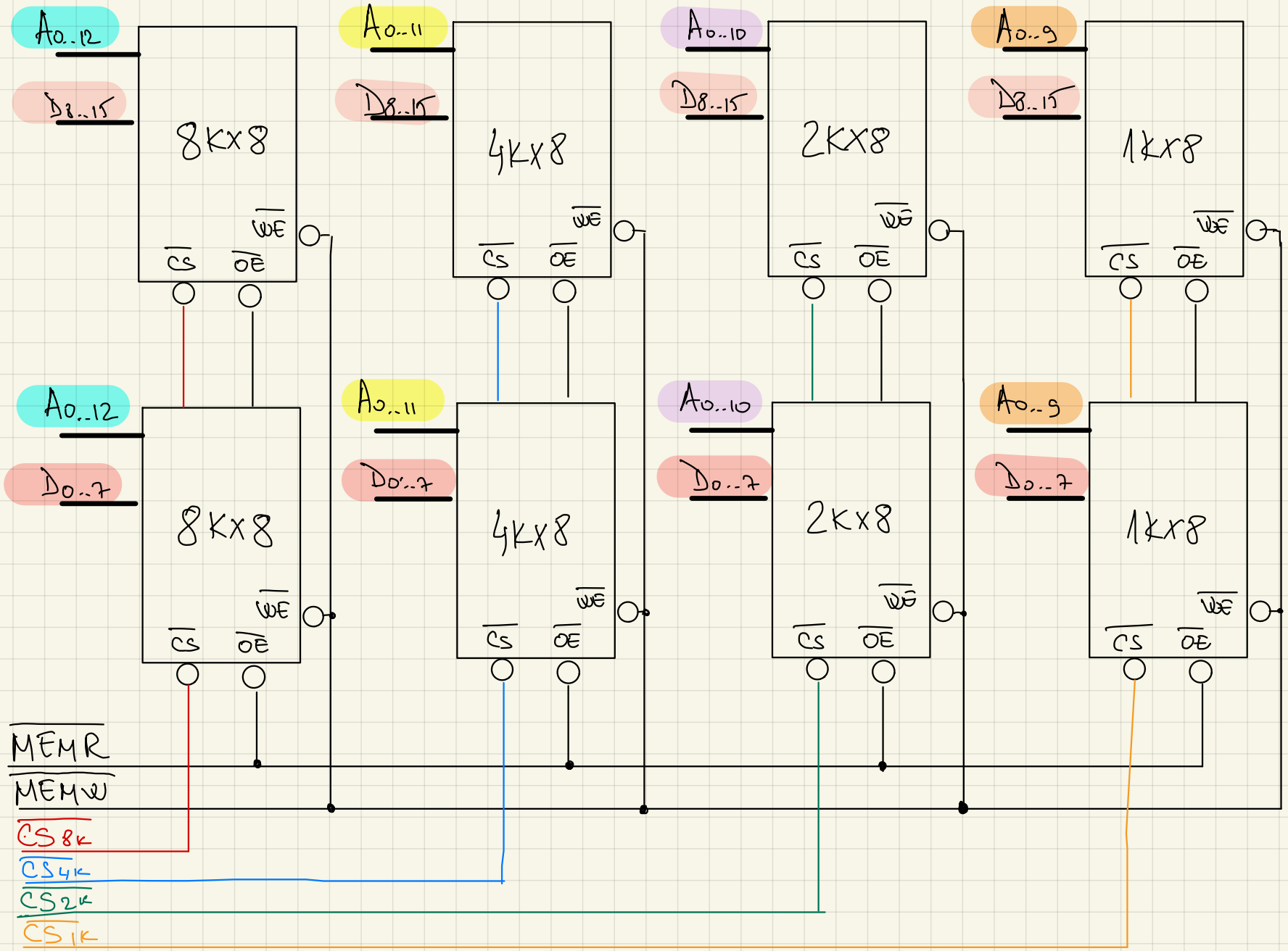
$CS_{8K} = \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}}$

$CS_{4K} = \overline{A_{15}} \cdot \overline{A_{14}} \cdot A_{13} \cdot \overline{A_{12}}$

$CS_{2K} = \overline{A_{15}} \cdot \overline{A_{14}} \cdot A_{13} \cdot A_{12} \cdot \overline{A_{11}}$

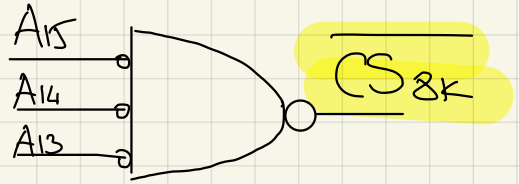
$CS_{1K} = \overline{A_{15}} \cdot \overline{A_{14}} \cdot A_{13} \cdot A_{12} \cdot A_{11} \cdot \overline{A_{10}}$

A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	8K
0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	4K
0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	
0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	2K
0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	1K
0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	

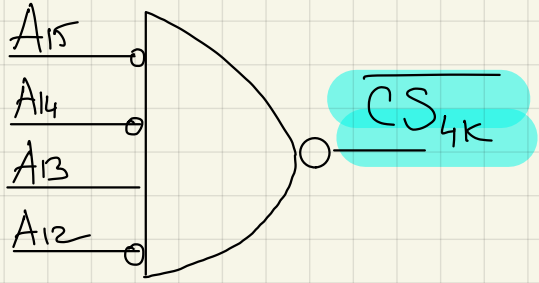

A0..12
D8..15
8Kx8
CS
OE
WE
A0..11
D8..15
4Kx8
A0..10
D8..15
2Kx8
A0..9
D8..15
1Kx8
A0..12
D0..7
8Kx8
A0..11
D0..7
4Kx8
A0..10
D0..7
2Kx8
A0..9
D0..7
1Kx8
MEMR
MEMW
CS 8K
CS 4K
CS 2K
CS 1K

Semmaale chip-select

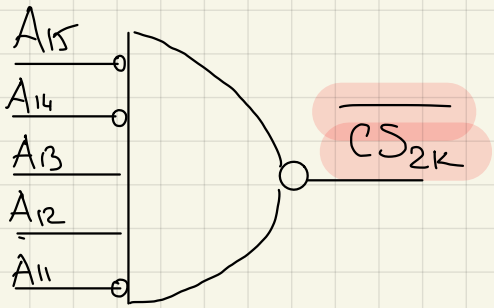
$$\overline{CS_{8K}} = \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}}$$



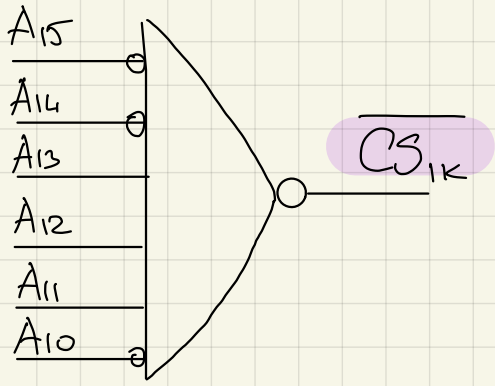
$$\overline{CS_{4K}} = \overline{A_{15}} \cdot \overline{A_{14}} \cdot A_{13} \cdot \overline{A_{12}}$$



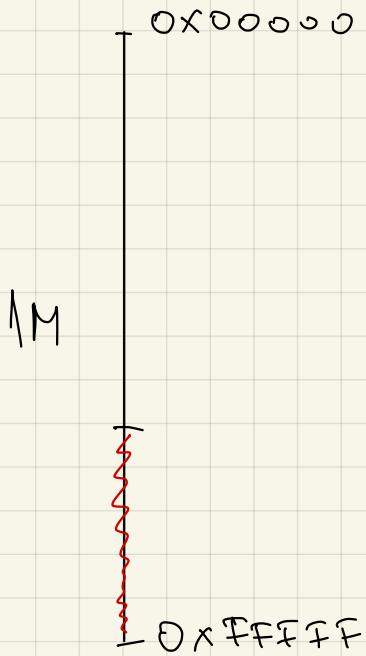
$$\overline{CS_{2K}} = \overline{A_{15}} \cdot \overline{A_{14}} \cdot A_{13} \cdot A_{12} \cdot \overline{A_{11}}$$



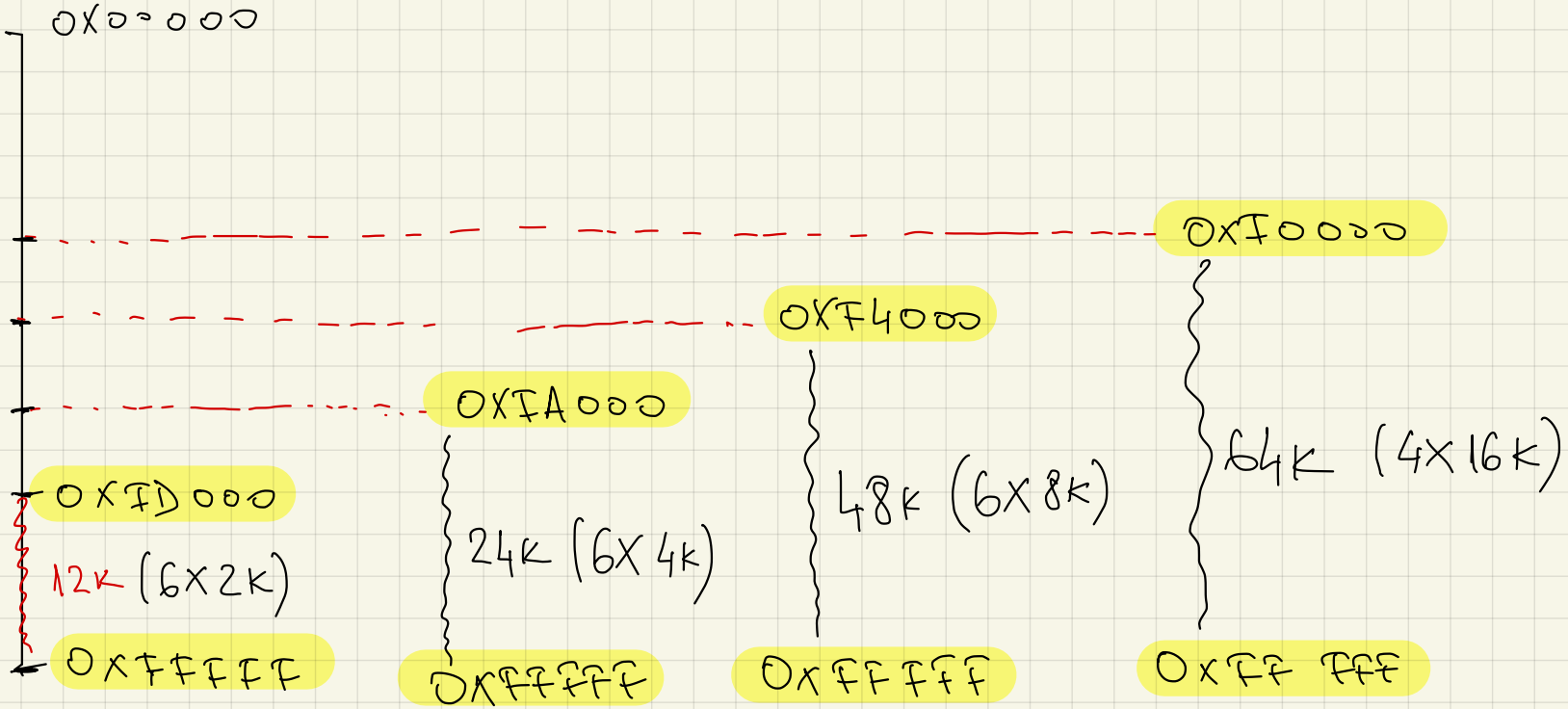
$$\overline{CS_{1K}} = \overline{A_{15}} \cdot \overline{A_{14}} \cdot A_{13} \cdot A_{12} \cdot A_{11} \cdot \overline{A_{10}}$$



8086 → Mapare EEPROM

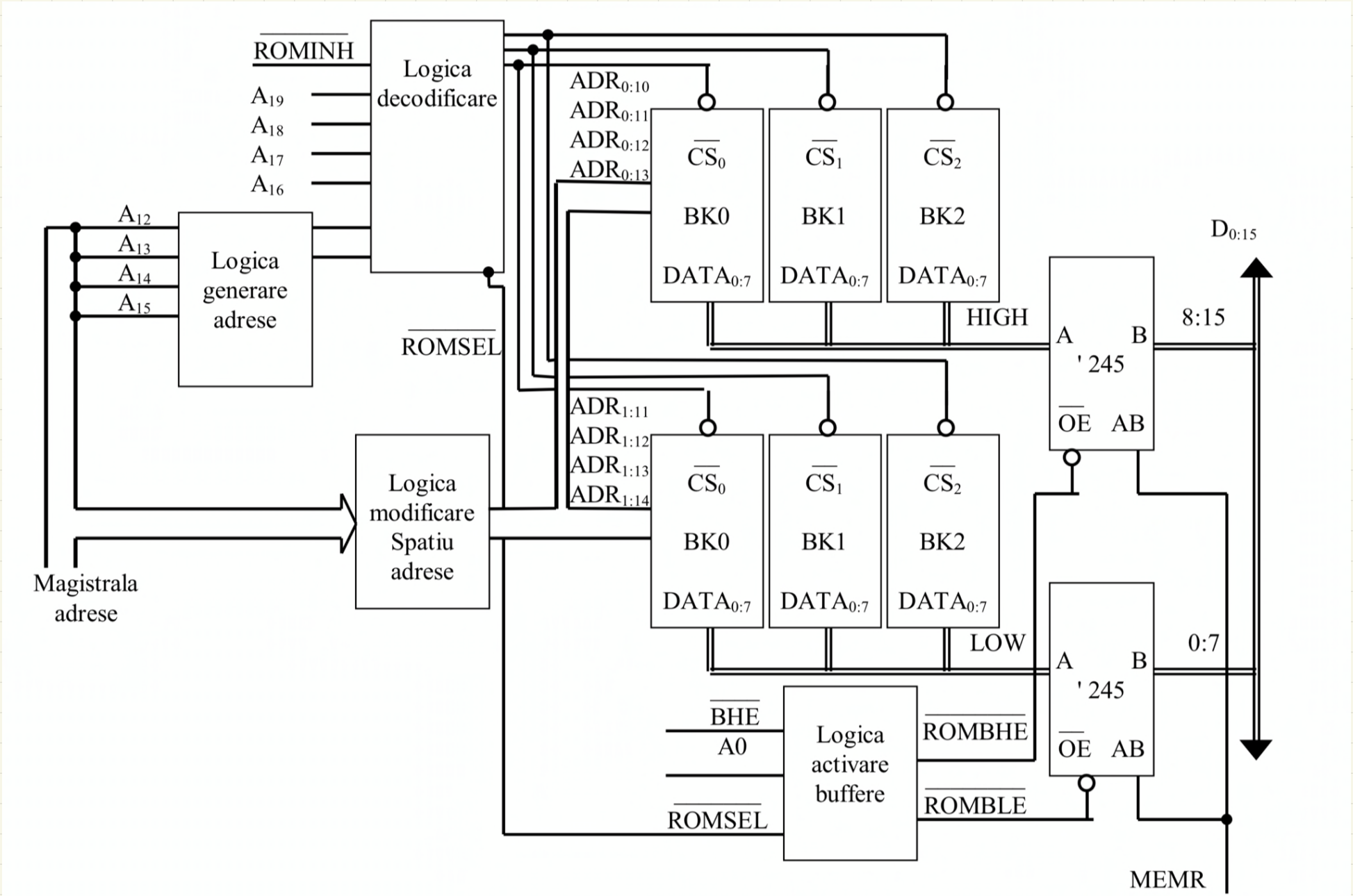


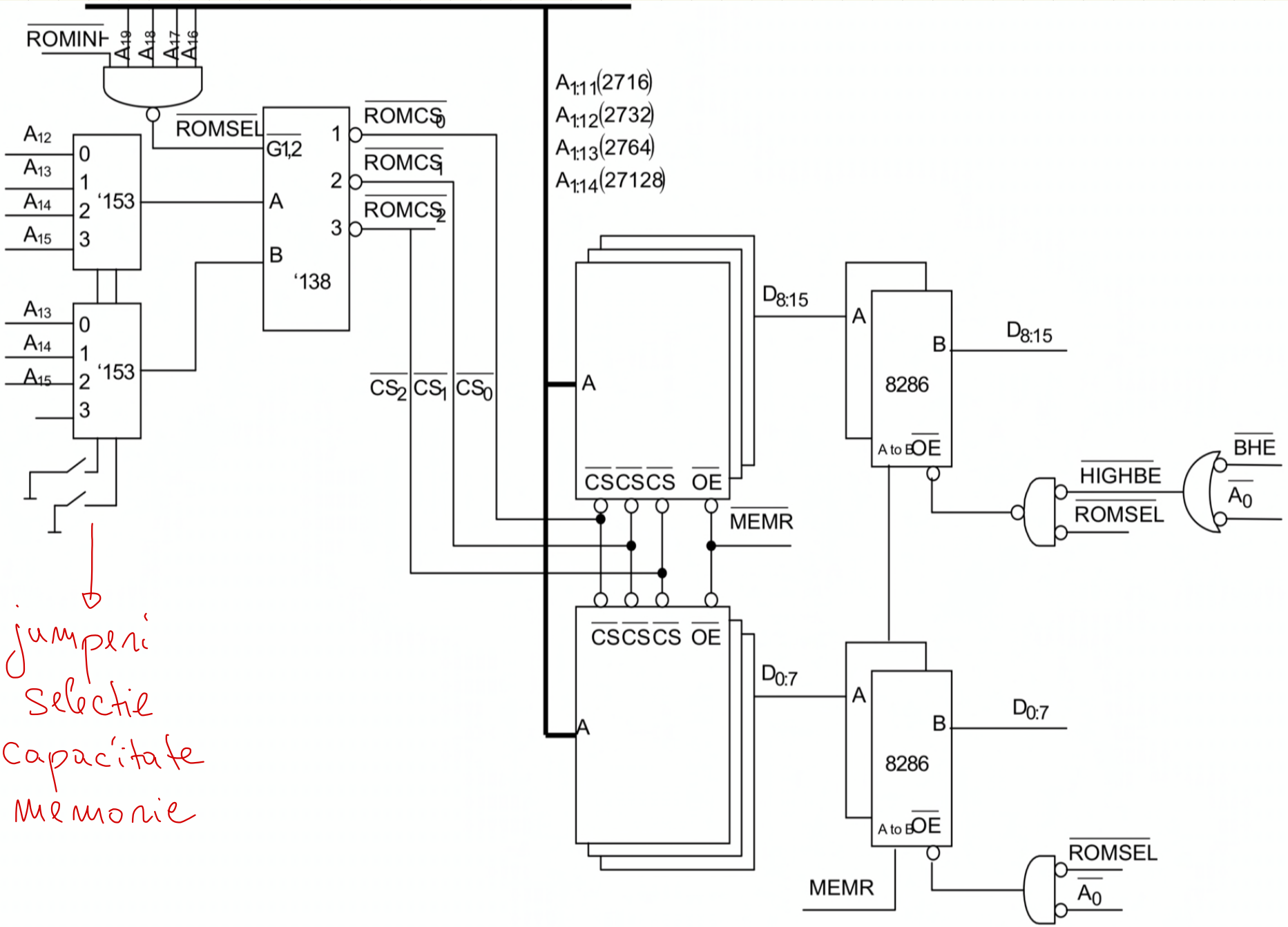
Chip memorie	Capacitate	Spatiu adresă
2716 (2k x 8)	12k	0xFD000 – 0xFFFFF
2732 (4k x 8)	24k	0xFA000 – 0xFFFFF
2764 (8k x 8)	48k	0xF4000 – 0xFFFFF
27128 (16k x 8)	64k	0xF0000 – 0xFFFFF



$\overline{BHE}$	A_0	
0	0	$D_{0..15}$
0	1	$D_{8..15}$
1	0	$D_{0..7}$
1	1	———

	A_{19}	A_{18}	A_{17}	A_{16}	A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
2716	1	1	1	1	X	X	Selectie capsulă		Adresa pt. capsulă											
2732	1	1	1	1	X	Selectie capsulă			Adresă pt. capsulă											
2764	1	1	1	1	Selectie capsulă				Adresă pt. capsulă											
27128	1	1	1	1	Sel. capsulă				Adresă pt. capsulă											


ROMINH
Logica decodificare
A19
A18
A17
A16
A12
A13
A14
A15
Logica generare adrese
ROMSEL
ADR0:10
ADR0:11
ADR0:12
ADR0:13
CS0
CS1
CS2
BK0
BK1
BK2
DATA0:7
HIGH
D0:15
8:15
A
B
' 245
OE
AB
ADR1:11
ADR1:12
ADR1:13
ADR1:14
Logica modificare Spatiu adrese
Magistrala adrese
LOW
0:7
BHE
A0
ROMSEL
Logica activare buffere
ROMBHE
ROMBLE
MEMR

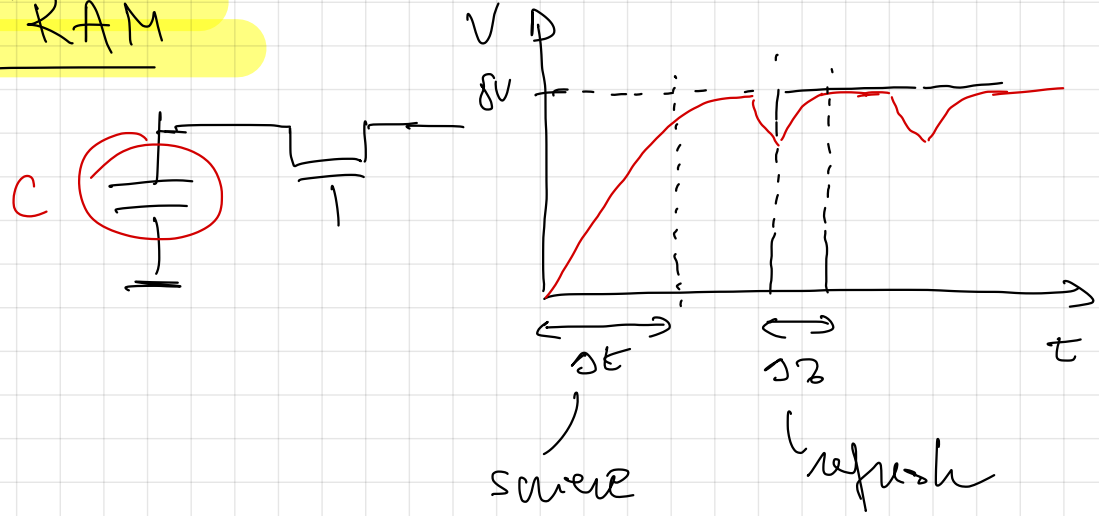

ROMINH
A19
A18
A17
A16
ROMSEL
G1,2
A
B
'138
1
2
3
ROMCS0
ROMCS1
ROMCS2
A12
A13
A14
A15
0
1
2
3
'153
A13
A14
A15
0
1
2
3
'153
A1:11(2716)
A1:12(2732)
A1:13(2764)
A1:14(27128)
CS2 CS1 CS0
A
CS CS CS OE
D8:15
MEMR
A
CS CS CS OE
D0:7
A
B
8286
A to B
OE
D8:15
A
B
8286
A to B
OE
D0:7
HIGHBE
ROMSEL
BHE
A0
MEMR
ROMSEL
A0
jumperi
Selectie
Capacitate
Memorie

PM - Cursul 9

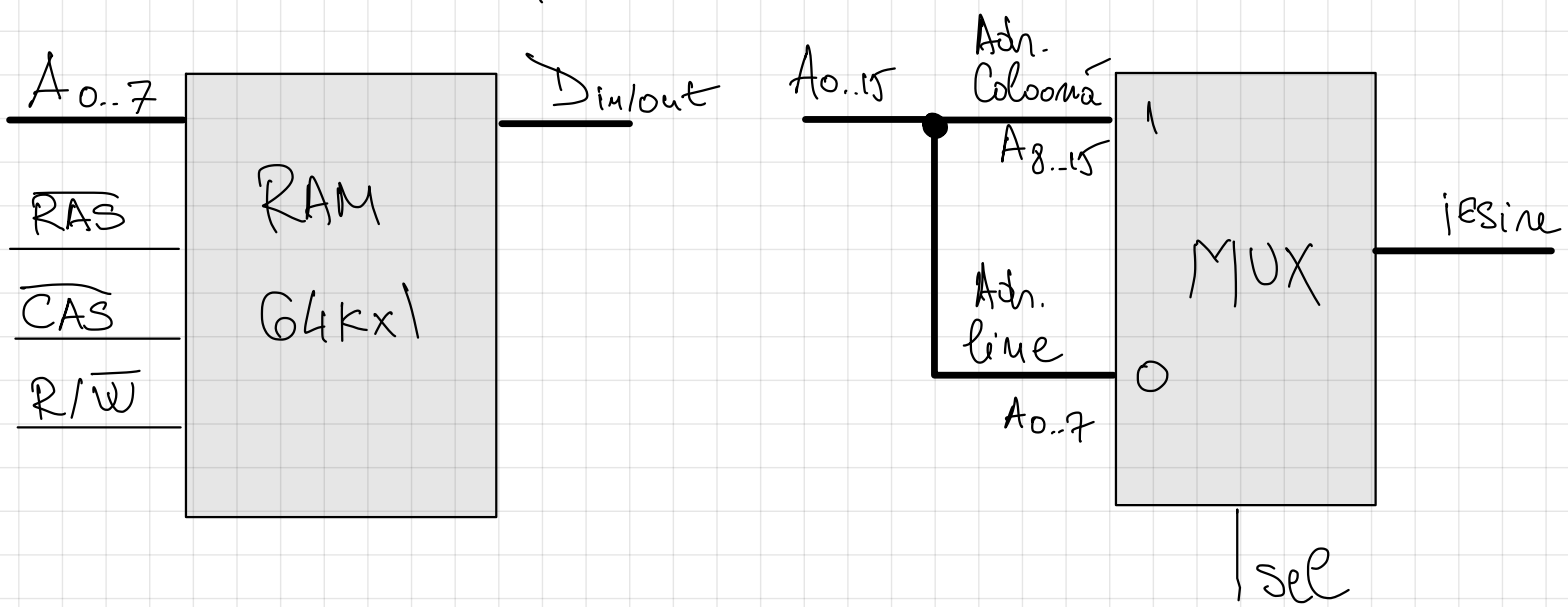
Notițe de curs

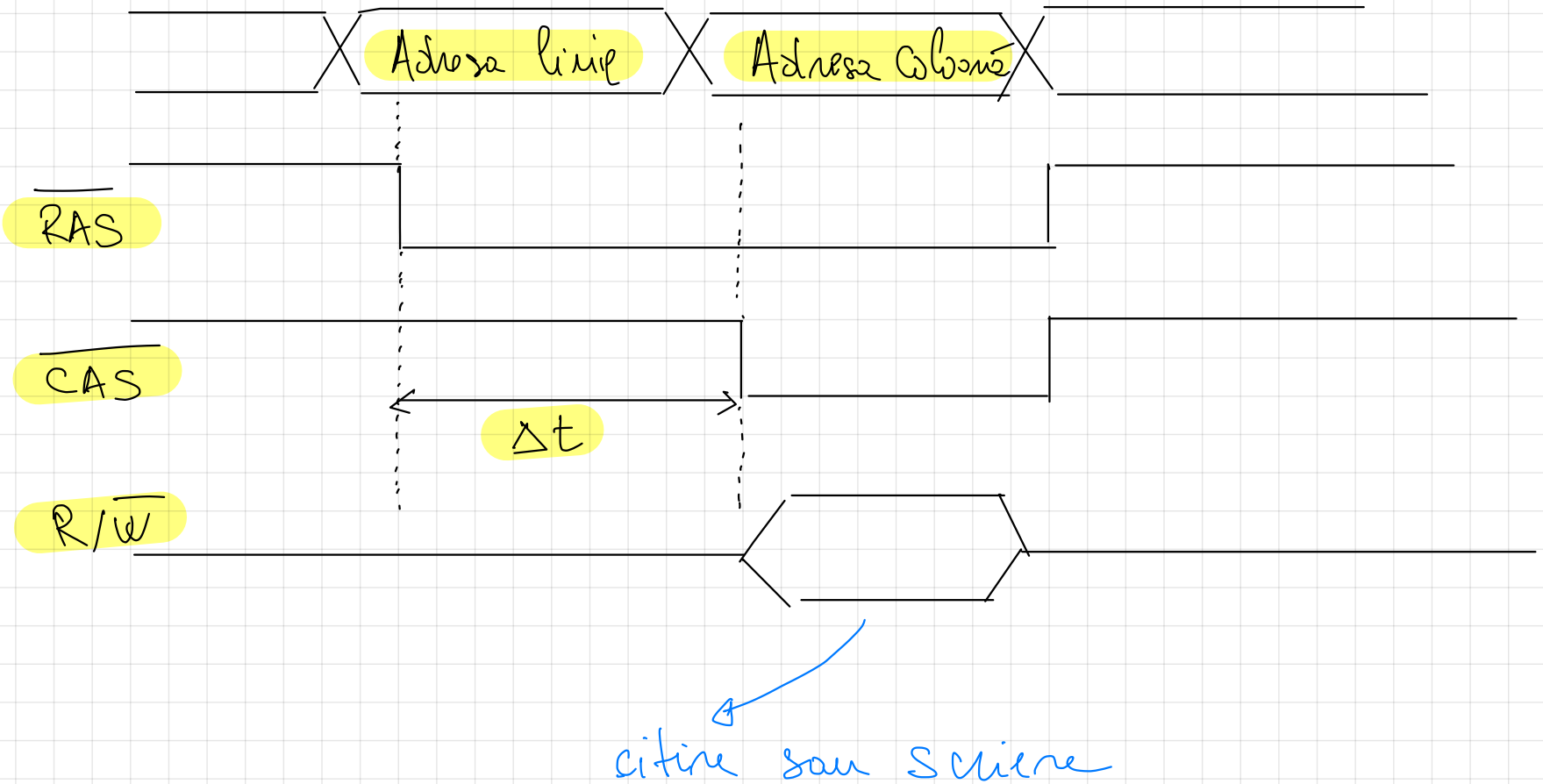
Interfațarea memoriei RAM

- RAM dinamic

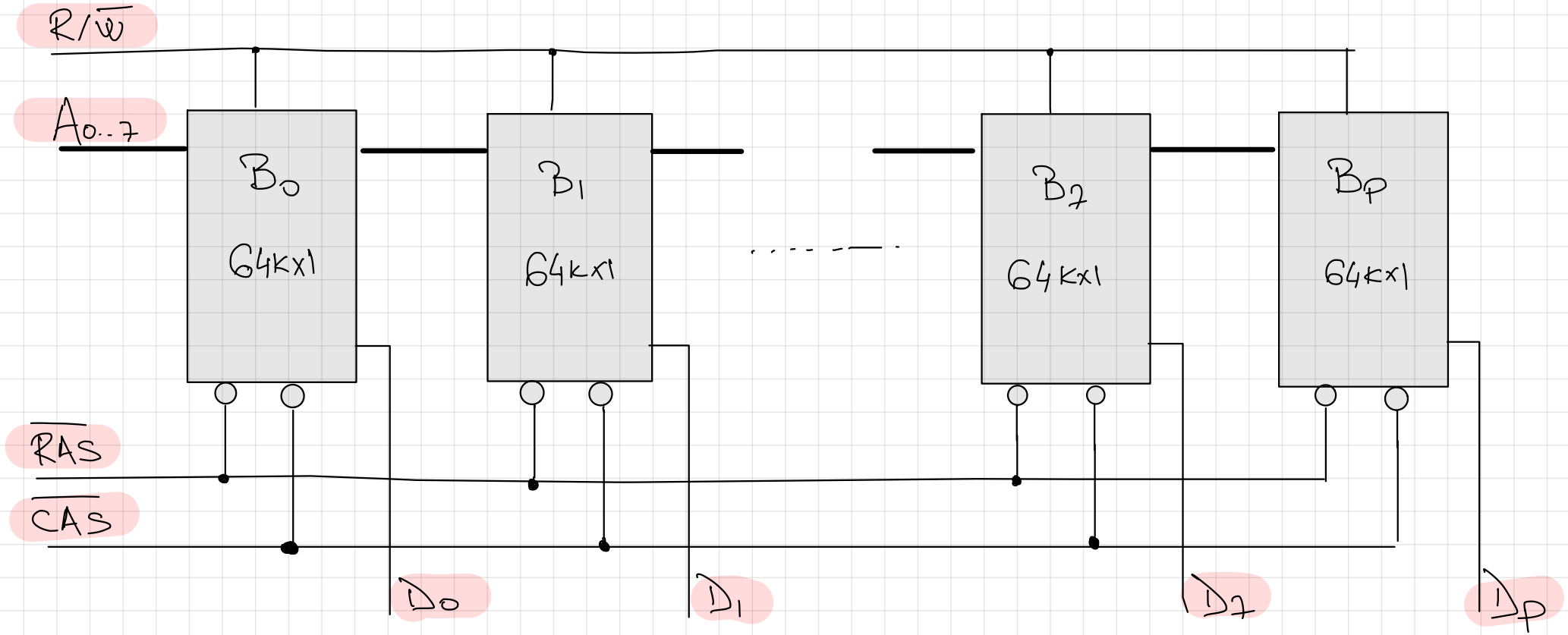


- capacitate 256K → 64K x 1 (4164)
- bit de paritate la nivel de octet (8 biți + 1 bit paritate)
- logică pt. verificarea parității
- Mapare în sp. adresă → la început 00000H ... +256K


A0..7
RAS
CAS
R/W
RAM
64kx1
Din/out
A0..15
Adr. Coloană
A8..15
Adr. linie
A0..7
1
0
MUX
ieșire
sel


Adresa linie
Adresa coloană
RAS
CAS
Δt
R/W
citire sau scriere

64K x 1 – Construim o memorie de (64K x (8+1 BP))

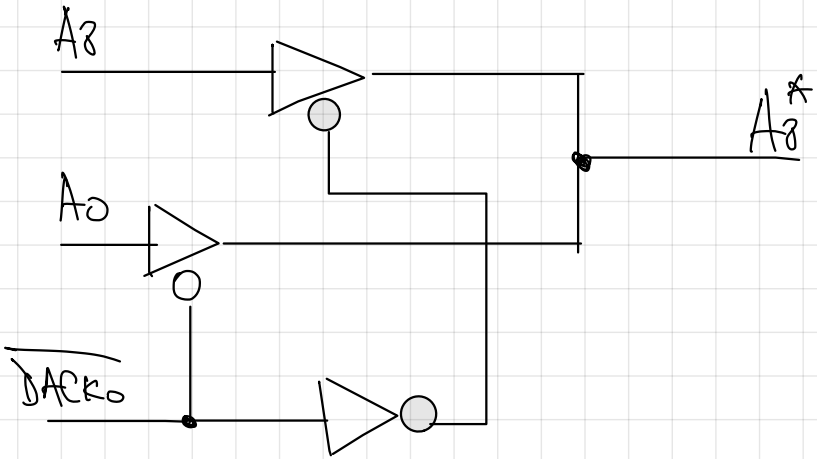


A8 sau A0 → A_8^*

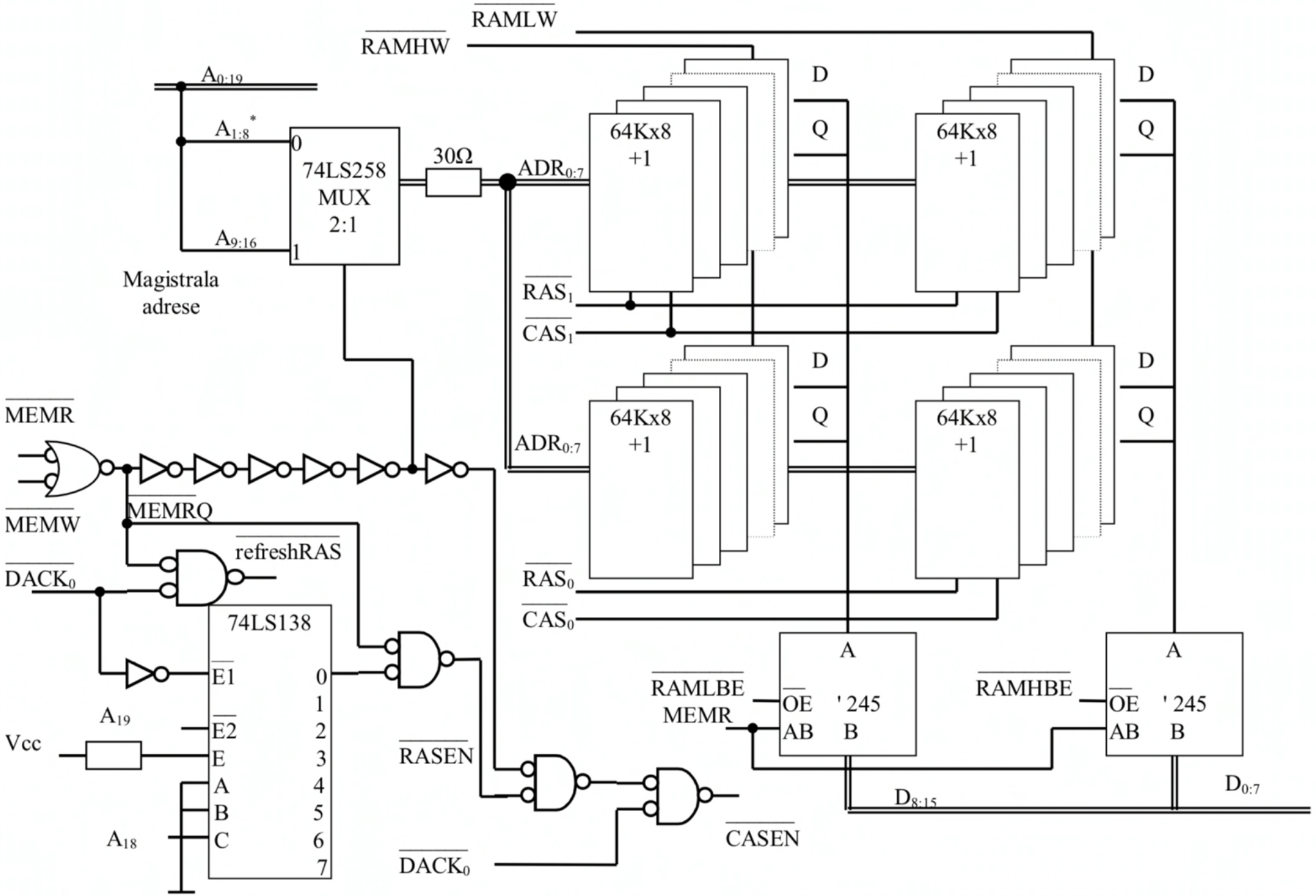
Dacă A8 – µP controlează mem.

Dacă A0 – DMA controlează mem.

$\overline{DACK_0} = 0$ → se utilizează A0



RAM 256K x (8+1)


RAMLW
RAMHW
A0:19
A1:8*
A9:16
0
1
74LS258
MUX
2:1
30Ω
ADR0:7
64Kx8
+1
D
Q
Magistrala
adrese
RAS1
CAS1
RAS0
CAS0
MEMR
MEMW
MEMRQ
refreshRAS
DACK0
74LS138
E1
E2
E
A
B
C
0
1
2
3
4
5
6
7
A19
Vcc
A18
RASEN
DACK0
CASEN
A
OE
AB
'245
B
RAMLBE
MEMR
RAMHBE
D8:15
D0:7

$A_{17} = 0 \rightarrow \overline{RAS_0}$

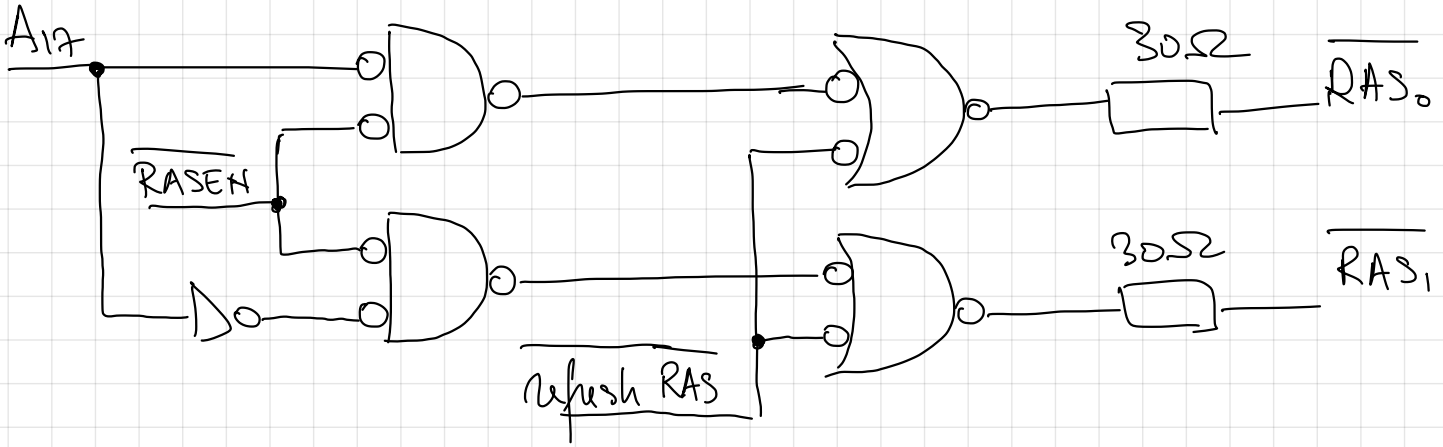
$A_{17} = 1 \rightarrow \overline{RAS_1}$

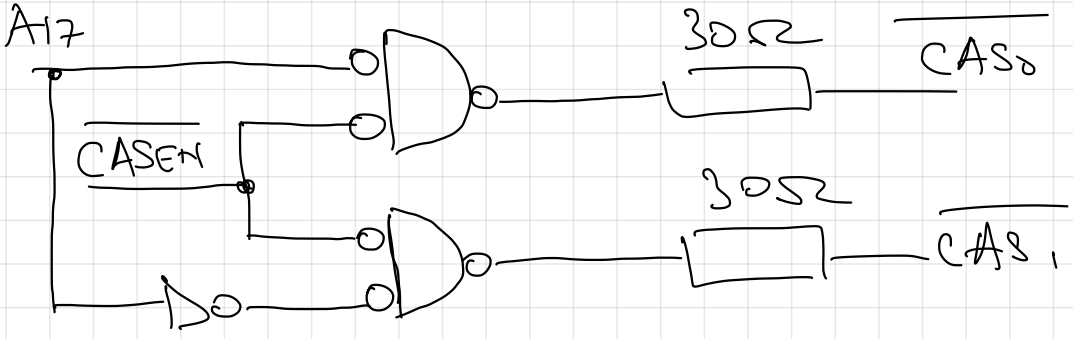
→ doar când $\overline{RASEN}$ este activ

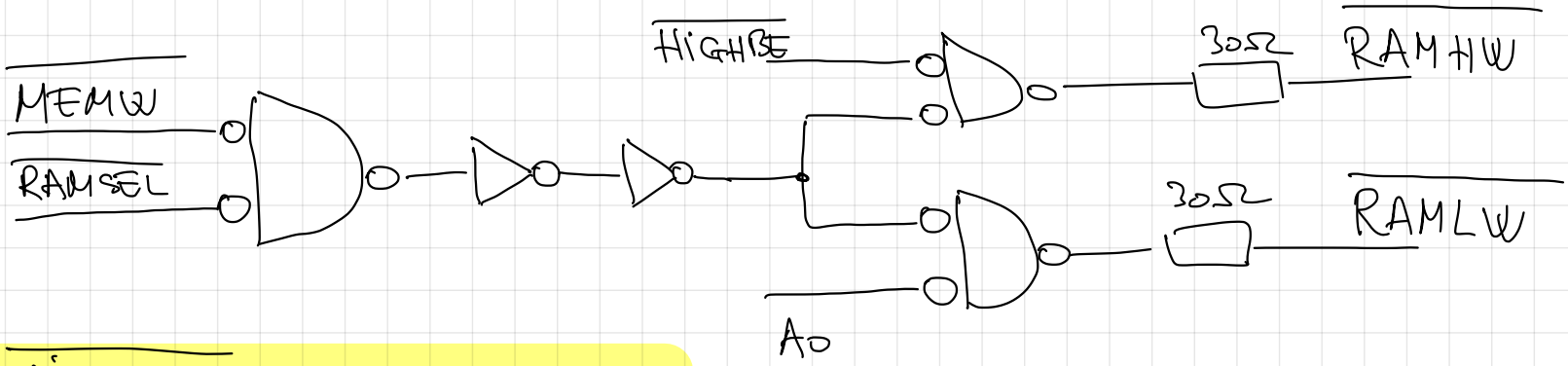
$A_{17} = 0 \rightarrow \overline{CAS_0}$

$A_{17} = 1 \rightarrow \overline{CAS_1}$

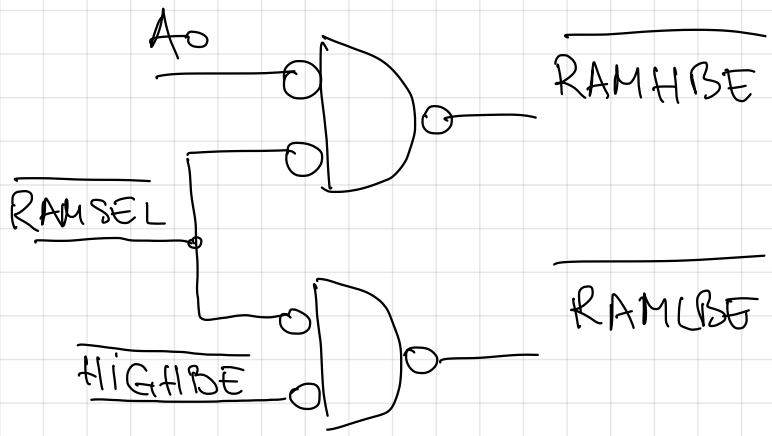
→ doar când $\overline{CASEN}$ este activ







$$\overline{HIGHBE} = \overline{A_0} \cdot \overline{BHE}$$



PM - Cursul 10

Notițe de curs

Porturi de i/O

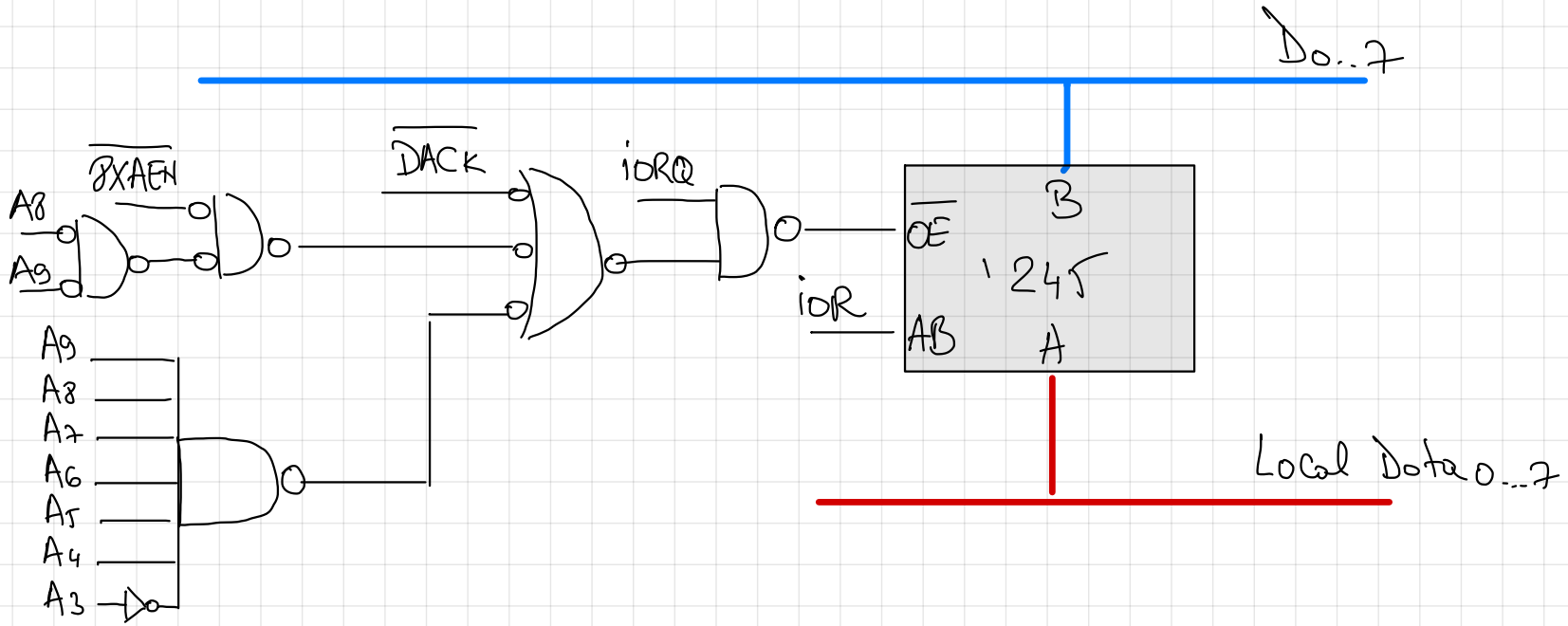
IBM XT/AT → MSDOS → ~~Q~~DOS

8086 ← X86-64

0 ... 0FFh	Rezervate pt placa de bază
100 ... 3FFh	Adrese pentru porturi de i/o
400h ... 0FFFh	Nu sunt disponibile pe magistrală

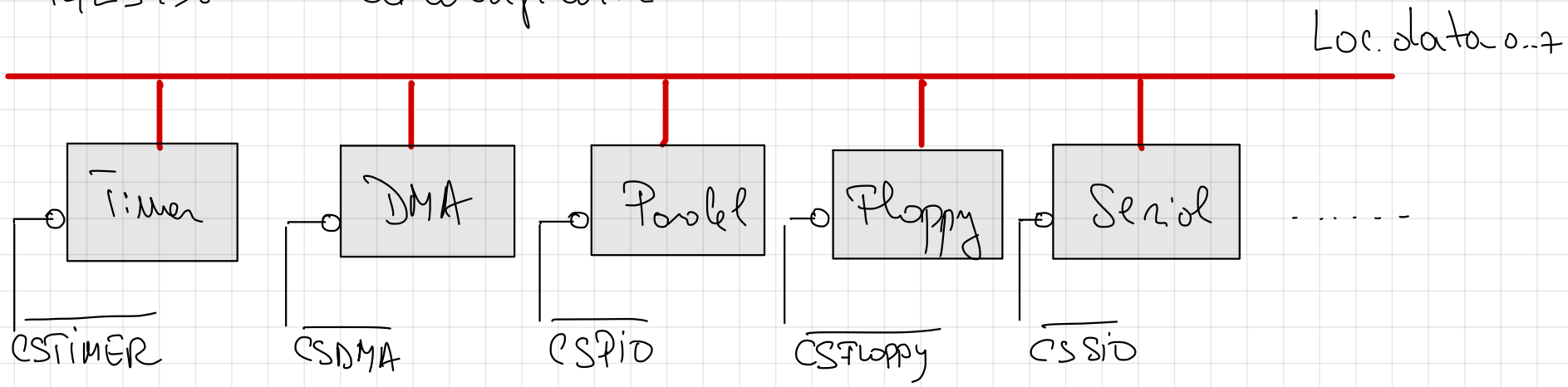
100 ... 10F h – rezervat pt DMA
120h .. 12Fh – sistemul de intreruperi (8259A) → P.I.C.
140h .. 14Fh – timer (8253)
160h .. 16Fh – interfata paralela
180h .. 18Fh – registre pagina modul DMA
3F0h ... 3F7h – floppy drive
378h .. 37Fh – port paralel (printer)

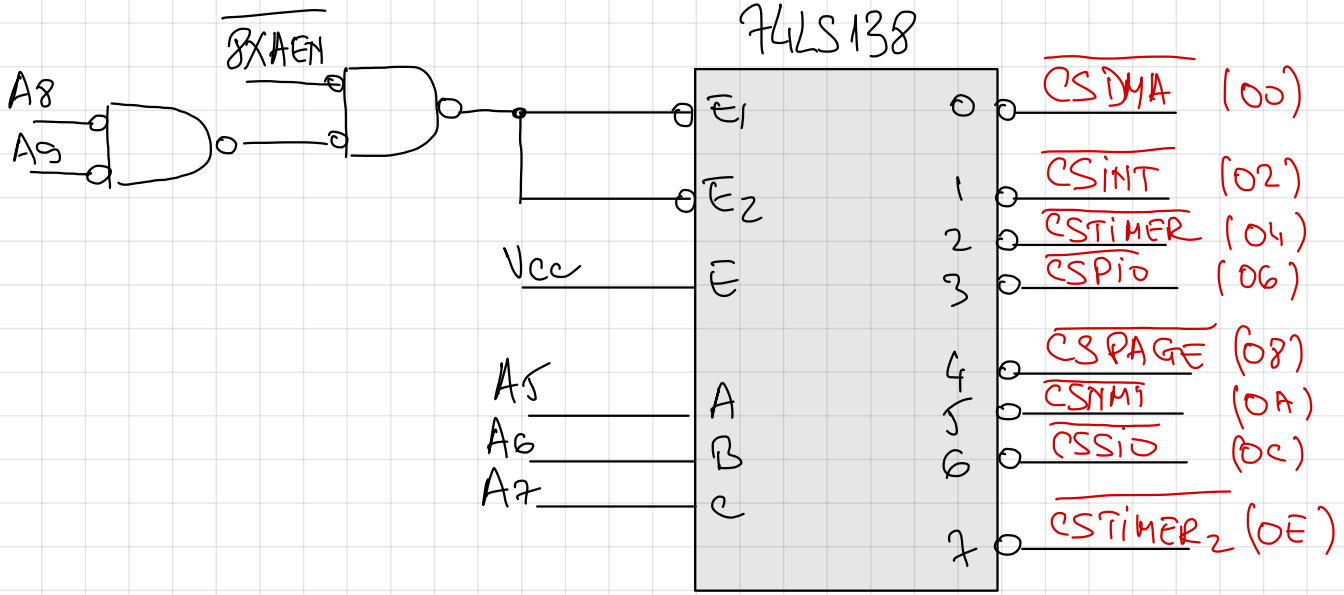
3F8h ... 3FFh – interfața serială (asincronă)
→ Adrese de pe magistrala locală slota



Decodificarea porturilor I/O

74LS138 → decodificator 3:8





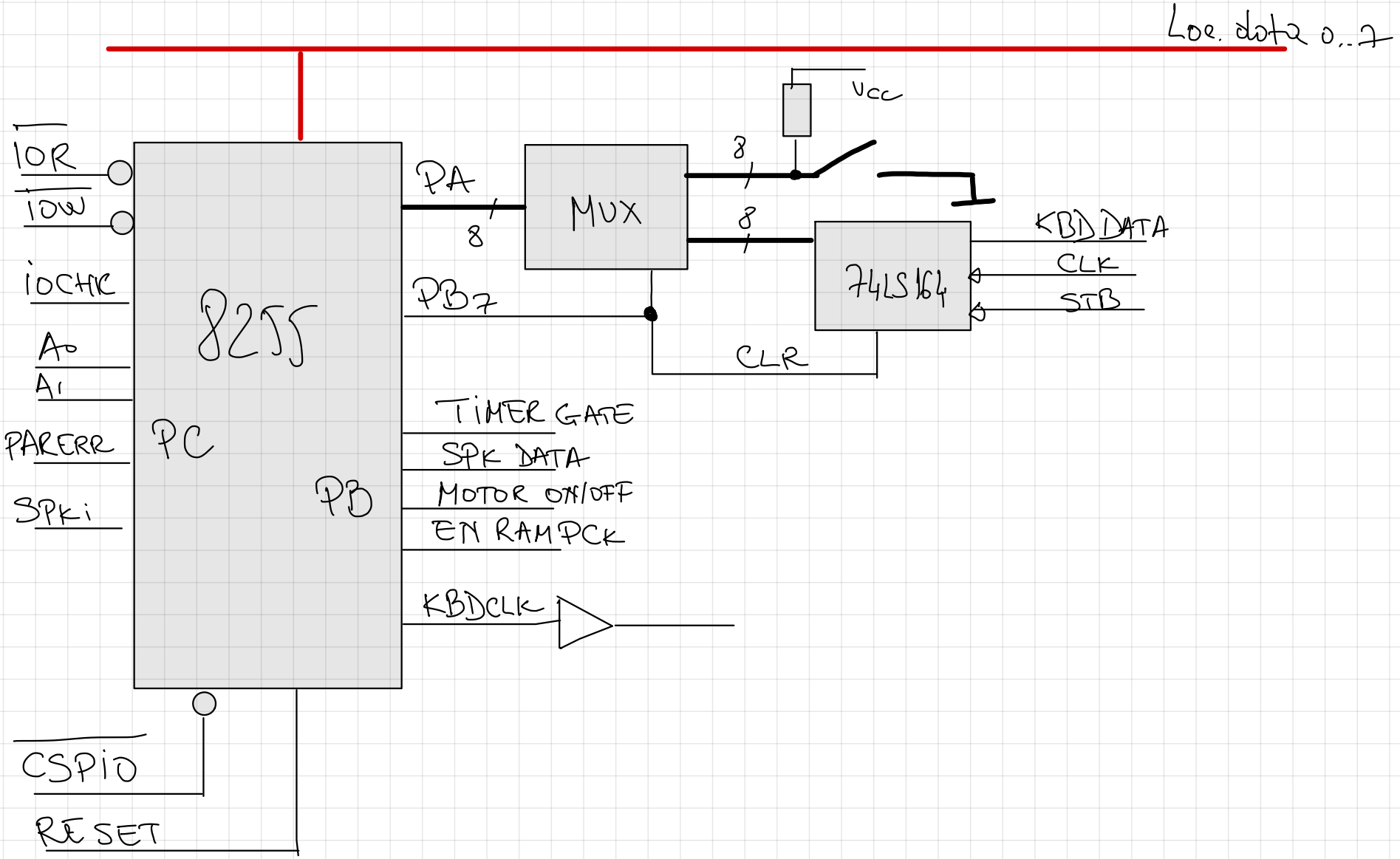
Cuplarea interfeței paralele

8255 – IC interfață paralelă
– trei porturi: PORT A, PORT B, PORT C

PORT A:
- Switchuri configurare sistem
- citirea info. tastaturii

PORT B:
- enable timer (TIMER GATE)
- speaker (PWM)
- activare / dezactivare motor floppy
- semnal activare / dezactivare logică protecție mem. RAM

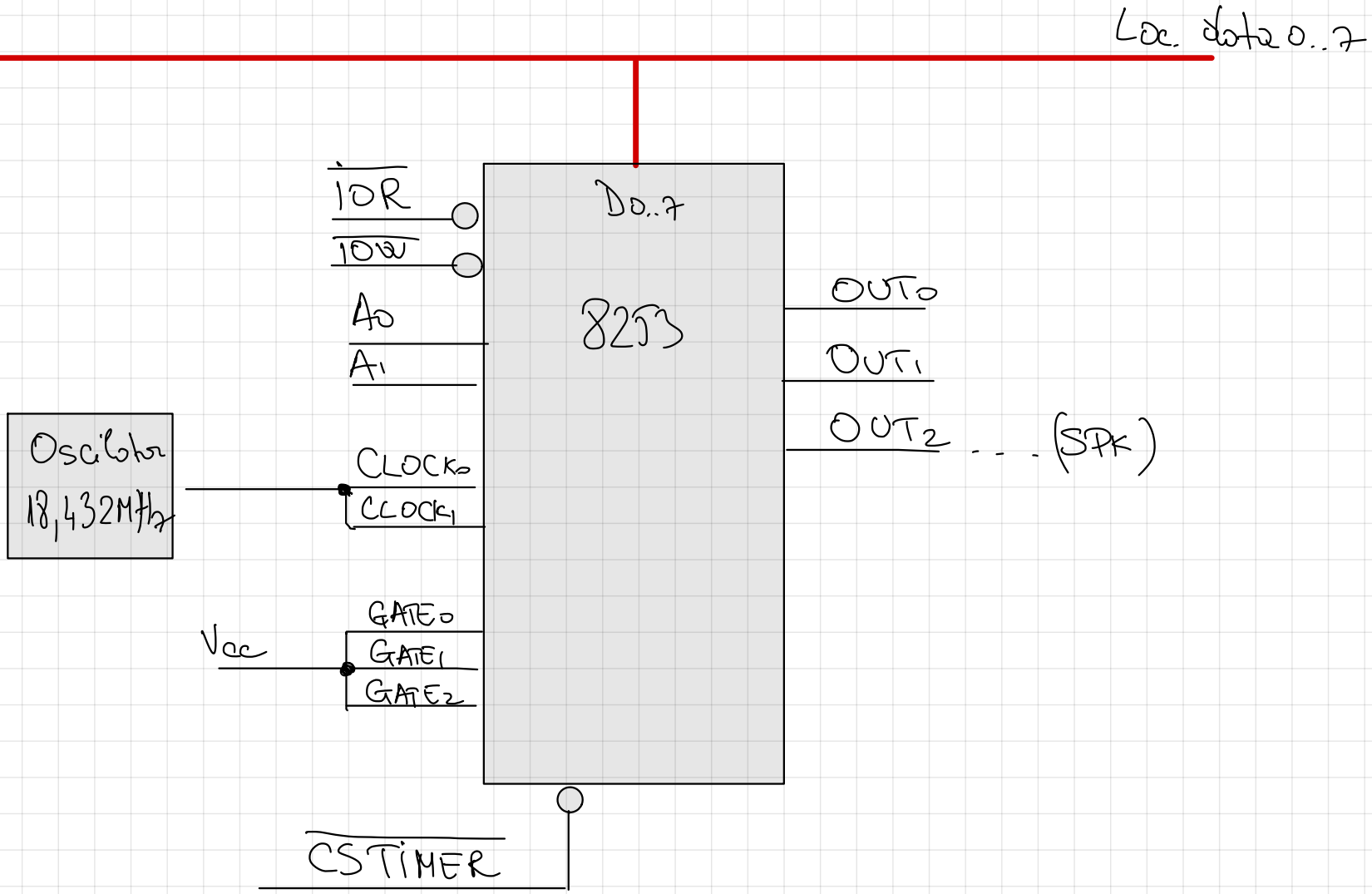
PORT C:
- erori de paritate RAM
- citire semnal comandă obturator



https://en.m.wikipedia.org/wiki/8255

Explorea timer-ului

8253 - circuit timer



https://en.m.wikipedia.org/wiki/Intel_8253

PM - Cursul 11

Notițe de curs

Controller DMA

MCS85 ← 8085 - CPU intel

64KB ← spatiu adresa

1,6 MB/s

4 canale de DMA

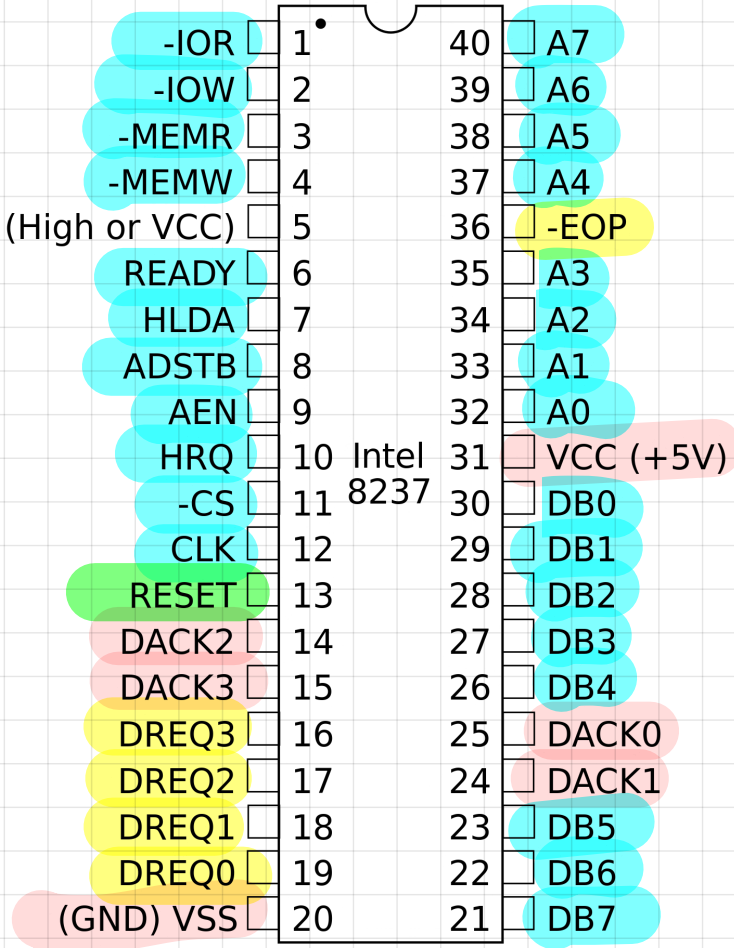
IBM PC/XT

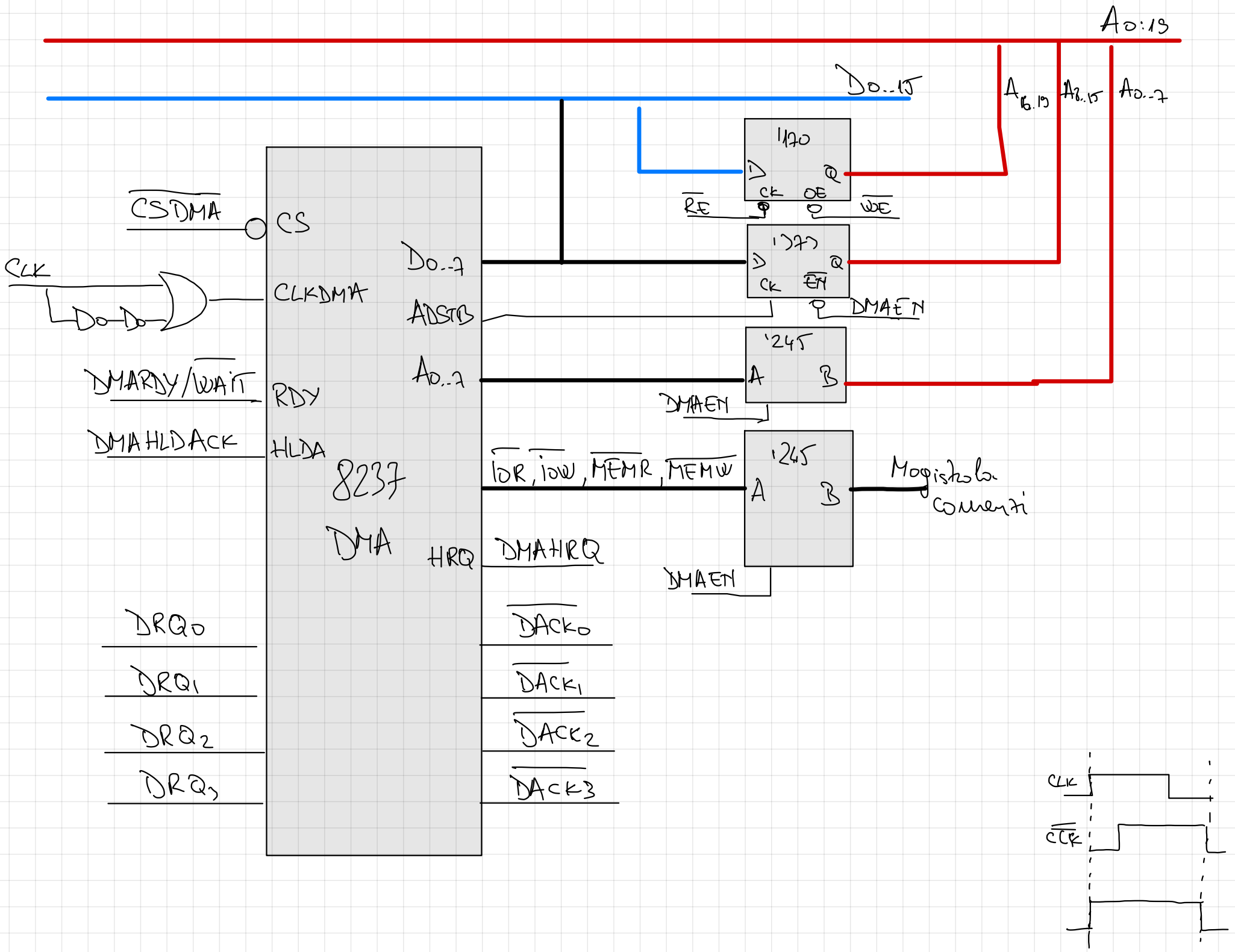
Cascadare in config Master-slave

4 moduri de operare:

- simple - intreaga cicli DMA si μP
- block
- demand

  DREQ$_i$ ↑ → ack DACK$_i$ ↑ → start transfer .... TC expires sau EOP ↓ sau DREQ$_i$ ↓
- cascade

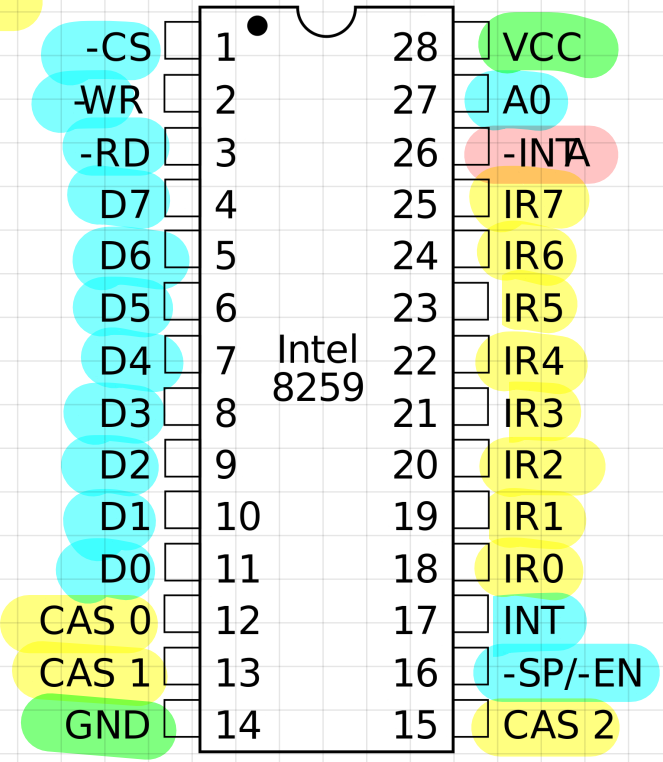


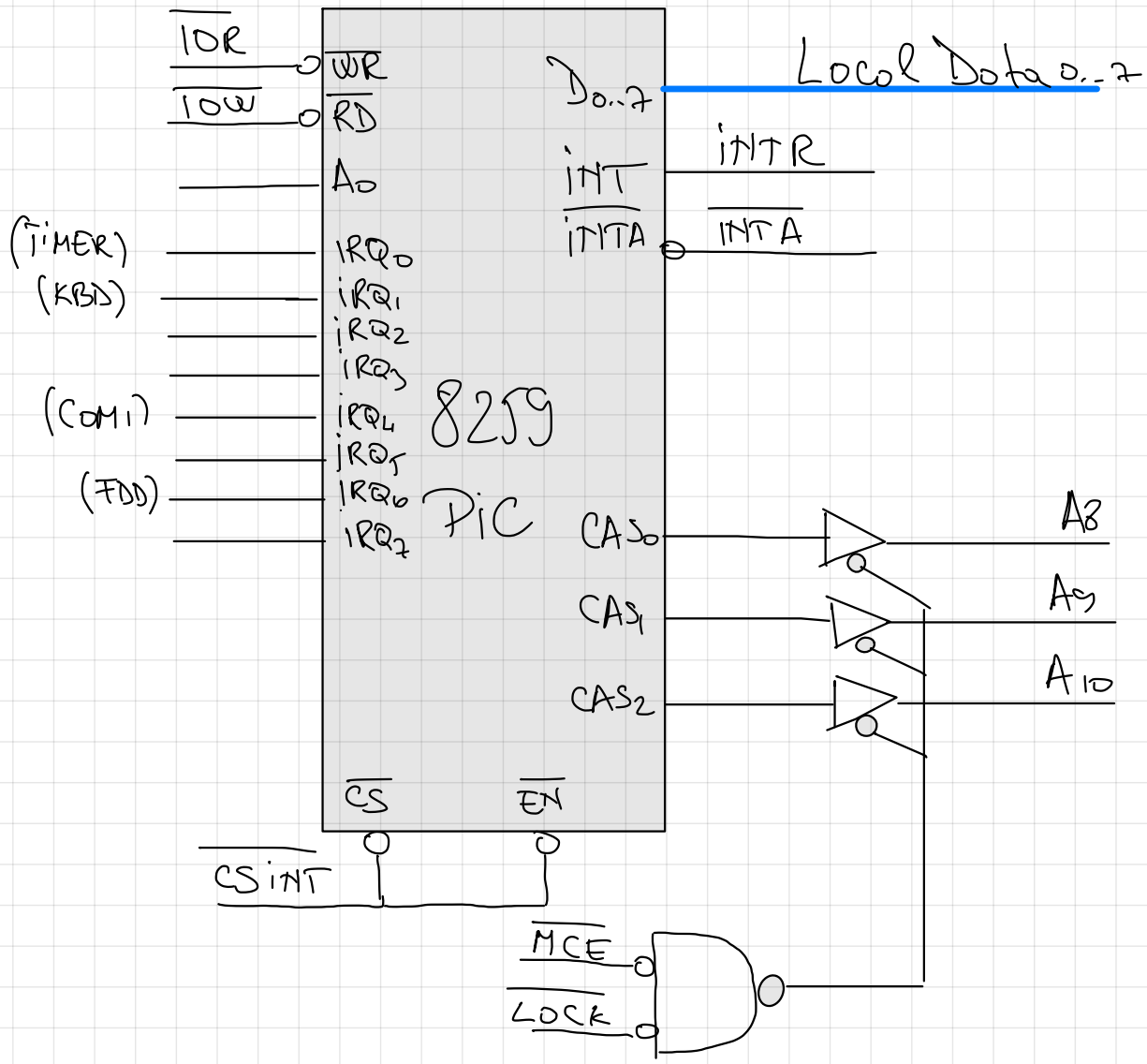

A0:19
D0..15
A16..19
A8..15
A0..7
'170
CK
OE
RE
WE
'377
CK
EN
DMAEN
'245
A
B
DMAEN
'245
A
B
Magistrale comandi
DMAEN
CSDMA
CS
CLK
CLKDMA
DMARDY/WAIT
RDY
DMAHLDACK
HLDA
8237
DMA
D0..7
ADSTB
A0..7
IOR, IOW, MEMR, MEMW
HRQ
DMAHRQ
DRQ0
DRQ1
DRQ2
DRQ3
DACK0
DACK1
DACK2
DACK3
CLK
CCK

Programmable Interrupt Controller (PIC)

8259 - 8 canale de intreruperi (IRQ)
Cascadabil in config master-slave
Maxim 64 IRQ-uri → 8 slave la 1 master
South Bridge → PC modern

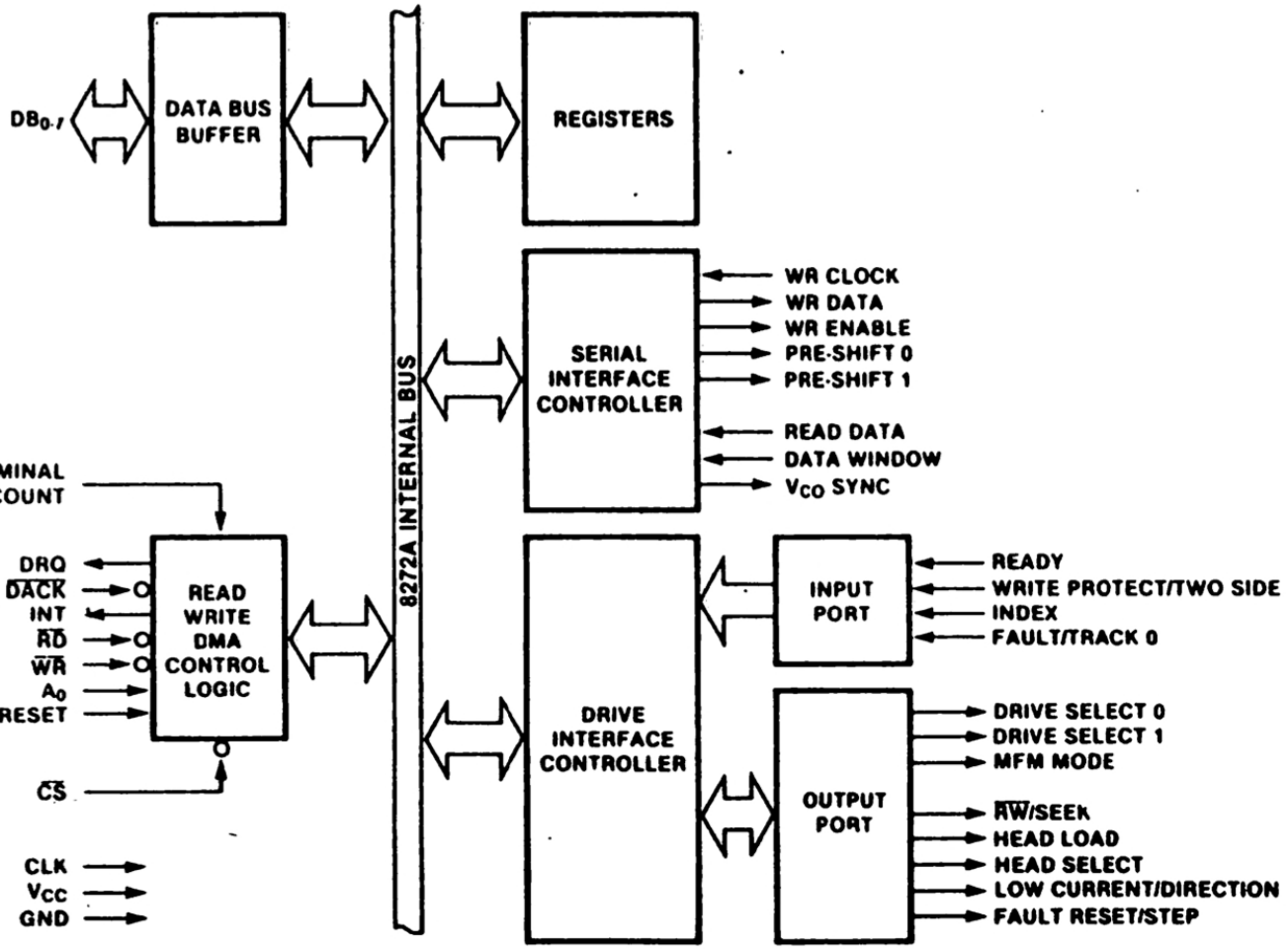
IMR - Interrupt Mask Register
IRR - Interrupt Request Register
ISR - In-Service Register

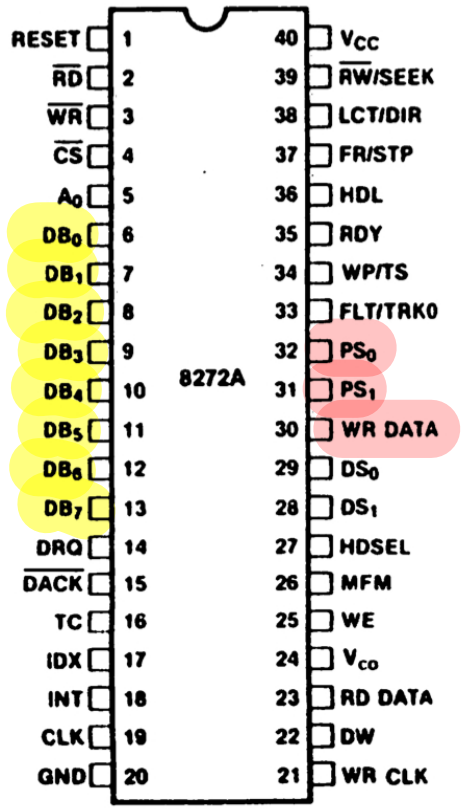


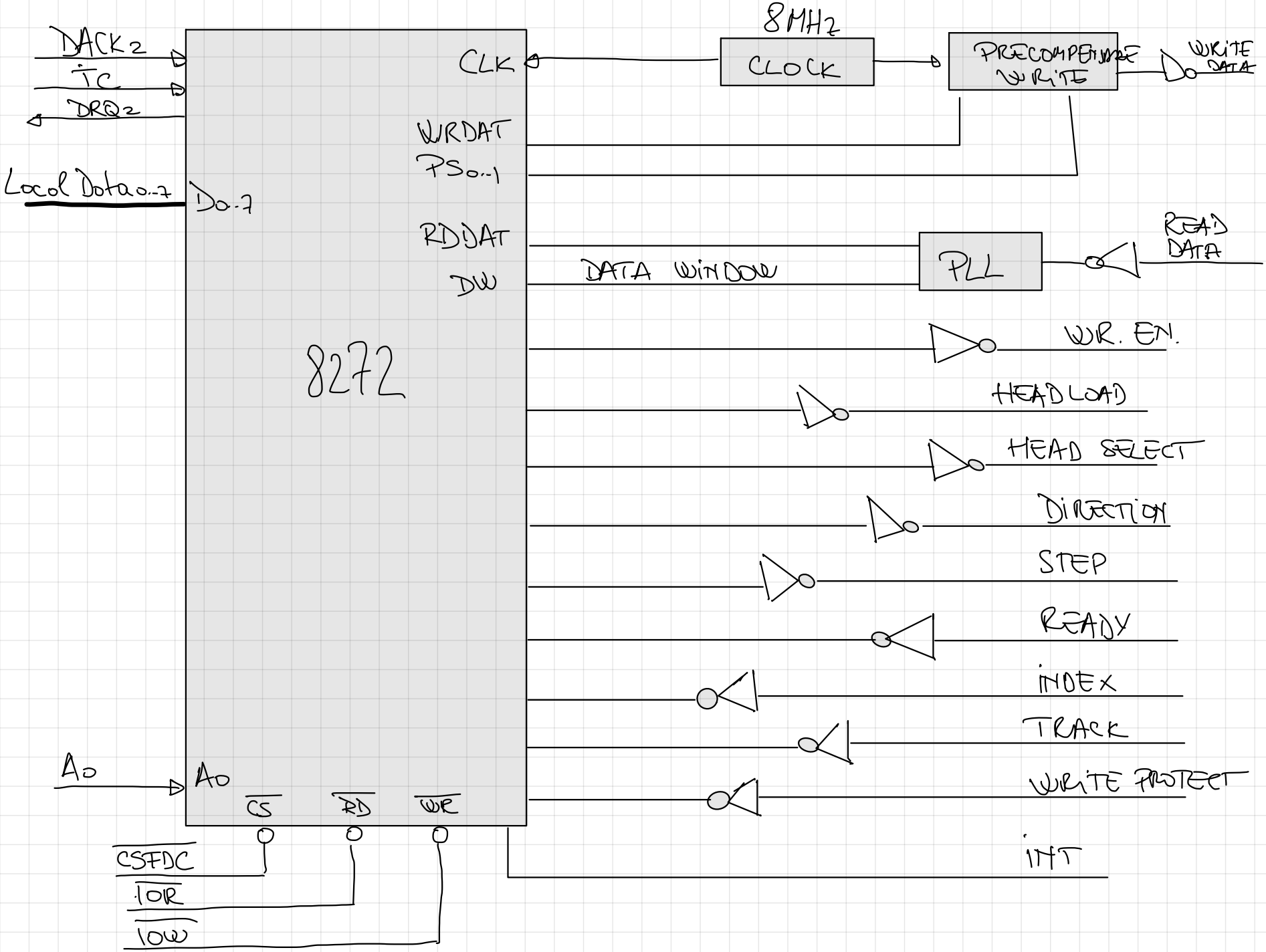

IOR
IOW
WR
RD
A0
(TIMER)
(KBD)
(COM1)
(FDD)
IRQ0
IRQ1
IRQ2
IRQ3
IRQ4
IRQ5
IRQ6
IRQ7
8259
PIC
D0..7
INT
INTA
Local Data 0..7
INTR
INTA
CAS0
CAS1
CAS2
A8
A9
A10
CS
EN
CS INT
MCE
LOCK

Controller Floppy Disk Drive

- capacità programmabile pt. transfer 128, 256, 512, 1024 Bytes/sector
- capacità cascadare 4 FDD
- transfer DMA
- Compatibil µP Intel






8MHz
CLOCK
PRECOMPENSATE WRITE
WRITE DATA
DACK2
TC
DRQ2
Local Data0-7
D0-7
CLK
WRDAT
PS0-1
RDDAT
DW
8272
DATA WINDOW
PLL
READ DATA
WR. EN.
HEADLOAD
HEAD SELECT
DIRECTION
STEP
READY
INDEX
TRACK
WRITE PROTECT
INT
A0
A0
CS
RD
WR
CSFDC
IOR
IOW